



AJ INSTITUTE OF ENGINEERING & TECHNOLOGY

A Unit of Laxmi Memorial Education Trust ®

(Approved by AICTE, New Delhi, Affiliated to Visvesvaraya Technological University, Belgavi)

Department of Electronics and Communication Engineering

NOTES ON
COURSE NAME: DIGITAL SYSTEM DESIGN
USING VERILOG

COURSE CODE: BEC302

22 SCHEME

Course Instructor: Ms. Pratheeksha Rai N

Assistant Professor

Department of ECE, AJIET



AJ INSTITUTE OF ENGINEERING & TECHNOLOGY

A Unit of Laxmi Memorial Education Trust ®

(Approved by AICTE, New Delhi, Affiliated to Visvesvaraya Technological University, Belgaavi)

Department of Electronics and Communication Engineering

Vision of the institute

“To produce top-quality engineers who are groomed for attaining excellence in their profession and competitive enough to help in the growth of nation and global society.”

Mission of the institute

- M1: To offer affordable high-quality graduate program in engineering with value education and make the students socially responsible.
- M2: To support and enhance the institutional environment to attain research excellence in both faculty and students and to inspire them to push the boundaries of knowledge base.
- M3: To identify the common areas of interest amongst the individuals for the effective industry- institute partnership in a sustainable way by systematically working together.
- M4: To promote the entrepreneurial attitude and inculcate innovative ideas among the engineering professionals.

Department Vision

To be recognized as a center of excellence in the area of Electronics and Communication Engineering by nurturing the young innovative minds into skillful and ethical professionals to cater the industrial and societal needs.

Department Mission

- M1. To establish state-of-the art laboratories to facilitate research and innovation to upgrade the knowledge and skills in healthcare sector and IoT.
- M2. To provide industry interaction for training programs on latest technology.
- M3. To provide ethical and value based education by promoting activities addressing the societal needs.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

- PEO1. Exhibit a desire for lifelong learning through professional and societal activities.
- PEO2. Exhibit and apply their technical skills and knowledge in Electronics and Communication Engineering for industry and societal needs.
- PEO3. Exhibit leadership qualities, professional skills, management skills and ethics needed for successful career.



AJ INSTITUTE OF ENGINEERING & TECHNOLOGY

A Unit of Laxmi Memorial Education Trust ®

(Approved by AICTE, New Delhi, Affiliated to Visvesvaraya Technological University, Belgaavi)

Department of Electronics and Communication Engineering

PROGRAM OUTCOMES (POs)

- 1. Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- 2. Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- 3. Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- 4. Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- 5. Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- 6. The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- 7. Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- 8. Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- 9. Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- 10. Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.



Department of Electronics and Communication Engineering

- 11. Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- 12. Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

- PSO1. **Embedded Systems:** Ability to apply the fundamental knowledge of core Electronics and Communication Engineering courses in the analysis, design, and development of integrated electronic systems and healthcare devices.
- PSO2. **Communication Systems:** Ability to apply the fundamental knowledge of signal processing in the analysis, design, and development of communication systems.
- PSO3. **Simulation:** Ability to use modern electronic tools such as MATLAB, Xilinx, Multisim etc, to design and analyze the complex electronics and communication systems.



AJ INSTITUTE OF ENGINEERING & TECHNOLOGY

A Unit of Laxmi Memorial Education Trust ®

(Approved by AICTE, New Delhi, Affiliated to Visvesvaraya Technological University, Belga))

Department of Electronics and Communication Engineering

COURSE OBJECTIVES: This course will enable students to:

Sl. No.	DESCRIPTION
1	To impart the concepts of simplifying Boolean expression using K-map techniques and Quine- McCluskey minimization techniques.
2	To impart the concepts of designing and analysing combinational logic circuits.
3	To impart design methods and analysis of sequential logic circuits.
4	To impart the concepts of Verilog HDL-data flow models for the design of digital systems.
5	To impart the concepts of Verilog HDL- behavioural models for the design of digital systems.

COURSE OUTCOMES (COs): After studying this course, students will be able to:

Sl. No.	DESCRIPTION	PO MAPPING	PSO MAPPING
CO1	Simplify Boolean functions using K-map and Quine-McCluskey minimization technique.	PO1, PO2,PO3, PO5	PSO1,PSO3
CO2	Analyse and design for combinational logic circuits.	PO1, PO2, PO3,PO5	PSO1, PSO3
CO3	Analyse the concepts of Flip Flops (SR, D, T and JK) and to design the synchronous sequential circuits using Flip Flops.	PO1, PO2, PO3,PO5	PSO1,PSO3
CO4	Combinational circuits (adders, subtractors, multiplexers) using Verilog descriptions.	PO1, PO2, PO3,PO5	PSO1,PSO3
CO5	Concepts of Flip Flops (SR, D, T and JK) and to design the synchronous sequential circuits using Verilog descriptions.	PO1, PO2, PO3,PO5	PSO1,PSO3



AJ INSTITUTE OF ENGINEERING & TECHNOLOGY

A Unit of Laxmi Memorial Education Trust

(Approved by AICTE, New Delhi, Affiliated to Visvesvaraya Technological University, Belga))

Department of Electronics and Communication Engineering

COURSE ARTICULATION MATRIX:

Course Name	Course Code					PO Mapped							PSO Mapped		
Digital System Design Using Verilog	BEC302					PO 1,2,3,5							PSO 1,3		
Course Outcomes	Program Outcomes												Program Specific Outcomes		
	1	2	3	4	5	6	7	8	9	10	11	12	1	2	3
Simplify Boolean functions using K-map and Quine-McCluskey minimization technique.	3	3	2		2								2		2
Analyze and design for combinational logic circuits.	3	3	2		2								2		2
Analyze the concepts of Flip Flops (SR, D, T and JK) and to design the synchronous sequential circuits using Flip Flops.	3	3	2		2								2		2
Combinational circuits (adders, subtractors, multiplexers) using Verilog descriptions.	2	3	3		3								2		3
Combinational sequential circuits using Verilog descriptions.	2	3	3		3								2		3

Course Outcome addresses program outcome: 1 - Lightly, 2 – Moderately, 3 - Highly

Module - 1

PRINCIPLES OF COMBINATIONAL LOGIC

Syllabus:

Principles of Combinational Logic: Definition of combinational logic, Canonical forms, Generation of switching equations from truth tables, Karnaugh maps- up to 4 variables, Quine-McCluskey Minimization Technique. Quine-McCluskey using Don't Care Terms. (Section 3.1 to 3.5 of Text 1).

Text1: Digital Logic Applications and Design by John M Yarbrough, Thomson Learning, 2001.

Introduction

- The logic networks described by a Boolean algebra are divided into two general categories namely combinational circuits and sequential circuits.
- Combinational networks are categorized by the fact that the outputs at any instant are functions of the inputs at any instants. Combinational network does not have a memory.
- Sequential networks are categorized by the fact that the outputs at any instant are not only a function of the current inputs but, in addition depend upon the past history of the inputs. Sequential network has a memory.

Definition of Combinational Logic

- Combinational logic deals with the technique of combining the basic gates, into circuits that perform desired function.
- Some of the useful combinational logic functions are adders, subtractors, decoders, encoders, multiplexers, demultiplexers, display drivers, keyboard encoders etc.
- Logic circuits without feedback from output to the input, constructed from a functionally complete gate set, are said to be combinational. Combinational network does not have a memory.
- Let X be the set of all input variables $\{x_0, x_1, \dots, x_n\}$ and Y be the set of all output variables $\{y_0, y_1, \dots, y_n\}$. Then the output is related to the input as

$$Y = F(x)$$

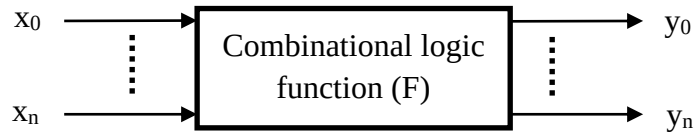


Fig.1.1: Combinational Logic Model

- For any combinational logic system can be designed it must be defined. The fig.1.2 illustrates the sequence of design tasks in general way.

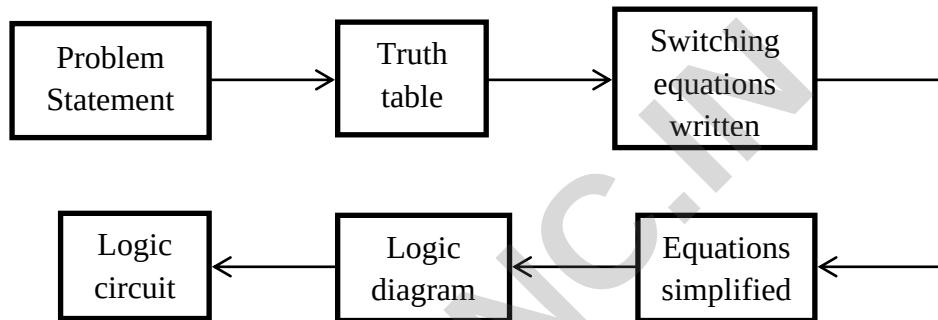


Fig.1.2: General Combinational Logic Design Sequence

- The first task is to define the problem to be solved. The problem is then rewritten in the form of truth table. From the truth table, the switching equations can be written, simplified and logic diagram drawn.
- Logic can be stated in several ways such as in truth table, in logic diagrams and by Boolean equations. Each input variable combination that produces logic 1 in the output column of the truth table can form a term in the Boolean switching Equations.

Truth Table Representation

- A truth table is a logical representation of a problem statements (or verbal descriptions of problems) in a tabular form which specifies number of inputs, outputs and a logic between input and output.

Problem:

- Develop a truth table for a system which accepts two 2-bit binary number and generates three outputs. The first output indicates when the two numbers differ by 2 or more, the second output indicates when the two numbers are identical and the third output indicates when the first number exceeds the second number.**

Let first number be AB (A is MSB (Most Significant Bit) and B is LSB (Least Significant Bit)) and second number be CD (C is MSB and D is LSB). Let X, Y, Z be the outputs. The truth table is represented as

Inputs				Outputs		
A	B	C	D	X	Y	Z
0	0	0	0	0	1	0
0	0	0	1	0	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	0	0	0
0	1	1	1	1	0	0
1	0	0	0	1	0	1
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	0	0	0
1	1	0	0	1	0	1
1	1	0	1	1	0	1
1	1	1	0	0	0	1
1	1	1	1	0	1	0

2. Give the truth table representation of a system which takes two 2-bit binary number as its inputs and generates an output to indicate when the sum of the two numbers is odd.

Let first number be AB (A is MSB (Most Significant Bit) and B is LSB (Least Significant Bit)) and second number be CD (C is MSB and D is LSB). Let Y be the output. The truth table is represented as

Inputs	Output
--------	--------

A	B	C	D	Y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	1
0	1	0	1	0
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0

3. Write a truth table for a four input system indicating when majority of inputs are true.

Let A, B, C, D be inputs and let Y be the output. The truth table is represented as

Inputs				Output
A	B	C	D	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0

0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Definitions of the logic terms related to boolean expression

Literal:

A literal is a Boolean variable or its complement.

For example: If X is a Boolean variable then both X and $\bar{X}$ are literal.

Product term:

A product term is a literal or logical product (AND) of multiple literals.

For example: If X, Y and Z are Boolean variables then X, XY, $\bar{X}YZ$ are some of the product terms

Sum term:

A sum term is a literal or logical OR of multiple literals.

For example: If X, Y and Z are Boolean variables then X, $X + Y$, $\bar{X} + Y + Z$ are some of the sum terms

Sum of products (SOP):

It is logical OR of multiple product terms.

For example: If X, Y and Z are Boolean variables then $F(X, Y, Z) = XY + \bar{X}\bar{Y} + YZ$ is a sum of product expression.

Product of Sum (POS):

It is logical AND of multiple sum terms.

For example: If X, Y and Z are Boolean variables then $F(X, Y, Z) = (X + Y)(X + Y + Z)$ is product of sum expression.

Minterms:

It is a product term where all the input variables appear once either in complemented or uncomplemented form.

For example: if X, Y and Z are Boolean variables then $XYZ, \bar{X}YZ$ are some of the minterms

Maxterms:

It is a sum term where all the input variables appear once either in complemented or uncomplemented form.

For example: if X, Y and Z are Boolean variables then $X+Y+Z, \bar{X} + Y + Z$ are some of the maxterms

Canonical sum of products:

It is a complete set of minterms that defines when an output variable is logic 1.

For example: if X, Y and Z are Boolean variables then

$F(X, Y, Z) = \bar{X}YZ + X\bar{Y}Z + XYZ$ is a canonical sum of product.

Canonical product of sums:

It is a complete set of maxterms that defines when an output variable is logic 0.

For example: if X, Y and Z are Boolean variables then

$F(X, Y, Z) = (\bar{X} + Y + Z)(X + \bar{Y} + \bar{Z})(X + Y + Z)$ is a canonical sum of product.

Prime Implicant:

It is a permitted group of minterms or maxterms

Essential prime implicant:

A prime implicant that contains one or more minterms or maxterms that is unique i.e. terms not present in any other implicant

Canonical Forms

- Canonical form or standard form of Boolean function will contain all the variables in either true form or complemented form
- Canonical form or standard form can be expressed as either Sum of minterms or Product of maxterms.

- Procedure to convert a SOP expression into standard canonical form
 - i. Identify the missing variables in the each AND term.
 - ii. AND the missing term and its complement with the original AND term.
 - iii. Expand the term by property of distribution.
- Procedure to convert a POS expression into standard canonical form
 - i. Identify the missing variables in the each OR term.
 - ii. OR the missing term and its complement with the original OR term.
 - iii. Expand the term by property of distribution.

Problems:

1. Convert the following SOP equation into proper or standard canonical form

$$Y = f(a, b, c) = a\bar{b} + bc + a\bar{c}$$

In the first term $a\bar{b}$ the missing variable is c , so AND $(c + \bar{c})$ with first term.

$$\text{i.e. } a\bar{b}(c + \bar{c}) = a\bar{b}c + a\bar{b}\bar{c}$$

In the second term bc the missing variable is a , so we AND $(a + \bar{a})$ with second term.

$$\text{i.e. } bc(a + \bar{a}) = bca + bc\bar{a}$$

In the third term $a\bar{c}$ the missing variable is b , so AND $(b + \bar{b})$ with third term.

$$\text{i.e. } a\bar{c}(b + \bar{b}) = a\bar{c}b + a\bar{c}\bar{b}$$

Therefore, the standard canonical form is,

$$Y = a\bar{b}c + a\bar{b}\bar{c} + abc + \bar{a}bc + ab\bar{c} + a\bar{b}\bar{c}$$

$$Y = a\bar{b}c + a\bar{b}\bar{c} + abc + \bar{a}bc + ab\bar{c} + a\bar{b}\bar{c}$$

2. Convert the following POS equation into proper or standard canonical form

$$Y = f(a, b, c) = (a + \bar{b})(\bar{b} + c)$$

In the first term $a + \bar{b}$ the missing variable is c , so OR $(c\bar{c})$ with first term.

$$\text{i.e. } a + \bar{b} + c\bar{c} = (a + \bar{b} + c)(a + \bar{b} + \bar{c}) \quad [x + yz = (x + y)(x + z)]$$

In the second term $\bar{b} + c$ the missing variable is a , so OR $(a\bar{a})$ with second term.

$$\text{i.e. } \bar{b} + c + a\bar{a} = (\bar{b} + c + a)(\bar{b} + c + \bar{a})$$

Therefore, the standard canonical form is,

$$Y = (a + \bar{b} + c)(a + \bar{b} + \bar{c})(\bar{a} + \bar{b} + c)$$

Generation of switching equation from truth tables

- Switching equations can be written more conveniently by using the minterm or maxterm representation. For an n-variable logical function there are 2^n minterms and 2^n maxterms.
- For example, consider a truth table for 3-variable logical function.

Table 1.1: Minterm and maxterm of 3 variables

i/p Variables			Minterm Representation	Maxterm Representation
A	B	C		
0	0	0	$\bar{A} \bar{B} \bar{C} = m_0$	$A + B + C = M_0$
0	0	1	$\bar{A} \bar{B} C = m_1$	$A + B + \bar{C} = M_1$
0	1	0	$\bar{A} B \bar{C} = m_2$	$A + \bar{B} + C = M_2$
0	1	1	$\bar{A} B C = m_3$	$A + \bar{B} + \bar{C} = M_3$
1	0	0	$A \bar{B} \bar{C} = m_4$	$\bar{A} + B + C = M_4$
1	0	1	$A \bar{B} C = m_5$	$\bar{A} + B + \bar{C} = M_5$
1	1	0	$A B \bar{C} = m_6$	$\bar{A} + \bar{B} + C = M_6$
1	1	1	$A B C = m_7$	$\bar{A} + \bar{B} + \bar{C} = M_7$

- Each minterm is represented by m_i and each maxterm by M_i where i is the decimal equivalent of the binary number.
- The logic expression in standard sum of product form can be written for the corresponding truth table by writing one product term for each input that produces an output of 1.

i/p Variables			o/p
A	B	C	
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0

1	1	0	1
1	1	1	0

- Sum of product form for the above truth table can be expressed as,

$$Y = f(A, B, C) = \bar{A}B\bar{C} + \bar{A}BC + AB\bar{C}$$

or

$$Y = f(A, B, C) = \sum m(2, 3, 6)$$

or

$$Y = f(A, B, C) = \sum (m2, m3, m6)$$

- Σ -Notation is used to represent the minterms in a simple way using decimal numbers.
- Similarly, the logic expression in standard product of sum form can be written for the corresponding truth table by writing one sum term for each input that produces an output of 0.
- Product of sum form can be expressed as,

$$Y = f(A, B, C) = (A + B + C)(A + B + \bar{C})(\bar{A} + B + C)(\bar{A} + B + \bar{C})(\bar{A} + \bar{B} + \bar{C})$$

or

$$Y = f(A, B, C) = \pi M(0, 1, 4, 5, 7)$$

or

$$Y = f(A, B, C) = \pi (M0, M1, M4, M5, M7)$$

- π -Notation is used to represent the maxterms in a simple way using decimal numbers.

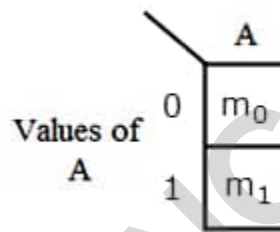
Karnaugh Maps (K-map)

- Simplification of switching equations reduces the amount of hardware needed to realize a given function.
- Boolean algebra can be used to simplify the boolean expression or switching equations but it is lengthy process. A better approach is to use Karnaugh Maps (K-map).
- K-map is a matrix of squares where each square represents a minterm or maxterm from a Boolean equation.
- K-map is a graphical method, which consists of 2^n cells for 'n' variables. The adjacent cells are differed only in single bit position.
- K-Map method is most suitable for minimizing Boolean functions of 2 variables to 5 variables.

- Literal or Product terms are assigned to the cells of K- map labelling each row and each column of the k-map with a variable, with its complement, or with a combination of variables and complements. Fig. 1.1 shows the way to label rows and columns of different K-maps.
- It is important to note that from one cell to the next cell along any row or from one cell to next cell along any column, only one variable in the product term should change.

1-Variable K-Map

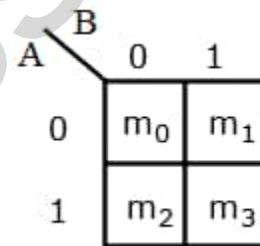
- For 1-variable there will be $2^1 = 2$ cells in the K-map as shown in the fig.



- Cells should be grouped separately.

2-Variable K-Map

- For 2-variables there will be $2^2 = 4$ cells in the K-map as shown in the fig.



- There is only one possibility of grouping 4 adjacent min terms.
- The possible combinations of grouping 2 adjacent min terms are $\{(m_0, m_1), (m_2, m_3), (m_0, m_2) \text{ and } (m_1, m_3)\}$.

3-Variable K-Map

- For 3-variables there will be $2^3 = 8$ cells in the K-map as shown in the fig.

Values of Variables
in Gray sequence

		BC			
		00	01	11	10
A	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6

- There is only one possibility of grouping 8 adjacent min terms.
- The possible combinations of grouping 4 adjacent min terms are $\{(m_0, m_1, m_3, m_2), (m_4, m_5, m_7, m_6), (m_0, m_1, m_4, m_5), (m_1, m_3, m_5, m_7), (m_3, m_2, m_7, m_6) \text{ and } (m_2, m_0, m_6, m_4)\}$.
- The possible combinations of grouping 2 adjacent min terms are $\{(m_0, m_1), (m_1, m_3), (m_3, m_2), (m_2, m_0), (m_4, m_5), (m_5, m_7), (m_7, m_6), (m_6, m_4), (m_0, m_4), (m_1, m_5), (m_3, m_7) \text{ and } (m_2, m_6)\}$.

4-Variable K-Map

- For 4-variables there will be $2^4 = 16$ cells in the K-map as shown in the fig.

		CD			
		00	01	11	10
AB	00	m_0	m_1	m_3	m_2
	01	m_4	m_5	m_7	m_6
	11	m_{12}	m_{13}	m_{15}	m_{14}
	10	m_8	m_9	m_{11}	m_{10}

- There is only one possibility of grouping 16 adjacent min terms.
- Let R_1, R_2, R_3 and R_4 represents the min terms of first row, second row, third row and fourth row respectively. Similarly, C_1, C_2, C_3 and C_4 represents the min terms of first column, second column, third column and fourth column respectively. The possible combinations of grouping 8 adjacent min terms are $\{(R_1, R_2), (R_2, R_3), (R_3, R_4), (R_4, R_1), (C_1, C_2), (C_2, C_3), (C_3, C_4), (C_4, C_1)\}$.

5-Variable K-Map

- For 5-variables there will be $2^5 = 32$ cells in the K-map as shown in the fig.

A = 0					A = 1				
BC \ DE					BC \ DE				
	00	01	11	10		00	01	11	10
00	m ₀	m ₁	m ₃	m ₂	00	m ₁₆	m ₁₇	m ₁₉	m ₁₈
01	m ₄	m ₅	m ₇	m ₆	01	m ₂₀	m ₂₁	m ₂₃	m ₂₂
11	m ₁₂	m ₁₃	m ₁₅	m ₁₄	11	m ₂₈	m ₂₉	m ₃₁	m ₃₀
10	m ₈	m ₉	m ₁₁	m ₁₀	10	m ₂₄	m ₂₅	m ₂₇	m ₂₆

- There is only one possibility of grouping 32 adjacent min terms.
- There are two possibilities of grouping 16 adjacent min terms. i.e., grouping of min terms from m₀ to m₁₅ and m₁₆ to m₃₁.

In the above all K-maps, we used exclusively the min terms notation. Similarly, you can use exclusively the Max terms notation.

Simplifying Minterm Equation (SOP expressions) using K-Maps

- The procedure to simplify SOP expressions are as follows:
1. Plot the K-map and place 1's in those cells corresponding to the 1's in the truth table or SOP expression. Place 0's in other cells.
 2. Check the K-map for all possible valid adjacent 1's and form a group of 2^n 1's and encircle those 1's which are not adjacent to any other 1's. They are called isolated 1's.
 3. While grouping make sure that there is minimum no. of groups.
 4. Then form the simplified expression by summing product terms of all the groups.

Problems:

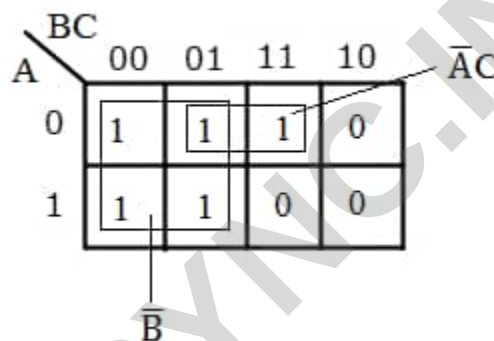
1. Simplify the following expression using K-map

$$Y = \overline{A}\overline{B}C + \overline{A}\overline{B}C + \overline{A}BC + A\overline{B}\overline{C} + \overline{A}\overline{B}\overline{C}$$

Procedure

- Plot 3-variable K-map for given expression.
- Place 1's in the cell 0, 1, 3, 4, 5 and 0's in the remaining cells.
- 1's in cell 3 and cell 1 are adjacent which can be grouped (group 1)
- 1's in cells 0, 1, 4 and 5 are adjacent which can be grouped (group 2)
- When all the 1's are grouped, the minimized expression can be generated. In group1 B is eliminated and in group2 variable A and C are eliminated.
- Therefore, the minimized expression is

$$Y = \bar{A}C + \bar{B}$$



2. Simplify the following expression using K-map

$$Y = \bar{A} \bar{B} \bar{C} \bar{D} + \bar{A} \bar{B} \bar{C} D + A \bar{B} \bar{C} \bar{D} + A \bar{B} \bar{C} D + A \bar{B} C \bar{D} + \bar{A} \bar{B} C \bar{D}$$

Procedure

- Plot 4-variable K-map for given expression.
- Place 1's in the cell 2, 4, 5, 9, 12, 13 and 0's in the remaining cells.
- 1's in cell 2 is not adjacent to other 1's therefore it is grouped separately (group 1)
- 1's in cells 9, and 13 are adjacent which can be grouped (group 2)
- 1's in cells 4, 5, 12 and 13 are adjacent which can be grouped (group 3)
- When all the 1's are grouped, the minimized expression can be generated. In group1 no variable can be eliminated, in group2 variable B is eliminated and in group3 variable A and D are eliminated.
- Therefore, the minimized expression is

$$Y = \bar{A}\bar{B}\bar{C}\bar{D} + A\bar{C}\bar{D} + \bar{B}\bar{C}$$

		CD			
		00	01	11	10
AB	00	0	0	0	1
	01	1	1	0	0
	11	1	1	0	0
	10	0	1	0	0

$\bar{A}\bar{B}C\bar{D}$ (points to cell 00, 10)
 $A\bar{C}D$ (points to cell 01, 10)

Problems:

1. Simplify the following function in SOP form using K-map

$$Y = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + \bar{A}BC + A\bar{B}\bar{C} + A\bar{B}C + ABC$$

		BC			
		00	01	11	10
A	0	1	1	1	0
	1	1	1	0	1

$\bar{A}C$ (points to cell 0, 01)
 $A\bar{C}$ (points to cell 1, 10)

Simplified boolean expression is

$$Y = \bar{B} + \bar{A}C + A\bar{C}$$

2. Simplify the following function in SOP form using K-map

$$Y = \bar{A}BC + A\bar{B}\bar{C} + ABC$$

		BC			
		00	01	11	10
A	0	0	0	1	0
	1	1	0	1	0

BC (points to cell 0, 11)
 $A\bar{B}\bar{C}$ (points to cell 1, 00)

Simplified boolean expression is

$$Y = A\bar{B}\bar{C} + BC$$

3. Simplify the following function in SOP form using K-map

$$Y = f(A, B, C) = \sum m(1, 3, 5, 6, 7)$$

		BC			
		00	01	11	10
A	0	0	1	1	0
	1	0	1	1	1

AB (points to the 1s in the row A=1)
C (points to the 1s in the column B=1)

Simplified boolean expression is

$$Y = AB + C$$

4. Simplify the following function in SOP form using K-map

$$Y = f(A, B, C) = \sum m(0, 2, 3, 4, 5, 7)$$

		BC			
		00	01	11	10
A	0	1		1	1
	1	1	1	1	

$\bar{A}B$ (points to the 1s in the row A=0)
AC (points to the 1s in the column B=1)
 $\bar{B}\bar{C}$ (points to the 1s in the column B=0)

Simplified boolean expression is

$$Y = \bar{B}\bar{C} + AC + \bar{A}B$$

5. Simplify the following function in SOP form using K-map

$$Y = f(A, B, C, D) = \sum m(0, 2, 8, 10)$$

AB \ CD	CD			
	00	01	11	10
00	1	0	0	1
01	0	0	0	0
11	0	0	0	0
10	1	0	0	1

$\overline{B}\overline{D}$
 $\overline{B}\overline{D}$

Simplified boolean expression is

$$Y = \overline{B}\overline{D}$$

6. Simplify the following function in SOP form using K-map

$$Y = f(A, B, C, D) = \sum m(0, 2, 5, 7, 8, 10, 13, 15)$$

AB \ CD	CD			
	00	01	11	10
00	1	0	0	1
01	0	1	1	0
11	0	1	1	0
10	1	0	0	1

$\overline{B}\overline{D}$
 $\overline{B}\overline{D}$
 BD

Simplified boolean expression is

$$Y = \overline{B}\overline{D} + BD$$

7. Simplify the following function in SOP form using K-map

$$Y = f(A, B, C, D) = \sum m(1, 3, 4, 6, 9, 11, 12, 14)$$

		CD				
		00	01	11	10	
AB	00	0	1	1	0	$\bar{B}D$
	01	1	0	0	1	
	11	1	0	0	1	$B\bar{D}$
	10	0	1	1	0	

Simplified boolean expression is

$$Y = \bar{B}D + B\bar{D}$$

Incompletely specified functions (don't care terms)

- When an output value is known for every possible combination of input variables, the function said to be completely specified.
- When an output value is not known for every possible combination of input variables, the function said to be incompletely specified i.e. the truth table does not generate an output value for every possible combination of input variables.
- The minterms or maxterms that are not used as a part of the output functions are called **don't care terms** denoted by 'x' or 'd' or '-' in the k-map.
- If a don't care term can be used to create a larger group of minterms then it is assigned 1 otherwise it is assigned 0.
- For example, consider the three variable truth table the output Y is incompletely specified for input conditions 110 and 111.

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0

1	0	1	1
1	1	0	-
1	1	1	-

- From the truth table the expression is written as

$$Y = f(A, B, C) = \sum m(1, 3, 5) + \sum d(6, 7)$$

		BC			
		00	01	11	10
A	0	0	1	1	0
	1	0	1	-	-

$$Y = C$$

- Don't care term in cell 7 is considered as 1 and group of four (cell 1, 3, 5, 7) is done to get expression $Y = C$.

Problems:

1. Simplify the following function in SOP form using K-map

$$Y = f(A, B, C, D) = \sum m(1, 2, 3, 5, 9, 12, 14, 15) + \sum d(4, 8, 11)$$

		CD			
		00	01	11	10
AB	00	0	1	1	1
	01	-	1	0	0
	11	1	0	1	1
	10	-	1	-	0

Simplified boolean expression is

$$Y = \bar{A} \bar{B} C + \bar{A} \bar{C} D + ABC + \bar{B} D + AB\bar{D}$$

2. Simplify the following function in SOP form using K-map

$$Y = f(A, B, C, D) = \sum m(0, 1, 4, 8, 9, 10) + d(2, 11)$$

CD \ AB		$\overline{B}\overline{C}$			
		00	01	11	10
00	1	1	0	—	$\overline{A}\overline{C}\overline{D}$
01	1	0	0	0	
11	0	0	0	0	$A\overline{B}$
10	1	1	—	1	

Simplified boolean expression is

$$Y = \overline{A}\overline{C}\overline{D} + \overline{B}\overline{C} + A\overline{B}$$

3. Simplify the following function in SOP form using K-map

$$Y = f(A, B, C, D) = \sum m(2, 3, 4, 5, 13, 15) + d(8, 9, 10, 11)$$

CD \ AB		00	01	11	10	$\overline{B}\overline{C}$
		00	01	11	10	
00				1	1	$\overline{A}\overline{B}\overline{C}$
01	1	1				
11			1	1		AD
10	—	—	—	—	—	

Simplified boolean expression is

$$Y = \overline{A}\overline{B}\overline{C} + \overline{B}\overline{C} + AD$$

4. Simplify the following function in SOP form using K-map

$$Y = f(A, B, C, D) = \sum m(1, 5, 7, 9, 13, 15) + d(8, 10, 11, 14)$$

AB \ CD		CD			
		00	01	11	10
AB	00	0	1	0	0
	01	0	1	1	0
	11	0	1	1	—
	10	—	1	—	—

$\bar{C}D$ (points to cell 00,01)
 BD (points to cell 01,11)

Simplified boolean expression is

$$Y = \bar{C}D + BD$$

5. Simplify the following function in SOP form using K-map

$$Y = f(A, B, C, D) = \sum m(0, 2, 4, 8, 10, 14) + d(5, 6, 7, 12)$$

AB \ CD		CD			
		00	01	11	10
AB	00	1			1
	01	1	—	—	—
	11	—			1
	10	1			1

$\bar{D}$ (points to column 00)

Simplified boolean expression is

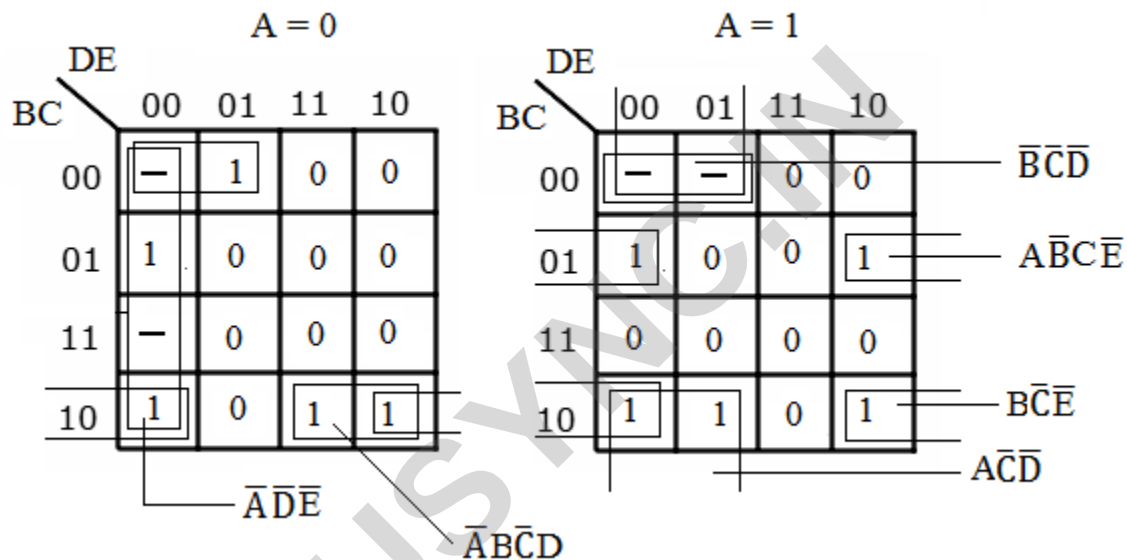
$$Y = \bar{D}$$

6. Simplify the following expression using K-map

$$Y = F(A, B, C, D, E) = \sum m(1, 4, 8, 10, 11, 20, 22, 24, 25, 26) + d(0, 12, 16, 17)$$

- Group 1 (cell 0, 4, 8, 12) is formed within the first 16 cell map.

- Group 2 (cell 10, 11) is formed within the first 16 cell map.
- Group 3 (cell 0, 1, 16, 17) is formed by combination of two pairs from two different 16 cell maps.
- Group 4 (cell 20, 22) is formed within the second 16 cell map.
- Group 5 (cell 8, 10, 24, 26) is formed by combination of two pairs from two different 16 cell maps.
- Group 6 (cell 16, 17, 24, 25) is formed within the second 16 cell map.



Simplified boolean expression is

$$Y = f(A, B, C, D, E) = \bar{A}\bar{D}\bar{E} + \bar{A}\bar{B}\bar{C}D + \bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}C\bar{E} + B\bar{C}\bar{E} + \bar{A}\bar{C}\bar{D}$$

7. Simplify the following function in SOP form using K-map

$$Y = f(A, B, C, D, E) = \sum m(1, 3, 4, 6, 9, 11, 12, 14, 17, 19, 20, 22, 25, 27, 28, 30) + d(8, 10, 24, 26)$$

		A = 0						A = 1					
BC	DE	00	01	11	10	BC	DE	00	01	11	10		
		0	1	1	0			0	1	1	0		
00	00	0	1	1	0	00	00	0	1	1	0	$\bar{C}E$	
01	01	1	0	0	1	01	01	1	0	0	1	$C\bar{E}$	
11	11	1	0	0	1	11	11	1	0	0	1		
10	10	—	1	1	—	10	10	—	1	1	—	$B\bar{C}$	

Simplified boolean expression is

$$Y = \bar{C}E + C\bar{E}$$

8. Simplify the following function in SOP form using K-map

$$Y = f(A, B, C, D, E) = \sum m(0, 2, 8, 10, 16, 18, 24, 26)$$

		A = 0						A = 1					
BC	DE	00	01	11	10	BC	DE	00	01	11	10		
		1	0	0	1			1	0	0	1		
00	00	1	0	0	1	00	00	1	0	0	1		
01	01	0	0	0	0	01	01	0	0	0	0		
11	11	0	0	0	0	11	11	0	0	0	0		
10	10	1	0	0	1	10	10	1	0	0	1	$\bar{C}\bar{E}$	

Simplified boolean expression is

$$Y = \bar{C}\bar{E}$$

Simplifying Maxterm Equations (POS expressions)

➤ The procedure to simplify POS expressions are as follows:

1. Plot the K-map and place 0s in those cells corresponding to the 0s in the truth table or POS expression. Place 1s in other cells.

2. Check the K-map for all possible valid adjacent 0s and form a group of 2^n 0s and encircle those 0s which are not adjacent to any other 0. They are called isolated 0s.
3. While grouping make sure that there are minimum no. of groups.
4. Then form the simplified expression by taking product of sum terms of all the groups.

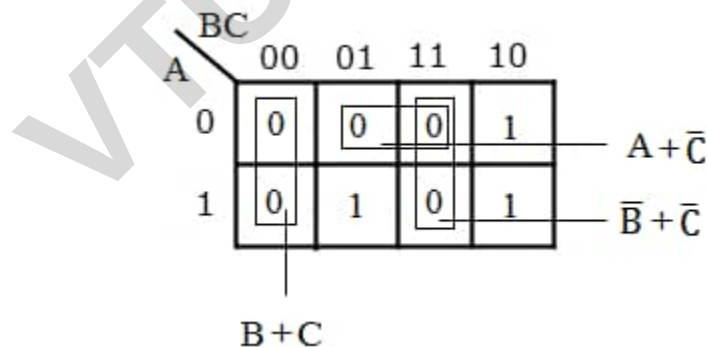
Problems:

1. Simplify the following expression using K-map

$$Y = (A + B + \bar{C})(A + \bar{B} + \bar{C})(\bar{A} + \bar{B} + \bar{C})(\bar{A} + B + C)(A + B + C)$$

Procedure

- Plot 3-variable K-map for given expression.
- Place 0s in the cell 0, 1, 3, 4, 7 and 1s in the remaining cells.
- 0s in cells 0, and 4 are adjacent which can be grouped (group 1)
- 0s in cells 1, and 3 are adjacent which can be grouped (group 2)
- 0s in cells 3 and 7 are adjacent which can be grouped (group 3)
- When all the 0s are grouped, the minimized expression can be generated. In group1 variable A can be eliminated, in group2 variable B is eliminated and in group3 variable A is eliminated.



- Therefore the minimized expression is

$$Y = (B + C)(A + \bar{C})(\bar{B} + \bar{C})$$

2. Simplify the following expression using K-map

$$Y = f(A, B, C, D) = \pi M(0, 2, 3, 8, 9, 12, 13, 15)$$

		CD				
		00	01	11	10	
AB	00	0	1	0	0	$A + B + \bar{C}$
	01	1	1	1	1	
	11	0	0	0	1	$\bar{A} + \bar{B} + \bar{D}$
	10	0	0	1	1	$\bar{A} + C$

Simplified boolean expression is

$$Y = (A + B + D)(A + B + \bar{C})(\bar{A} + \bar{B} + \bar{D})(\bar{A} + C)$$

Quine McCluskey Minimization technique

- For many applications the number of variables in a problem is too large to simplify manually using K-maps.
- The Quine McCluskey Minimization technique is an algorithm that uses the same Boolean algebra postulates that were used with k-map but in a form suitable for computer solution.

Procedure to use Quine McCluskey Minimization technique:

1. Arrange all the minterms, in a list of increasing order, make groups such that it contains same number of 1s.
2. Create a new table showing the minterms in group n that matched with those from group n + 1 such that they differ in only one position. This process is repeated until all of the minterms in each group have been compared to those in the next higher group. When a minterm in a group is combined with a minterm in an adjacent group, a dash (-) is used to indicate an eliminated variable. As each minterm from a group combines with it is now part of a larger group. If a minterm did not combine with another, then no check would be made. If a term doesn't simplify it is a prime implicant.
3. All of the adjacent minterm groups, are compared to see if groups of four can be made.
4. Repeat the process until step 3. In this case both dashes must be in the same bit position with only one other variable allowed to change. The creation of a new table further

groups the sets of minterms. This same process is repeated until no further combination of minterm group is possible.

5. All non checked minterm groups are now considered to be prime implicants.
6. All of the prime implicants are formed into a prime implicant table.
7. Evaluate the prime implicants by circling those minterms that are contained in only one prime implicant. Circled minterms represent essential prime implicants.

Problems:

1. **Simplify the following Boolean function by using a Quine McCluskey minimization technique**

$$Y = f(A, B, C, D) = \sum m(0, 1, 2, 3, 6, 7, 8, 9, 14, 15)$$

- Arrange all the minterms in increasing order and make a group such that each minterm in the group contain same number of 1s as shown in table 1.2. Group 0 contains only those minterms with no 1s, Group 1 contains only those minterms with single 1s, Group 2 contains only those minterms with two 1s, Group 3 contains only those minterms with three 1s, Group 4 contains only those minterms with four 1s as shown in table below.

Table 1.2: Grouping of minterms according to numbers of 1s

Group	Minterm	Binary representation	Check
0	0	0000	✓
1	1	0001	✓
	2	0010	✓
	8	1000	✓
2	3	0011	✓
	6	0110	✓
	9	1001	✓
3	7	0111	✓
	14	1110	✓
4	15	1111	✓

- Create a new table showing the minterms in group n that matched with those from group n + 1 such that they differ in only one position. This process is repeated until all of the

minterms in each group have been compared to those in the next higher group. When a minterm in a group is combined with a minterm in an adjacent group, a dash (-) is used to indicate an eliminated variable. The combined minterm are grouped together as shown in table 1.3.

$$\text{i.e. } \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}\bar{C}D = \bar{A}\bar{B}\bar{C}(D + \bar{D}) = \bar{A}\bar{B}\bar{C} -$$

- When all of the minterm in group 0 is compared with those in group 1 then compare the minterms in group1 with those in group 2.
- This process is repeated until all the minterm in each group is compared with those in the next higher group.
- When a minterm from a group combines with a minterm in the next higher group then it is checked (✓). If a minterm does not combine with another then no check would be made. If a term does not simplify then it is a prime implicant

Table 1.3: Creation of minterm groups of two

Group	Minterm	Binary representation	Check
0	0, 1	0 0 0 –	✓
	0, 2	0 0 – 0	✓
	0, 8	– 0 0 0	✓
1	1, 3	0 0 – 1	✓
	1, 9	– 0 0 1	✓
	2, 3	0 0 1 –	✓
	2, 6	0 – 1 0	✓
	8, 9	1 0 0 –	✓
2	3, 7	0 – 1 1	✓
	6, 7	0 1 1 –	✓
	6, 14	– 1 1 0	✓
3	7, 15	– 1 1 1	✓
	14, 15	1 1 1 –	✓

- All of the adjacent minterm groups in table 1.4 is compared with the next higher group to see if groups of four can be made. When group of four is made the dashes (–) in the

groups of two must be in the same bit position and only one variable change (0 in one group and 1 in another group) is allowed

Table 1.4: Creation of minterm groups of four

Group	Minterm	Binary representation
0	0, 1, 2, 3	0 0 – –
	0, 1, 8, 9	– 0 0 –
1	2, 6, 3, 7	0 – 1 –
2	6, 7, 14, 15	– 1 1 –

- Repeat the previous steps until no further combination of minterms groups is possible. When further grouping is made the dashes must be in the same bit position with only one other variable is allowed to change. The creation of a new table further groups the sets of minterms.
- All the non-checked minterm groups are prime implicants.
- Prime implicant table is prepared using all the non-checked minterm groups as shown in table 1.5.
- Each prime implicant is listed vertically in two forms: PI terms and the decimal notation of minterms.

Table 1.5: Prime implicant table

PI terms	Minterm	0	1	2	3	6	7	8	9	14	15
$\bar{A} \bar{B}$	0, 1, 2, 3	x	x	x	x						
$\bar{B} \bar{C}$	0, 1, 8, 9	x	x					⊗	⊗		
$\bar{A} C$	2, 6, 3, 7			x	x	x	x				
BC	6, 7, 14, 15					x	x			⊗	⊗

- Evaluate the prime implicants by circling those minterms that are present in only one prime implicant i.e. only one x in a column. Circled minterm represent essential prime implicant (EPI).
- To represent the simplified boolean expression select all the essential prime implicants are considered along with the other prime implicants such that all the minterms should be

covered. Minterm (2, 3) are present in both the prime implicants so select any one of these prime implicant to cover all the minterms in the equation but not both.

- Therefore, the simplified boolean expression is

$$Y = \bar{B} \bar{C} + B C + \bar{A} \bar{B}$$

or

$$Y = \bar{B} \bar{C} + B C + \bar{A} C$$

2. Simplify the following Boolean function by using a Quine McCluskey minimization technique

$$Y = f(A, B, C, D) = \sum m(0, 2, 3, 6, 7, 8, 10, 12, 13)$$

Grouping of minterms according to numbers of 1s

Group	Minterm	Binary representation	Check
0	0	0000	✓
1	2	0010	✓
	8	1000	✓
2	3	0011	✓
	6	0110	✓
	10	1010	✓
	12	1100	✓
3	7	0111	✓
	13	1101	✓

Creation of minterm groups of two

Group	Minterm	Binary representation	Check
0	0, 2	0 0 – 0	✓
	0, 8	– 0 0 0	✓
1	2, 3	0 0 1 –	✓
	2, 6	0 – 1 0	✓
	2, 10	– 0 1 0	✓
	8, 10	1 0 – 0	✓

	8, 12	1 – 0 0	
2	3, 7	0 – 1 1	✓
	6, 7	0 1 1 –	✓
	12, 13	1 1 0 –	

Creation of minterm groups of four

Group	Minterm	Binary representation
0	0, 2, 8, 10	– 0 – 0
1	2, 3, 6, 7	0 – 1 –

Prime implicant table

PI terms	Minterm	0	2	3	6	7	8	10	12	13
$\bar{B} \bar{D}$	0, 2, 8, 10	⊗	x				x	⊗		
$\bar{A} C$	2, 3, 6, 7		x	⊗	⊗	⊗				
$A \bar{C} \bar{D}$	8, 12						x		x	
$A B \bar{C}$	12, 13								x	⊗

Therefore, the simplified boolean expression is

$$Y = \bar{B} \bar{D} + \bar{A} C + A B \bar{C}$$

Quine McCluskey using don't care terms

- The same rules that are applied for using don't care terms with the K-map can be used for Quine McCluskey.
1. Construct a list of minterms and don't care terms classified according to the number of 1s. Indicate the don't care terms by *.
 2. Compare the terms in group n, including don't care terms, with terms in group n+1, looking for a single variable change. Treat don't care terms as 1 in finding prime implicants.
 3. Repeat step2 for creating an additional table indicating groups of four minterms/don't care term groups. Repeat step3 until no further grouping can occur.

4. Construct a prime implicant chart and determine essential prime implicants. The don't care terms are used to find the prime implicants but it is not compulsory to include don't care terms in the final expression.

Problems:

1. Simplify the following Boolean function by using a Quine McCluskey minimization technique

$$Y = f(A, B, C, D) = \sum m(1, 3, 13, 15) + \sum d(8, 9, 10, 11)$$

Grouping of minterms according to numbers of 1s

Group	Minterm	Binary representation	Check
1	1	0001	✓
	8*	1000	✓
2	3	0011	✓
	9*	1001	✓
	10*	1010	✓
3	11*	1011	✓
	13	1101	✓
4	15	1111	✓

Creation of minterm groups of two

Group	Minterm	Binary representation	Check
1	1, 3	0 0 – 1	✓
	1, 9*	– 0 0 1	✓
	8*, 9*	1 0 0 –	✓
	8*, 10*	1 0 – 0	✓
2	3, 11*	– 0 1 1	✓
	9*, 11*	1 0 – 1	✓
	9*, 13	1 – 0 1	✓
	10*, 11*	1 0 1 –	✓
3	11*, 15	1 – 1 1	✓

	13, 15	1 1 – 1	✓
--	--------	---------	---

Creation of minterm groups of four

Group	Minterm	Binary representation
1	1, 3, 9*, 11*	– 0 – 1
	8*, 9*, 10*, 11*	1 0 – –
2	9*, 13, 11*, 15	1 – – 1

Prime implicant table

PI terms	Minterm	1	3	13	15
$\bar{B} D$	1, 3, 9*, 11*	⊗	⊗		
AD	9*, 13, 11*, 15			⊗	⊗

Since the all minterms in the minterm group (8, 9, 10, 11) are don't cares it is not considered for creating prime implicant table.

Therefore the simplified boolean expression is

$$Y = \bar{B} D + A D$$

2. Simplify the following Boolean function by using a Quine McCluskey minimization technique

$$Y = f(A, B, C, D) = \sum m(1, 2, 3, 5, 9, 12, 14, 15) + \sum d(4, 8, 11)$$

Grouping of minterms according to numbers of 1s

Group	Minterm	Binary representation	Check
1	1	0001	✓
	2	0010	✓
	4*	0100	✓
	8*	1000	✓
2	3	0011	✓
	5	0101	✓
	9	1001	✓
	12	1100	✓
3	11*	1011	✓

	14	1110	✓
4	15	1111	✓

Creation of minterm groups of two

Group	Minterm	Binary representation	Check
1	1, 3	0 0 – 1	✓
	1, 5	0 – 0 1	
	1, 9	– 0 0 1	✓
	2, 3	0 0 1 –	
	4*, 5	0 1 0 –	
	4*, 12	– 1 0 0	
	8*, 9	1 0 0 –	
	8*, 12	1 – 0 0	
2	3, 11*	– 0 1 1	✓
	9, 11*	1 0 – 1	✓
	12, 14	1 1 – 0	
3	11*, 15	1 – 1 1	
	14, 15	1 1 1 –	

Creation of minterm groups of four

Group	Minterm	Binary representation
1	1, 3, 9, 11*	– 0 – 1

Prime implicant table

PI terms	Minterm	1	2	3	4	5	8	9	11	12	14	15
$\bar{B} D$	1, 3, 9, 11*	x		x				x	x			
$\bar{A} \bar{C} D$	1, 5	x				x						
$\bar{A} \bar{B} C$	2, 3		⊗	x								
$\bar{A} B \bar{C}$	4*, 5				x	x						
$B \bar{C} \bar{D}$	4*, 12				x					x		
$A \bar{B} \bar{C}$	8*, 9						x	x				
$A \bar{C} \bar{D}$	8*, 12						x			x		

A B $\bar{D}$	12, 14									X	X	
A C D	11*, 15								X			X
A B C	14, 15										X	X

Essential prime implicant has two minterms (2, 3) and is included in the final expression.

Minterm 1 is present in two prime implicant group therefore to add minterm 1 to the final expression select the prime implicant group which has more number of minterms (i.e. 1, 3, 9, 11).

Minterm 5 is present in two prime implicant group therefore to add minterm 5 to the final expression select the minterm group (1, 5).

Minterm 12 is present in three prime implicant group therefore to add minterm 12 to the final expression select the minterm group (12, 14).

Minterm 15 is present in two prime implicant group therefore to add minterm 15 to the final expression select the minterm group (14, 15).

Therefore, the simplified boolean expression is

$$Y = \bar{B}D + AB\bar{D} + ABC + \bar{A}\bar{C}D + \bar{A}\bar{B}C$$

Question Bank

1. Define the following terms:
 - i. Minterm
 - ii. Maxterm
 - iii. Canonical SOP
 - iv. Canonical POS
 - v. Prime implicants
 - vi. Essential prime implicants
 - vii. Incompletely specified functions
2. Design a combinational logic circuit which takes two, 2-bit binary numbers as its input and generates an output equal to 1, when the sum of the two number is odd.
3. Design a 3-input,1-output minimal Combinational network that has a logical-1 output when the majority of its inputs are logic-1 and has a logic-0 when majority of inputs are logic-0
4. Convert the given boolean function into canonical minterm form and canonical maxterm form
 - i. $f(A, B, C) = (A + B)(B + C)$
 - ii. $f(A, B, C) = A + AC(B + C)$
 - iii. $f(A, B, C) = \bar{A}B + BC$
 - iv. $f(A, B, C) = (\bar{A} + B)(B + \bar{C})$
 - v. $f(A, B, C) = A + \bar{B}C$
5. Find all the prime implicants and essential prime implicants for the given function using k-Map method.
 - i. $f(A, B, C, D) = \sum m(0, 3, 6, 7, 8, 9, 10) + d(2, 5, 11, 12, 15)$
 - ii. $f(A, B, C, D) = \sum m(0, 2, 5, 7, 8, 10, 13, 15) + d(1, 4, 11, 14)$
 - iii. $f(A, B, C, D) = \pi M(4, 5, 6, 7, 8, 12, 13) + d(1, 15)$
 - iv. $f(A, B, C, D) = \pi M(0, 1, 4, 5, 6, 7, 9, 14) + d(13, 15)$
 - v. $f(A, B, C, D, E) = \sum m(3, 7, 11, 12, 13, 14, 15, 16, 18)$
 $+ d(24, 25, 26, 27, 28, 29, 30, 31)$
 - vi. $f(A, B, C, D, E) = \sum m(3, 7, 8, 10, 11, 12, 14, 15, 17, 19, 21, 23, 25, 27, 29, 31) +$

$$d(2, 6, 26, 30)$$

$$\text{vii. } f(A, B, C, D, E) = \sum m(0, 2, 4, 6, 8, 16, 18, 20, 22, 24, 26, 28, 30) + \\ d(3, 7, 11, 15, 19, 23, 27, 31)$$

6. Find the prime implicants and essential prime implicants using Quine-McCusky method and verify the result using k-map.

$$\text{i. } F(A, B, C, D) = \sum m(7, 9, 12, 13, 14, 15) + d(4, 11)$$

$$\text{ii. } Y = \sum m(1, 3, 8, 6, 10, 12, 14) + d(7, 13)$$

$$\text{iii. } f(A, B, C, D) = \sum m(1, 2, 3, 5, 9, 12, 14, 15) + d(4, 8, 11)$$

$$\text{iv. } Y = \sum m(1, 4, 7, 9, 12, 14) + d(2, 13)$$

7. Simplify the following Expression using 3-variable MEV

$$\text{i. } f(A, B, C, D) = \sum m(1, 3, 7, 11, 15) + d(0, 2, 5)$$

$$\text{ii. } f(A, B, C, D) = \sum m(1, 4, 7, 9, 12, 14) + d(2, 13)$$

$$\text{iii. } f(A, B, C, D) = \sum m(0, 2, 3, 8, 9, 12, 13, 15) + d(2, 13)$$

$$\text{iv. } f(A, B, C, D) = \sum m(1, 5, 7, 10, 11) + d(2, 3, 6, 13)$$

$$\text{v. } f(A, B, C, D) = \sum m(2, 3, 4, 10, 13, 14, 15) + d(7, 9, 11)$$

8. Simplify the following Expression using K-map

$$\text{i. } Y = \bar{A}\bar{B} + \bar{A}B\bar{C} + A\bar{B}D$$

$$\text{ii. } Y = \bar{A}BC + A\bar{B}\bar{C} + ABC$$

$$\text{iii. } Y = \bar{A}BC + A\bar{D} + AB\bar{D}$$

Module 2

Logic Design with MSI Components and Programmable Logic Devices:

Complexity of single chip is called **scale of integration**.

- 1 to 10 gates → SSI (small scale integration)
- 10 to 100 gates → MSI (Medium scale integration)
- 100 to 1000 gates → LSI (Large scale integration)
- More than 1000 gates → VLSI (Very Large scale integration)

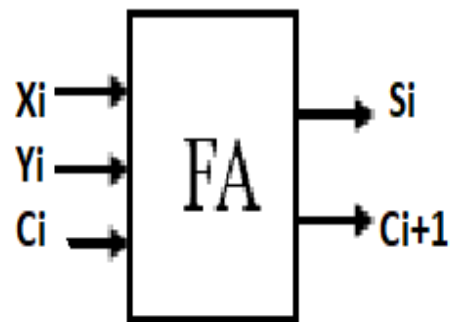
Binary Adders

Binary adder is one of the basic combinational logic circuits. The outputs of a combinational logic circuit depend on the present input only. In other words, outputs of combinational logic circuit do not depend upon any previously applied inputs. It does not require any memory like component. Binary adder is one of the basic combinational logic circuits as present state of input variables.

Truth table

X_i	Y_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Block schematic



kmap

Yi, Ci		Xi			
		00	01	11	10
Xi	0	0	1	3	2
	1	1	4	5	6
		K-map for Sum (Si)			

Yi, Ci		Xi			
		00	01	11	10
Xi	0	0	1	3	2
	1	4	5	7	6
		K-map for Carry (Ci+1)			

From k map for

$$S_i = (X_i' Y_i' C_i) + (X_i' Y_i C_i') + (X_i Y_i' C_i') + (X_i Y_i C_i)$$

$$= X_i' [(Y_i' C_i) + (Y_i C_i')] + X_i [(Y_i' C_i') + (Y_i C_i)] \text{---equation 1}$$

Let $[(Y_i' C_i) + (Y_i C_i')]$ be A then becomes $a'b + ab' = a \oplus b$ and $a'b' + ab = (a \oplus b)'$ now equation 1 can be written as

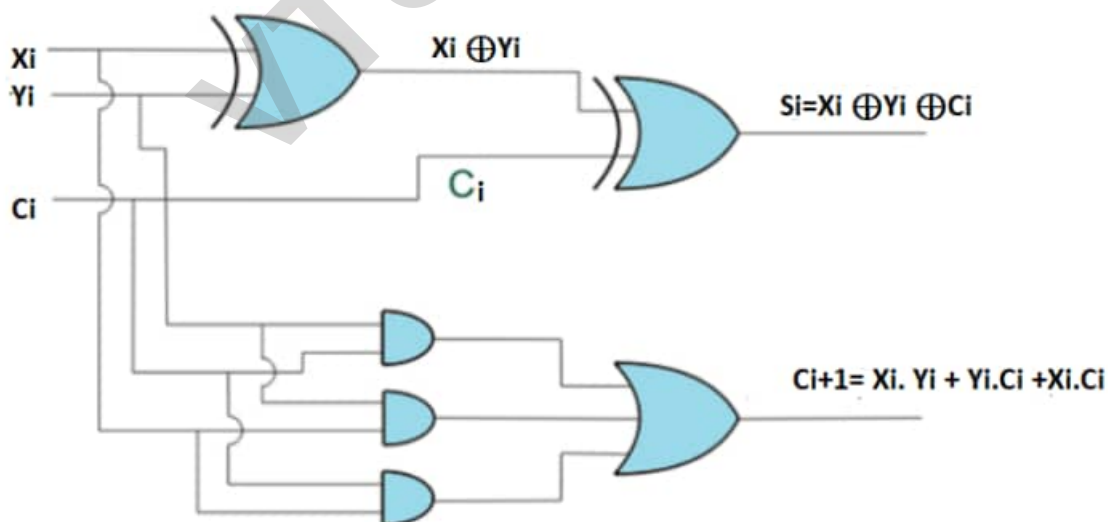
$$S_i = X_i' (Y_i \oplus C_i) + X_i (Y_i \oplus C_i)' \text{---equation 2}$$

The above equation again in the form $a'b + ab' = a \oplus b$ and $a'b' + ab = (a \oplus b)'$ then S_i becomes

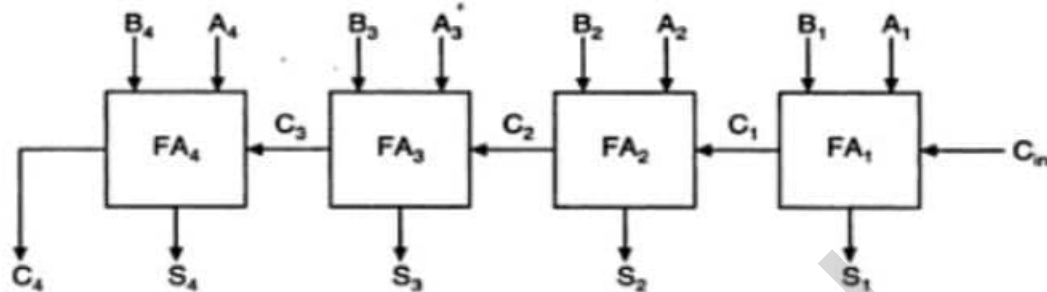
$$S_i = X_i \oplus Y_i \oplus C_i$$

$$C_{i+1} = Y_i C_i + X_i C_i + X_i Y_i$$

Circuit



4 bit ripple carry adder



Working of 4-bit Ripple Carry Adder

- Let's take an example of two input sequences 1011 and 0011. These are representing the A4 A3 A2 A1 and B4 B3 B2 B1.
- As per this adder concept, input carry is 0.
- When A1 & B1 are applied at 1st full adder along with input carry 0.
- Here A1 = 1 ; B1 = 1 ; Cin = 0
- Sum (S1) and carry (C1) will be generated as per the Sum and Carry equations of this adder. As per its theory, the output equation for the Sum = $A1 \oplus B1 \oplus Cin$ and Carry = $A1B1 \oplus B1Cin \oplus CinA1$
- As per this equation, for 1st full adder S1 = 0 and Carry output i.e., C1 = 1.
- Same like for next input bits A2 and B2, output S2 = 1 and C2 = 0. Here the important point is the second stage full adder gets input carry i.e., C1 which is the output carry of initial stage full adder.
- Like this will get the final output sequence (S4 S3 S2 S1) = (1 1 1 0) and Output carry C4 = 0
- This is the addition process for 4-bit input sequences when it's applied to this carry adder.

Example:

A + B

(A = 1011) and (B = 0011)

Subscript i	4	3	2	1	
Input Carry	0	1	1	0	C _i
A	1	0	1	1	A _i
+					
B	0	0	1	1	B _i
Sum	1	1	1	0	S _i
Output Carry	0	0	1	1	C _{i+1}

C₀ = 0

Carry Propagation

Parrel binary adder/ripple binary adder



Figure 5.3. Parallel (ripple) binary adder.

Subtractors

Binary Subtraction can take many forms but the rules for subtraction are the same whichever process you use. As binary notation only has two digits, subtracting a “0” from a “0” or a “1” leaves the result unchanged as $0-0 = 0$ and $1-0 = 1$. Subtracting a “1” from a “1” results in a “0”, but subtracting a “1” from a “0” requires a borrow. In other words $0 - 1$ requires a borrow.

Binary Subtraction of Two Bits

0	1	1 (borrow)	1 → 0
<u>- 0</u>	<u>- 0</u>	<u>- 1</u>	<u>- 1</u>
0	1	0	1

Table 5.2 Truth table for a binary full subtractor

x_i	y_i	b_i	b_{i+1}	d_i
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

K-map
equation

$y_i b_i$					
x_i		00	01	11	10
		0	1	3	2
0		0	1		1
1		1		1	

K-map for d_i

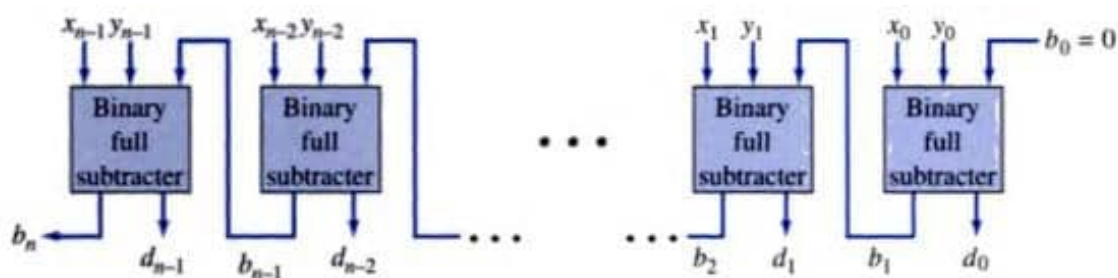
$y_i b_i$					
x_i		00	01	11	10
		0	1	3	2
0		0	1	1	1
1		0	1	1	0

K-map for b_{i+1}

$$d_i = x_i \oplus (y_i \oplus b_i)$$

$$b_{i+1} = x_i' b_i + y_i b_i + x_i' y_i$$

n bit parallel binary subtracter

**Figure 5.4.** Parallel (ripple) binary subtracter.

Parallel binary subtracter using parallel binary adder

This circuit performs the subtraction operation by considering the principle that the addition of minuend and the complement of the subtrahend is equivalent to the subtraction process.

The subtraction of A by B is obtained by taking 2's complement of B and adding it to A. The 2's complement of B is obtained by taking 1's complement and adding 1 to the least significant pair of bits.

Hence, in this circuit 1's complement of B is obtained with the inverters (NOT gate) as shown in figure and a 1 can be added to the sum through the input carry.

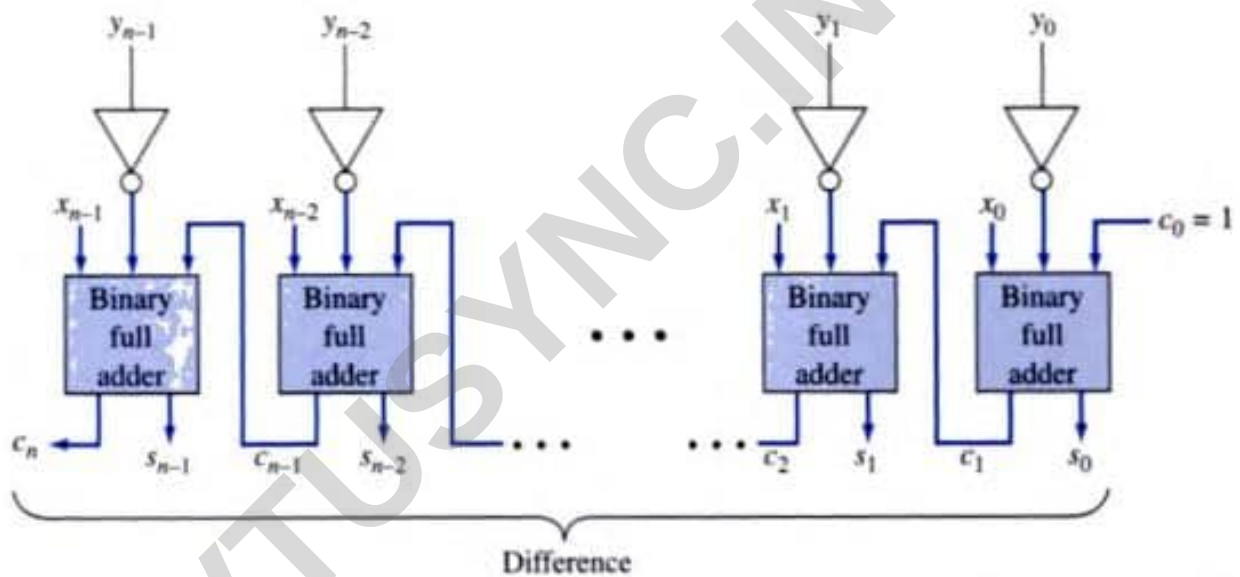


Figure 5.5. Parallel binary subtracter constructed using a parallel binary adder.

Parallel binary adder /subtractor

Here we can use XOR gate at each input B_1, B_2, B_3 and B_4 with control input M (either 1 or 0). Now, if $M = 1$, B_1, B_2, B_3 and B_4 will be complemented. At the same time if third input of FAI is 1, the circuit becomes subtractor. So, $M = 1$ is also to be fed to the third input of FAI in subtractor.

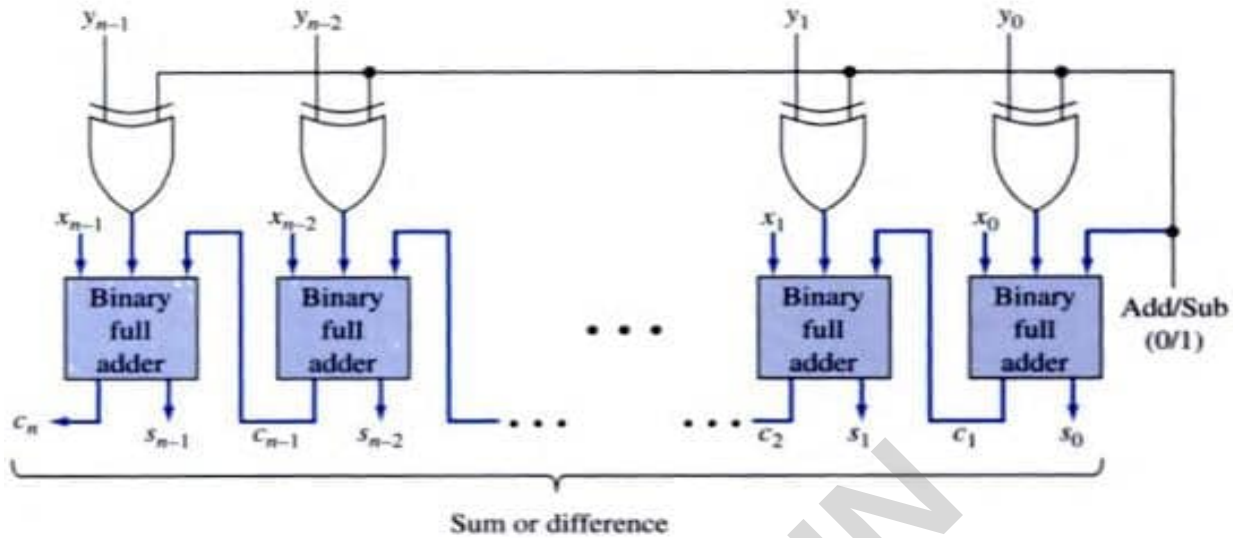


Figure 5.6. Parallel binary adder/subtractor.

CLA

To perform these operations ‘Adder circuits’ are implemented using basic logic gates. Adder circuits are evolved as Half-adder, Full-adder, Ripple-carry Adder, and Carry Look-ahead Adder. Among these Carry Look-ahead Adder is the faster adder circuit. It reduces the propagation delay, which occurs during addition, by using more complex hardware circuitry. It is designed by transforming the ripple-carry Adder circuit such that the carry logic of the adder is changed into two-level logic.

To reduce the computation time, there are faster ways to add two binary numbers by using carry lookahead adders. They work by creating two signals P and G known to be Carry Propagator and Carry Generator. The carry propagator is propagated to the next level whereas the carry generator is used to generate the output carry, regardless of input carry.

The number of gate levels for the carry propagation can be found from the circuit of full adder. The signal from input carry C_{in} to output carry C_{out} requires an AND gate and an OR gate, which constitutes two gate levels. So if there are four full adders in the parallel adder, the output carry C_5 would have $2 \times 4 = 8$ gate levels from C_1 to C_5 . For an n -bit parallel adder, there are $2n$ gate levels to propagate through.

Consider the full adder circuit shown above with corresponding truth table. We define two variables as ‘carry generate’ and ‘carry propagate’ then, the sum output and carry output can be expressed in terms of carry generate G_i and carry propagate P_i as

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = G_i + P_i C_i$$

where produces the carry when both A_i, B_i are 1 regardless of the input carry. P_i is associated with the propagation of carry from C_i to C_{i+1} .

The carry output Boolean function of each stage in a 4 stage carry look-ahead adder can be expressed as

$$\begin{aligned} C_1 &= G_0 + P_0 C_{in} \\ C_2 &= G_1 + P_1 C_1 = G_1 + P_1 G_0 + P_1 P_0 C_{in} \\ C_3 &= G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{in} \\ C_4 &= G_3 + P_3 C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{in} \end{aligned}$$

$$g_i = x_i y_i \quad (5.3)$$

$$p_i = x_i + y_i \quad (5.4)$$

the output carry equation for the i th stage is given by

$$c_{i+1} = g_i + p_i c_i$$

$$c_1 = g_0 + p_0 c_0 \quad (5.5)$$

$$\begin{aligned} c_2 &= g_1 + p_1 c_1 \\ &= g_1 + p_1 (g_0 + p_0 c_0) \\ &= g_1 + p_1 g_0 + p_1 p_0 c_0 \end{aligned} \quad (5.6)$$

$$\begin{aligned} c_3 &= g_2 + p_2 c_2 \\ &= g_2 + p_2 (g_1 + p_1 g_0 + p_1 p_0 c_0) \\ &= g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0 \end{aligned} \quad (5.7)$$

$$\begin{aligned} c_4 &= g_3 + p_3 c_3 \\ &= g_3 + p_3 (g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0) \\ &= g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 c_0 \end{aligned} \quad (5.8)$$

⋮

$$c_{i+1} = g_i + p_i g_{i-1} + p_i p_{i-1} g_{i-2} + \dots + p_i p_{i-1} \dots p_1 g_0 + p_i p_{i-1} \dots p_0 c_0 \quad (5.9)$$

The general organization of carry look ahead adder is shown in below figure 5.7a. sigma block corresponds to logic needed to form sum bit, the carry generate function and the carry propagate function at each block.

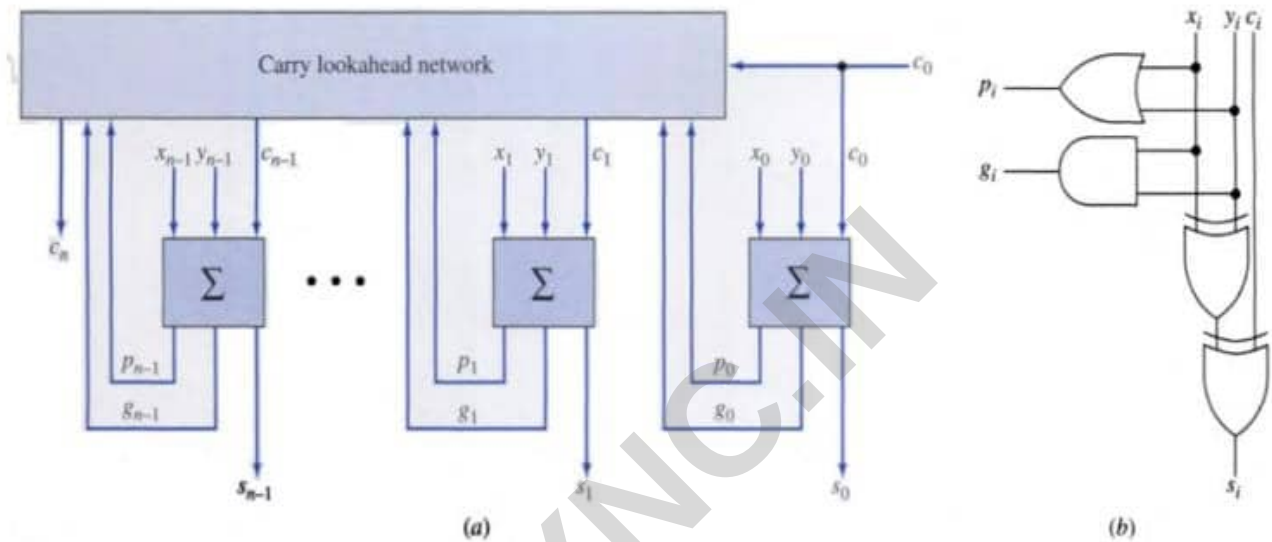


Figure 5.7 A carry lookahead adder. (a) General organization. (b) Sigma block.

Large high-speed adders using CLA

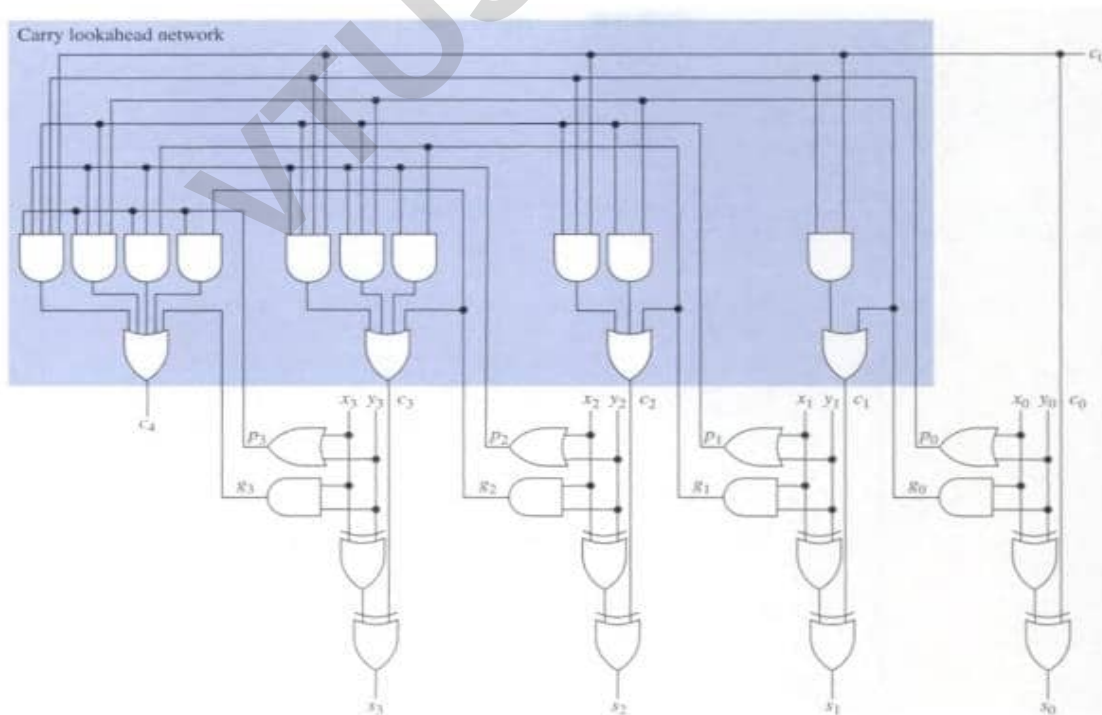


Figure 5.8 A 4-bit carry lookahead adder.

Advantages of CLA

- Minimizing propagation delay
- High speed addition

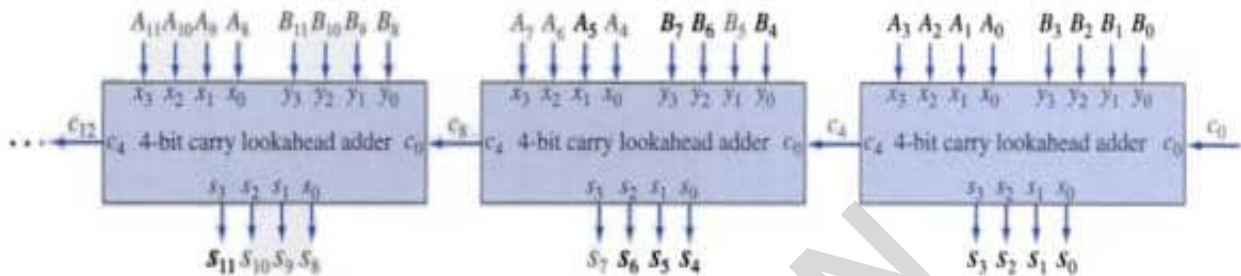


Figure 5.9 Cascade connection of 4-bit carry lookahead adders.

Another approach of large speed adders is on partitioning the operands into blocks. It is shown in fig 5.10a it is same as CLA only additional is it has 2 output G and P

$$G = g_3 + p_3g_2 + p_3p_2g_1 + p_3p_2p_1g_0$$

$$P = p_3p_2p_1p_0$$

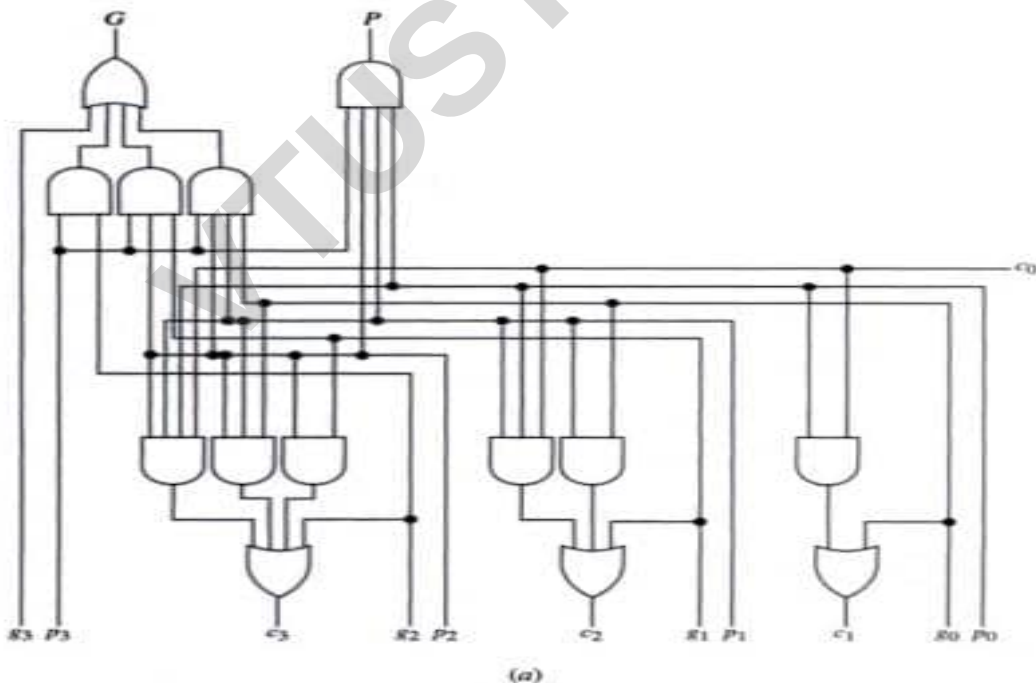


Figure 5.10 (a) A carry lookahead generator.

Decimal adders

From the below table 5.3, it is clear that if the produced sum is between 1 to 9, the Binary and the BCD code is the same. But for 10 to 19 decimal numbers, both the codes are different. In the above table, the binary sum combinations from 10 to 19 give invalid BCD. There are the following points that help the circuit to identify the invalid BCD.

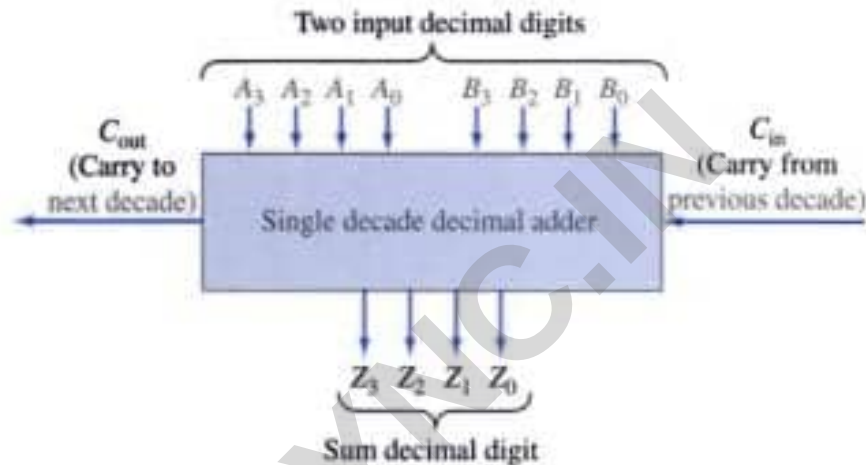


Figure 5.11 Organization of a single-decade decimal adder.

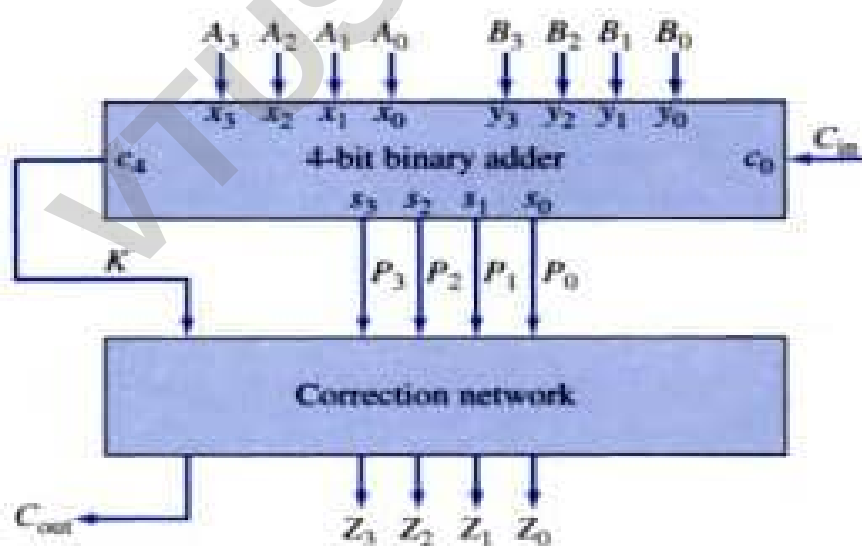
1. It is obvious from the table that a correction is needed when the '**Binary Sum**' has an output carry $K=1$.
2. The other six combinations from 10 to 15 need correction in which the bit on the Z_8 position is 1.
3. In the Binary sum of 8 and 9, the bit on the Z_8 position is also 1. So, the second step fails, and we need to modify it.
4. To distinguish these two numbers, we specify that the bit on the Z_4 or Z_2 position also needs to be 1 with the bit of Z_8
5. The condition for a correction and an output carry can be expressed by the Boolean function:

$$C = K + Z_8 \cdot Z_4 + Z_8 \cdot Z_2$$

Once the circuit found the invalid BCD, the circuit adds the binary number of 6 into the invalid BCD code to make it valid.

Table 5.3 Comparing binary and BCD sums

Decimal sum	Binary sum					Required BCD sum				
	K	P_3	P_2	P_1	P_0	C_{out}	Z_3	Z_2	Z_1	Z_0
0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0	0	0	1
2	0	0	0	1	0	0	0	0	1	0
3	0	0	0	1	1	0	0	0	1	1
4	0	0	1	0	0	0	0	1	0	0
5	0	0	1	0	1	0	0	1	0	1
6	0	0	1	1	0	0	0	1	1	0
7	0	0	1	1	1	0	0	1	1	1
8	0	1	0	0	0	0	1	0	0	0
9	0	1	0	0	1	0	1	0	0	1
10	0	1	0	1	0	1	0	0	0	0
11	0	1	0	1	1	1	0	0	0	1
12	0	1	1	0	0	1	0	0	1	0
13	0	1	1	0	1	1	0	0	1	1
14	0	1	1	1	0	1	0	1	0	0
15	0	1	1	1	1	1	0	1	0	1
16	1	0	0	0	0	1	0	1	1	0
17	1	0	0	0	1	1	0	1	1	1
18	1	0	0	1	0	1	1	0	0	0
19	1	0	0	1	1	1	1	0	0	1

**Figure 5.12** Organization of a single-decade BCD adder.

		P_1P_0			
		00	01	11	10
P_3P_2	00	0	0	0	0
	01	0	0	0	0
	11	1	1	1	1
	10	0	0	1	1

Figure 5.13 Karnaugh map to detect the combinations $P_3P_2P_1P_0 = 1010, 1011, \dots, 1111$.

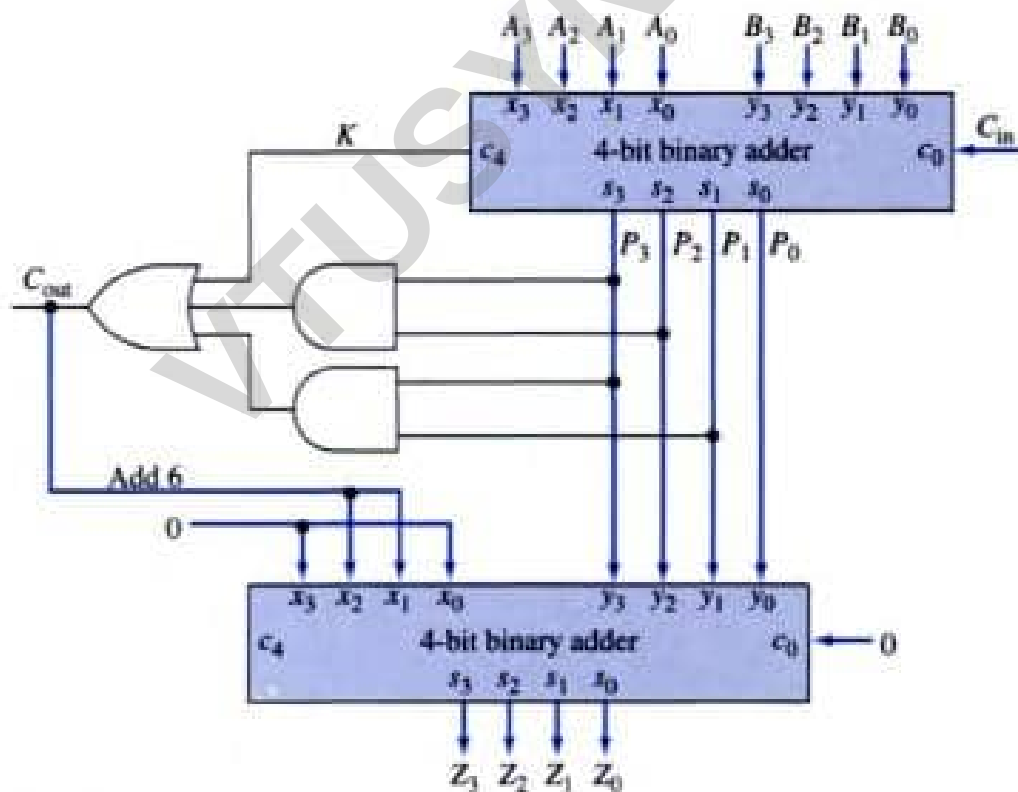


Figure 5.14 A single-decade BCD adder.

In the above diagram,

1. We take a 4-bit Binary-Adder, which takes addend and augend bits as an input with an input carry '**Carry in**'.
2. The Binary-Adder produces five outputs, i.e., Z₈, Z₄, Z₂, Z₁, and an output carry K.
3. With the help of the output carry K and Z₈, Z₄, Z₂, Z₁ outputs, the logical circuit is designed to identify the C_{out}
4. The Z₈, Z₄, Z₂, and Z₁ outputs of the binary adder are passed into the 2nd 4-bit binary adder as an Augend.
5. The addend bit of the 2nd 4-bit binary adder is designed in such a way that the 1st and the 4th bit of the addend number are 0 and the 2nd and the 3rd bit are the same as C_{out}. When the value of C_{out} is 0, the addend number will be 0000, which produce the same result as the 1st 4-bit binary number. But when the value of the C_{out} is 1, the addend bit will be 0110, i.e., 6, which adds with the augend to get the valid BCD number.

Example: 1001+1000

1. First, add both the numbers using a 4-bit binary adder and pass the input carry to 0.
2. The binary adder produced the result 0001 and carried output 'K' 1.
3. Then, find the C_{out} value to identify that the produced BCD is invalid or valid using the expression

$$C_{out} = K + Z_8 \cdot Z_4 + Z_8 \cdot Z_2$$

$$K = 1$$

$$Z_8 = 0$$

$$Z_4 = 0$$

$$Z_2 = 0$$

$$C_{out} = 1 + 0 \cdot 0 + 0 \cdot 0$$

$$C_{out} = 1 + 0 + 0$$

$$C_{out} = 1$$

4. The value of C_{out} is 1, which expresses that the produced BCD code is invalid. Then, add the output of the 1st 4-bit binary adder with 0110.

$$= 0001 + 0110$$

$$= 0111$$

5. The BCD is represented by the carry output as:
 $BCD = C_{out} Z_8 Z_4 Z_2 Z_1 = 1\ 0\ 1\ 1\ 1$

Comparators

A Comparator is a [combinational circuit](#) that gives output in terms of $A > B$, $A < B$, and $A = B$. This is entirely expected from the name. A digital comparator's purpose is to compare numbers and represent their relationship with each other.

The below figure shows 1 bit comparator with 5 inputs and 3 outputs.

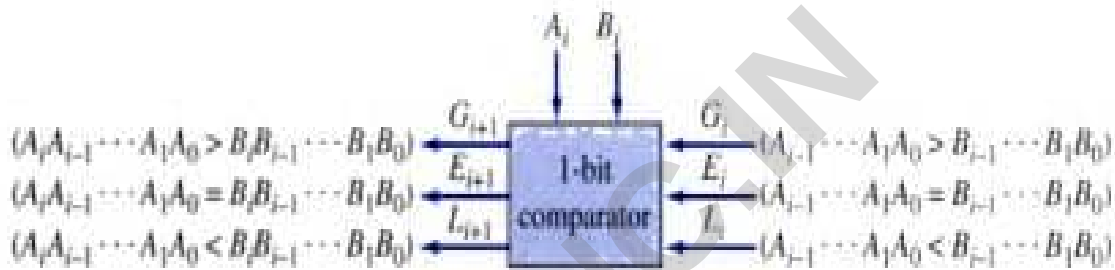


Figure 5.15 Organization of a 1-bit comparator.

Table 5.4 Truth table for a 1-bit comparator

A_i	B_i	G_i	E_i	L_i	G_{i+1}	E_{i+1}	L_{i+1}	A_i	B_i	G_i	E_i	L_i	G_{i+1}	E_{i+1}	L_{i+1}
0	0	0	0	0	—	—	—	1	0	0	0	0	—	—	—
0	0	0	0	1	0	0	1	1	0	0	0	1	1	0	0
0	0	0	1	0	0	1	0	1	0	0	1	0	1	0	0
0	0	0	1	1	—	—	—	1	0	0	1	1	—	—	—
0	0	1	0	0	1	0	0	1	0	1	0	0	1	0	0
0	0	1	0	1	—	—	—	1	0	1	0	1	—	—	—
0	0	1	1	0	—	—	—	1	0	1	1	0	—	—	—
0	0	1	1	1	—	—	—	1	0	1	1	1	—	—	—
0	1	0	0	0	—	—	—	1	1	0	0	0	—	—	—
0	1	0	0	1	0	0	1	1	1	0	0	1	0	0	1
0	1	0	1	0	0	0	1	1	1	0	1	0	0	1	0
0	1	0	1	1	—	—	—	1	1	0	1	1	—	—	—
0	1	1	0	0	0	0	1	1	1	1	0	0	1	0	0
0	1	1	0	1	—	—	—	1	1	1	0	1	—	—	—
0	1	1	1	0	—	—	—	1	1	1	1	0	—	—	—
0	1	1	1	1	—	—	—	1	1	1	1	1	—	—	—

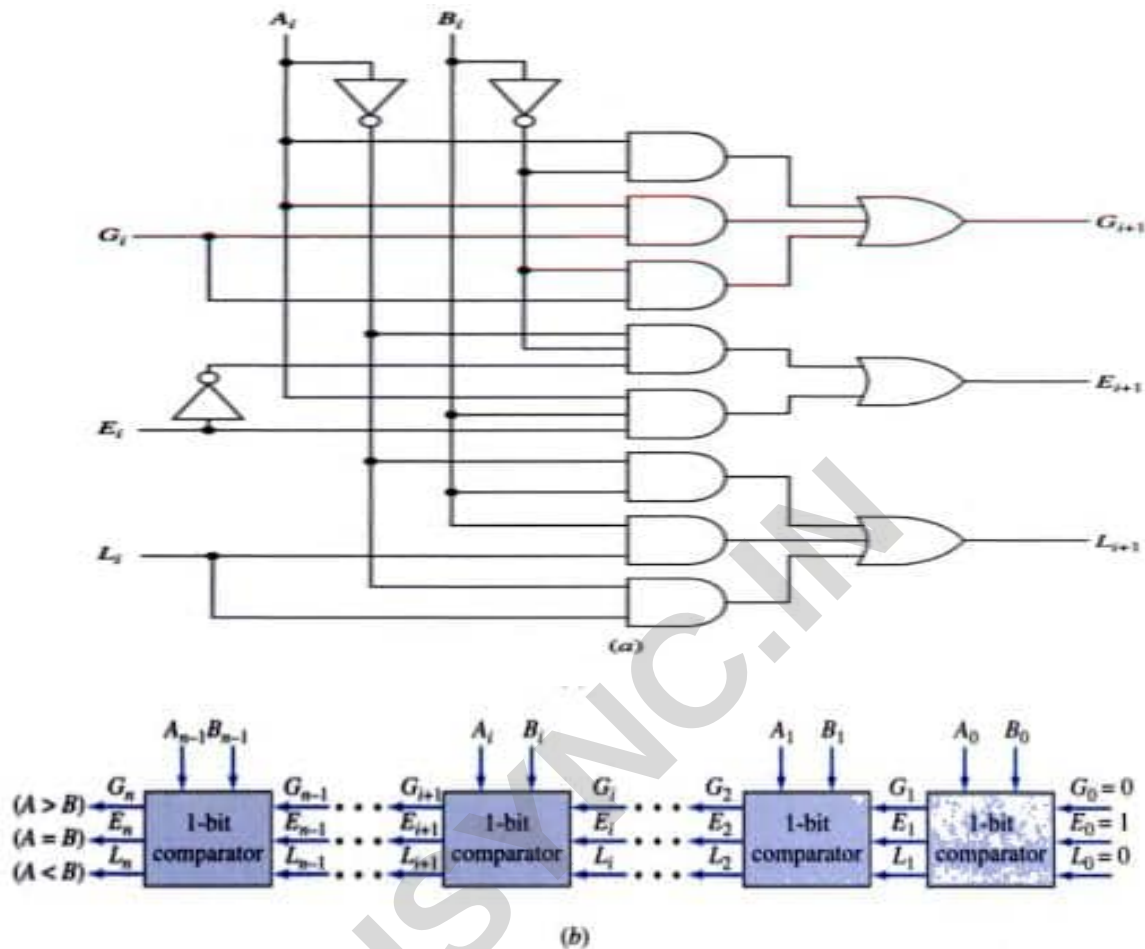


Figure 5.16 Comparing two binary numbers A and B . (a) 1-bit comparator network. (b) Cascade connection of 1-bit comparators.

Binary comparators

Magnitude Comparator

- A magnitude Comparator is a combinational circuit that compares two digital or binary numbers in order to find out whether one binary number is equal, less than or greater than the other binary number.
- To design a logical circuit for comparator it requires two inputs: one for A and other for B and three output, one for $A > B$ condition, one for $A = B$ condition and one for $A < B$ condition.

1-Bit Magnitude Comparator

- A comparator used to compare two bits is called a single bit comparator.
- It consists of two inputs each for two single bit numbers and three outputs to generate less than, equal to and greater than between two binary numbers as shown in the fig.2.25.

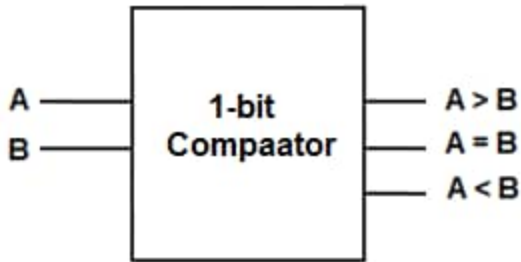


Fig.2.25: Block diagram of 1-bit comparator

- The truth table for a 1-bit comparator is given below

Inputs		Outputs		
A	B	A = B	A > B	A < B
0	0	1	0	0
0	1	0	0	1
1	0	0	1	0
1	1	1	0	0

- To design the logic for a 1-bit comparator, find the Boolean equation for the three outputs.
- The equations for each output are:

$$A \text{ EQ } B = \overline{A}\overline{B} + AB = \overline{A \oplus B}$$

$$A > B = A\overline{B}$$

$$A < B = \overline{A}B$$

- The resulting logic diagram is shown in the fig.2.26

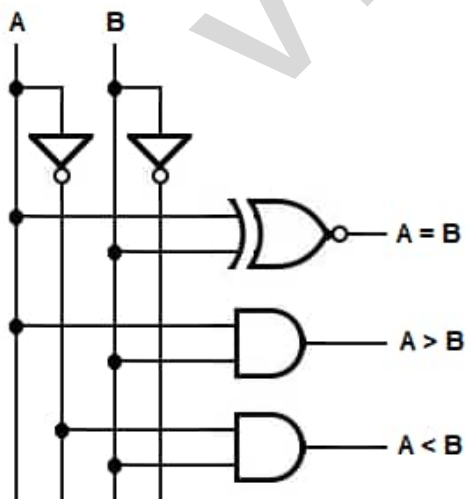


Fig.2.26: Logic diagram of 1-bit comparator

2-Bit Magnitude Comparator

- 2-bit Magnitude comparator is used to compare two binary numbers, each input is of two bits as shown in the fig.2.27.
- It consists of two inputs and three outputs to generate less than, equal to and greater than between two binary numbers.

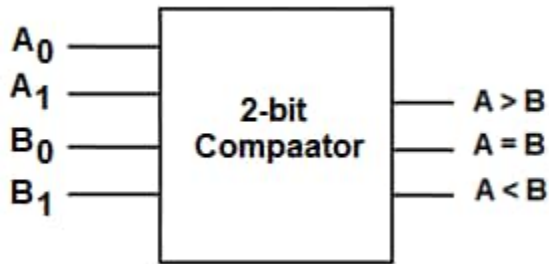


Fig.2.27: Block diagram of 1-bit comparator

- The truth table for a 2-bit comparator is given below

Inputs				Outputs		
A		B				
a ₁	a ₀	b ₁	b ₀	A = B	A > B	A < B
0	0	0	0	1	0	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	0	1	0
0	1	0	1	1	0	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	0	1	0
1	0	0	1	0	1	0
1	0	1	0	1	0	0
1	0	1	1	0	0	1
1	1	0	0	0	1	0
1	1	0	1	0	1	0

1	1	1	0	0	1	0
1	1	1	1	1	0	0

➤ To design the logic for a 2-bit comparator, find the Boolean equation for the three outputs.

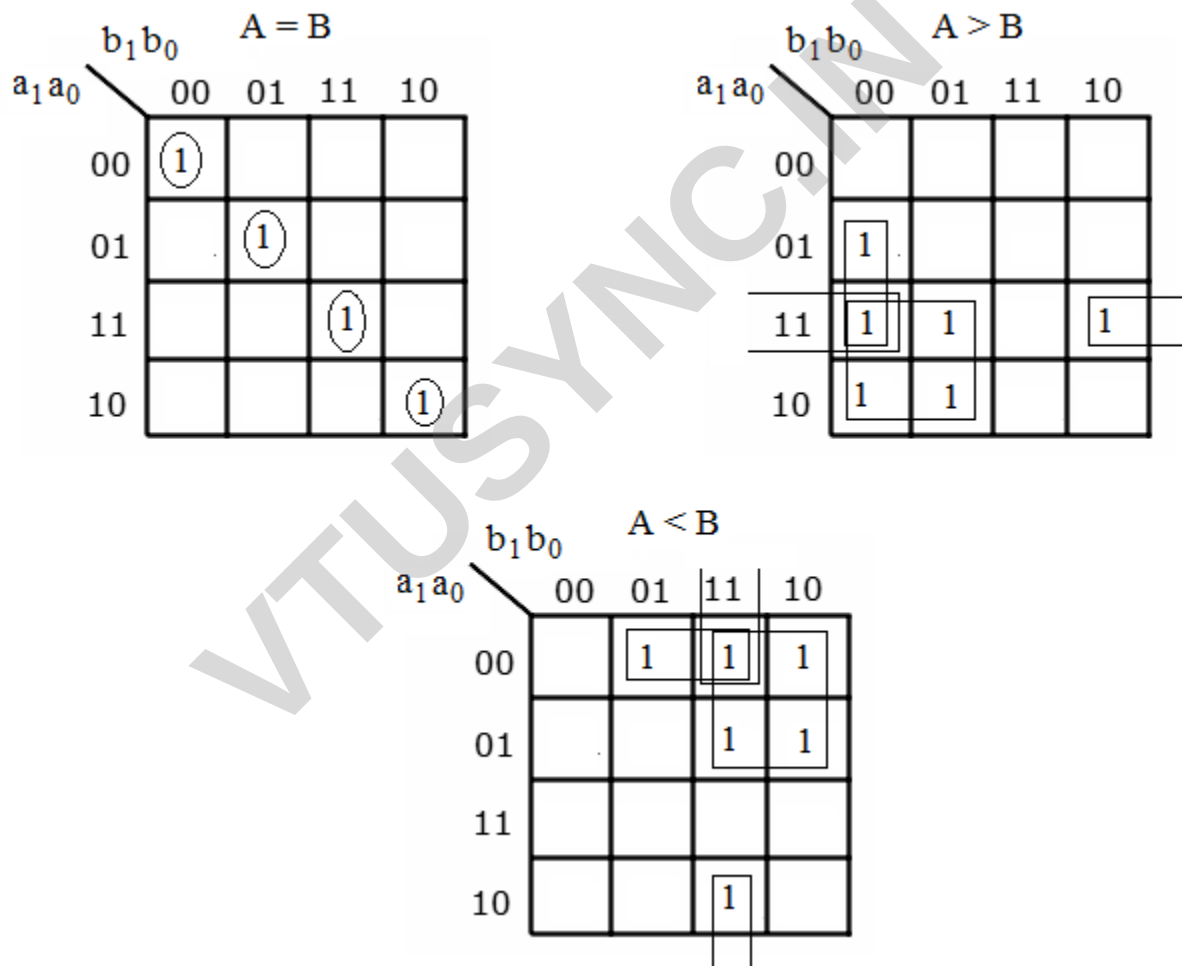
➤ The equations for each output are:

$$A = B = f(a_1, a_0, b_1, b_0) = \sum m(0, 5, 10, 15)$$

$$A > B = f(a_1, a_0, b_1, b_0) = \sum m(4, 8, 9, 12, 13, 14)$$

$$A < B = f(a_1, a_0, b_1, b_0) = \sum m(1, 2, 3, 6, 7, 11)$$

➤ The boolean expression for each output terminal by using K-maps.



The resulting equations are:

$$A = B = \bar{a}_1\bar{a}_0\bar{b}_1\bar{b}_0 + \bar{a}_1a_0\bar{b}_1b_0 + a_1a_0b_1b_0 + a_1\bar{a}_0b_1\bar{b}_0$$

$$A > B = a_1\bar{b}_1 + a_0\bar{b}_1\bar{b}_0 + a_1a_0\bar{b}_0$$

$$A < B = \bar{a}_1b_1 + \bar{a}_1\bar{a}_0b_0 + \bar{a}_0b_1b_0$$

Decoders

- A decoder is a combinational circuit which has n inputs and 2^n outputs, and outputs 1 on the wire corresponding to the binary number represented by the inputs.
- Fig.2.1 shows the general structure of the decoder.

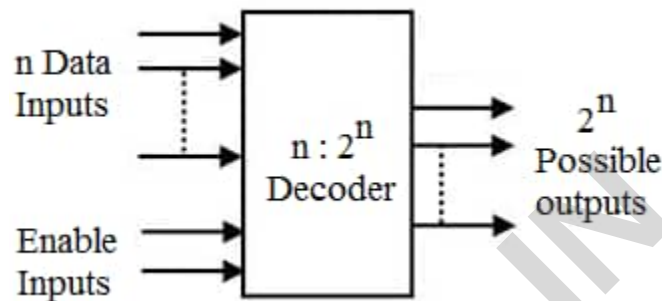


Fig.2.1: General structure of decoder

- One of these outputs will be active High based on the combination of inputs present, when the decoder is enabled. The outputs of the decoder are nothing but the min terms of ' n ' input variables (lines), when it is enabled.

2:4 Decoder with Active high enable input

- 2:4 Decoder has two inputs (A & B) and four outputs (Y_3 , Y_2 , Y_1 & Y_0) as shown in the fig.2.2. It also consists of an active high enable input E. All the inputs and outputs are active high signals.

Block Diagram:

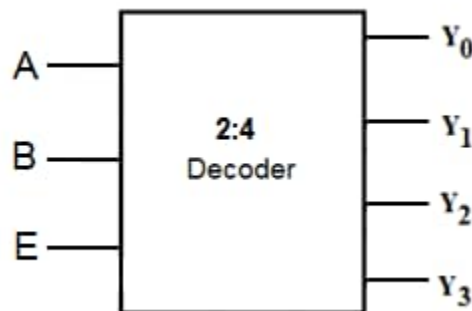


Fig.2.2: Block diagram of 2:4 Decoder with Active high enable input

- If enable, E is '0' all the four outputs will be '0'. When enable, E is '1' one of these four outputs will be '1' for each combination of inputs as shown in the truth table.

Truth Table:

INPUT		OUTPUT			
A	B	Y ₃	Y ₂	Y ₁	Y ₀
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

- From Truth table, **Boolean functions** for each output can be written as

$$Y_0 = \bar{A}\bar{B}$$

$$Y_1 = \bar{A}B$$

$$Y_2 = A\bar{B}$$

$$Y_3 = AB$$

- Each output is having one product term. So, there are four product terms in total which can be implemented using four AND gates having three inputs each & two inverters as shown in the fig.2.3.

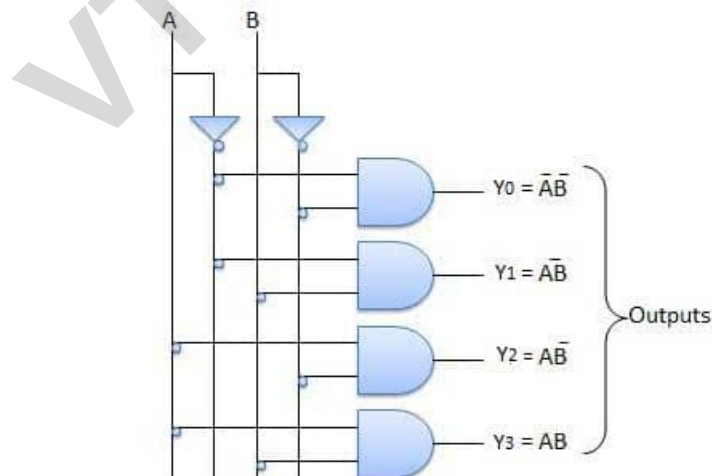
Logic Diagram:

Fig.2.3: Logic diagram of 2:4 Decoder with Active high enable input

3:8 Decoder with Active high enable input

- 3:8 Decoder has three inputs (A, B and C) and four outputs ($Y_7, Y_6, Y_5, Y_4, Y_3, Y_2, Y_1$ & Y_0) as shown in the fig.2.6. It also consists of an active high enable input E. All the inputs and outputs are active high signals.

Block Diagram:

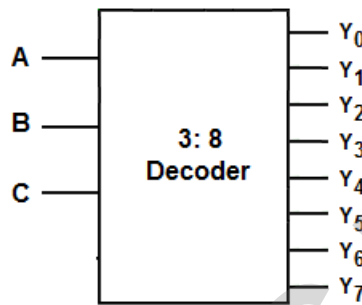


Fig.2.6: Block diagram of 3:8 Decoder with Active high enable input

- If enable, E is '0' all the eight outputs will be '0'. When enable, E is '1' one of these eight outputs will be '1' for each combination of inputs as shown in the truth table.

Truth Table:

INPUT			OUTPUT							
A	B	C	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

- From Truth table, **Boolean functions** for each output can be written as

$$Y_0 = \overline{A}\overline{B}\overline{C}$$

$$Y_1 = \overline{A}\overline{B}C$$

$$Y_2 = \overline{A}B\overline{C}$$

$$Y_3 = \bar{A}BC$$

$$Y_4 = A\bar{B}\bar{C}$$

$$Y_5 = A\bar{B}C$$

$$Y_6 = AB\bar{C}$$

$$Y_7 = ABC$$

Logic Diagram:

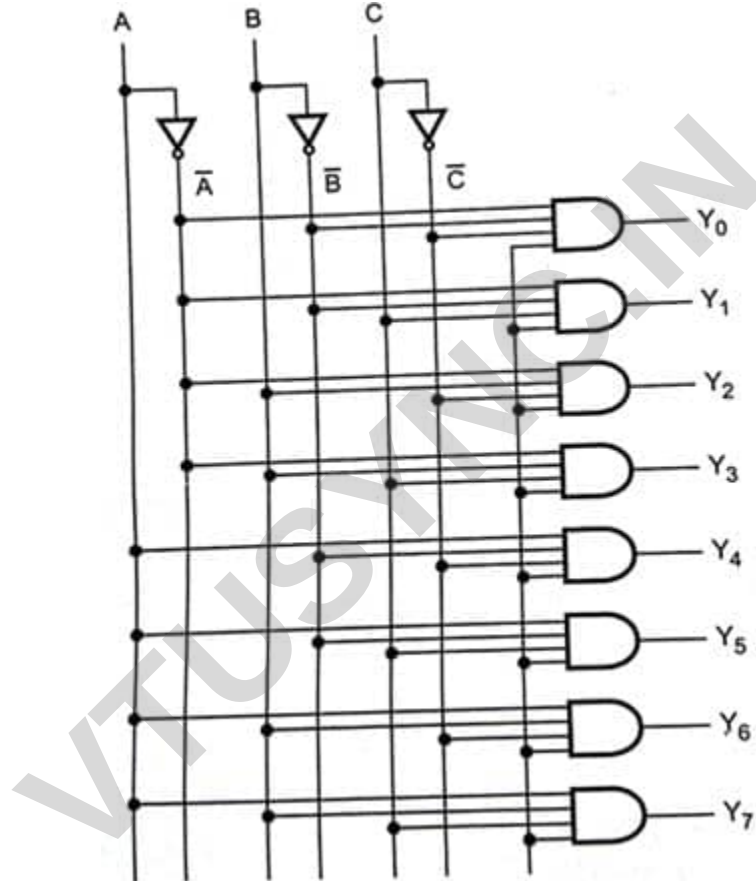


Fig.2.7: Logic diagram of 3:8 Decoder with Active high enable input

- Each output is having one product term. So, there are eight product terms in total which can be implemented using eight AND gates having three inputs each & three inverters as shown in the fig.2.7.

Realization of multiple output function using Binary decoder

- The combination of decoder and external logic gates can be used to implement single or multiple output functions.

Implementation of SOP function for Active high output:

- When decoder output is active high, it generates minterms for input variables. In order to implement SOP function, take the sum of selected products terms generated by the decoder. This can be implemented by **ORing** the selected decoder outputs.

Implementation of POS function for Active high output:

- When decoder output is active high, it generates minterms for input variables. In order to implement POS function, take the sum of selected products terms generated by the decoder and complement it. This can be implemented by **NORing** the selected decoder outputs.

Implementation of SOP function for Active low output:

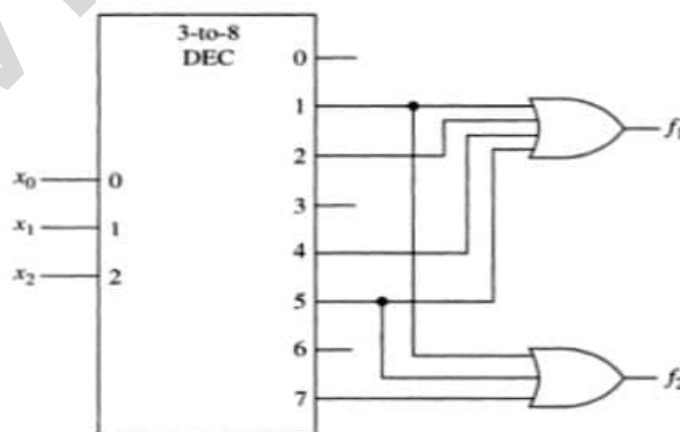
- When decoder output is active low, it generates maxterms for input variables. In order to implement SOP functions, take the product of selected sum terms generated by the decoder and complement it. This can be implemented by **NANDing** the selected decoder outputs.

Implementation of POS function for Active low output:

- When decoder output is active low, it generates maxterms for input variables. In order to implement POS functions, take the product of selected sum terms generated by the decoder. This can be implemented by **ANDing** the selected decoder outputs.

Realization of Boolean function using 3:8 decoder for sop

Q) Using single 3:8 decoder realize $f_1(x_2, x_1, x_0) = \sum m(1, 2, 4, 5)$ and $f_2(x_2, x_1, x_0) = \sum m(1, 5, 7)$

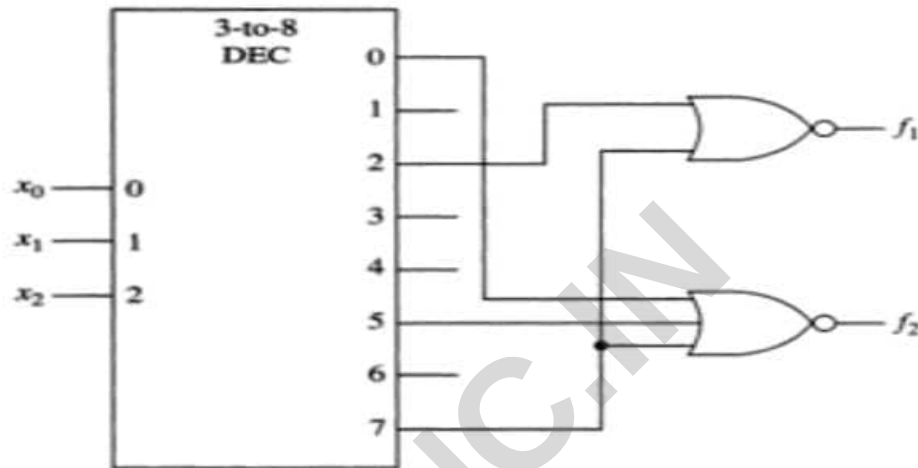


Q) Using single 3:8 decoder realize $f_1(x_2, x_1, x_0) = \sum m(0, 1, 3, 4, 5, 6)$ and $f_2(x_2, x_1, x_0) = \sum m(1, 2, 3, 4, 6)$

Sol:

$$f_1(x_2, x_1, x_0) = \sum m(0, 1, 3, 4, 5, 6) \rightarrow f_1'(x_2, x_1, x_0) = \overline{\sum m(2, 7)} = \prod (2, 7)$$

$$f_2(x_2, x_1, x_0) = \sum m(1, 2, 3, 4, 6) \rightarrow f_2'(x_2, x_1, x_0) = \overline{\sum m(0, 5, 7)} = \prod (0, 5, 7)$$

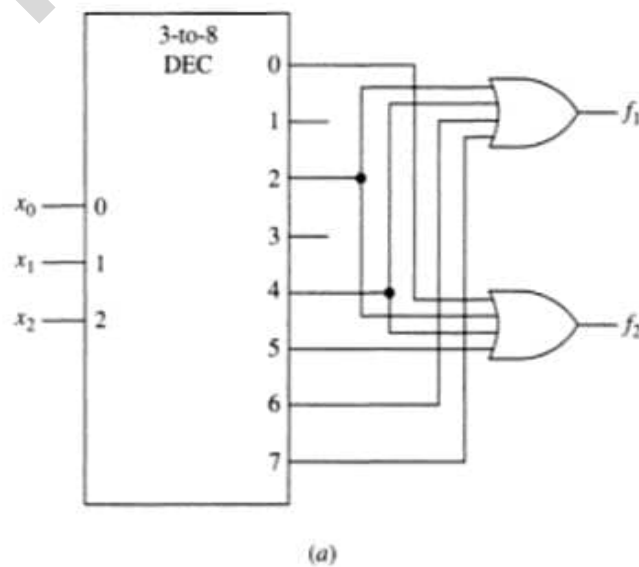


Realization of Boolean function using 3:8 decoder for POS

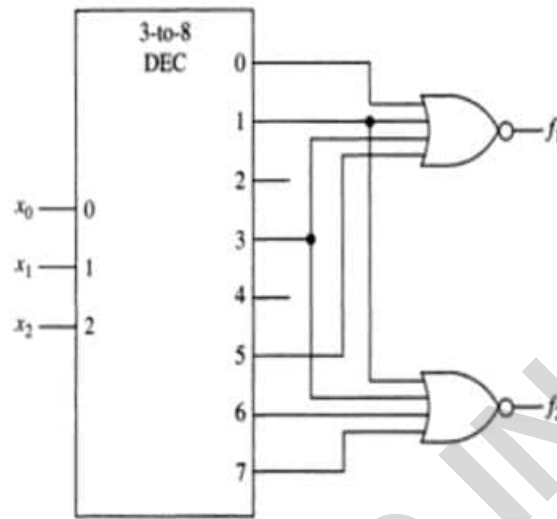
Q) Using single 3:8 decoder realize $f_1(X_2, X_1, X_0) = \prod M(0, 1, 3, 5)$ and $f_2(X_2, X_1, X_0) = \prod M(1, 3, 6, 7)$

Sol: $f_1(X_2, X_1, X_0) = \prod M(0, 1, 3, 5) \rightarrow = \sum m(2, 4, 6, 7)$

$$f_2(X_2, X_1, X_0) = \prod M(1, 3, 6, 7) \rightarrow = \sum m(0, 2, 4, 5)$$



Q) Using single 3:8 decode realize $f_1(X_2, X_1, X_0) = \sum M(0, 1, 3, 5)$ and $f_2(X_2, X_1, X_0) = \sum M(2, 3, 4)$

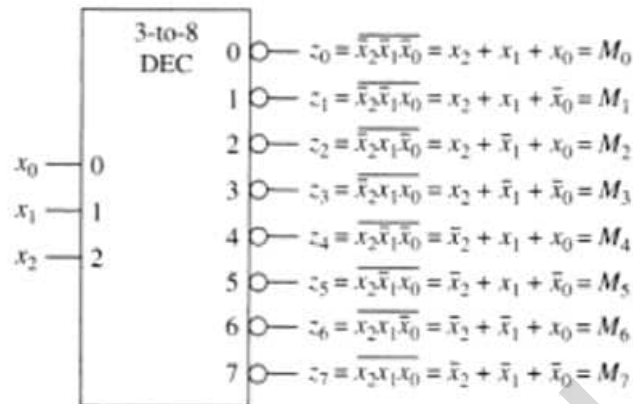


Realization of 3:8 decoder using nand gates

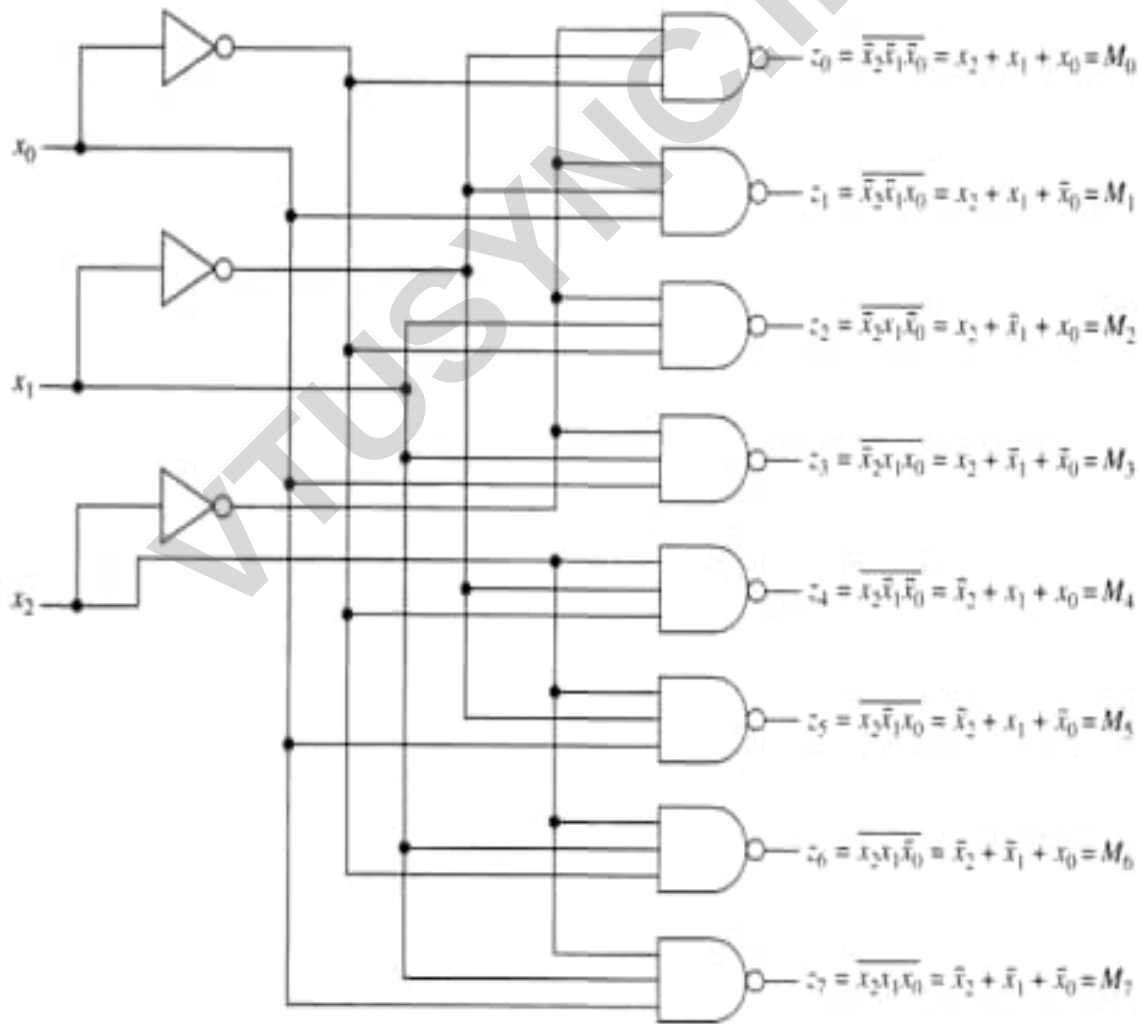
Truth table

Inputs			Outputs							
x_2	x_1	x_0	z_0	z_1	z_2	z_3	z_4	z_5	z_6	z_7
0	0	0	0	1	1	1	1	1	1	1
0	0	1	1	0	1	1	1	1	1	1
0	1	0	1	1	0	1	1	1	1	1
0	1	1	1	1	1	0	1	1	1	1
1	0	0	1	1	1	1	0	1	1	1
1	0	1	1	1	1	1	1	0	1	1
1	1	0	1	1	1	1	1	1	0	1
1	1	1	1	1	1	1	1	1	1	0

Logic diagram



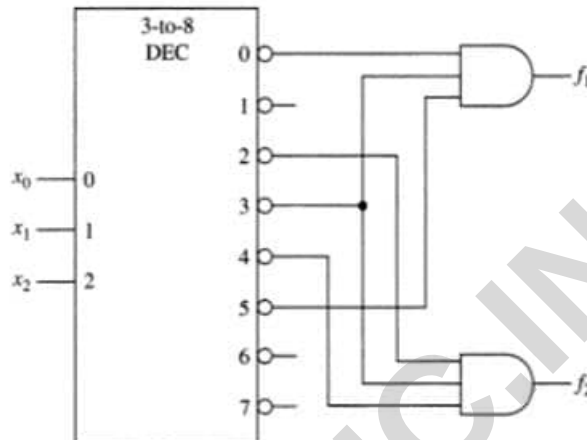
3:8 decoder using NAND gate



Realize the pair of maxterm canonical expression $f_1(X_2, X_1, X_0) = \prod M(0, 3, 5)$ and $f_2(X_2, X_1, X_0) = \prod M(2, 3, 4)$ with a 3:8 decoder and two and gates

Sol: $f_1(X_2, X_1, X_0) = \prod M(0, 3, 5)$

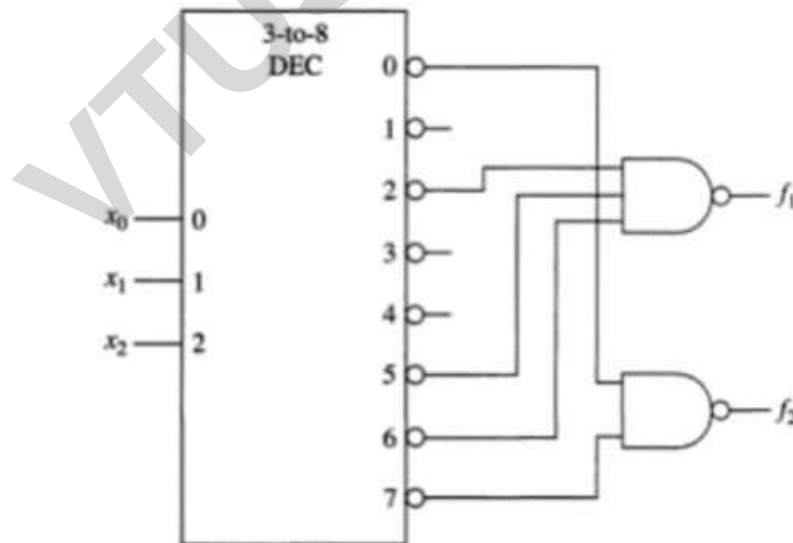
$f_2(X_2, X_1, X_0) = \prod M(2, 3, 4)$



Realize the pair of Boolean expression $f_1(X_2, X_1, X_0) = \prod M(0, 1, 3, 4, 7)$ and $f_2(X_2, X_1, X_0) = \prod M(1, 2, 3, 4, 5, 6)$ with a 3:8 decoder and two and gates

Sol: $f_1(X_2, X_1, X_0) = \prod M(0, 1, 3, 4, 7) = \overline{\prod M(2, 5, 6)} = \Sigma(2, 5, 7)$

$f_2(X_2, X_1, X_0) = \prod M(1, 2, 3, 4, 5, 6) = \overline{\prod M(0, 7)} = \Sigma(0, 7)$



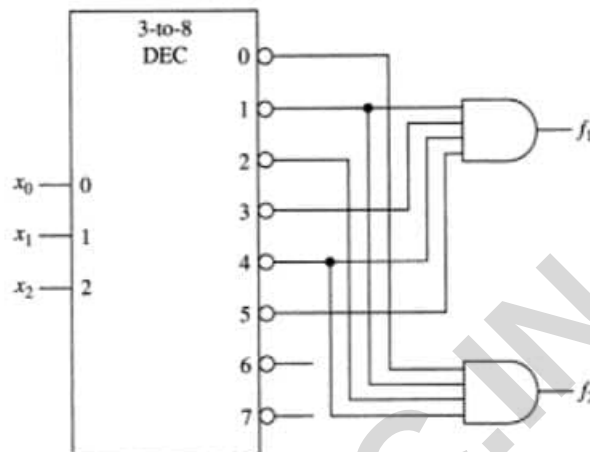
Realize the pair of Boolean expression $f_1(X_2, X_1, X_0) = \Sigma m(0, 2, 6, 7)$ and $f_2(X_2, X_1, X_0) = \Sigma m(3, 5, 6, 7)$ with a 3:8 decoder and gates and nand gates

Sol:

Using AND

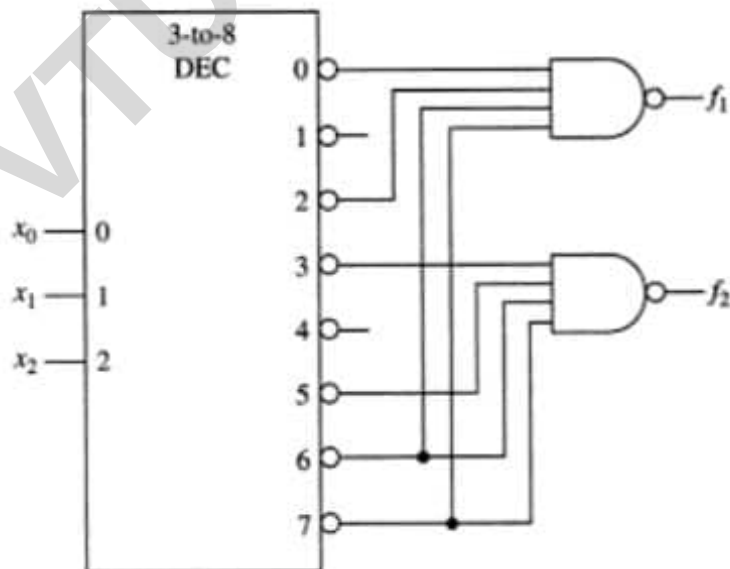
$$f_1(X_2, X_1, X_0) = \sum m(0, 2, 6, 7) = \prod M(1, 3, 4, 5)$$

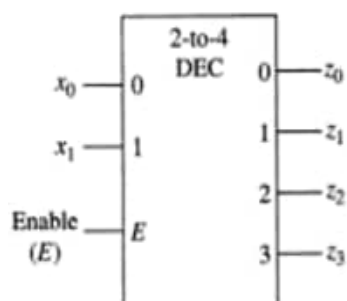
$$f_2(X_2, X_1, X_0) = \sum m(3, 5, 6, 7) = \prod M(0, 1, 2, 4)$$

**Using NAND**

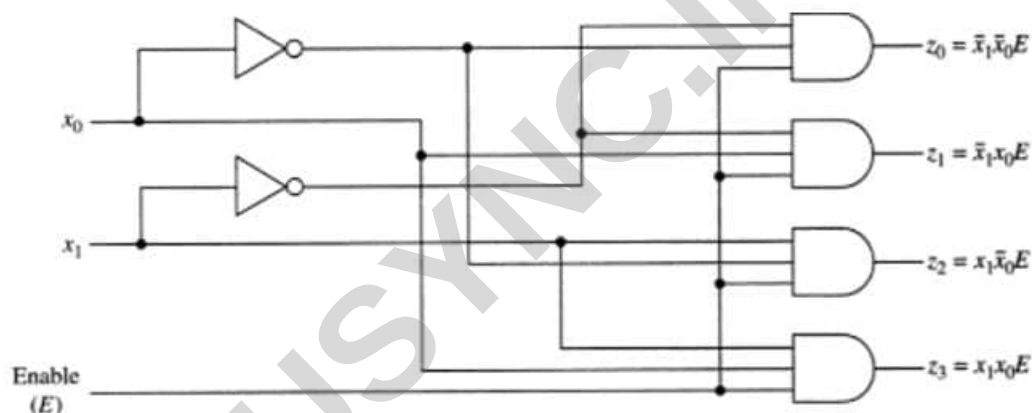
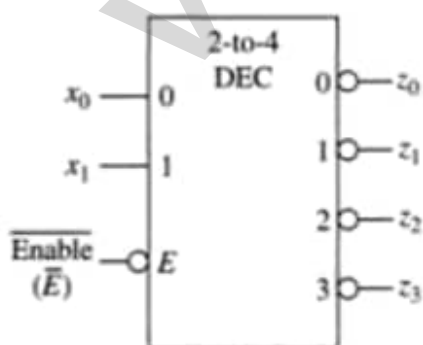
$$f_1(X_2, X_1, X_0) = \sum m(0, 2, 6, 7) = \overline{\prod M(0, 2, 6, 7)}$$

$$f_2(X_2, X_1, X_0) = \sum m(3, 5, 6, 7) = \overline{\prod M(3, 5, 6, 7)}$$



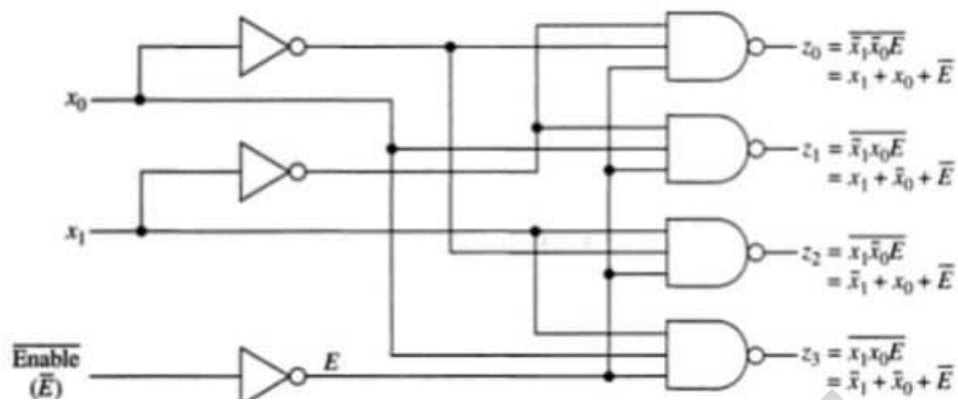
DECODER WITH AN ENABLE INPUT**Symbol****Truth table**

Inputs			Outputs			
E	x_1	x_0	z_0	z_1	z_2	z_3
0	x	x	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

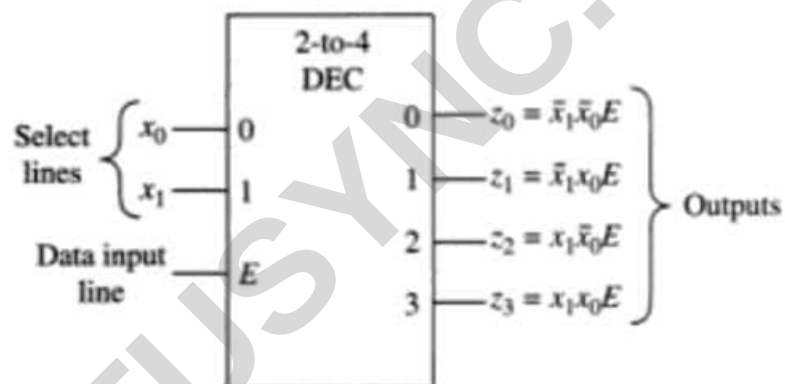
Logic diagram**2:4 decoder with enable input using NAND gate****Symbol****Truth table**

Inputs			Outputs			
$\bar{E}$	x_1	x_0	z_0	z_1	z_2	z_3
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0
1	x	x	1	1	1	1

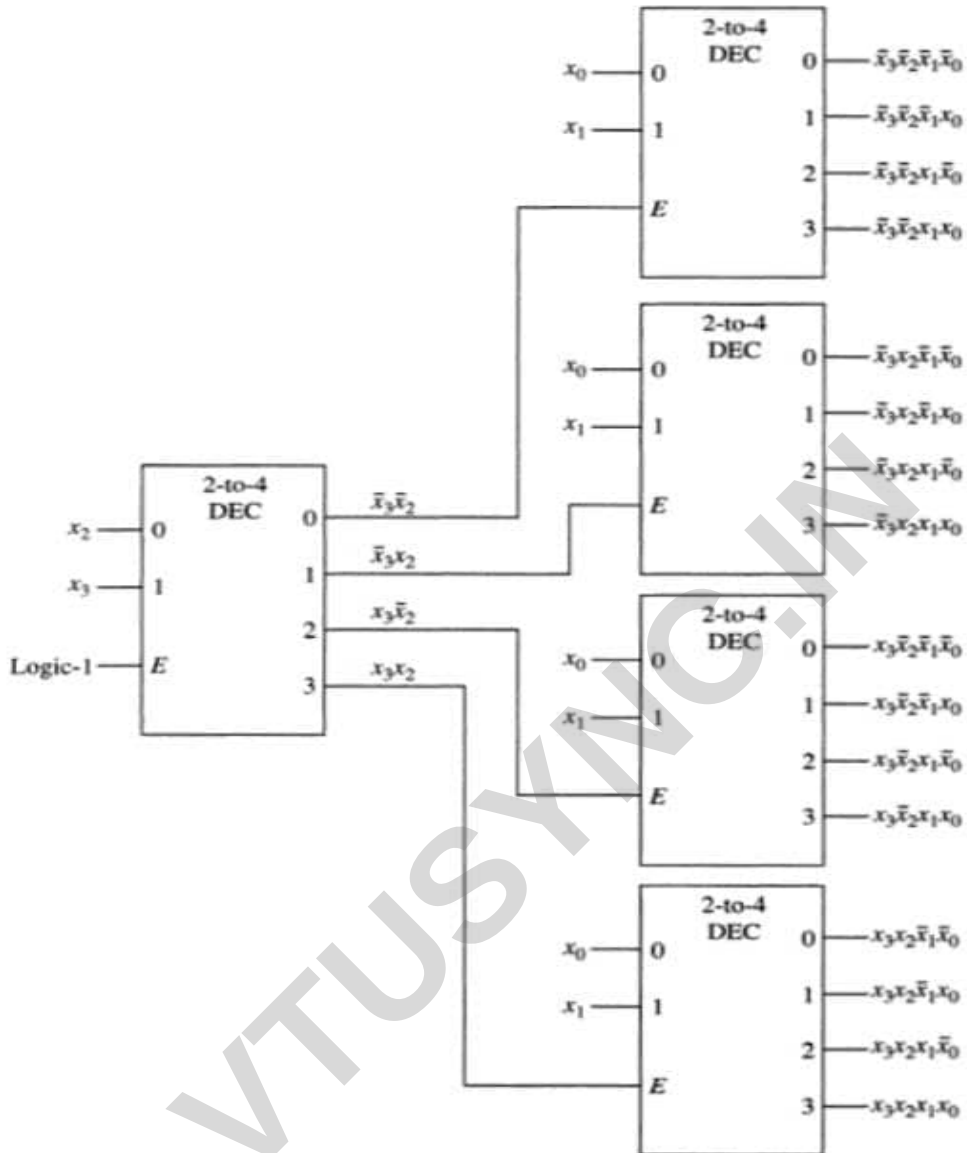
Logic diagram



DEMULTIPLEXER



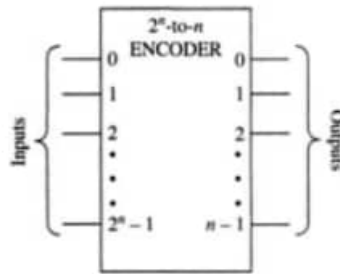
4:16 LINE DECODER USING 2:4 LINE DECODER



Encoders

- An **Encoder** is a combinational circuit that performs the reverse operation of Decoder.
- It has maximum of 2^n input lines and 'n' output lines.
- It will produce a binary code equivalent to the input, which is active High.
- Therefore, the encoder encodes 2^n input lines with 'n' bits. It is optional to represent the enable signal in encoders.

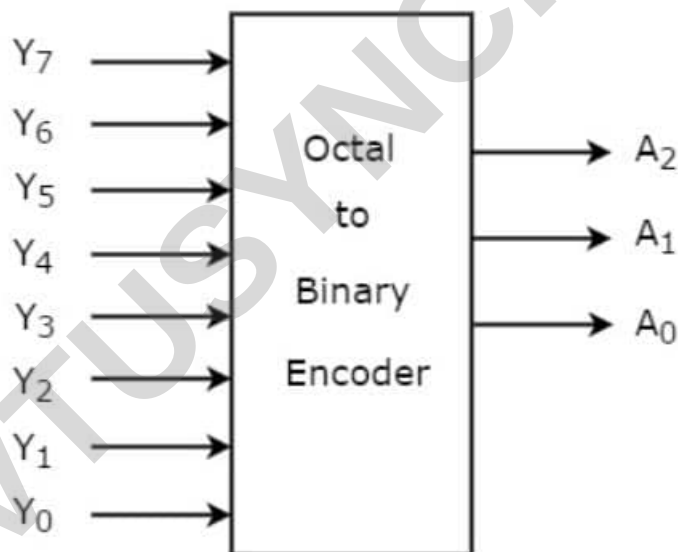
Symbol



8:3 line encoder/octal to binary encoder

Octal to binary Encoder has eight inputs, Y_7 to Y_0 and three outputs A_2 , A_1 & A_0 . Octal to binary encoder is nothing but 8 to 3 encoder. The **block diagram** of octal to binary Encoder is shown in the following figure.

Block diagram



At any time, only one of these eight inputs can be '1' in order to get the respective binary code. The **Truth table** of octal to binary encoder is shown below.

Truth table

Inputs								Outputs		
Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0	A_2	A_1	A_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

From Truth table, we can write the **Boolean functions** for each output as

Output equation :

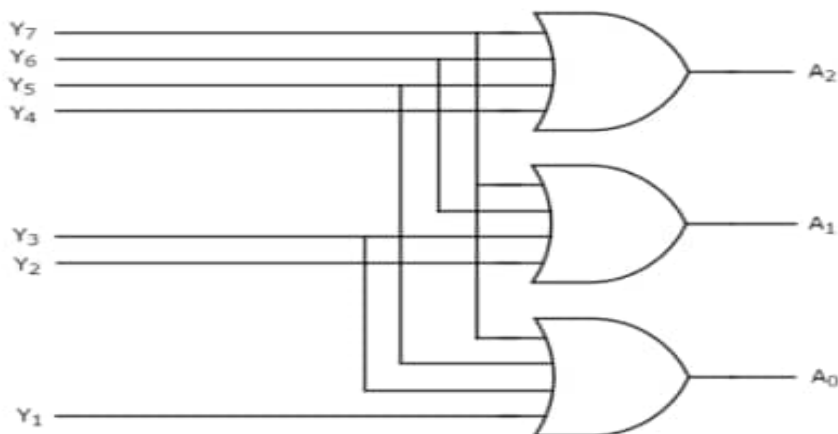
$$A_2 = Y_7 + Y_6 + Y_5 + Y_4$$

$$A_1 = Y_7 + Y_6 + Y_3 + Y_2$$

$$A_0 = Y_7 + Y_5 + Y_3 + Y_1$$

We can implement the above Boolean functions by using four input OR gates. The **circuit diagram** of octal to binary encoder is shown in the following figure.

Circuit diagram:



Drawbacks of Encoder

Following are the drawbacks of normal encoder.

- There is an ambiguity, when all outputs of encoder are equal to zero. Because, it could be the code corresponding to the inputs, when only least significant input is one or when all inputs are zero.
- If more than one input is active High, then the encoder produces an output, which may not be the correct code. For **example**, if both Y_3 and Y_6 are '1', then the encoder produces 111 at the output. This is neither equivalent code corresponding to Y_3 , when it is '1' nor the equivalent code corresponding to Y_6 , when it is '1'.

So, to overcome these difficulties, we should assign priorities to each input of encoder. Then, the output of encoder will be the binary code corresponding to the active High inputs, which has higher priority. This encoder is called as **priority encoder**.

Priority Encoder

A 4 to 2 priority encoder has four inputs Y_3, Y_2, Y_1 & Y_0 and two outputs A_1 & A_0 . Here, the input, Y_3 has the highest priority, whereas the input, Y_0 has the lowest priority. In this case, even if more than one input is '1' at the same time, the output will be the binary code corresponding to the input, which is having **higher priority**.

We considered one more **output, V** in order to know, whether the code available at outputs is valid or not.

- If at least one input of the encoder is '1', then the code available at outputs is a valid one. In this case, the output, V will be equal to 1.
- If all the inputs of encoder are '0', then the code available at outputs is not a valid one. In this case, the output, V will be equal to 0.

Table 5.5 Condensed truth table for an 8-to-3 line priority encoder

Inputs								Outputs			
x_0	x_1	x_2	x_3	x_4	x_5	x_6	x_7	z_2	z_1	z_0	Valid
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
x	1	0	0	0	0	0	0	0	0	1	1
x	x	1	0	0	0	0	0	0	1	0	1
x	x	x	1	0	0	0	0	0	1	1	1
x	x	x	x	1	0	0	0	1	0	0	1
x	x	x	x	x	1	0	0	1	0	1	1
x	x	x	x	x	x	1	0	1	1	0	1
x	x	x	x	x	x	x	1	1	1	1	1

$$Z_0 = x_3 + x_5 + x_7$$

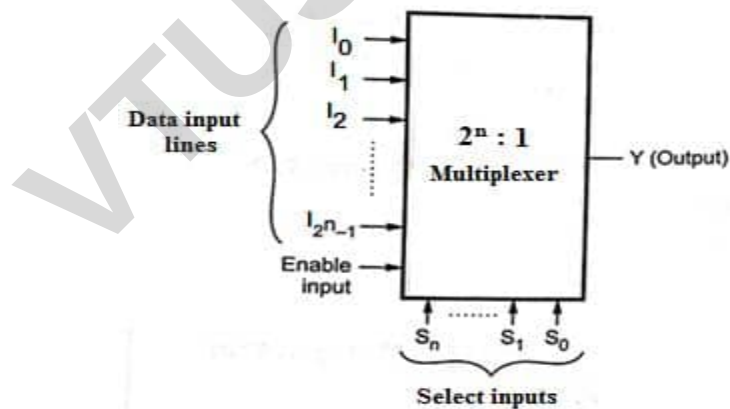
$$Z_1 = x_2 + x_3 + x_6 + x_7$$

$$Z_2 = x_1 + x_3 + x_5 + x_7$$

$$V = x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7$$

Multiplexers

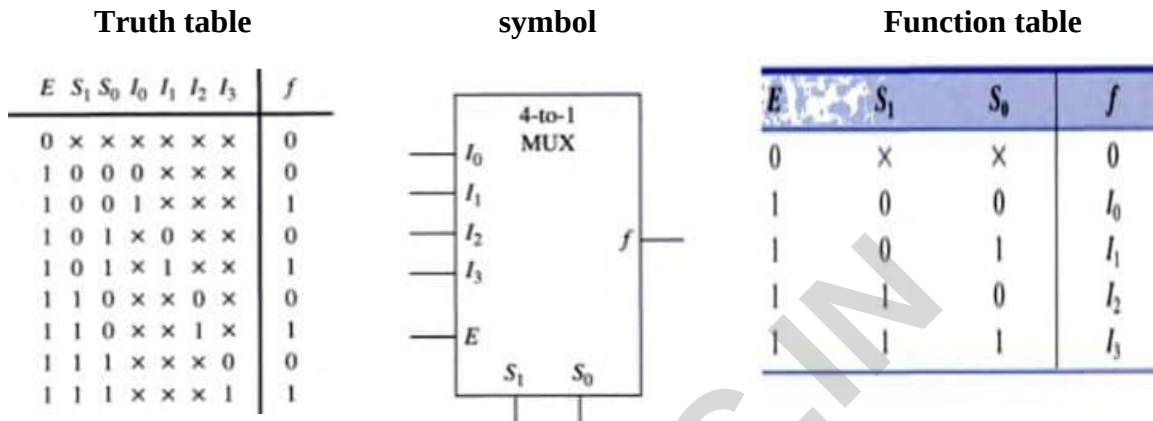
- A Digital Multiplexer connects one of m inputs to a single output line, so that the logical value of the input is transferred to the output.

**Fig.2.11: Block diagram of $2^n : 1$ multiplexer**

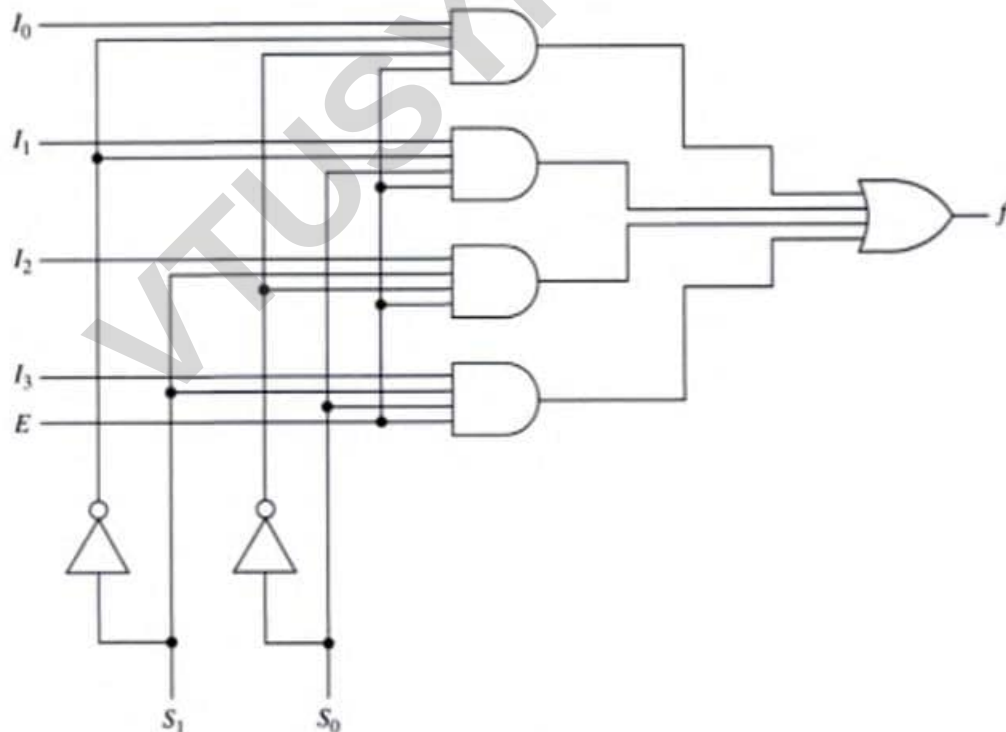
- The one of m input selection is determined by n select inputs, where $m = 2^n$. Since multiplexer selects one of inputs and routes it to the output, it is also known as data selector.
- A general block diagram of $2^n : 1$ multiplexer is shown in the fig.2.11.
- A 2 to 1 multiplexer requires 1 select input, 4 to 1 multiplexers require 2 select inputs and 8 to 1 multiplexers require 3 select inputs.

- Several digital multiplexer ICs are available. The 74xx153 is a dual 4 to 1 multiplexer, the 74xx151 is a single 8 to 1.

4:1 MULTIPLEXER



Logic diagram



4:1 Multiplexer with Active HIGH enable input

- A 4 to 1 multiplexer has 4 data inputs, 2 select inputs and 1 output. Fig.2.14 shows 4:1 multiplexer with an active low enable input.

Symbol:

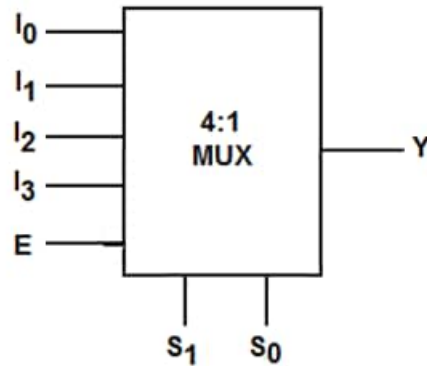


Fig.2.14: Block diagram of 4:1 Multiplexer with Active HIGH enable input

Compressed truth table

E	S1	S0	I0	I1	I2	I3	Y
0	X	X	X	X	X	X	0
1	0	0	0	X	X	X	0
1	0	0	1	X	X	X	1
1	0	1	X	0	X	X	0
1	0	1	X	1	X	X	1
1	1	0	X	X	0	X	0
1	1	0	X	X	1		1
1	1	1	X	X	X	0	0
1	1	1	X	X	X	1	1

Functional Table:

Enable (E)	Select (S1)	Select (S0)	Output Y
0	X	X	0
1	0	0	I ₀
1	0	1	I ₁
1	1	0	I ₂

1	1	1	I_3
---	---	---	-------

- When $E = 0$, output $Y = 0$ irrespective of any input condition.
- When $E = 1$ output selects any one of the four data input depending on the select input value.
- From Truth table, **Boolean functions** for the output can be written as

$$Y = E\bar{S}_1\bar{S}_0I_0 + E\bar{S}_1S_0I_1 + ES_1\bar{S}_0I_2 + ES_1S_0I_3$$

Logic Diagram:

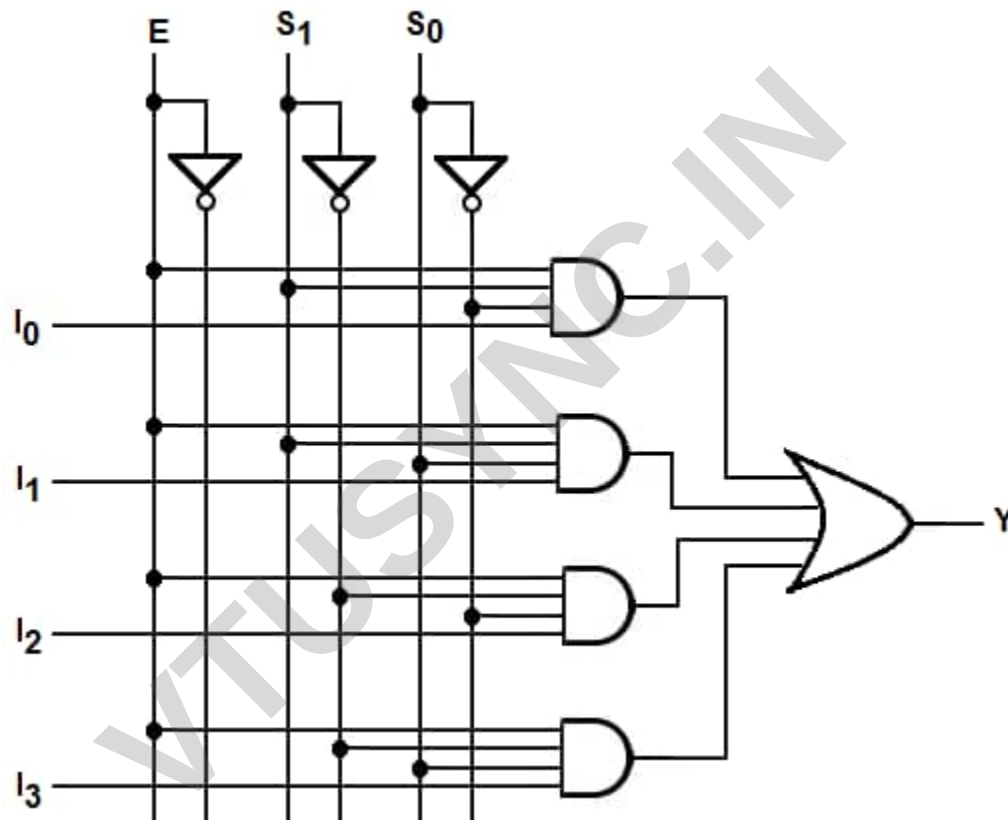
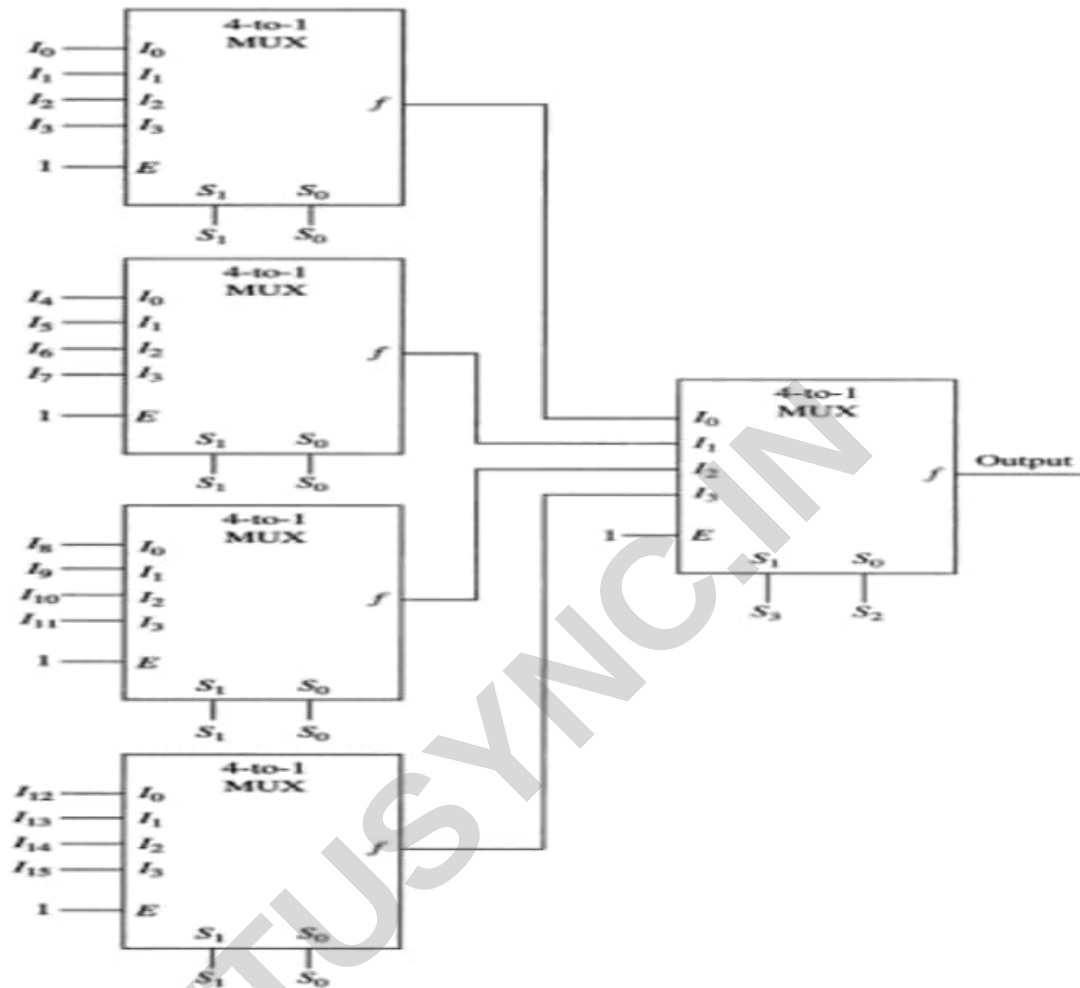
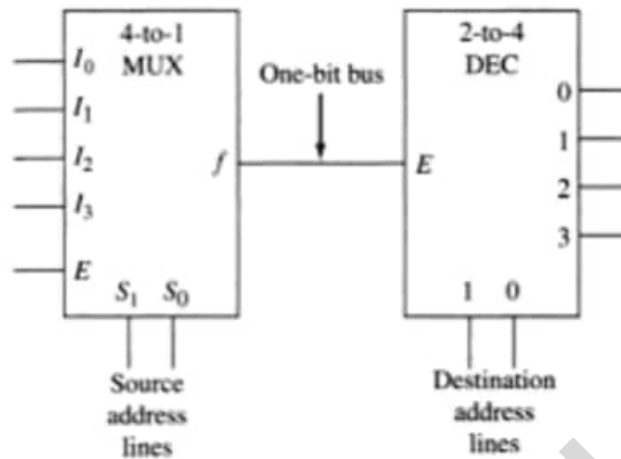


Fig.2.15: Logic diagram of 4:1 Multiplexer with Active High enable input

- The output is having four product terms which can be implemented using four AND gates having four inputs each, three inverters and an OR gate having four inputs as shown in the fig.2.15.

Multiplexer tree to form 16:1 multiplexer using 4:1 mux**Multiplexer or demultiplexer arrangement for information transmission**

It has a single input of data and signals. **A Multiplexer generates a single output for data and signals. A Demultiplexer generates multiple outputs for data and signals.** It processes the digital data and info by collecting them from multiple sources and integrating them into a single source as the output.



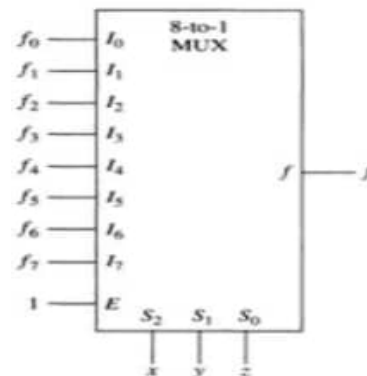
Logic design with mux

Realize 3 variable function using an 8:1 mux

Truth table

x	y	z	f
0	0	0	f_0
0	0	1	f_1
0	1	0	f_2
0	1	1	f_3
1	0	0	f_4
1	0	1	f_5
1	1	0	f_6
1	1	1	f_7

General realization



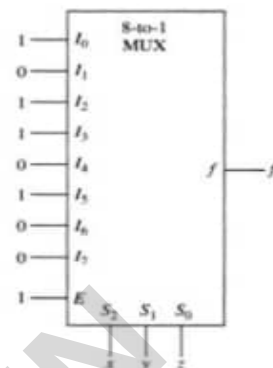
Q) Realize $f(x,y,z) = \sum m(0,2,3,5)$

Sol:

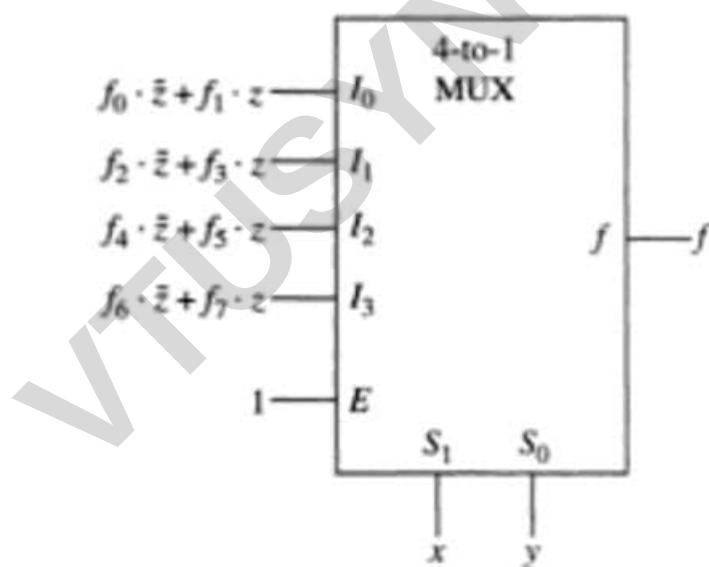
Truth table

x	y	z	f
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	0

8:1 realization

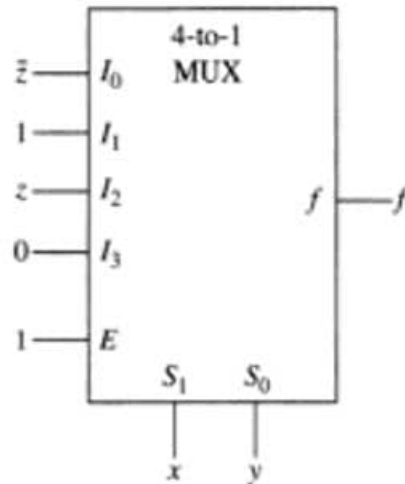


Realization of 3 variable function using 4:1 mux



Realization of 3 variable function using 4:1 mux

q) $f(x,y,z) = \sum m(0,2,3,5)$

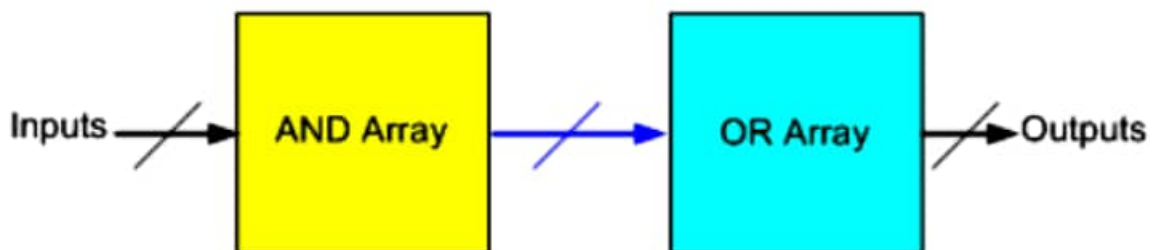


Programmable Logic Devices (PLDs)

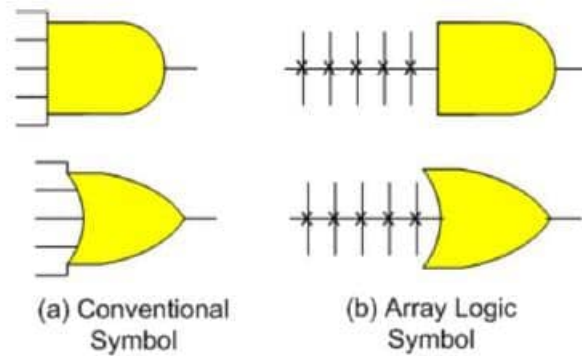
Programmable Logic Devices (PLDs) are the integrated circuits. They contain an array of AND gates & another array of OR gates. There are three kinds of PLDs based on the type of arrays, which has programmable feature.

- Programmable Read Only Memory
- Programmable Array Logic
- Programmable Logic Array

The process of entering the information into these devices is known as **programming**. Basically, users can program these devices or ICs electrically in order to implement the Boolean functions based on the requirement. Here, the term programming refers to hardware programming but not software programming.



In order to show the internal logic diagram for such technologies in a concise form, it is necessary to have special symbols for array logic. Figure shows the conventional and array logic symbols for a multiple input AND and a multiple input OR gate.



Programmable Logic Devices (PLDs)

PLD's are Standard logic devices that can be **programmed** to implement any combinational logic circuit. **Programmable** refers to a hardware process used to specify the logic that a PLD implements.

There are various types of PLD devices based on which array is programmable

The Device names and the type of array are listed in the table below.

Types of PLDs

DEVICE	AND array	OR array
PROM	Fixed	Programmable
PLA	Programmable	Programmable
PAL	Programmable	Fixed

Programmable Logic Arrays (PLAs)

Similar concept as in PROM, except that a PLA does not necessarily generate all possible minterms (ie. the decoder is not used). More precisely, in PLAs both the AND and OR arrays can be programmed (in PROM, the AND array is fixed – the decoder – and only the OR array can be programmed).

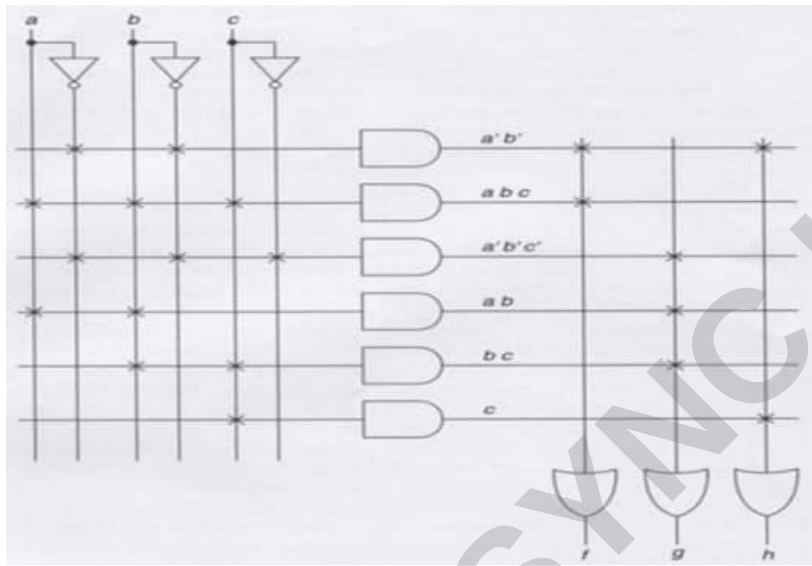
PLA Example

$$f(a,b,c) = a'b' + abc$$

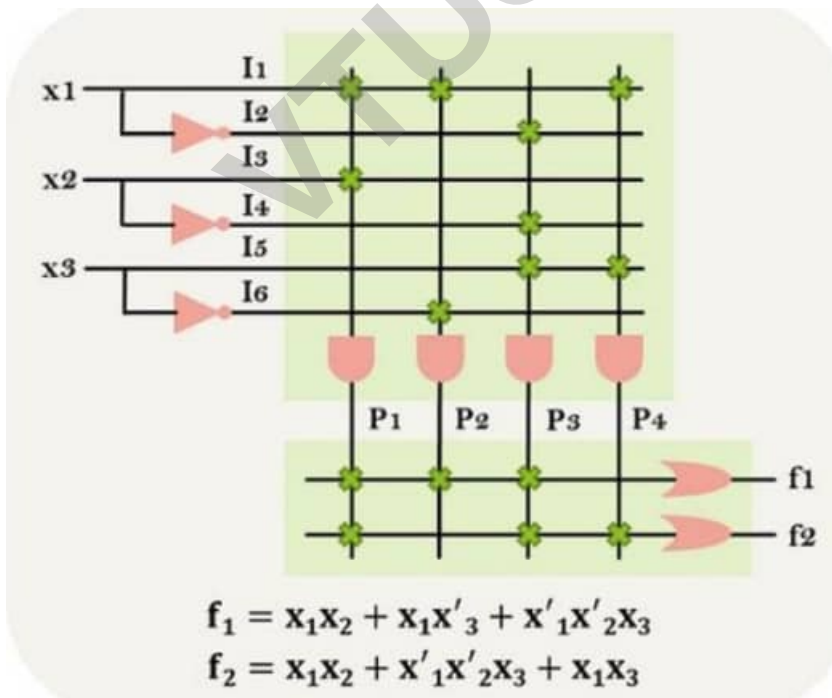
$$g(a,b,c) = a'b'c' + ab + bc$$

$$h(a,b,c) = c$$

PLAs can be more compact implementations than ROMs, since they can benefit from minimizing the number of products required to implement a function.



Example 2

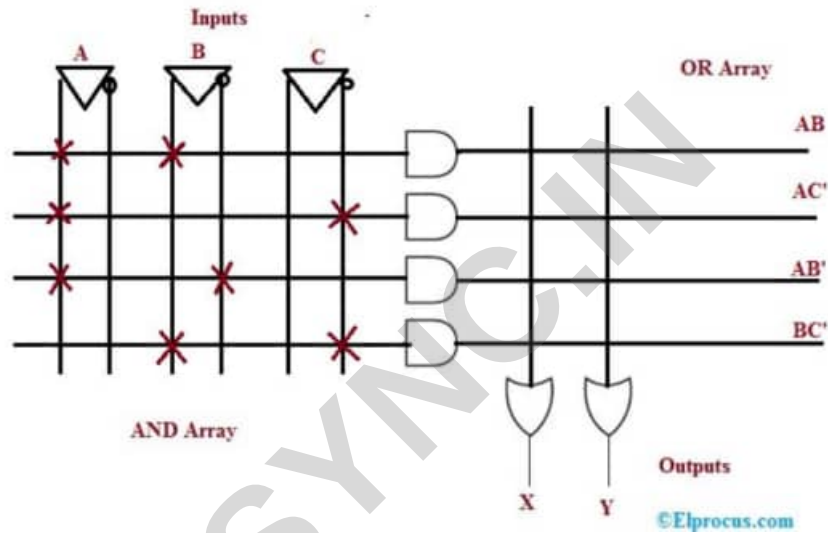


Programmable Array Logic (PAL)

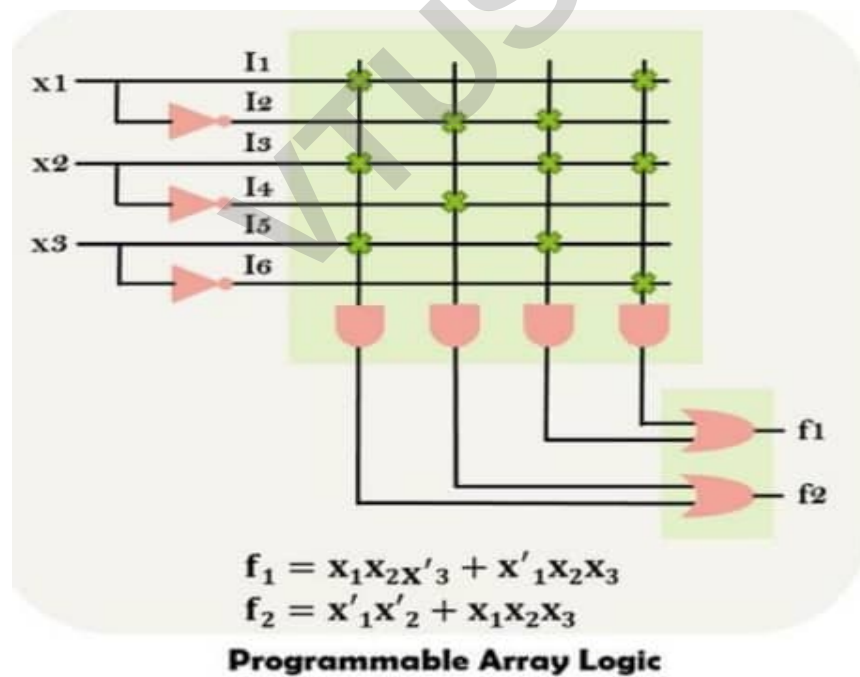
- OR plane (array) is fixed, AND plane can be programmed
- A PAL is less flexible than PLA
- Number of product terms available per function (OR outputs) is limited

Example 1:

- $X = AB + AC'$
 $Y = AB' + BC'$

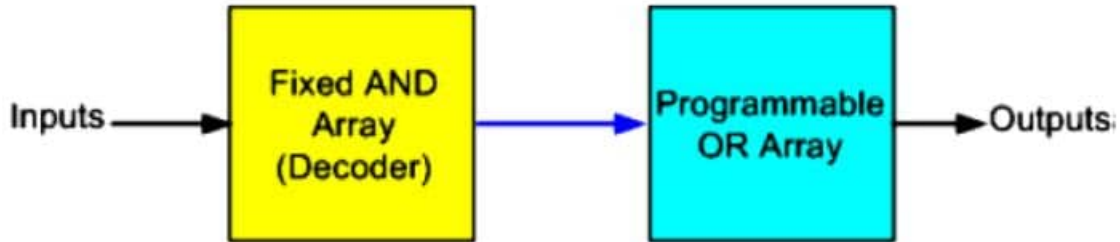


- Example 2

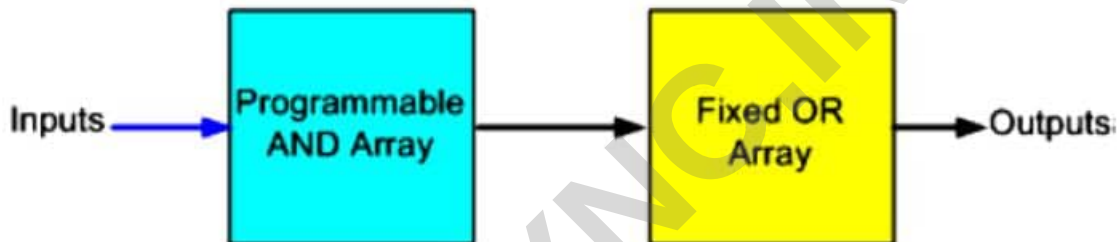


Three Fundamental Types of PLDs:

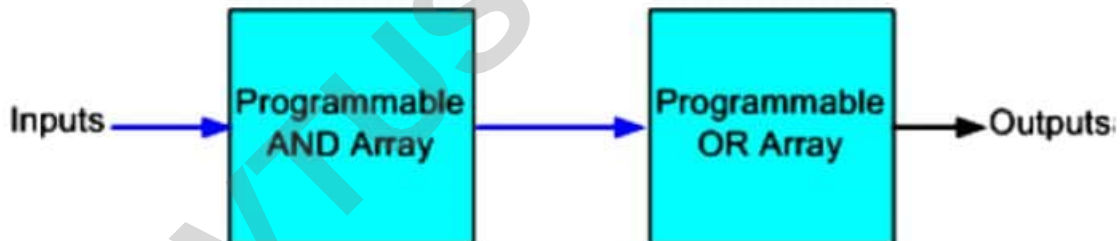
The three fundamental types of PLDs differ in the placement of programmable connections in the AND-OR arrays. Figure shows the locations of the programmable connections for the three types.



(a) Programmable Read Only Memory (PROM)



(b) Programmable Array Logic (PAL) Device



(c) Programmable Logic Array (PLA) Device

- The PROM (Programmable Read Only Memory) has a fixed AND array (constructed as a decoder) and programmable connections for the output OR gates array. The PROM implements Boolean functions in sum-of-min terms form.

The PAL (Programmable Array Logic) device has a programmable AND array and fixed connections for the OR array.

The PLA (Programmable Logic Array) has programmable connections for both AND and OR arrays. So it is the most flexible type of PLD.

The ROM (Read Only Memory) or PROM (Programmable Read Only Memory):

The input lines to the AND array are hard-wired and the output lines to the OR array are programmable.

Each AND gate generates one of the possible AD products (.e., i minterms).

The PLA (Programmable Logic Array):

In PLAs, instead of using a decoder as in PROMs, a number (k) of AND gates is used where $k < 2^n$, (n is the number of inputs).

Each of the AND gates can be programmed to generate a product term of the input variables and does not generate all the minterms as in the ROM.

The AND and OR gates inside the PLA are initially fabricated with the links (fuses) among them.

VTUSYNC.IN

Question bank

1. What is magnitude comparator? Design a two bit digital comparator by writing TT, relevant expression and logic diagram.
2. Implement the following functions using 3:8 decoder(IC-74138)
 - i. $f(a, b, c, d) = \pi M(2, 4, 5, 7, 9, 10, 13, 14)$
 - ii. $f(a, b, c, d) = \sum m(1, 3, 5, 8, 12, 14, 15)$
3. Explain Carry look ahead adder with neat diagram and relevant expressions.
4. Implement $f(a, b, c, d) = \sum m(0, 1, 5, 6, 10, 12, 14, 15)$ using
 - i. 8:1 MUX with a, b, c as select lines.
 - ii. 4:1 MUX with a, b as select lines.
5. Implement the following function using 74138 Decoder
 - i. $f_1(a, b, c) = \pi M(2, 3, 4, 5, 7)$
 - ii. $f_2(a, b, c) = \sum m(1, 3, 5)$
6. Implement $f(a, b, c, d) = \sum m(0, 1, 5, 6, 7, 10, 15)$ using 8:1 MUX with a, b, c as select lines
7. Design a 4 to 16 decoder by cascading 2 to 4 decoders.
8. Design an 8:1 MUX using only 2:1 Multiplexers.
9. Design a full adder by constructing the truth table and simplify the output equations.
10. Design one-bit comparator circuit, represent truth table, k-maps and logic diagram.
11. Implement 4-bit parallel adder/subtractor using 4-full adders blocks. Explain its operation, if $C_{in} = 0$ the circuit should act as adder and if $C_{in} = 1$ the circuit act as subtractor.
12. Implement the function using 8:1 MUX,
$$F(a, b, c, d) = \sum m(0, 1, 3, 4, 7, 10, 11, 14, 15)$$
13. Construct an 16:1 MUX using 4:1 and 2:1 multiplexers and hence analyze using truth table

Module - 3

Flip-Flops and its Applications

Introduction

- Logic circuit can be classified as combinational circuits and sequential circuits
- Logic circuits whose outputs at any instant of time depend on the input signals present at that time are known as combinational circuits as shown in fig.3.1.

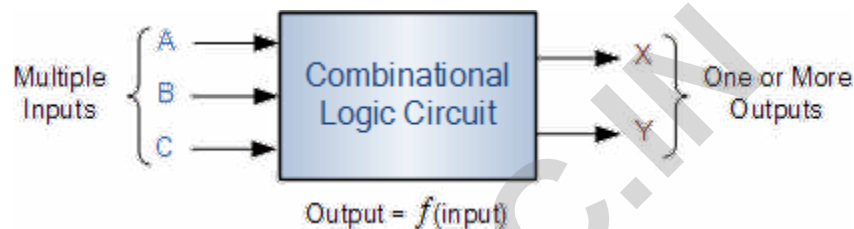


Fig.3.1: Combinational circuit

- Logic circuits whose outputs at any instant of time depend not only on the present inputs but also on past outputs are known as sequential circuits
- In sequential circuits the outputs are fed back to input side thus the output depends on the present inputs and past outputs.

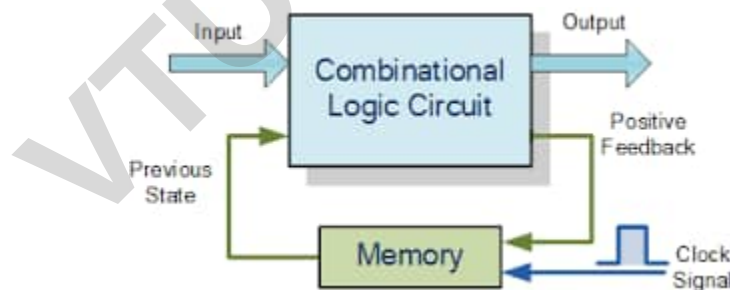


Fig.3.2: Sequential circuit

- A sequential circuit consists of a combinational circuit to which memory elements are added to form a feedback path as shown in fig.3.2.
- Memory elements are the devices which are capable to store binary information (1 or 0).
- Memory elements used in sequential circuit are flip-flops or latch.
- A **latch** is a device with exactly two stable states. These states are Q and $\bar{Q}$. These states Q and $\bar{Q}$ are always complement to each other.

- A latch has a feedback path, so information can be retained by the device. The latches can store one bit of data for as long as the device is powered.
- A **flip-flop** is a device very similar to a latch with exactly two stable states.
- The difference between a latch and a flip-flop is that a latch is asynchronous circuit, and flip-flop is a synchronous circuit.
- In latch the outputs can change as soon as the inputs are applied (level triggered). In flip-flop the outputs can change only when a control signal (clock) goes from high to low or low to high. i.e. latch does not have a clock signal, whereas a flip-flop always does.

Sequential Circuit:

A sequential circuit refers to a special type of circuit. It consists of a series of various inputs and outputs. Here, the outputs depend on a combination of both the present inputs as well as the previous outputs. This previous output gets treated in the form of the present state. Thus, the sequential circuit consists of the combinational circuit along with its memory storage elements. It doesn't need to consist of a combinational circuit all the time. A sequential circuit can basically contain only the memory element.

Types of Sequential Circuits

There are basically two types of sequential circuits:

1. Asynchronous sequential circuits
2. Synchronous sequential circuits

1. Asynchronous Sequential Circuits

The asynchronous sequential circuits don't make use of the clock signals. This type of circuit is operated through various pulses. Thus, the changes in our input can easily make a change in the state of our circuit. Clock pulses are not used in the case of asynchronous circuits. Here, the internal state gets altered whenever there is a change in the input variable. The time-delayed or the unclocked flip-flops form the memory elements of the asynchronous sequential circuits. This type of circuit is very similar to that of the combinational circuits along with the feedback.

2. Synchronous Sequential Circuits

The clock signal performs the synchronization of the state of memory elements in the case of synchronous sequential circuits. The output, in this case, is either stored in latches (memory devices) or flip-flops. The output gets synchronized with the clock's only positive edges or only the negative edges.

Clock Signal and Triggering

- The clock signal refers to a periodic signal where the ON and OFF times do not need to be the same. Thus, whenever the ON and OFF times of the clock signal happen to be the same, we use a square wave to represent the clock signal. Here is a diagram that represents a typical clock signal:
- Remember that a clock signal is often considered to be the square wave. The signal here sometimes stays at logic. It's either 5V (high) or 0V (low), to an equal share of time. It basically gets repeated with a certain time, which is equal to twice the 'OFF time' or 'ON time'. Let us now know more about triggering.

Types of Triggering

- A sequential circuit has two types of triggering:
 1. Level triggering
 2. Edge Triggering

1. Level Triggering

- There are two levels present in the clock signal – the logic Low and the logic High. The circuit is only activated in the case of a level triggering whenever the clock pulse happens to be at any particular level. Thus, there are the following types of level triggering in a sequential circuit:

1.1. Positive Level Triggering

- In the case of a positive level triggering, the signals with Logic High would occur. Thus, in this type of triggering, the circuit gets operated with such a type of clock signal. Here is a diagrammatic representation of positive level triggering:



1.2. Negative Level Triggering

In the case of a negative level triggering, a signal occurs with Logic Low. Thus, in this type of triggering, the available circuit gets operated with such a type of clock signal. Here is a diagrammatic representation of Negative level triggering:

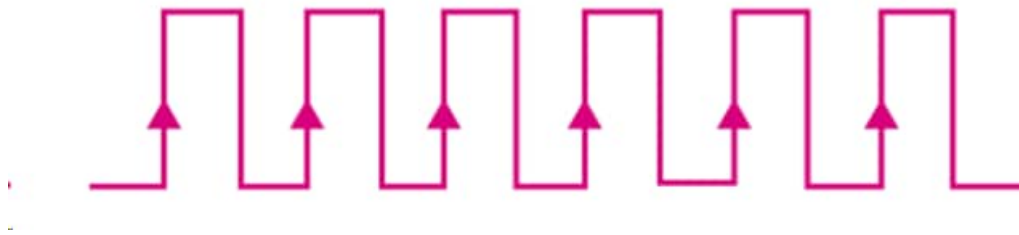


2. Edge Triggering

Two major types of transitions occur in the case of edge triggering clock signals. This transition either occurs from Logic High to Logic Low or from Logic Low to Logic High. Now, on the basis of the transitions of a given clock signal, the edge triggering can be of the following types:

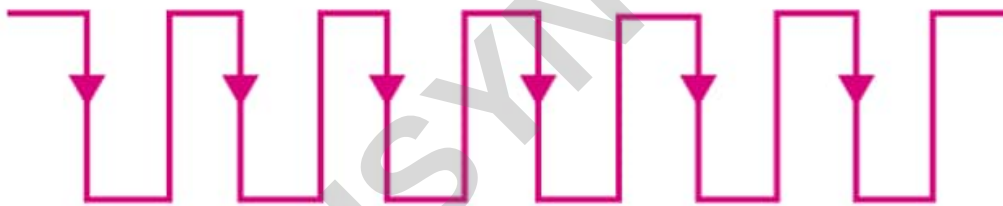
2.1. Positive Edge Triggering

In the clock signal of the positive edge triggering, a transition occurs from the Logic Low to the Logic High. Thus, the circuit of a positive edge triggering is operated using such a type of clock signal. Below is the diagram of the positive edge triggering:



2.2. Negative Edge Triggering

In the clock signal of the negative edge triggering, a transition occurs from the Logic High to the Logic Low. Thus, the circuit of a negative edge triggering is operated using such a type of clock signal. Below is the diagram of the negative edge triggering:



Flip-Flop:

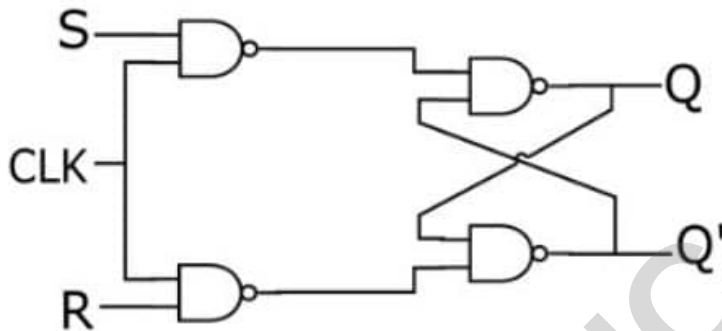
A **flip-flop** is a **sequential digital electronic circuit** having two stable states that can be used to store one bit of binary data. Flip-flops are the fundamental building blocks of all **memory devices**.

Types of Flip-Flops

- S-R flip-flop
- J-K flip-flop
- D flip-flop
- T flip-flop

S-R Flip-flop

- This is the simplest flip-flop circuit. It has a set input (S) and a reset input (R). When in this circuit when S is set as active, the output Q would be high and the Q' will be low. If R is set to active then the output Q is low and the Q' is high. Once the outputs are established, the results of the circuit are maintained until S or R get changed, or the power is turned off.



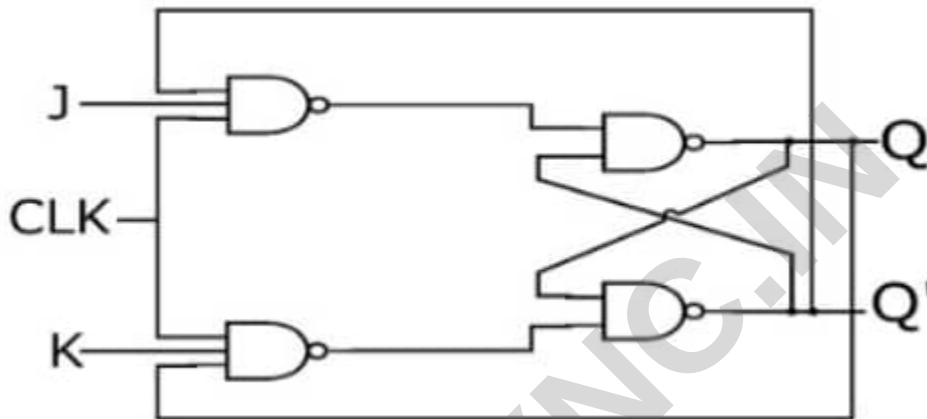
- Truth table of S-R flip-flop**

S	R	Q	State
0	0	0	No Change
0	1	0	Reset
1	0	1	Set
1	1	X	

J-K Flip-flop

- Because of the invalid state corresponding to $S=R=1$ in the SR flip-flop, there is a need of another flip-flop.
- A **JK flip flop** is a type of 1-bit memory element having inputs namely J and K, one clock input, and two output specified by Q and Q'. The JK flip flop is an improved version of SR flip flop which does not have forbidden state. To avoid the forbidden or indeterminate state, the outputs of the JK flip flop are fed back to its inputs.

- The JK flip-flop operates with only positive or negative clock transitions. The operation of the JK flip-flop is similar to the SR flip-flop. When the input J and K are different then the output Q takes the value of J at the next clock edge. When J and K both are low then NO change occurs at the output. If both J and K are high, then at the clock edge, the output will toggle from one state to the other.



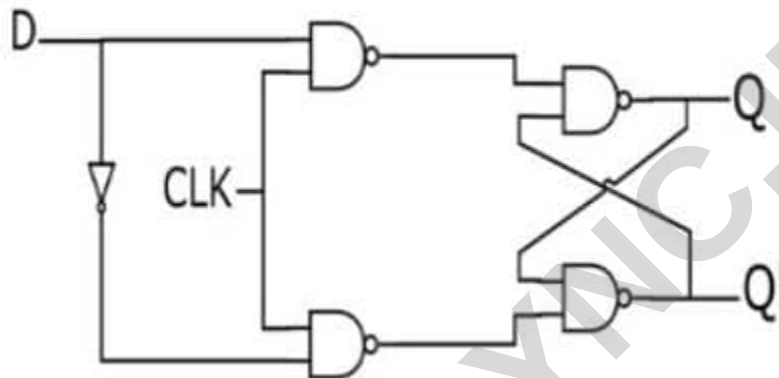
J	K	Q	State
0	0	0	No Change
0	1	0	Reset
1	0	1	Set
1	1	Toggles	Toggle

However, due to these feedback paths, a new problem is raised in the circuit, which is called race around condition. Race around condition in the JK flip is a major problem in which the outputs of flip flop are toggled continuously till the end of applied clock signal.

To avoid the problem of race around condition in JK flip flop, we use the JK flip flop in the **Master and Slave Mode**

D Flip-flop

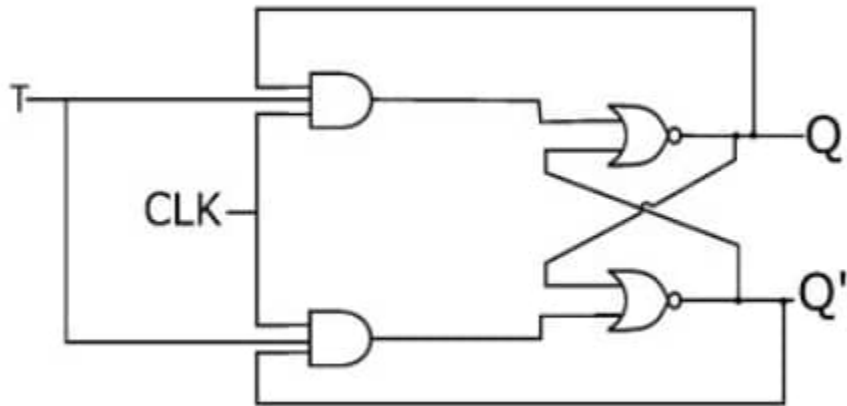
- In a D flip-flop, the output can only be changed at positive or negative clock transitions, and when the inputs changed at other times, the output will remain unaffected. The D flip-flops are generally used for shift-registers and counters. The change in output state of D flip-flop depends upon the active transition of clock. The output (Q) is same as input and changes only at active transition of clock



D	Q
0	0
1	1

T Flip-flop

- A T flip-flop (Toggle Flip-flop) is a simplified version of JK flip-flop. The T flop is obtained by connecting the J and K inputs together. The flip-flop has one input terminal and clock input. These flip-flops are said to be T flip-flops because of their ability to toggle the input state. Toggle flip-flops are mostly used in counters.



T	Q(t)	Q(t+1)
0	0	0
0	1	1
1	0	1
1	1	0

Applications of Flip-flops

- Counters
- Shift Registers
- Storage Registers, etc.

MASTER-SLAVE JK FLIPFLOP

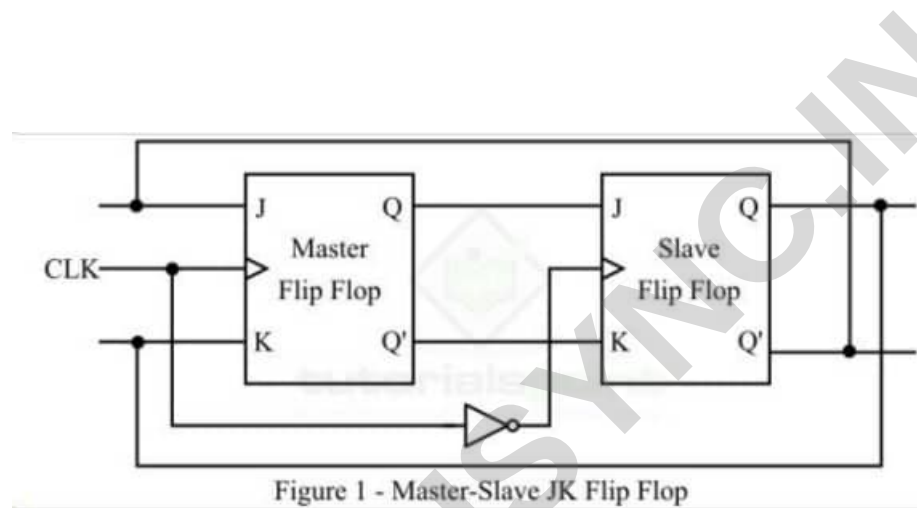
A **JK flip flop** is a type of 1-bit memory element having inputs namely J and K, one clock input, and two output specified by Q and Q'. The JK flip flop is an improved version of SR flip flop which does not have forbidden state. To avoid the forbidden or indeterminate state, the outputs of the JK flip flop are fed back to its inputs.

However, due to these feedback paths, a new problem is raised in the circuit, which is called race around condition. Race around condition in the JK flip is a major problem in which the outputs of flip flop are toggled continuously till the end of applied clock signal.

To avoid the problem of race around condition in JK flip flop, we use the JK flip flop in the **Master and Slave Mode**. Hence, the JK flip flop is called **Master-Slave Flip Flop**.

So, let us start with the basic construction of the master-slave JK flip flop.

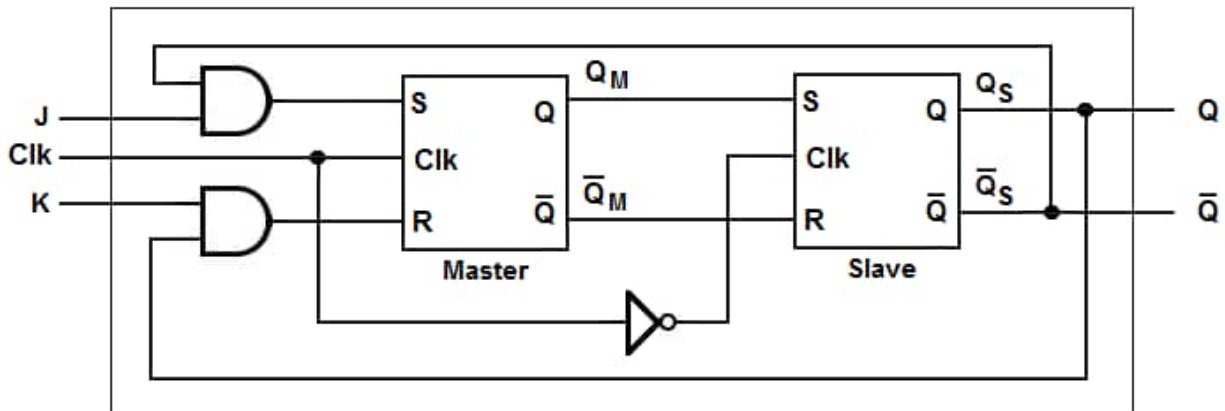
Master Slave JK Flip Flop is a combination of two JK flip flops which are connected in the cascaded manner as shown in Figure-1.



In this combination of two JK flip flop, one acts as a **master flip flop** and the other acts as a **slave flip flop**. In this master-slave flip flop, the outputs of the master JK flip flop are connected to the inputs of the slave JK flip flop. The outputs of the slave flip flop are fed back to the inputs of the master JK flip flop.

In the master-slave JK flip flop, a NOT gate (Inverter) is also used which is connected to clock signal in a manner that the inverted clock signal is applied to the slave flip flop.

Therefore, when clock signal to master flip flop is 0, then for slave flip flop the clock signal is 1, and if the clock signal to master flip flop is 1, then for the slave flip flop it 0.



Operation of Master-Slave JK Flip Flop

When the clock pulse goes to high, the slave flip flop becomes inactive and the inputs J and K can control the state of the system.

When the clock pulse goes back to low, the information is transferred from master flip flop to the slave flip flop, and the final output of the system is obtained.

From the circuit, it is clear that the master flip flop is positive level triggered and the slave flip flop is negative level triggered. Consequently, the master flip flop responds before the slave flip flop.

When $J = 0$ and $K = 0$, both JK flip flops remains inactive and hence the output Q remains unchanged. This is called Hold State of the master-slave JK flip flop.

- When $J = 0$ and $K = 1$, the output Q' of the master flip flop is high and goes to the input K of the slave flip flop. The clock signal forces the slave flip flop to reset. Therefore, the slave flip flop has the same output as the master flip flop, i.e., high Q' and low Q. This is called reset state of the master-slave JK flip flop.
- When $J = 1$ and $K = 0$, the output Q of the master flip flop is high and goes to the input J of the slave flip flop, the negative transition of the clock signal sets the slave flip flop. Hence, this is called the set state of the master-slave JK flip flop.

- When $J = 1$ and $K = 1$, for this input combination, the master flip flop toggles on the positive transition of the clock pulse and the slave flip flop toggles on the negative transition of the clock pulse. Hence, the problem of the race around condition of the JK flip flop is solved.

Inputs		Output	Comment
J	K	Q_{n+1}	
0	0	Q_n	No Change
0	1	0	Reset
1	0	1	Set
1	1	Q_n'	Toggle

Timing Diagram of Master Slave JK Flipflop:

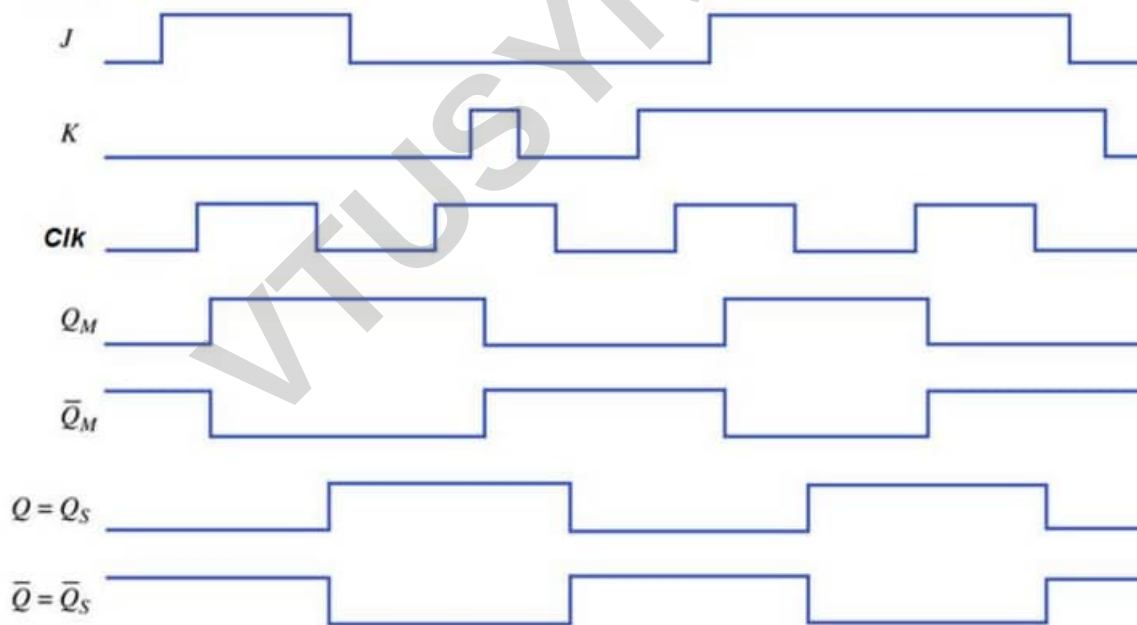


Fig.3.8: Timing diagram for a master slave JK flip flop

Characteristic Table and Equations

- The characteristic table defines the next state output of flip-flop as a function of its inputs and present state output. Q_n refers to the present state and Q_{n+1} refers to the next state after the occurrence of the clock pulse.
- The characteristic table for the SR flip-flop shows that the next state is equal to the present state when both inputs S and R are equal to 0. When $R = 1$, the next clock pulse clears the flip-flop output Q to 0. When $S = 1$, the next clock pulse sets the flip-flop output Q to 1. When S and R are equal to 1, the next state output of SR flipflop is indeterminate.
- The characteristic equation is the functional Boolean equation that is derived from the characteristic table.

Characteristics table of SR Flipflop:

Inputs		Outputs	
		Present State	Next State
S	R	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	—
1	1	1	—

RQ		00	01	11	10	
S	0		1			$\bar{R}Q$
	1	1	1	-	-	S

Characteristics Equation:

$$Q_{n+1} = S + \bar{R}Q_n$$

Characteristics table of JK Flipflop:

Inputs		Outputs	
		Present State	Next State
J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

KQ		00	01	11	10	
J	0		1			$\bar{K}Q$
	1	1	1		1	$J\bar{Q}$

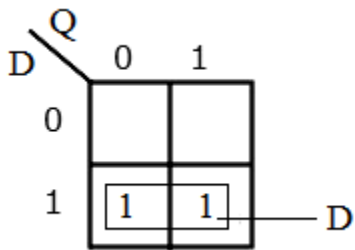
Characteristics Equation:

$$Q_{n+1} = J\bar{Q}_n + \bar{K}Q_n$$

Characteristics table of D Flipflop:

PRATHEKSHA RAI N, ASST. PROF,ECE

Inputs	Outputs	
	Present State	Next State
D	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

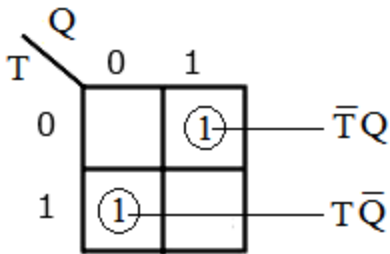


Characteristics Equation:

$$Q_{n+1} = D$$

Characteristics table of T Flipflop:

Inputs	Outputs	
	Present State	Next State
T	Q_n	Q_{n+1}
0	0	0
0	1	1
1	0	1
1	1	0



Characteristics Equation:

$$Q_{n+1} = \bar{T}Q_n + T\bar{Q}_n = T \oplus Q_n$$

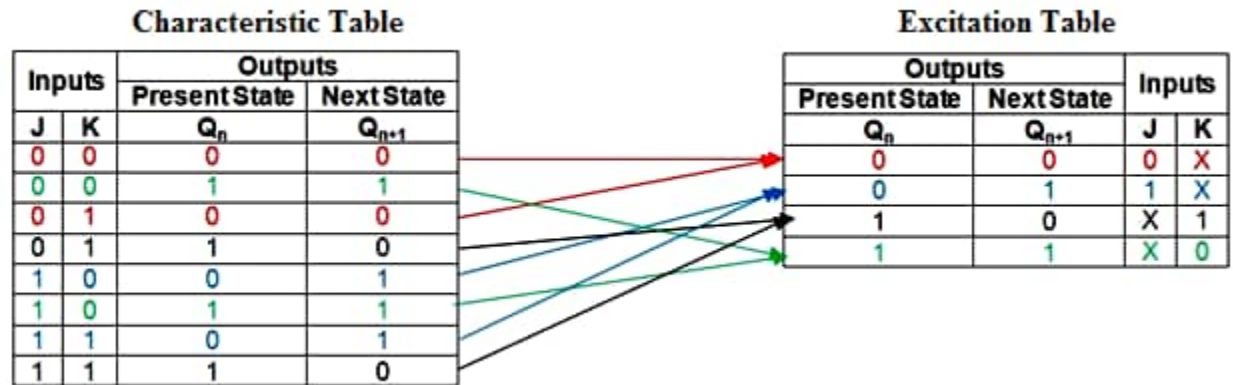
Excitation Table

- The excitation table gives the value of the flip-flop's inputs that are necessary to change the flip-flop's current state to the desired next state at the next active edge of the clock signal.
- The excitation table is derived from the characteristic table and is used in the synthesis of sequential circuits.

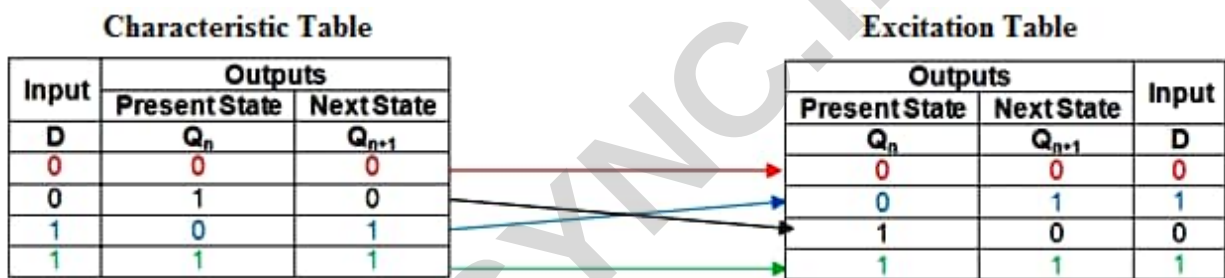
Excitation table of SR Flipflop:

Characteristic Table				Excitation Table			
Inputs		Outputs		Outputs		Inputs	
S	R	Present State	Next State	Present State	Next State	S	R
0	0	0	0	0	0	0	X
0	0	1	1	0	1	1	0
0	1	0	0	1	0	0	1
0	1	1	0	1	1	X	0
1	0	0	1				
1	0	1	1				
1	1	invalid					
1	1	invalid					

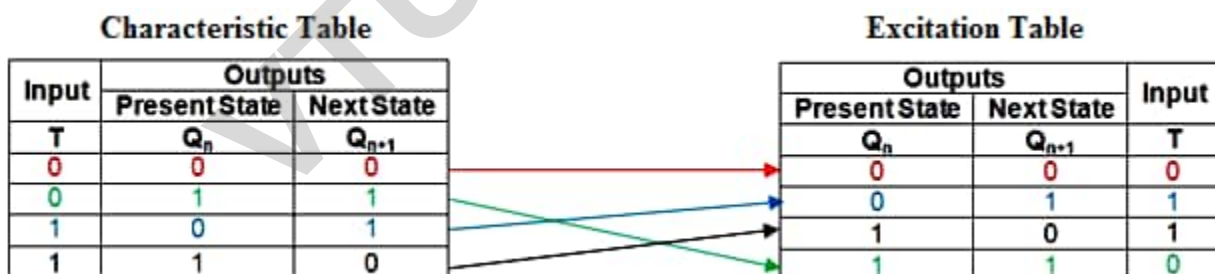
Excitation table of JK Flipflop:



Excitation table of D Flipflop:



Excitation table of T Flipflop:



Registers

Shift Register

- Shift Register is a group of flip flops used to store multiple bits of data. The bits stored in such registers can be made to move within the registers and in/out of the registers by applying clock pulses.
- An n-bit shift register can be formed by connecting n flip-flops where each flip flop stores

a single bit of data.

- The registers are connected in such a way that the output of one of the flip flop forms in input to other.
- The data in a shift register is moved serially (one bit at a time).
- There are two ways to shift data into a register (serial or parallel) and similarly two ways to shift the data out of the register (serial or parallel).
- Depending on the type of shift, four basic types of registers can be constructed
 1. Serial in/Serial out (SISO)
 2. Serial in/Parallel out (SIPO)
 3. Parallel in/Serial out (PISO)
 4. Parallel in/Parallel out (PIPO)
- i. **Serial In - Serial Out (SISO):** The data is shifted in and out of the register serially, one bit at a time in either a left or right direction under clock control.
- ii. **Serial in - Parallel Out (SIPO):** The register is loaded with serial data, one bit at a time, with the stored data being available at the output in parallel form.
- iii. **Parallel In - Serial Out (PISO):** The parallel data is loaded into the register simultaneously and is shifted out of the register serially one bit at a time under clock control.
- iv. **Parallel In - Parallel Out (PIPO):** The parallel data is loaded simultaneously into the register, and transferred together to their respective outputs at the same clock pulse.

Serial In - Serial Out shift register

- Serial In – Serial Out (SISO) shift register allows serial input and produces serial output. as shown in the fig.3.13.

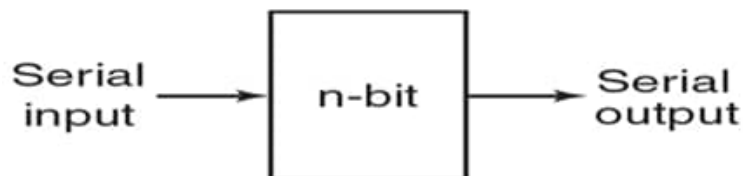


Fig.3.13: Block diagram of SISO Shift register

- Fig.3.14 shows a 4-bit Serial In – Serial Out (SISO) shift register which consists of four D flip-flops connected in cascaded form i.e., output of one D flip-flop is connected as the

input of next D flip-flop. All these flip-flops are synchronous with each other as same clock signal is applied to each one.

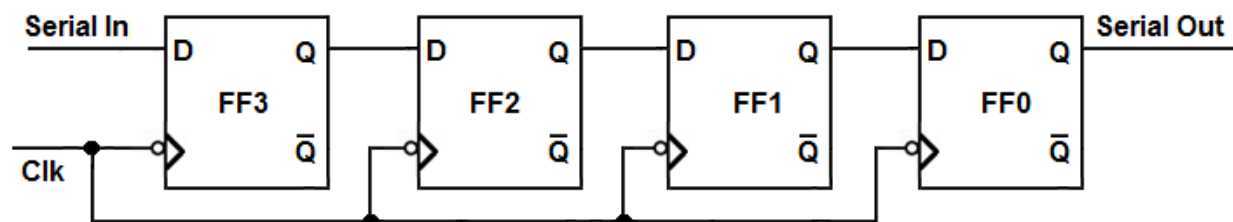


Fig.3.14: 4-bit SISO Shift register

- In this shift register, bits can be sent serially from the input of left most D flip-flop. Hence, this input is also called as serial input. For every negative edge triggering of clock signal, the data shifts from one stage to the next. So, the bits are received serially from the output of right most D flip-flop. Hence, this output is also called as serial output.
- Consider the serial input 1010 for the SISO shift register. The input is applied to shift register by taking LSB first and lastly the MSB.
- On the first clock pulse LSB (logic 0) is applied as serial input to the serial input pin of FF3 therefore Q_3 will be set (logic 0) and all the other outputs still remains at logic 0.
- On the second clock pulse the next bit (logic 1) is applied as serial input to the serial input pin of FF3 therefore Q_3 will be set (logic 1). The previous output of Q_3 is connected to FF2 therefore Q_2 will be set (logic 0) and all the other outputs still remains at logic 0.
- On the third clock pulse the next bit (logic 0) is applied as serial input to the serial input pin of FF3 therefore Q_3 will be reset (logic 0). The previous output of Q_3 is connected to FF2 therefore Q_2 will be set (logic 1). The previous output of Q_2 is connected to FF1 therefore Q_1 will be set (logic 0) and all the other outputs still remains at logic 0.
- On the fourth clock pulse the next bit (logic 1) is applied as serial input to the serial input pin of FF3 therefore Q_3 will be set (logic 1). The previous output of Q_3 is connected to FF2 therefore Q_2 will be reset (logic 0). The previous output of Q_2 is connected to FF1 therefore Q_1 will be set (logic 1) and the previous output of Q_1 is connected to FF0 therefore Q_0 will be set (logic 0).

Clk Pulse	Serial in	Q_3	Q_2	Q_1	Q_0 (Serial Out)
1	0	0	—	—	—

2	1	1	0	–	–
3	0	0	1	0	–
4	1	1	0	1	0
5	–	–	1	0	1
6	–	–	–	1	0
7	–	–	–	–	1

- Therefore, the 4-bit SISO shift register requires seven clock pulses in order to produce the valid output. Similarly, the N-bit SISO shift register requires $2N-1$ clock pulses in order to shift 'N' bit information.

Serial in - Parallel Out shift register

- Serial In – Parallel Out (SIPO) shift register allows serial input and produces parallel output as shown in the fig.3.15.

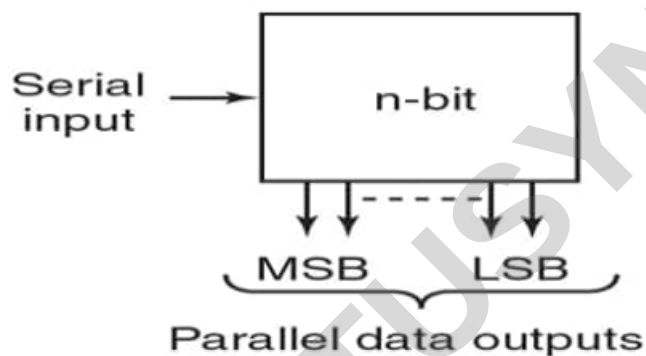


Fig.3.15: Block diagram of SIPO Shift register

- Fig.3.16 shows a 4-bit Serial In – Parallel Out (SIPO) shift register which consists of four D flip-flops connected in cascaded form i.e., output of one D flip-flop is connected as the input of next D flip-flop. All these flip-flops are synchronous with each other as same clock signal is applied to each one.

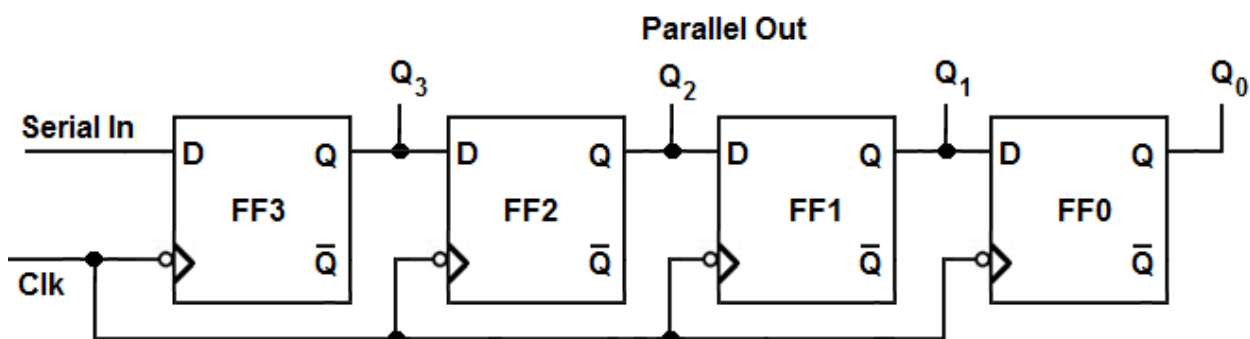


Fig.3.16: 4-bit SIPO Shift register

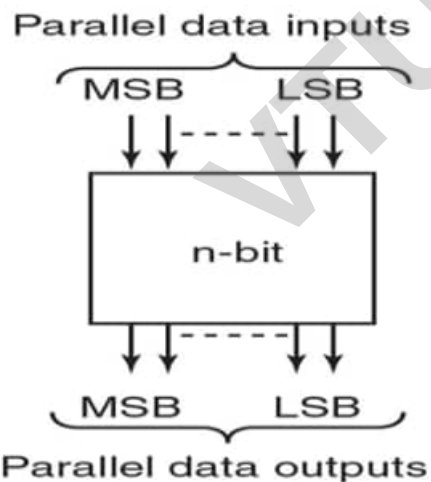
- In this shift register, bits can be sent serially from the input of left most D flip-flop. Hence, this input is also called as serial input. For every negative edge triggering of clock signal, the data shifts from one stage to the next. In this case, the outputs of each D flip-flop can be accessed in parallel. So, parallel outputs are obtained from this shift register.

Clk Pulse	Serial In	Q ₃	Q ₂	Q ₁	Q ₀
1	0	0	—	—	—
2	1	1	0	—	—
3	0	0	1	0	—
4	1	1	0	1	0

- The 4-bit SIPO shift register requires four clock pulses in order to produce the valid output. Similarly, the **N-bit SIPO shift register** requires N clock pulses in order to shift 'N' bit information.

Parallel In – Parallel Out shift register

- Parallel In – Parallel Out (**PISO**) shift register allows parallel input and produces parallel output as shown in fig.3.17.

**Fig.3.17: Block diagram of PIPO Shift register**

- Fig.3.18 shows a 4-bit Parallel In – Parallel Out (PIPO) shift register which consists of four D flip-flops.

Clk Pulse	Parallel Data inputs	Parallel Data Outputs
-----------	----------------------	-----------------------

	D ₃	D ₂	D ₁	D ₀	Q ₃	Q ₂	Q ₁	Q ₀
1	1	0	1	1	1	0	1	1

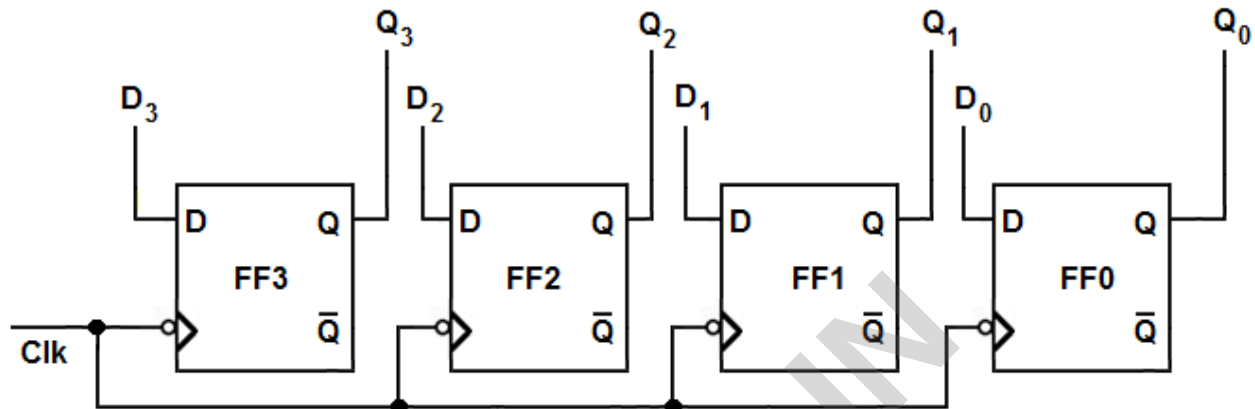


Fig.3.18: 4-bit PIPO Shift register

Parallel In – Serial Out shift register

- Parallel In – Serial Out (**PISO**) shift register allows parallel input and produces serial output as shown in fig.3.19.

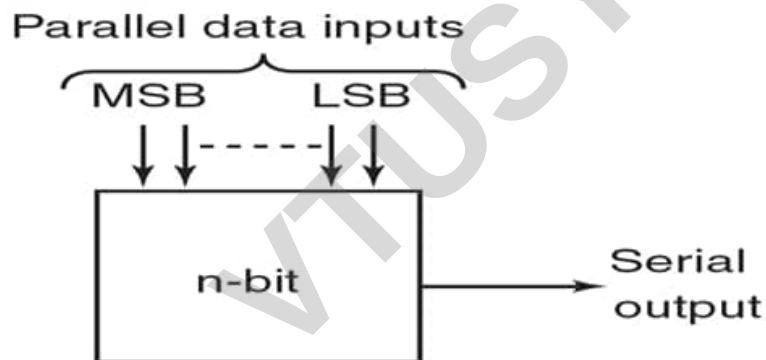


Fig.3.19: Block diagram of PIPO Shift register

- Fig.3.20 shows a 4-bit Parallel In – Serial Out (SIPO) shift register which consists of four D flip-flops and an AND-OR logic to determine whether data will load in parallel, or stored data will shift to right.

Clk Pulse	Shift / $\overline{\text{Load}}$	Parallel Data Inputs				Output
		D ₃	D ₂	D ₁	D ₀	Serial Out
1	0	1	0	1	0	0
2	1	–	1	0	1	1
3	1	–	–	1	0	0
4	1	–	–	–	1	1

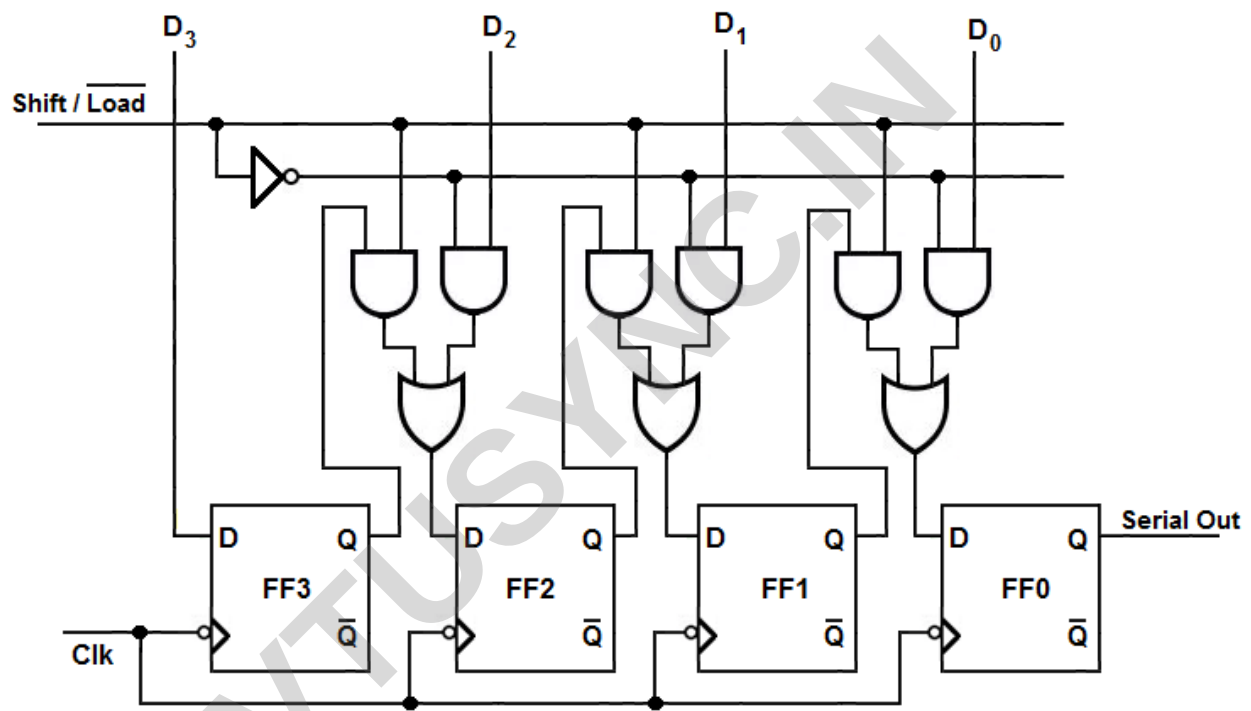
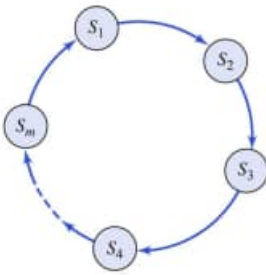


Fig.3.20: PISO Shift register

Binary Ripple Counters

A counter is an example of a register. Its primary function is to produce a specified output pattern sequence. This sequence might correspond to the number of occurrences of an event or it might be used to control various parts of a digital system. The counting sequence is often depicted by a graph called a state diagram. A modulus-m counter (i.e., a counter with m states) has the following state diagram: 3.21. Counters Each node S_i denotes the states of the counter and the arrows in the graph denote the order in which the states occur. Counters are available in two categories: ripple counters and synchronous counter.

**Fig.3.21: state diagram:**

Each node S_i denotes the states of the counter and the arrows in the graph denote the order in which the states occur. Counters are available in two categories: ripple counters and synchronous counters.

Binary ripple counter

In a ripple counter, the flip-flop output transition serves as a source for triggering other flip-flops. A 4-bit binary ripple counter (mod-16) is as follows:

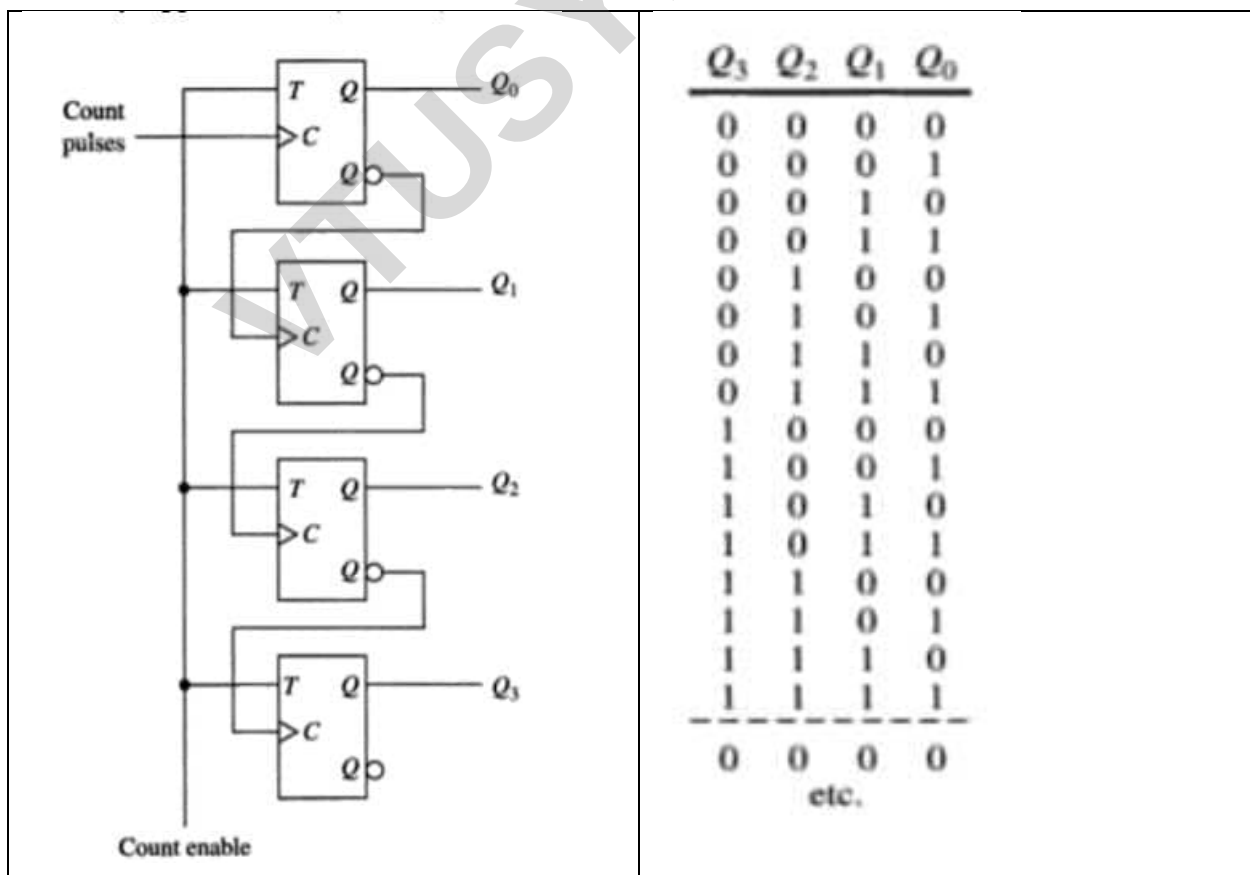


Fig.3.22. a) :4 bit ripple counter	Fig.3.22. b): counting sequence
------------------------------------	---------------------------------

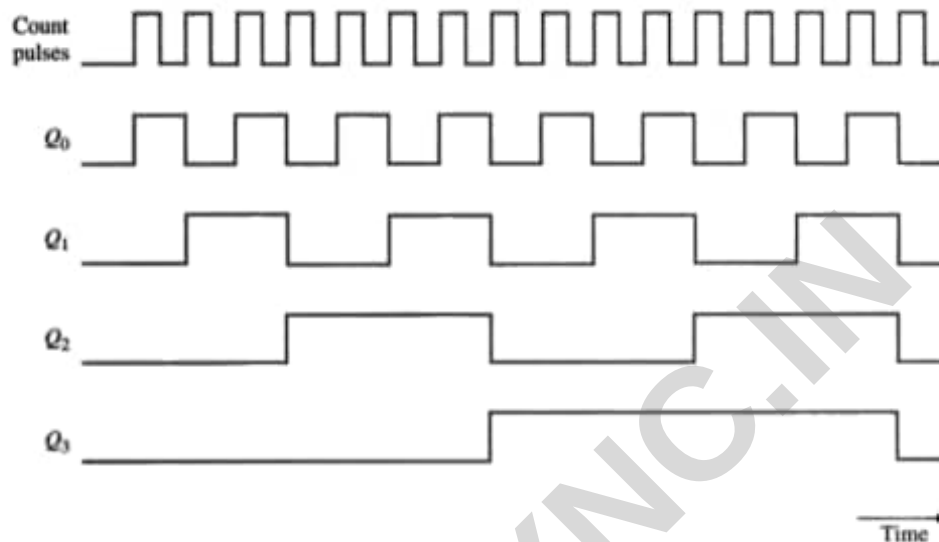


Fig.3.23: timing diagram

Synchronous Binary Counters

- For synchronous counter the count pulse are applied directly to control input All T inputs are connected to Logic 1 as shown in the fig.3.24. Therefore, each flip flop will toggle with negative transition at its clock input. As long as counter is enabled counter follows binary counting sequence.
- Lowest order of FF, Q_0 toggles on the positive edge of the each count pulse ,
- The AND gate preceding each T input terminal of the remaining FF detects if all the lower order FF in logic 1- states. If this condition is satisfied , then the FF toggles upon the occurrence of positive edge of clock pulse.

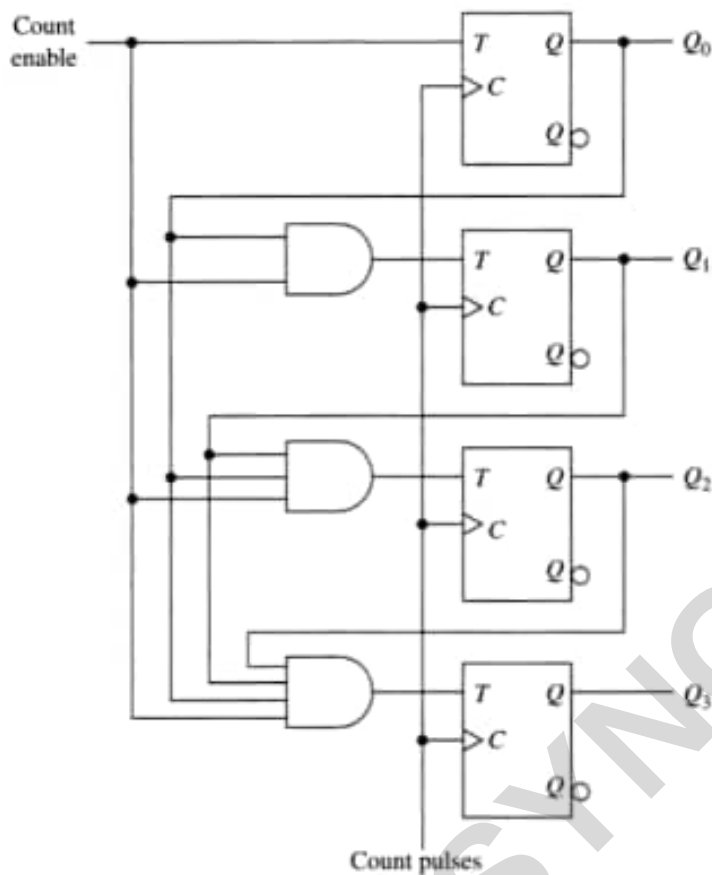


Fig.3.24: 4 bit synchronous binary counter

Counters based on Shift Registers

Ring Counter

- An 'N' bit Ring counter performs the Serial In - Parallel Out shift operation but, the only difference is that the output of rightmost D flip-flop is given as input to leftmost D flip-flop instead of applying data from outside. Therefore, Ring counter produces a sequence of states (pattern of zeros and ones) and it repeats for every 'N' clock cycles.
- The block diagram of 4-bit Ring counter is shown in the fig.4.10.

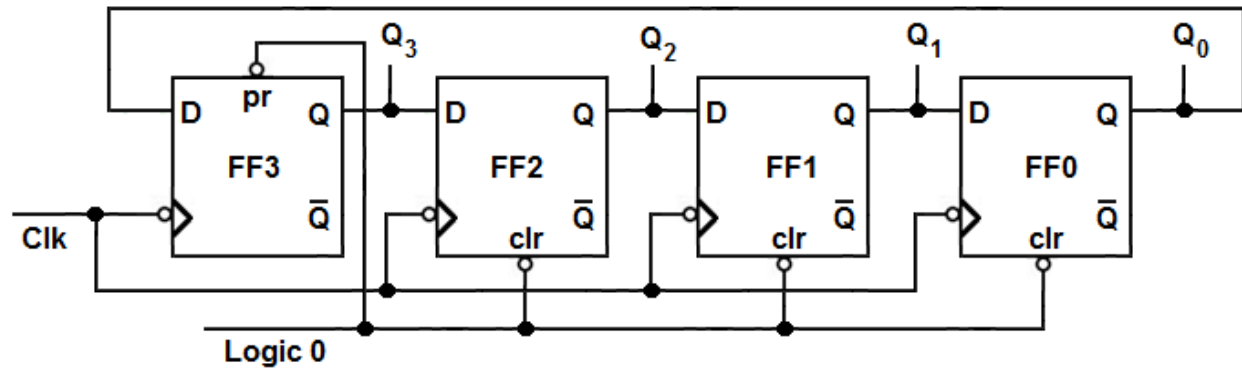


Fig.4.10: 4-bit Ring counter

- Using preset and clear input initial value of the output is defined. The first flipflop output is set to logic 1 and all other flipflop output is set to logic 0.
- The initial status of the D flip-flops in the absence of clock signal is 1000. This status repeats for every four negative edge transitions of clock signal.

Clk Pulse	Q ₃	Q ₂	Q ₁	Q ₀
Initial Value	1	0	0	0
1	0	1	0	0
2	0	0	1	0
3	0	0	0	1
4	1	0	0	0

Johnson Counter

- The operation of Johnson counter is similar to that of Ring counter but, the only difference is that the complemented output of rightmost D flip-flop is given as input to leftmost D flip-flop instead of normal output. Therefore, 'N' bit Johnson Ring counter produces a sequence of states (pattern of zeros and ones) and it repeats for every '2N' clock cycles.
- Johnson counter is also called as Twisted Ring counter and switch tail Ring counter.
- The block diagram of 3-bit Johnson Ring counter is shown in the fig.4.11.
- Using preset and clear input initial value of the output is defined. The first flipflop output is set to logic 1 and all other flipflop output is set to logic 0.
- The initial status of the D flip-flops in the absence of clock signal is 1000. This status repeats for every eight negative edge transitions of clock signal.

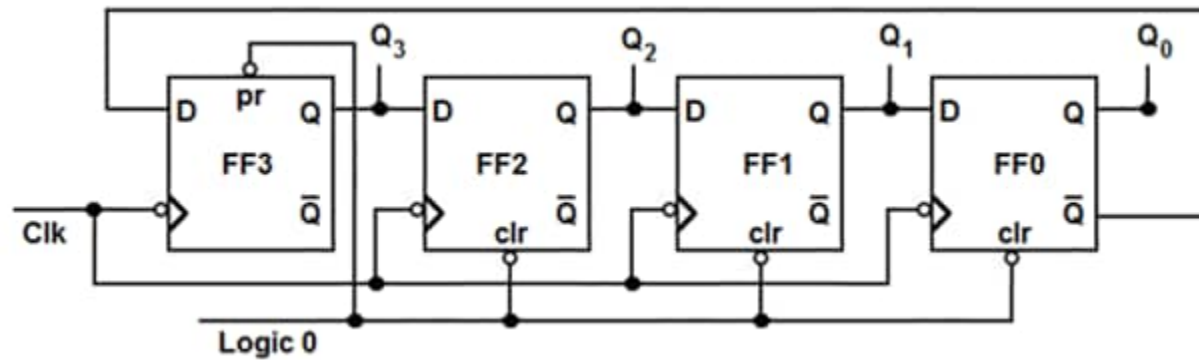


Fig.4.11: 4-bit Johnson counter

Clk Pulse	Q ₃	Q ₂	Q ₁	Q ₀
Initial Value	1	0	0	0
1	1	1	0	0
2	1	1	1	0
3	1	1	1	1
4	0	1	1	1
5	0	0	1	1
6	0	0	0	1
7	0	0	0	0
8	1	0	0	0

Universal Shift Register

- A Universal shift register is a register which has both the right shift and left shift with parallel load capabilities.
- Universal shift registers are used as memory elements in computers. A Unidirectional shift register is capable of shifting in only one direction. A bidirectional shift register is capable of shifting in both the directions.
- The Universal shift register is a combination design of bidirectional shift register and a unidirectional shift register with parallel load provision.
- A 4-bit universal shift register shown in the fig.4.12 consists of four flip-flops and four 4×1 multiplexers. All the 4 multiplexers share the same select lines (S₁ and S₀) to select the mode in which the shift register operates. The select inputs select the suitable input for the flip-flops.

- The working of the Universal shift register depends on the inputs given to the select lines.

S ₁	S ₀	Register Operation
0	0	No changes
0	1	Shift right
1	0	Shift left
1	1	Parallel load

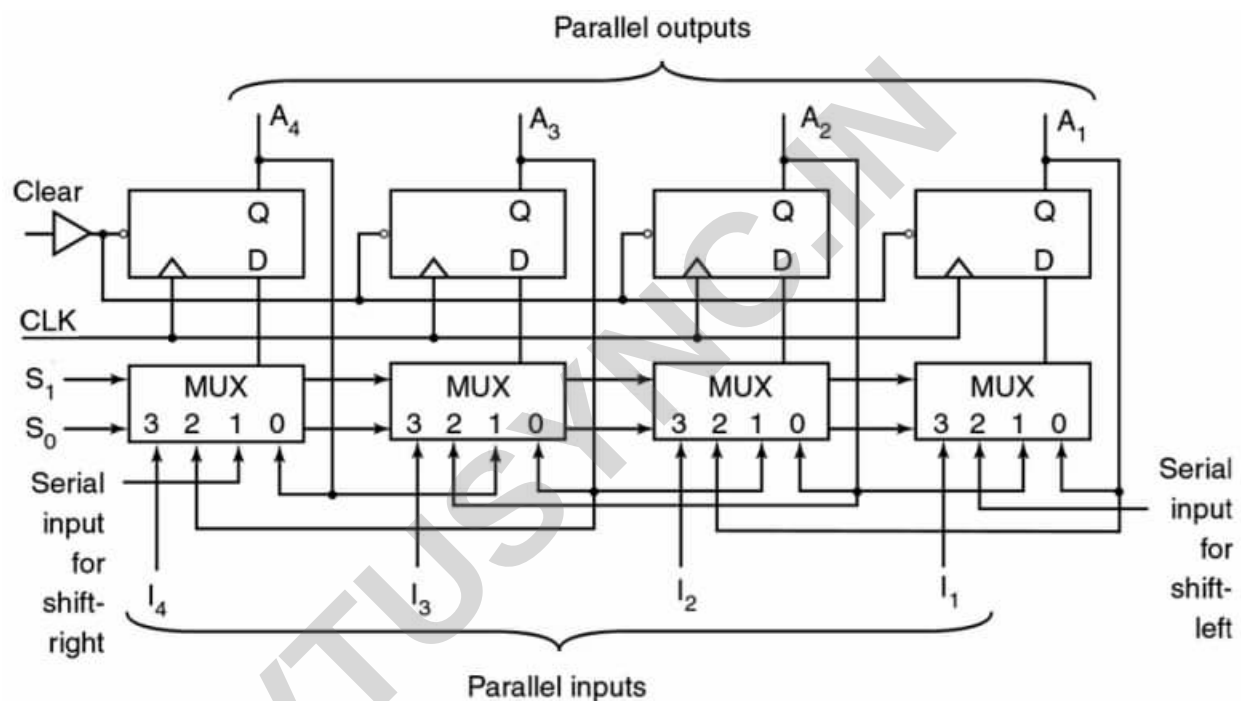


Fig.4.12: 4-bit universal shift register

Basic connections of universal shift register

1. The first input (0th pin of multiplexer) is connected to the output pin of the corresponding flip-flop.
2. The second input (1st pin of multiplexer) is connected to the output of the very-previous flip flop which facilitates the right shift.
3. The third input (2nd pin of multiplexer) is connected to the output of the very-next flip-flop which facilitates the left shift.
4. The fourth input (3rd pin of multiplexer) is connected to the individual bits of the input data which facilitates parallel loading.

Counters

- Counting is frequently required in digital computers and other digital systems to record the number of events occurring in a specified interval of time.
- Normally an electronic counter is used for counting the number of pulses coming at the input line in a specified time period. The counter must possess memory since it has to remember its past states. Counters can be synchronous or asynchronous.
- In synchronous counter all the flip-flops are driven simultaneously by a common clock pulse.
- In an asynchronous counter (also known as ripple counters), the first flip-flop is driven by a clock pulse and each successive flip-flop is driven by the output of the preceding flip-flop in the sequence.

Design of Synchronous Counters

Design steps for Synchronous MOD – N counter

- i. Determine the number of states
- ii. Determine the number of flipflops required
- iii. Choose the type of flipflop
- iv. Using excitation table of selected flipflop, determine the excitation table of counter. Excitation table of counter includes present state output, next state output and flipflop inputs.
- v. Use K-map simplification method to derive flipflop input expression
- vi. Draw the logic diagram

Design of Synchronous mod-n Counter using clocked T, JK, D and SR flip-flops.

Design of synchronous mod-6 up counter using JK-Flipflop

Number of states: 6 [0 – 1 – 2 – 3 – 4 – 5]

Number of flipflops required: 03

Type of flipflop: JK

Excitation table of JK flipflop

Q_n	Q_{n+1}	J	K
-------	-----------	---	---

0	0	0	–
0	1	1	–
1	0	–	1
1	1	–	0

Excitation table for the synchronous mod-6 counter:

Present State			Next State			Flip Flop Inputs					
Q_C	Q_B	Q_A	Q_C	Q_B	Q_A	J_C	K_C	J_B	K_B	J_A	K_A
0	0	0	0	0	1	0	–	0	–	1	–
0	0	1	0	1	0	0	–	1	–	–	1
0	1	0	0	1	1	0	–	–	0	1	–
0	1	1	1	0	0	1	–	–	1	–	1
1	0	0	1	0	1	–	0	0	–	1	–
1	0	1	0	0	0	–	1	0	–	–	1

Boolean expression for flipflop inputs using k-map simplification:

Boolean expression for inputs of flip flop C

Q_C	$Q_B Q_A$			
	00	01	11	10
0	0	0	1	0
1	–	–	–	–

Q_C	$Q_B Q_A$			
	00	01	11	10
0	–	–	–	–
1	0	1	–	–

$$J_C = Q_B Q_A$$

$$K_C = Q_A$$

Boolean expression for inputs of flip flop B

Q_C	$Q_B Q_A$			
	00	01	11	10
0	0	1	-	-
1	0	0	-	-

Q_C	$Q_B Q_A$			
	00	01	11	10
0	-	-	1	0
1	-	-	-	-

$$J_B = \bar{Q}_C Q_A$$

$$K_B = Q_A$$

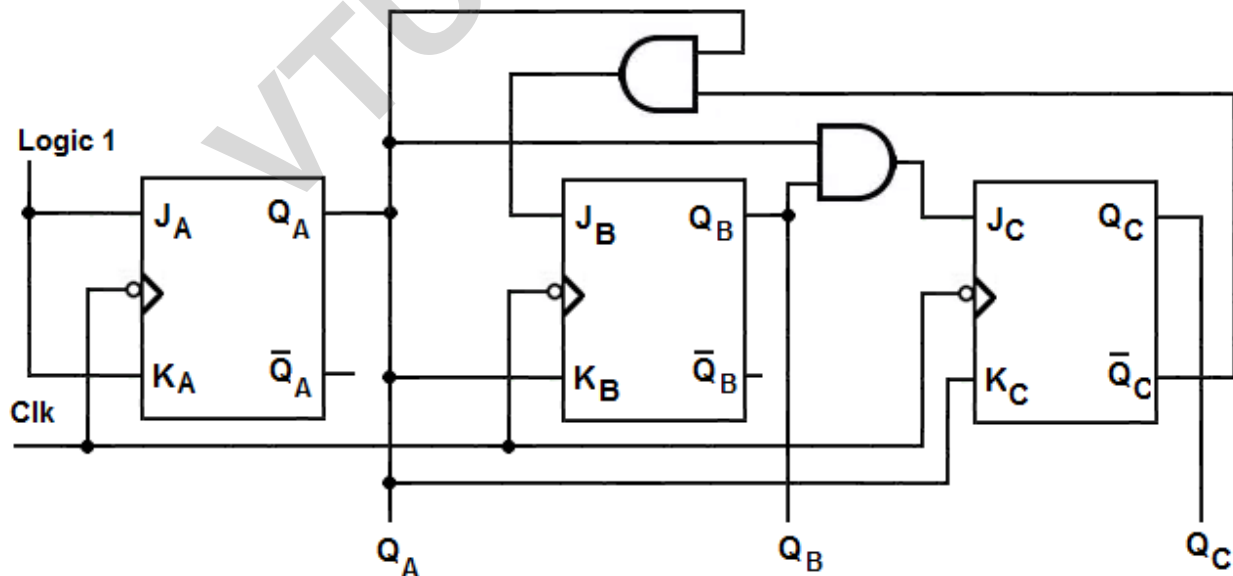
Boolean expression for inputs of flip flop A

Q_C	$Q_B Q_A$			
	00	01	11	10
0	1	-	-	1
1	1	-	-	-

Q_C	$Q_B Q_A$			
	00	01	11	10
0	-	1	1	-
1	-	1	-	-

$$J_A = K_A = 1$$

Logic diagram:



Design of synchronous mod-6 counter using D-Flipflop

Number of states: 6 [0 – 1 – 2 – 3 – 4 – 5]

Number of flipflops required: 03

Type of flipflop: D

Excitation table of D flipflop

Q_n	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

Excitation table for the synchronous mod-6 counter:

Present State			Next State			Flip Flop Inputs		
Q_C	Q_B	Q_A	Q_C	Q_B	Q_A	D_C	D_B	D_A
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	0
0	1	0	0	1	1	0	1	1
0	1	1	1	0	0	1	0	0
1	0	0	1	0	1	1	0	1
1	0	1	0	0	0	0	0	0

Boolean expression for flipflop inputs using k-map simplification:

Boolean expression for inputs of flip flop C

Q_C	$Q_B Q_A$			
	00	01	11	10
0	0	0	1	0
1	1	0	–	–

$$D_C = Q_C \bar{Q}_A + Q_B Q_A$$

Boolean expression for inputs of flip flop B

Q_C	$Q_B Q_A$			
	00	01	11	10
0	0	1	0	1
1	0	0	-	-

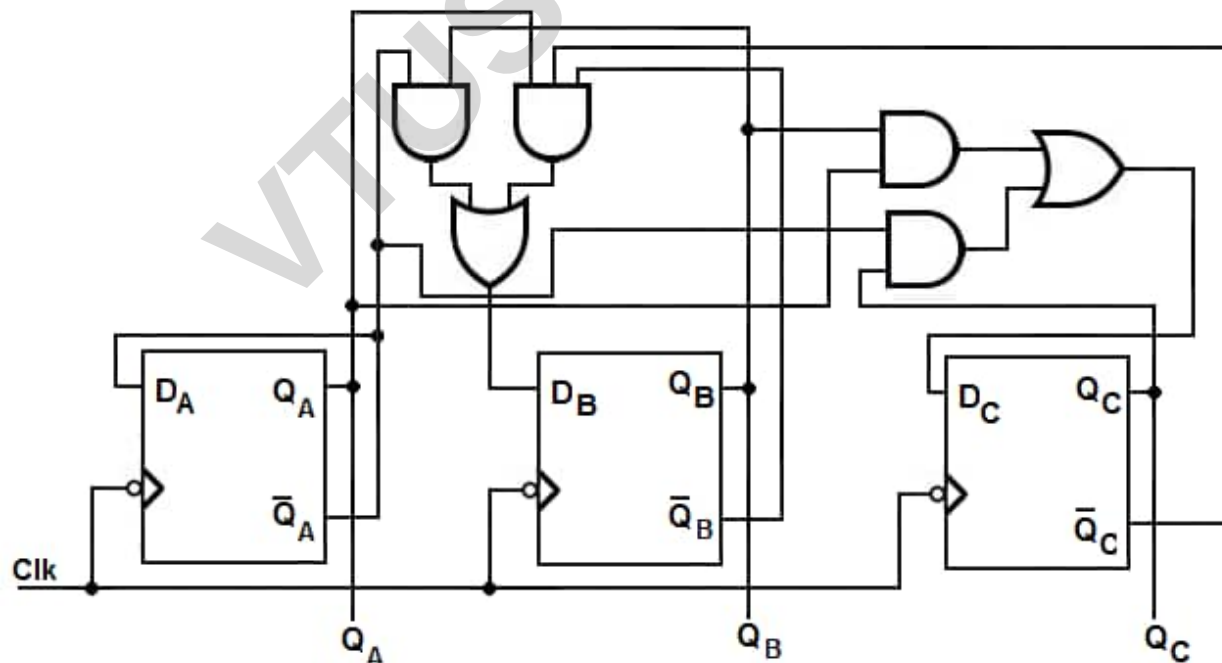
$$D_B = \bar{Q}_C \bar{Q}_B Q_A + Q_B \bar{Q}_A$$

Boolean expression for inputs of flip flop A

Q_C	$Q_B Q_A$			
	00	01	11	10
0	1	0	0	1
1	1	0	-	-

$$D_A = \bar{Q}_A$$

Logic diagram:



Design of synchronous mod-6 counter using T-Flipflop

Number of states: 6 [0 – 1 – 2 – 3 – 4 – 5]

Number of flipflops required: 03

Type of flipflop: T

Excitation table of T flipflop

Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

Excitation table for the synchronous mod-6 counter:

Present State			Next State			Flip Flop Inputs		
Q_C	Q_B	Q_A	Q_C	Q_B	Q_A	T_C	T_B	T_A
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	1
0	1	0	0	1	1	0	0	1
0	1	1	1	0	0	1	1	1
1	0	0	1	0	1	0	0	1
1	0	1	0	0	0	1	0	1

Boolean expression for flipflop inputs using k-map simplification:

Boolean expression for inputs of flip flop C

		$Q_B Q_A$			
		00	01	11	10
Q_C	0	0	0	1	0
	1	0	1	–	–

$$T_C = Q_B Q_A + Q_C Q_A$$

$$T_C = Q_A (Q_B + Q_C)$$

Boolean expression for inputs of flip flop B

Q_C	$Q_B Q_A$			
	00	01	11	10
0	0	1	1	0
1	0	0	-	-

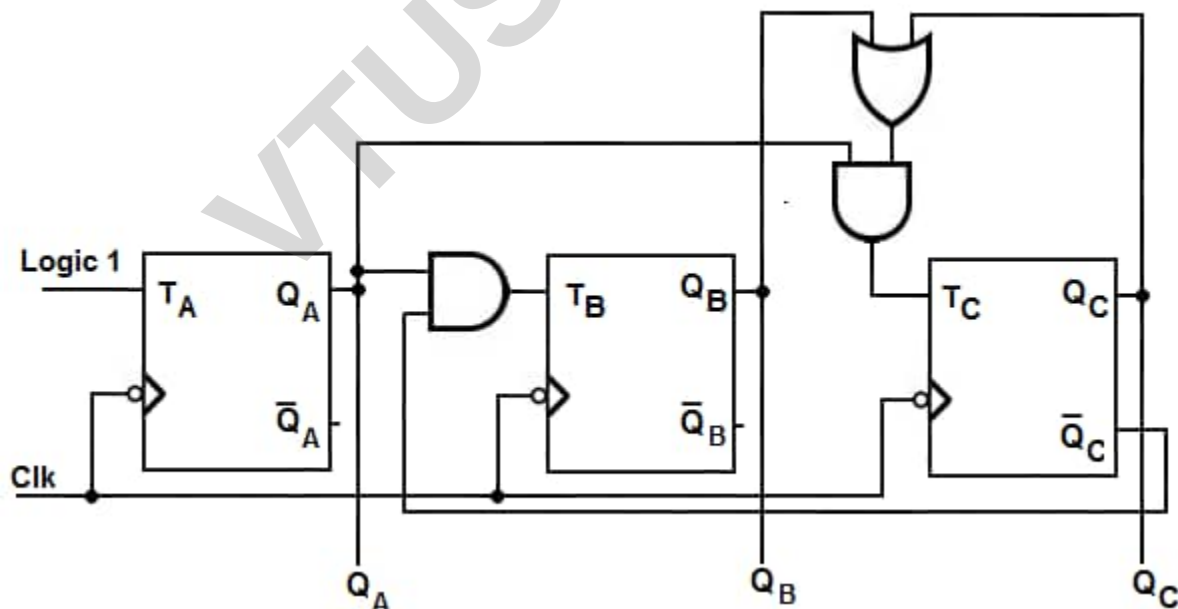
$$T_B = \bar{Q}_C Q_A$$

Boolean expression for inputs of flip flop A

Q_C	$Q_B Q_A$			
	00	01	11	10
0	1	1	1	1
1	1	1	-	-

$$T_A = 1$$

Logic diagram:



Design of synchronous mod-6 counter using SR-Flipflop

Number of states: 6 [0 – 1 – 2 – 3 – 4 – 5]

Number of flipflops required: 03

Type of flipflop: SR

Excitation table of SR flipflop

Q_n	Q_{n+1}	S	R
0	0	0	–
0	1	1	0
1	0	0	1
1	1	–	0

Excitation table for the synchronous mod-6 counter:

Present State			Next State			Flip Flop Inputs					
Q_C	Q_B	Q_A	Q_C	Q_B	Q_A	S_C	R_C	S_B	R_B	S_A	R_A
0	0	0	0	0	1	0	–	0	–	1	0
0	0	1	0	1	0	0	–	1	0	0	1
0	1	0	0	1	1	0	–	–	0	1	0
0	1	1	1	0	0	1	0	0	1	0	1
1	0	0	1	0	1	–	0	0	–	1	0
1	0	1	0	0	0	0	1	0	–	0	1

Boolean expression for flipflop inputs using k-map simplification:

Boolean expression for inputs of flip flop C

Q_C	$Q_B Q_A$			
	00	01	11	10
0	0	0	1	0
1	–	0	–	–

Q_C	$Q_B Q_A$			
	00	01	11	10
0	–	–	0	–
1	0	1	–	–

$$S_C = Q_B Q_A$$

$$R_C = Q_C Q_A$$

Boolean expression for inputs of flip flop B

Q_C	$Q_B Q_A$			
	00	01	11	10
0	0	1	0	-
1	0	0	-	-

Q_C	$Q_B Q_A$			
	00	01	11	10
0	-	0	1	0
1	-	-	-	-

$$S_B = \bar{Q}_C \bar{Q}_B Q_A$$

$$R_B = Q_B Q_A$$

Boolean expression for inputs of flip flop A

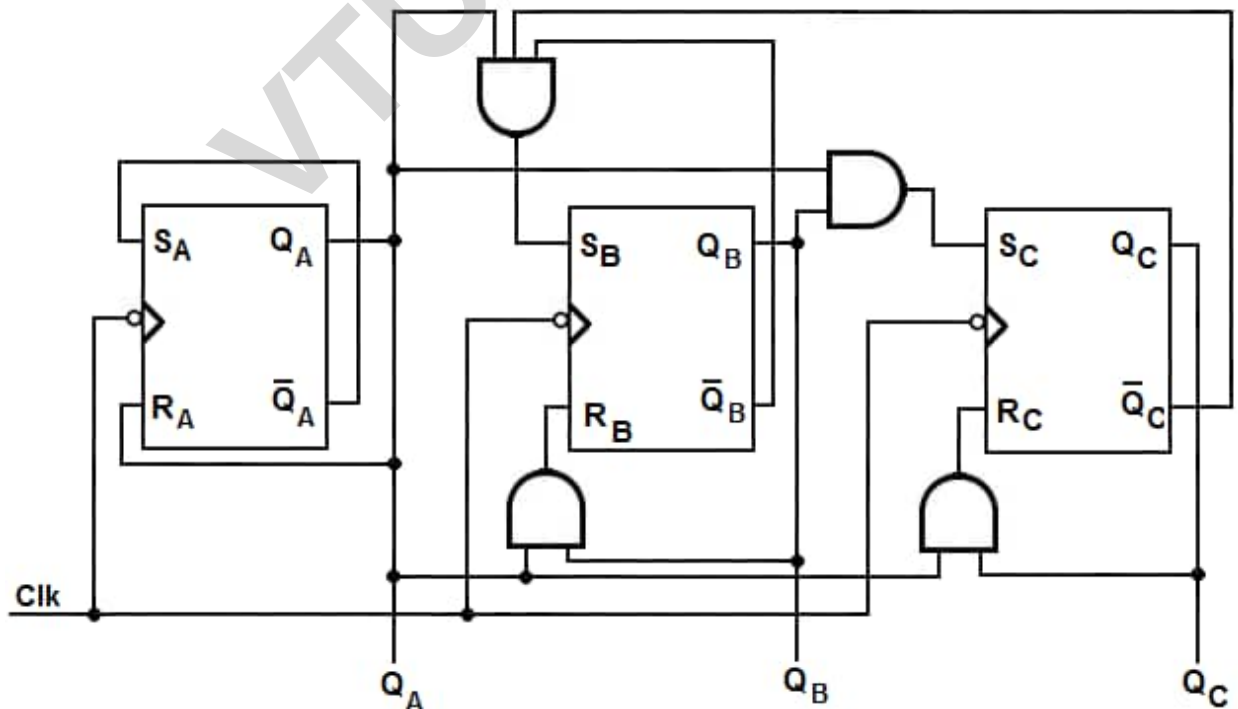
Q_C	$Q_B Q_A$			
	00	01	11	10
0	1	0	0	1
1	1	0	-	-

Q_C	$Q_B Q_A$			
	00	01	11	10
0	0	1	1	0
1	0	1	-	-

$$S_A = \bar{Q}_A$$

$$R_A = Q_A$$

Logic diagram:



Design synchronous mod-6 counter for the sequence 0 – 5 – 4 – 2 – 3 – 1 using JK-Flipflop

Number of states: 6 [0 – 5 – 4 – 2 – 3 – 1]

Number of flipflops required: 03

Type of flipflop: JK

Excitation table of JK flipflop

Cell No	Present State			Next State			Flip Flop Inputs					
	Q _C	Q _B	Q _A	Q _C	Q _B	Q _A	J _C	K _C	J _B	K _B	J _A	K _A
0	0	0	0	1	0	1	1	–	0	–	1	–
5	1	0	1	1	0	0	–	0	0	–	–	1
4	1	0	0	0	1	0	–	1	1	–	0	–
2	0	1	0	0	1	1	0	–	–	0	1	–
3	0	1	1	0	0	1	0	–	–	1	–	0
1	0	0	1	0	0	0	0	–	0	–	–	1

Boolean expression for inputs of flip flop C

Q _C	Q _B Q _A			
	00	01	11	10
0	1	0	0	0
1	–	–	–	–

Q _C	Q _B Q _A			
	00	01	11	10
0	–	–	–	–
1	1	0	–	–

$$J_C = \bar{Q}_B \bar{Q}_A$$

$$K_C = \bar{Q}_A$$

Boolean expression for inputs of flip flop B

Q _C	Q _B Q _A			
	00	01	11	10
0	0	0	–	–
1	1	0	–	–

Q _C	Q _B Q _A			
	00	01	11	10
0	–	–	1	0
1	–	–	–	–

$$J_B = Q_C \bar{Q}_A$$

$$K_B = Q_A$$

Boolean expression for inputs of flip flop A

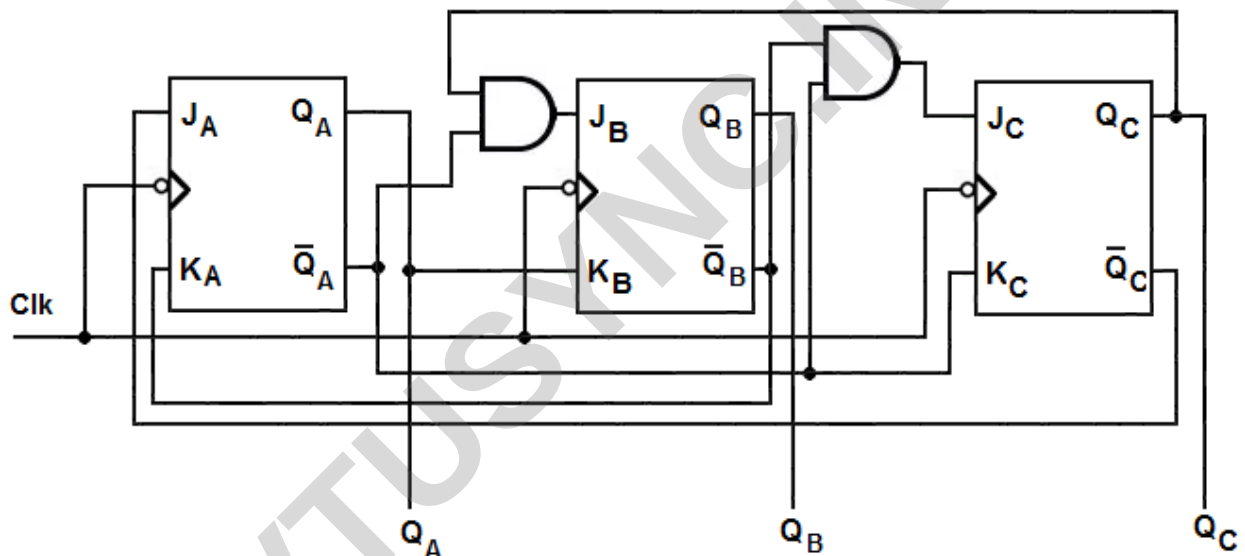
		Q_B	Q_A	
		00	01	11
Q_C	0	1	-	-
	1	0	-	-

		Q_B	Q_A	
Q_C		00	01	11
	0	–	1	–
	1	–	1	–

$$J_A = \overline{Q}_C$$

$$K_A = \bar{Q}_B$$

Logic diagram



Question Bank

1. What is flip-flop? What are the different types of Flipflops.
2. What is significance of edge triggering? Explain working of negative edge triggered D flip-flop with their functional table and waveforms.
3. Explain the working of Master-Slave JK flip-flop with functional table and timing diagram.
4. Explain Positive Edge Triggered D Flip-flop with the help of circuit diagram and waveforms
5. Obtain the Characteristic Equations for the following Flip-flops
(i) JK (ii) SR (iii) D (iv) T
6. With logic diagram and truth table explain the operation of a SR latch.
7. Explain the working of a master-slave JK flip-flop with the help of logic diagram, function table, logic symbol and timing diagram.
8. Explain the working principle of four-bit ripple counter, with the help of logic diagram and timing diagram.
9. Explain the working principle of four-bit synchronous counter, with the help of logic diagram and timing diagram.

MODULE – 4

Syllabus: *Introduction to Verilog: Structure of Verilog module, Operators, Data Types, Styles of Description. (Section 1.1 to 1.6.2, 1.6.4 (only Verilog), 2 of Text 3) Verilog Data flow description: Highlights of Data flow description, Structure of Data flow description. (Section 2.1 to 2.2 (only Verilog) of Text 3)*

Text book: 3. HDL Programming VHDL and Verilog by Nazeih M Botros, 2009 reprint, Dream techpress.

Hardware Description Language:

- Hardware Description Language (HDL) is an essential computer-aided design (CAD) tool for the modern design and synthesis of digital systems.
- The advancement in the semiconductor technology, the power and complexity of digital systems has increased. Due to this, such digital systems cannot be realized using discrete integrated circuits (IC's).
- Complex digital systems can be realized using high-density programmable chips such as application specific integrated circuits (ASIC's) and field programmable gate arrays (FPGA's). To design such systems, we require sophisticated CAD tool such as HDL.
- HDL is used by designer to describe the system in a computer language that is similar to other software Language like C. Debugging the design is easy, since HDL package implement simulators and test benches. The two widely used Hardware description languages are VHDL and Verilog
- VHDL, which stands for very-high-speed integrated circuit (VHSIC) hardware description language, was developed in the early 1980s
- After writing and testing the HDL code, the user can synthesize the code into digital logic components such as gates and flip-flops that can be downloaded into FPGAs or compatible electronic components.
- Usually, the CAD package that has HDL will also have a synthesizer. The HDL and synthesizer have made the task of designing complex systems much easier and faster than before.

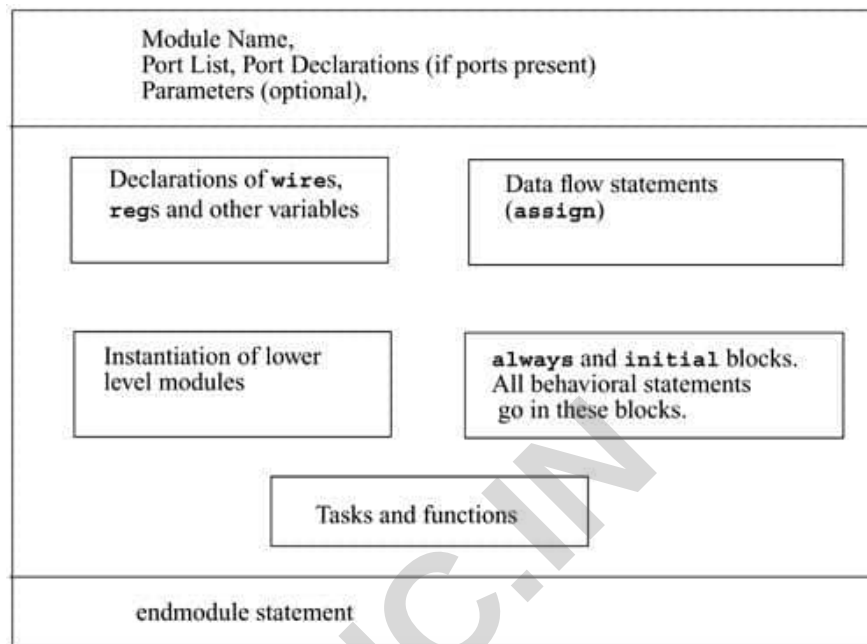
4.1 INTRODUCTION TO VERILOG

- **Verilog** was introduced in 1980's and has gone through several iterations and standardization by the Institute of Electrical and Electronic Engineers (IEEE), such as in December 1995 when Verilog HDL became IEEE Standard 1364-1995, in 2001 when IEEE Std. 1364-2001 was introduced, and in 2005 when IEEE 1800-2005 was introduced.
- Verilog HDL has evolved as a standard hardware description language. Verilog HDL offers many useful features.
- Verilog HDL is a **general-purpose hardware description language** that is easy to learn and easy to use. It is similar in syntax to the C programming language. Designers with C programming experience will find it easy to learn Verilog HDL.
- Verilog HDL **allows different levels of abstraction** to be mixed in the same model. Thus, a designer can define a hardware model in terms of switches, gates, RTL, or behavioral code. Also, a designer needs to learn only one language for stimulus and hierarchical design.
- Most popular logic synthesis tools support Verilog HDL. This makes it the **language of choice** for designers.
- All fabrication vendors provide Verilog HDL libraries for postlogic synthesis simulation. Thus, designing a chip in Verilog HDL allows the widest choice of vendors.
- The Programming Language Interface (PLI) is a powerful feature that allows the user to write custom C code to interact with the internal data structures of Verilog. Designers can customize a Verilog HDL simulator to their needs with the PLI.

4.2 STRUCTURE OF VERILOG MODULE:

- Verilog module has declaration and body. In the declaration, the name, inputs, and outputs of the module are entered.
- The body shows the relationship between the inputs and the outputs.

- **Structure of Veilog Module:**



- A module definition always begins with the keyword **module**.
- The module name, port list, port declarations, and optional parameters must come first in a module definition.
- Port list and port declarations are present only if the module has any ports to interact with the external environment.
- The five components within a module are: **variable declarations, dataflow statements, instantiation of lower modules, behavioral blocks, and tasks or functions**.
- These components can be in any order and at any place in the module definition.
- The endmodule statement must always come last in a module definition.
- All components except module, module name, and endmodule are optional and can be mixed and matched as per design needs.
- Verilog allows multiple modules to be defined in a single file. The modules can be defined in any order in the file.

Example: Design a Verilog Code for Half adder:

Half Adder: A half adder is a combinational circuit, which is a circuit whose output depends only on its input and which adds two input bits and outputs the result as two bits, one bit for the sum and one bit for the carry out.

Truth table:

Input		Output	
a	b	S	C
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1

Boolean/Logic Expression:

The Boolean function of the output of the adder is obtained from the truth table. The Boolean function of the output is generated using min-terms (where the output has a value of 1) or maxterms (where the output has a value of 0).

The Boolean function using minterms in the sum of products (SOP) form is

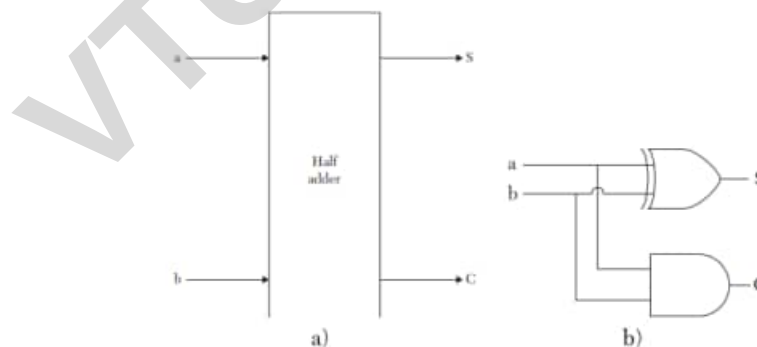
$$S = a'b + a b' = a \oplus b$$

$$C = a b$$

Using the maxterms in the product of sums (POS) forms

$$S = (a' + b)(a + b') = a \oplus b$$

$$C = (a + b)(a + b)(a + b) = ab$$

Block diagram and Logic diagram:**Verilog Design Code:**

```

module Half_adder(a,b,S,C);
input a,b; output S, C; // Blank lines are allowed
assign S = a ^ b; // statement 1
assign C= a & b; // statement 2
endmodule

```

Description:

- Verilog is case sensitive. Halfadder and halfadder are two different modules in verilog.
- The declaration starts with predefined word module.
- The name of the module should start with alphabetical letter and can include special character underscore (_). It is user selected.
- Semicolon (;) is a line separator.
- “=” is assignment operator, and symbols ^ and & are used for: “xor” and “and” respectively.
- The doubles slashes (//) signal a comment command or /*.....*/ the pair is used to write a comment of any length.
- The program ends with predefined word endmodule

Verilog ports:

- Ports provide the interface by which a module can communicate with its environment. For example, the input/output pins of an IC chip are its ports.
- The environment can interact with the module only through its ports. The internals of the module are not visible to the environment. This provides a very powerful flexibility to the designer.
- The internals of the module can be changed without affecting the environment as long as the interface is not modified. Ports are also referred to as terminals.

input: The port is only an input port. In any assignment statement, the port should appear only on the right hand side of the assignment statement.(i.e., port is read.)

output: the port is an output port.

inout: this port can be used as both an input and output. The inout port represents a bidirectional bus.

Verilog Keyword	Type of Port
input	Input port
output	Output port
<u>inout</u>	Bidirectional port

ANSI C Style Port Declaration Syntax

```
module Half_addr(output sum, carry, input a, b);  
...  
<module internals>  
...  
endmodule
```

- This syntax avoids the duplication of naming the ports in both the module definition statement and the module port list definitions.
- If a port is declared but no data type is specified, then, under specific circumstances, the signal will default to a wire data type.

Port Connection Rules

- One can visualize a port as consisting of two units, one unit that is internal to the module and another that is external to the module. The internal and external units are connected. There are rules governing port connections when modules are instantiated within other modules. The Verilog simulator complains if any port connection rules are violated. These rules are summarized in below Figure:

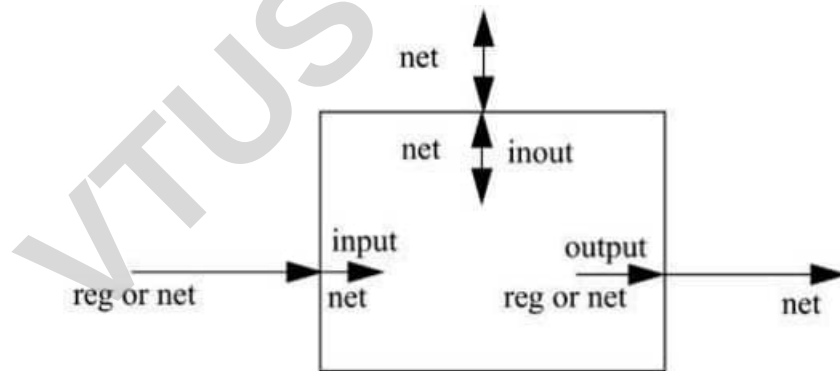


Figure 4.1. Port Connection Rules

i)Inputs

- Internally, input ports must always be of the type net. Externally, the inputs can be connected to a variable which is a reg or a net.

ii)Outputs

- Internally, outputs ports can be of the type reg or net. Externally, outputs must always be connected to a net. They cannot be connected to a reg.

iii)Inouts

- Internally, inout ports must always be of the type net. Externally, inout ports must always be connected to a net.

iv)Width matching

- It is legal to connect internal and external items of different sizes when making inter-module port connections. However, a warning is typically issued that the widths do not match.

v)Unconnected ports

- Verilog allows ports to remain unconnected. For example, certain output ports might be simply for debugging, and you might not be interested in connecting them to the external signals.

Connecting Ports to External Signals

- There are two methods of making connections between signals specified in the module instantiation and the ports in a module definition. These two methods cannot be mixed. These methods are discussed in the following sections.

i)Connecting by ordered list

- Connecting by ordered list is the most intuitive method for most beginners. The signals to be connected must appear in the module instantiation in the same order as the ports in the port list in the module definition.

Eg: fulladd4 fa_ordered(SUM, C_OUT, A, B, C_IN);

ii)Connecting ports by name

- For large designs where modules have, say, 50 ports, remembering the order of the ports in the module definition is impractical and error-prone.
- Verilog provides the capability to connect external signals to ports by the port names, rather than by position.

Eg: fulladd4 fa_byname(.c_out(C_OUT), .sum(SUM), .b(B), .c_in(C_IN), .a(A),);

Verilog Gate Types:

Verilog supports basic logic gates as predefined primitives.

- These primitives are instantiated like modules except that they are predefined in Verilog and do not need a module definition.
- All logic circuits can be designed by using basic gates.
- There are two classes of basic gates: and/or gates and buf/not gates.

And/Or Gates

- And/or gates have one scalar output and multiple scalar inputs.
- The first terminal in the list of gate terminals is an output and the other terminals are inputs.
- The output of a gate is evaluated as soon as one of the inputs changes.
- The and/or gates available in Verilog are shown below.

and or xor
nand nor xnor

The corresponding logic symbols for these gates are shown below. We consider gates with two inputs. The output terminal is denoted by out. Input terminals are denoted by i1 and i2

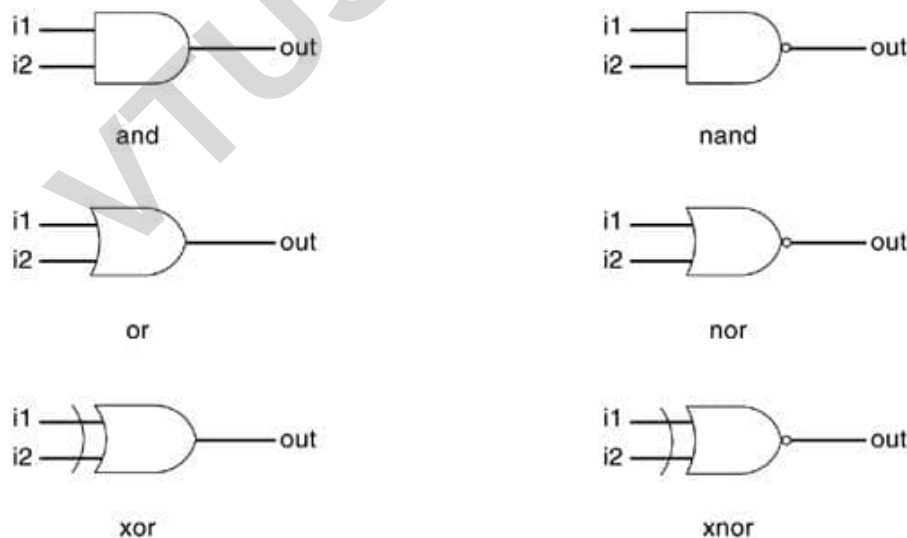


Figure 4-2. Basic Gates

- These gates are instantiated to build logic circuits in Verilog. Examples of gate instantiations are shown below.
- In example, for all instances, OUT is connected to the output out, and IN1 and IN2 are connected to the two inputs i1 and i2 of the gate primitives.
- Note that the instance name does not need to be specified for primitives. This lets the designer instantiate hundreds of gates without giving them a name.
- More than two inputs can be specified in a gate instantiation. Gates with more than two inputs are instantiated by simply adding more input ports in the gate instantiation .
- Verilog automatically instantiates the appropriate gate.

Example :Gate Instantiation of And/Or Gates

wire OUT, IN1, IN2; \\Input declared as IN1,IN2 and Output as OUT

Basic gate instantiations.

and a1(OUT, IN1, IN2);

nand na1(OUT, IN1, IN2);

or or1(OUT, IN1, IN2);

nor nor1(OUT, IN1, IN2);

xor x1(OUT, IN1, IN2);

xnor nx1(OUT, IN1, IN2)

More than two inputs;

3 input nand gate :

nand na1_3inp(OUT, IN1, IN2, IN3);

Gate instantiation without instance name

and (OUT, IN1, IN2); // legal gate instantiation

The truth tables for these gates define how outputs for the gates are computed from the inputs. Truth tables are defined assuming two inputs. The truth tables for these gates are shown in Table 4-1. Outputs of gates with more than two inputs are computed by applying the truth table iteratively.

		i1			
		0	1	x	z
i2	and	0	0	0	0
	0	0	0	0	0
	1	0	1	x	x
	x	0	x	x	x
	z	0	x	x	x

		i1			
		0	1	x	z
i2	nand	0	1	1	1
	0	1	1	1	1
	1	1	0	x	x
	x	1	x	x	x
	z	1	x	x	x

		i1			
		0	1	x	z
i2	or	0	0	1	x
	0	0	1	x	x
	1	1	1	1	1
	x	x	1	x	x
	z	x	1	x	x

		i1			
		0	1	x	z
i2	nor	0	1	0	x
	0	1	0	x	x
	1	0	0	0	0
	x	x	0	x	x
	z	x	0	x	x

		i1			
		0	1	x	z
i2	xor	0	0	1	x
	0	0	1	x	x
	1	1	0	x	x
	x	x	x	x	x
	z	x	x	x	x

		i1			
		0	1	x	z
i2	xnor	0	1	0	x
	0	1	0	x	x
	1	0	1	x	x
	x	x	x	x	x
	z	x	x	x	x

Table 4-1. Truth Tables for Logic Gates

OPERATORS

Expressions

Expressions are constructs that combine operators and operands to produce a result.

Operators act on the operands to produce desired results. Verilog provides various types of operators.

d1 && d2 // && is an operator on operands d1 and d2

!a[0] // ! is an operator on operand a[0]

B >>1 // >> is an operator on operands B and 1

4.4 Operator Types

Verilog provides many different operator types. Operators can be arithmetic, logical, relational, equality, bitwise, reduction, shift, concatenation, or conditional.

Some of these operators are similar to the operators used in the C programming language.

Each operator type is denoted by a symbol. Table below shows the complete listing of operator symbols classified by category.

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Arithmetic	*	multiply	two
	/	divide	two
	+	add	two
	-	subtract	two
	%	modulus	two
	**	power (exponent)	two
Logical	!	logical negation	one
	&&	logical and	two
		logical or	two
Relational	>	greater than	two
	<	less than	two
	>=	greater than or equal	two

	<code><=</code>	less than or equal	two
Equality	<code>==</code>	equality	two
	<code>!=</code>	inequality	two
	<code>===</code>	case equality	two
	<code>!==</code>	case inequality	two

Bitwise	<code>~</code>	bitwise negation	one
	<code>&</code>	bitwise and	two
	<code> </code>	bitwise or	two
	<code>^</code>	bitwise xor	two
	<code>^~</code> or <code>~^</code>	bitwise xnor	two
Reduction	<code>&</code>	reduction and	one
	<code>~&</code>	reduction nand	one
	<code> </code>	reduction or	one
	<code>~ </code>	reduction nor	one
	<code>^</code>	reduction xor	one
	<code>^~</code> or <code>~^</code>	reduction xnor	one

Shift	>>	Right shift	Two
	<<	Left shift	Two
	>>>	Arithmetic right shift	Two
	<<<	Arithmetic left shift	Two
Concatenation	{ }	Concatenation	Any number
Replication	{ { } }	Replication	Any number
Conditional	?:	Conditional	Three

1) Arithmetic Operators

There are two types of arithmetic operators: Binary and Unary.

i) Binary operators

Binary arithmetic operators are

- multiply (*),
- divide (/),
- add (+),
- subtract (-),
- power (**),
- modulus (%).

Binary operators take two operands.

`A = 4'b0011; B = 4'b0100; // A and B are register vectors`

`D = 6; E = 4; F=2// D and E are integers`

`A * B // Multiply A and B. Evaluates to 4'b1100`

`D / E` // Divide D by E. Evaluates to 1. Truncates any fractional part.

`A + B` // Add A and B. Evaluates to 4'b0111

`B - A` // Subtract A from B. Evaluates to 4'b0001

`F = E ** F`; //E to the power F, yields 16

If any operand bit has a value x, then the result of the entire expression is x. This seems intuitive because if an operand value is not known precisely, the result should be an unknown.

`in1 = 4'b101x`;

`in2 = 4'b1010`;

`sum = in1 + in2`; // sum will be evaluated to the value 4'bx

Modulus operators produce the remainder from the division of two numbers.

They operate similarly to the modulus operator in the C programming language.

`13 % 3`// Evaluates to 1

`16 % 4`// Evaluates to 0

`-7 % 2`// Evaluates to -1, takes sign of the first operand

`7 % -2`// Evaluates to +1, takes sign of the first operand

ii)Unary operators

The operators + and - can also work as unary operators. They are used to specify the positive or negative sign of the operand. Unary + or ? operators have higher precedence than the binary + or ? operators.

`-4` // Negative 4

`+5` // Positive 5

Negative numbers are represented as 2's complement internally in Verilog.

It is advisable to use negative numbers only of the type integer or real in expressions.

Designers should avoid negative numbers of the type `<sss> '<base><nnn>` in expressions because they are converted to unsigned 2's complement numbers and hence yield unexpected results.

```
//Advisable to use integer or real numbers -10 / 5//  
Evaluates to -2
```

```
//Do not use numbers of type
```

```
<sss> '<base><nnn>
```

`-'d10 / 5//` Is equivalent $(2\text{'s complement of } 10)/5 = (232 - 10)/5$ where 32 is the default machine word width.

This evaluates to an incorrect and unexpected result

2) Logical Operators

Logical operators are

- logical-and (&&),
- logical-or (||)
- logical- not (!).

Operators && and || are binary operators.

Operator ! is a unary operator.

Logical operators follow these conditions:

- Logical operators always evaluate to a 1-bit value, **0 (false), 1 (true), or x (ambiguous)**.
- If an operand is not equal to zero, it is equivalent to a logical 1 (true condition).
- If it is equal to zero, it is equivalent to a logical 0 (false condition).
- If any operand bit is x or z, it is equivalent to x (ambiguous condition) and is normally treated by simulators as a false condition.
- Logical operators take variables or expressions as operands.
- Use of parentheses to group logical operations is highly recommended to improve readability. Also, the user does not have to remember the precedence of operators.

Logical operations A = 3; B = 0;

A && B // Evaluates to 0. Equivalent to (logical-1 && logical-0)

A || B // Evaluates to 1. Equivalent to (logical-1 || logical-0)

!A// Evaluates to 0. Equivalent to not(logical-1)

!B// Evaluates to 1. Equivalent to not(logical-0)

Unknowns

A = 2'b0x; B = 2'b10;

A && B // Evaluates to x. Equivalent to (x && logical 1)

// Expressions

(a == 2) && (b == 3) // Evaluates to 1 if both a == 2 and b == 3 are true.

// Evaluates to 0 if either is false.

3) Relational Operators

Relational operators are

- greater-than (>),
- less-than (<),
- greater-than-or-equal-to (>=),
- less-than-or-equal-to (<=).

If relational operators are used in an expression, the expression returns a logical value of 1 if the expression is true and 0 if the expression is false

If there are any unknown or z bits in the operands, the expression takes a value x.

These operators function exactly as the corresponding operators in the C programming language.

A = 4, B = 3

X = 4'b1010, Y = 4'b1101, Z = 4'b1xxx

A <= B // Evaluates to a logical 0

A > B // Evaluates to a logical 1

Y >= X // Evaluates to a logical 1

Y < Z // Evaluates to an x

4) Equality Operators

Equality operators are logical equality (==), logical inequality (!=), case equality (===), and case inequality (!==) .

When used in an expression, equality operators return logical value 1 if true, 0 if false.

These operators compare the two operands bit by bit, with zero filling if the operands are of unequal length. lists the operators.

Table: Equality Operators

Expression	Description	Possible Logical Value
a == b	a equal to b, result unknown if x or z in a or b	0, 1, x
a != b	a not equal to b, result unknown if x or z in a or b	0, 1, x
a === b	a equal to b, including x and z	0, 1
a !== b	a not equal to b, including x and z	0, 1

- It is important to note the difference between the logical equality operators (==, !=) and case equality operators (===, !==).
- The logical equality operators (==, !=) will yield an x if either operand has x or z in its bits.

- However, the case equality operators (`===`, `!==`) compare both operands bit by bit and compare all bits, including x and z. The result is 1 if the operands match exactly, including x and z bits.
- The result is 0 if the operands do not match exactly. Case equality operators never result in an x.

Example:

- `A = 4, B = 3`
- `X = 4'b1010, Y = 4'b1101`
- `Z = 4'b1xxz, M = 4'b1xxz, N = 4'b1xxx`
- `A == B` // Results in logical 0
- `X != Y` // Results in logical 1
- `X == Z` // Results in x
- `Z === M` // Results in logical 1 (all bits match, including x and z)
- `Z === N` // Results in logical 0 (least significant bit does not match)
- `M !== N` // Results in logical 1

5) Bitwise Operators

Bitwise operators are

- negation (`~`)
- and(`&`),
- or (`|`),
- xor (`^`),
- xnor (`^~`, `~^`).

Bitwise operators perform a bit-by-bit operation on two operands.

They take each bit in one operand and perform the operation with the corresponding bit in the other operand.

If one operand is shorter than the other, it will be bit-extended with zeros to match the length of the longer operand.

Logic tables for the bit-by-bit computation are shown below

A z is treated as an x in a bitwise operation.

The exception is the unary negation operator ($\sim$), which takes only one operand and operates on the bits of the single operand.

bitwise and	0	1	x
0	0	0	0
1	0	1	x
x	0	x	x

bitwise or	0	1	x
0	0	1	x
1	1	1	1
x	x	1	x

bitwise xor	0	1	x
0	0	1	x
1	1	0	x
x	x	x	x

bitwise xnor	0	1	x
0	1	0	x
1	0	1	x
x	x	x	x

bitwise negation	result
0	1
1	0
x	x

Table Truth Tables for Bitwise Operators

Examples of bitwise operators are shown below.

- $X = 4'b1010, Y = 4'b1101, Z = 4'b10x1$
- $\sim X$ // Negation. Result is $4'b0101$
- $X \& Y$ // Bitwise and. Result is $4'b1000$
- $X | Y$ // Bitwise or. Result is $4'b1111$
- $X \wedge Y$ // Bitwise xor. Result is $4'b0111$
- $X \wedge \sim Y$ // Bitwise xnor. Result is $4'b1000$
- $X \& Z$ // Result is $4'b10x0$

It is important to distinguish bitwise operators $\sim$, $\&$, and $|$ from logical operators $!$, $\&\&$, $||$. Logical operators always yield a logical value 0, 1, x, whereas bitwise operators yield a bit-by-bit value. Logical operators perform a logical operation, not a bit-by-bit operation.

- `// X = 4'b1010, Y = 4'b0000`
- `X | Y` // bitwise operation. Result is `4'b1010`
- `X || Y` // logical operation. Equivalent to `1 || 0`. Result is `1`.

6) Reduction Operators

Reduction operators are

- and (`&`),
- nand (`~&`),
- or (`|`),
- nor (`~|`),
- xor (`^`),
- xnor (`~^`, `^^`).

Reduction operators take only one operand.

Reduction operators perform a bitwise operation on a single vector operand and yield a 1-bit result.

The difference is that bitwise operations are on bits from two different operands, whereas reduction operations are on the bits of the same operand.

Reduction operators work bit by bit from right to left. Reduction nand, reduction nor, and reduction xnor are computed by inverting the result of the reduction and, reduction or, and reduction xor, respectively.

`// X = 4'b1010`

- `&X` //Equivalent to `1 & 0 & 1 & 0`. Results in `1'b0`
- `|X` //Equivalent to `1 | 0 | 1 | 0`. Results in `1'b1`
- `^X` //Equivalent to `1 ^ 0 ^ 1 ^ 0`. Results in `1'b0`

A reduction xor or xnor can be used for even or odd parity //generation of a vector.

The use of a similar set of symbols for logical (`!`, `&&`, `||`), bitwise (`~`, `&`, `|`, `^`), and reduction operators (`&`, `|`, `^`) is somewhat confusing initially. The difference lies in the number of operands each operator takes and also the value of results computed.

7)Shift Operators

Shift operators are

- right shift ($\gg$),
- left shift ($\ll$),
- arithmetic right shift ($\ggg$),
- arithmetic left shift ($\lll$)

Regular shift operators shift a vector operand to the right or the left by a specified number of bits.

The operands are the vector and the number of bits to shift. When the bits are shifted, the vacant bit positions are filled with zeros.

Shift operations do not wrap around. Arithmetic shift operators use the context of the expression to determine the value with which to fill the vacated bits.

- `// X = 4'b1100`
- `Y = X >> 1; //Y is 4'b0110. Shift right 1 bit. 0 filled in MSB position.`
- `Y = X << 1; //Y is 4'b1000. Shift left 1 bit. 0 filled in LSB position.`
- `Y = X << 2; //Y is 4'b0000. Shift left 2 bits.`
- `integer a, b, c; //Signed data types`
- `a = 0;`
- `b = -10; // 00111...10110 binary`
- `c = a + (b >>> 3); //Results in -2 decimal, due to arithmetic shift`

Shift operators are useful because they allow the designer to model shift operations, shift-and-add algorithms for multiplication, and other useful operations.

Operation	Description Before Shift	Operand A After Shift	Operand A
A << 1	Shift A one position left logical	1110	1100
A << 2	Shift A two positions left logical	1110	1000
A >> 1	Shift A one position right logical	1110	0111
A >> 2	Shift A two positions right logical	1110	0011
A.>>> 2	Shift A two positions right arithmetic	1110	1111
A.<<< 2	Shift A two positions left arithmetic	1110	1000

8) Concatenation Operator

The concatenation operator ({ , }) provides a mechanism to append multiple operands. The operands must be sized. Unsized operands are not allowed because the size of each operand must be known for computation of the size of the result.

Concatenations are expressed as operands within braces, with commas separating the operands. Operands can be scalar nets or registers, vector nets or registers, bit-select, part-select, or sized constants.

```
// A = 1'b1, B = 2'b00, C = 2'b10, D = 3'b110
```

```
Y = {B, C} // Result Y is 4'b0010
```

```
Y = {A, B, C, D, 3'b001} // Result Y is 11'b10010110001
```

```
Y = {A, B[0], C[1]} // Result Y is 3'b101
```

9) Replication Operator

Repetitive concatenation of the same number can be expressed by using a replication constant.

A replication constant specifies how many times to replicate the number inside the brackets ({ }).

```
reg A;
```

```
reg [1:0] B, C;
```

```
reg [2:0] D;
```

```
A = 1'b1; B = 2'b00; C = 2'b10; D = 3'b110;
```

```
Y = { 4{A} } // Result Y is 4'b1111
```

```
Y = { 4{A} , 2{B} } // Result Y is 8'b11110000
```

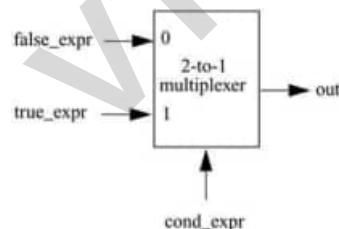
```
Y = { 4{A} , 2{B} , C } // Result Y is 8'b1111000010
```

10) Conditional Operator

The conditional operator(**?:**) takes three operands.

- Usage: **condition_expr ? true_expr : false_expr ;**
- The condition expression (condition_expr) is first evaluated.
- If the result is true (logical 1), then the true_expr is evaluated. If the result is false (logical 0), then the false_expr is evaluated.
- If the result is x (ambiguous), then both true_expr and false_expr are evaluated and their results are compared, bit by bit, to return for each bit position an x if the bits are different and the value of the bits if they are the same.

The action of a conditional operator is similar to a multiplexer. Alternately, it can be compared to the if-else expression.



Conditional operators are frequently used in dataflow modeling to model conditional assignments. The conditional expression acts as a switching control.

```
//model functionality of a tristate buffer
assign addr_bus = drive_enable ? addr_out : 36'bz;
```

```
//model functionality of a 2-to-1 mux
```

```
assign out = control ? in1 : in0;
```

Conditional operations can be nested. Each true_expr or false_expr can itself be a conditional operation.

In the example that follows, convince yourself that (A==3) and control are the two select signals of 4-to-1 multiplexer with n, m, y, x as the inputs and out as the output signal.

```
assign out = (A == 3) ? ( control ? x : y ) : ( control ? m : n );
```

Operator Precedence

Operators listed in Table 4-4 are in order from highest precedence to lowest precedence

It is recommended that parentheses be used to separate expressions except in case of unary operators or when there is no ambiguity.

Operators	Operator Symbols	Precedence
Unary	+ - ! ~	Highest precedence
Multiply, Divide, Modulus	* / %	
Add, Subtract	+ -	
Shift	<< >>	
Relational	<<= >>=	
Equality	== != === !==	
	&, ~&	

Reduction	$\wedge$ $\wedge \sim$ $ $, $\sim $	
Logical	$\&\&$ $ $	
Conditional	$?:$	Lowest precedence

Table : Operator Precedence

4.1.4 Examples

Having understood the various types of gates available in Verilog, we will discuss a real example that illustrates design of gate-level digital circuits.

We will design a 4-to-1 multiplexer with 2 select signals. Multiplexers serve a useful purpose in logic design. They can connect two or more sources to a single destination. They can also be used to implement boolean functions. We will assume for this example that signals s1 and s0 do not get the value x or z. The I/O diagram and the truth table for the multiplexer are shown in Figure 4-4. The I/O diagram will be useful in setting up the port list for the multiplexer.

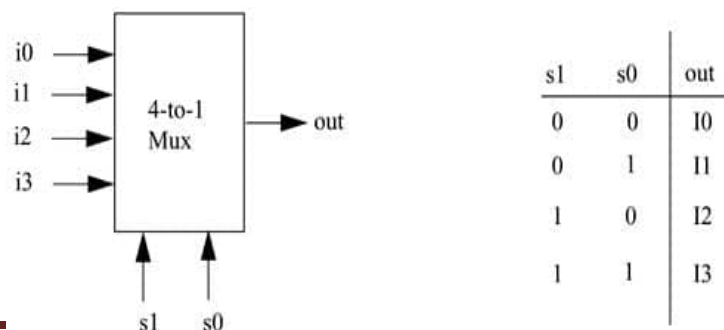
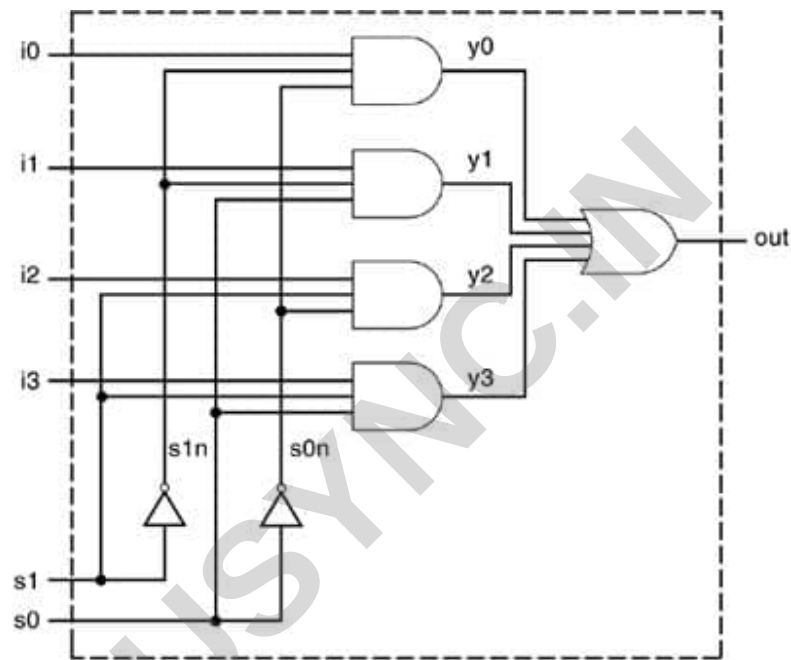


Figure 3-4. 4-to-1 Multiplexer

We will implement the logic for the multiplexer using basic logic gates. The logic diagram for the multiplexer is shown in below:

**Figure 3-5. Logic Diagram for Multiplexer**

The logic diagram has a one-to-one correspondence with the Verilog description. The Verilog description for the multiplexer is shown in [Example 5-5](#). Two intermediate nets, s0n and s1n, are created; they are complements of input signals s1 and s0. Internal nets y0, y1, y2, y3 are also required. Note that instance names are not specified for primitive gates, not, and, and or. Instance names are optional for Verilog primitives but are mandatory for instances of user-defined modules.

Example :Verilog Description of Multiplexer

Module 4-to-1 multiplexer. Port list is taken exactly from the I/O diagram.

```
module mux4_to_1 (out, i0, i1, i2, i3, s1, s0); //Port declarations from the I/O diagram
output out;
```

```
input i0, i1, i2, i3;

input s1, s0;

//Internal wire declarations

wire s1n, s0n;

wire y0, y1, y2, y3;

//Gate instantiations

//Create s1n and s0n signals.
not (s1n, s1);

not (s0n, s0);

//3-input and gates instantiated
and (y0, i0, s1n, s0n);

and (y1, i1, s1n, s0);

and (y2, i2, s1, s0n);

and (y3, i3, s1, s0);

//4-input or gate instantiated
or (out, y0, y1, y2, y3);

endmodule
```

This multiplexer can be tested with the stimulus shown in Example 4-6.

The stimulus checks that each combination of select signals connects the appropriate input to the output.

The signal OUTPUT is displayed one time unit after it changes.

System task \$monitor could also be used to display the signals when they change values.

Example 3-6 Stimulus for Multiplexer

```
//Define the stimulus module (no ports)

module stimulus;
```

```
//Declare variables to be connected to inputs
```

```
reg IN0, IN1, IN2, IN3;
```

```
reg S1, S0;
```

```
    //Declare output wire
```

```
wire OUTPUT
```

```
    //Instantiate the multiplexer
```

```
mux4_to_1 mymux(OUTPUT, IN0, IN1, IN2, IN3, S1, S0);
```

```
    //Stimulate the inputs
```

```
    //Define the stimulus module (no ports)
```

```
initial
```

```
begin
```

```
//set input lines
```

```
IN0 = 1; IN1 = 0; IN2 = 1; IN3 = 0;
```

```
#1 $display("IN0= %b, IN1= %b, IN2= %b, IN3= %b\n",IN0,IN1,IN2,IN3);
```

```
    //choose IN0
```

```
S1 = 0; S0 = 0;
```

```
#1 $display("S1 = %b, S0 = %b, OUTPUT = %b \n", S1, S0, OUTPUT);
```

```
    //choose IN1
```

```
S1 = 0; S0 = 1;
```

```
#1 $display("S1 = %b, S0 = %b, OUTPUT = %b \n", S1, S0, OUTPUT);
```

```
    //choose IN2
```

```
S1 = 1; S0 = 0;
```

```
#1 $display("S1 = %b, S0 = %b, OUTPUT = %b \n", S1, S0, OUTPUT);
```

```
    //choose IN3
```

```
S1 = 1; S0 = 1;
```

```
#1 $display("S1 = %b, S0 = %b, OUTPUT = %b \n", S1, S0, OUTPUT);
```

```
end
```

```
endmodule
```

The output of the simulation is shown below. Each combination of the select signals is tested.

IN0= 1, IN1= 0, IN2= 1, IN3= 0

S1 = 0, S0 = 0, OUTPUT = 1

S1 = 0, S0 = 1, OUTPUT = 0

S1 = 1, S0 = 0, OUTPUT = 1

S1 = 1, S0 = 1, OUTPUT = 0

Verilog Data Types

- Data or operands used in the language must have several types to match the need for describing the hardware.
- For example, if we are describing a signal, we need to specify its type (i.e., the values that the signal can take), such as type bit, which means that the signal can assume only 0 or 1, or type.
- Examples of types include integer, real, vector, bit, and array.
- Verilog supports several data types including nets, registers, vectors, integer, real, parameters, and arrays.

1) Nets

Nets are declared by the predefined word wire. Nets have values that change continuously by the circuits that are driving them. Verilog supports four values for nets, as shown in Table 1.13.

TABLE 1: Verilog Net Values

Value	Definition
0	Logic 0 (false)
1	Logic 1 (true)
X	Unknown
Z	High impedance

Examples of **net types** are as follows:

```
wire sum;
```

```
wire S1 = 1'b0;
```

The first statement declares a net by the name sum. The second statement declares a net by the name of S1; its initial value is 1'b0, which represents 1 bit with value 0.

2. Register

Register, in contrast to nets, stores values until they are updated. Register, as its name suggests, represents data-storage elements. Register is declared by the predefined word reg. Verilog supports four values for register, as shown in Table:

TABLE 2: Verilog Register Values

Value	Definition
0	Logic 0 (false)
1	Logic 1 (true)
X	Unknown
Z	High impedance

An example of register is:

```
reg Sum_total;
```

The above statement declares a register by the name Sum_total.

3)Vectors

Vectors are multiple bits. A register or a net can be declared as a vector. Vectors are declared by brackets [].

Examples of vectors are:

```
wire [3:0] a = 4'b1010;
```

```
reg [7:0] total = 8'd12;
```

The first statement declares a net a. It has four bits, and its initial value is 1010 (b stands for bit). The second statement declares a register total. Its size is eight bits, and its value is decimal 12 (d stands for decimal).

4) Integers

Integers are declared by the predefined word integer. An example of integer declaration is:

```
integer no_bits;
```

The above statement declares no_bits as an integer.

4) Real

Real (floating-point) numbers are declared with the predefined word real. Examples of real values are 2.4, 56.3, and 5e12. The value 5e12 is equal to 5×10^{12} . The following statement declares the register weight as

```
real;
```

```
real weight;
```

5) Parameter

Parameter represents a global constant. It is declared by the pre-defined word parameter. The following is an example of implementing parameters:

Example: module compr_genr (X, Y, xgty, xlty, xeqy);

```
parameter N = 3;
```

```
input [N:0] X, Y;
```

```
output xgty, xlty, xeqy;
```

```
wire [N:0] sum, Yb;
```

To change the size of the inputs x and y, the size of the nets sum, and the size of net Yb to eight bits, the value of N is changed to seven as:

parameter N = 7

6) Arrays

Verilog, in contrast to VHDL, does not have a predefined word for array. Registers and integers can be written as arrays.

Consider the following

statements:

parameter N = 4;

parameter M = 3;

reg signed [M:0] carry [0:N];

The above statements declare an array by the name carry. The array carry has five elements, and each element is four bits.

The four bits are in two's complement form.

For example, if the value of a certain element is 1001, then it is equivalent to decimal -7.

Arrays can be multidimensional.

STYLES OF DESCRIPTION:

- Several styles of code writing can be used to describe the system. Selection of the styles depends on the available information on the system.
- For example, some systems may be easily described by the Boolean function of the output; for other systems, such as biological mechanisms, it will be hard to obtain the Boolean function of the output, but they can be described if the relationship between the changes of the output with the input is known.
- In the following section, six styles will be discussed: data flow, behavioral, structural, switch level, mixed type, and mixed language.

1)Behavioral or algorithmic level

- This is the highest level of abstraction provided by Verilog HDL. A module can be implemented in terms of the desired design algorithm without concern for the hardware implementation details. Designing at this level is very similar to C programming.
- Design Code:

```
module Half_adder(a,b,S,C);  
  input a,b;  
  output S, C;  
  reg S,C; // output declared as reg  
  always @ (a,b)  
  begin  
    if (a != b)  
      S = 1'b1;  
    else  
      S = 1'b0;  
  end  
endmodule
```

2)Dataflow level

- At this level, the module is designed by specifying the data flow. The designer is aware of how data flows between hardware registers and how the data is processed in the design.
- Code:

```
module Half_adder(a,b,S,C);  
  input a,b; output S, C; // Blank lines are allowed  
  assign S = a ^ b; // statement 1  
  assign C= a & b; // statement 2  
endmodule
```

3)Gate level/Structural Description:

- The module is implemented in terms of logic gates and interconnections between these gates. Design at this level is similar to describing a design in terms of a gate-level logic diagram.

Design Code:

```
module Half_adder1(a,b,S,C);  
  input a, b;  
  output S,C;  
  and a1(C,a,b); //The above statement is AND gate  
  xor x1(S,a,b); //The above statement is EXCLUSIVE-OR gate  
endmodule
```

4)Switch level

- This is the lowest level of abstraction provided by Verilog. A module can be implemented in terms of switches, storage nodes, and the interconnections between them. Design at this level requires knowledge of switch-level implementation details.
- Design Code:

```
module invert(y,a);  
  input a;  
  output y;  
  supply1 vdd;  
  supply0 gnd;  
  pmos p1(y, vdd, a);  
  nmos n1(y, gnd, a); //The above two statement are using the two primitives pmos and  
  nmos.  
endmodule
```

- Verilog allows the designer to mix and match all four levels of abstractions in a design. In the digital design community, the term register transfer level (RTL) is frequently used for a Verilog description that uses a combination of behavioral and dataflow constructs and is acceptable to logic synthesis tools.
- Normally, the higher the level of abstraction, the more flexible and technology-independent the design. As one goes lower toward switch-level design, the design becomes technology-dependent and inflexible. A small modification can cause a significant number of changes in the

design. Consider the analogy with C programming and assembly language programming. It is easier to program in a higher-level language such as C. The program can be easily ported to any machine. However, if you design at the assembly level, the program is specific for that machine and cannot be easily ported to another machine.

5)Mixed-Type Description

Mixed-type or mixed-style descriptions are those that use more than one type or style of the above-mentioned descriptions. In fact, most of the descriptions of moderate to large systems are mixed. Some parts of the system may be described using one type and others using other types of description.

6) Mixed-Language Description

The mixed-language description is a newly added tool to HDL description. The user now can write a module in one language (VHDL or Verilog) and invoke or import a construct (entity or module) written in the other language. If a design contains four modules, Verilog allows each of the modules to be written at a different level of abstraction. As the design matures, most modules are replaced with gate-level implementations.

DATA FLOW DESCRIPTION

Data flow is one type (style) of hardware description. Other types include behavioral, structural, switch level, mixed type, and mixed language.

Data-flow description simulates the system to be described by showing how the signal flows from the system inputs to its outputs. For example, the Boolean function of the output or the logical structure of the system shows such signal flow.

Signal-assignment statements are concurrent. At any simulation time, all signal-assignment statements that have an event are executed concurrently .

Signal Declaration And Assignment Statement:

Figure below shows an AND-OR circuit. Signals a, b, c, and d are the inputs, signal y is the output, and signals s1 and s2 are intermediates.

The Boolean function of the output y can be written as:

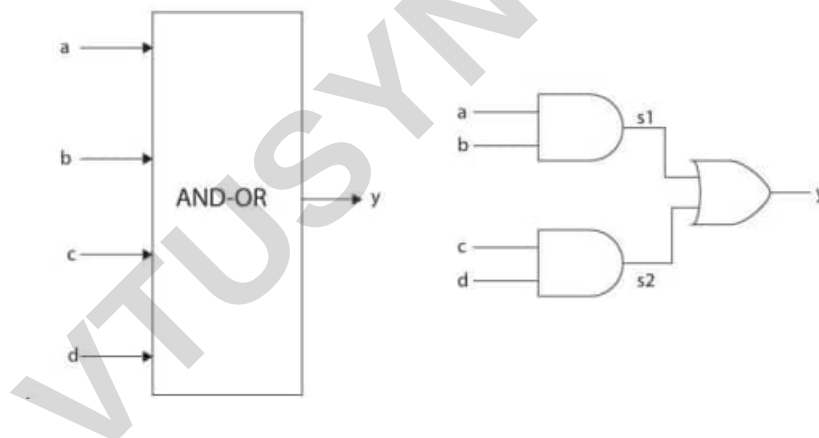


Fig :AND-OR circuit. a) Symbol diagram. b) Logic diagram.

Verilog description

```
module andor (a,b,c,d, y );
input a,b,c,d;
output y;
wire s1, s2; //wire statement here is not necessarily needed since s1 and s2 are single bit
    assign s1 = a & b; //statement 1.
    assign s2 = c & d; //statement 2.
    assign y = s1 | s2; //statement 3.
```

endmodule

Using a CAD package with HDL simulator the code can be simulated on the screen of the computer, and a wave form showing a graphical relationship between the input and the output can be obtained. Figure 2.2 shows such a waveform.

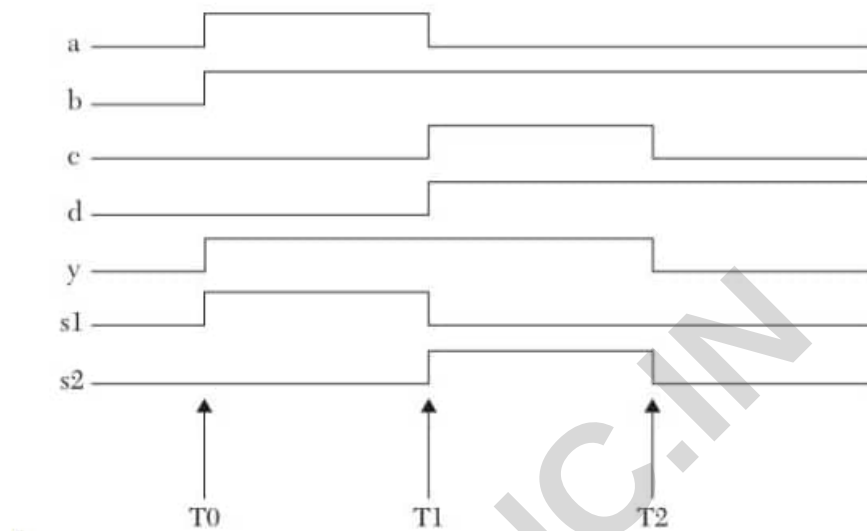


Fig : Simulation waveform for the AND-OR circuit

A signal-assignment statement is used to assign a value to a signal. The left-hand side of the statement should be declared as a signal. The right hand side can be a signal, a variable, or a constant. The operator for signal assignment is the predefined word assign in Verilog.

- The execution of the signal-assignment statement in HDL is somehow different in concept from that of software languages such as C.
- Statements 1–3 need an event to occur on its right-hand side to start execution. If no event occurred on any statement, this statement would not be executed.
- An event is a change in the value of a signal or variable such as a change from 0 to 1 (from low to high) or from 1 to 0 (from high to low).
- The statement that receives an event first will be executed first regardless of the order of its placement in the HDL code. If more than one statement have an event at the same time, all of these statements will be executed concurrently (i.e., simultaneously).
- Accordingly, statement 3, for example, could have been written before statement 1 in Listing 2.1, and the order of execution would not be affected.
- The signal-assignment statement is executed in two phases: calculation and assignment.

- If an event occurs on the right-hand side of a statement, then this side is calculated at the time of the event; after calculation, the value obtained from the calculation is assigned to the left-hand side, taking into consideration any timing information given in the statement.
- At T0, an event has occurred in signal a and signal b (both signals changed their value from 0 to 1, which is an event). Accordingly, an event occurred in statement 1; the value of (a and b) is calculated as (1 and 1 = 1).
- Because no delay time is specified, the value 1 is assigned immediately to s1, changing s1 from 0 to 1. Changing the value of s1 from 0 to 1 constitutes an event in s1 and in statement 3, which is executed as a result of the event in its right-hand side.
- The right-hand side of statement 3 is calculated at T0 as (s1 [1] or s2 [0] = 1). The value of 1 is assigned to y; all at T0 because no delay time is specified. At T1, there is an event on signals a (1 to 0), c (0 to 1), and d (0 to 1).
- Statements 1 and 2 will be executed concurrently because an event occurred on their right-hand side. The right-hand side of statement 1 and 2 is calculated at T1 as (0 and 1 = 0) and (1 and 1 = 1); the value of 0 is assigned to s1, and the value of 1 is assigned to s2 at T1.
- Changing the value of s1 and s2 constitutes an event on s1 and s2, which selects statement 3 for execution at T1; statement 3 is executed (calculation, s1 or s2 = 0 or 1 = 1), and accordingly, 1 is assigned to signal y.
- At T2, an event occurred on signal c, statement 2 is executed at T2, and the calculation results in 0 and 1 = 0; the value 0 is assigned to s2, changing its value from 1 to 0 and generating an event in s2.
- Statement 3 is executed because an event (changing the value of s2 from 1 to 0) occurred on the right-hand side. The calculation results in 0 or 0 = 0; the value 0 is assigned to y at T2.

Constant Declaration and Constant Assignment Statements:

A constant in HDL is treated as it is in C language; its value is constant within the segment of the program where it is visible.

In Verilog, a constant can be declared by its type such as time or integer. For example, the following statements declare period as a constant of type time:

time period; // Verilog To assign a value to a constant, use the assignment operator = in Verilog. For example, to assign a value of 100 nanoseconds to the constant period described above: period = 100; // Verilog

In the above Verilog statement, there are no explicit units of time; 100 means 100 simulation screen time units.

The declaration and assignment can be combined in one statement as:

```
time period = 100 //Verilog
```

Assigning a Delay Time to the Signal-Assignment Statement:

To assign a delay time to a signal-assignment statement, the predefined word # in Verilog is used. For example, the following statement assigns a 10 ns delay time to signal S1:

```
assign #10 S1 = a & b // Verilog
```

In Verilog, the delay is in simulation screen unit time. Let us assume that there is a delay of 10 ns between the output of each statement 1–3 and its input in Listing 2.1. This is equivalent to saying that operation (and) or (or) takes 10 ns to be completed. Listing 2.2 shows the HDL code for Figure 2.1 with a 10 ns delay for the (and) and (or) operations.

Verilog description

```
module and_orDlyVr( a,b,c,d, y );  
  
input a,b,c,d;  
  
output y;  
  
time dly = 10;  
  
wire s1, s2; // wire above is not necessarily needed since s1 and s2 are single bit/  
  
    assign # dly s1 = a & b; //statement 1.  
  
    assign # dly s2 = c & d; //statement 2.  
  
    assign # dly y = s1 | s2; //statement 3.  
  
endmodule
```

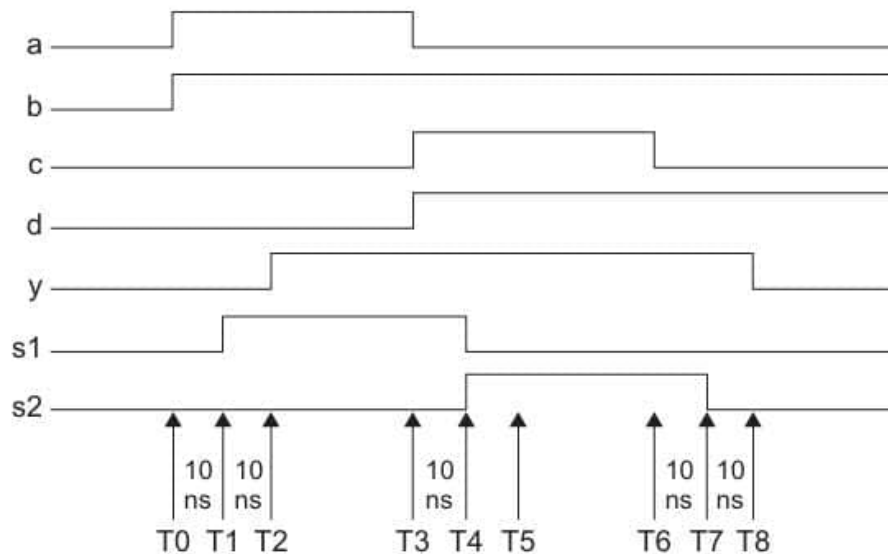


Figure: Simulation waveform of Listing 2.2.

EXAMPLE : DATA-FLOW DESCRIPTION OF A FULL ADDER

- A full adder is a combinational circuit (output depends only on the input) that adds three input bits ($a + b + c$) and outputs the result as two bits; one bit for the sum and one bit for the carryout. Examples of full addition are: $1 + 0 + 1 = 10$ (in decimal $1 + 0 + 1 = 2$) and $1 + 1 + 1 = 11$ (in decimal $1 + 1 + 1 = 3$). Table below shows the truth table of the full adder.

Input			Output	
a	b	c	Carryout	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

The Boolean function of the Sum and Carryout can be obtained from K-maps as shown in Figure

bc \ a	00	01	11	10
0		1		1
1	1		1	

Sum = $f(a,b,c) = m(1,2,4,7)$

bc \ a	00	01	11	10
0			1	
1		1	1	1

Carryout = $f(a,b,c) = m(3,5,6,7)$

The Boolean functions can be written as:

$$\text{Sum} = \bar{a}\bar{b}c + \bar{a}b\bar{c} + a\bar{b}\bar{c} + abc$$

$$\text{Carryout} = ab + ac + bc$$

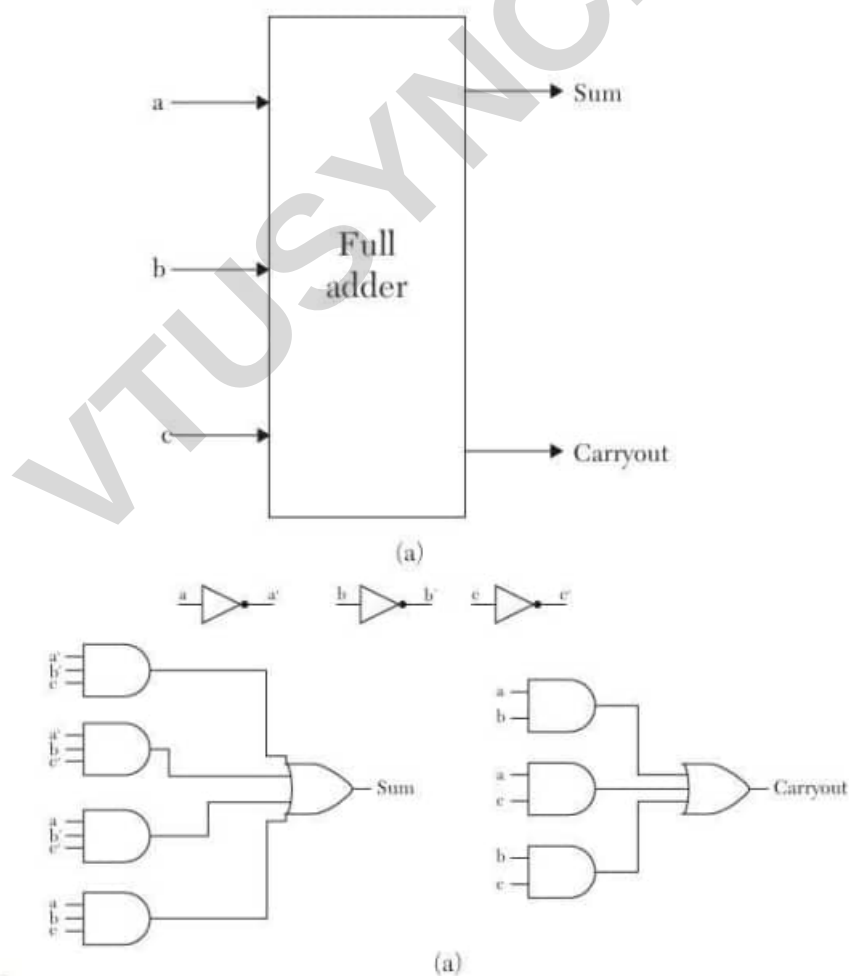


Fig: A full Adder. a) Logic symbol. b) Logic diagram.

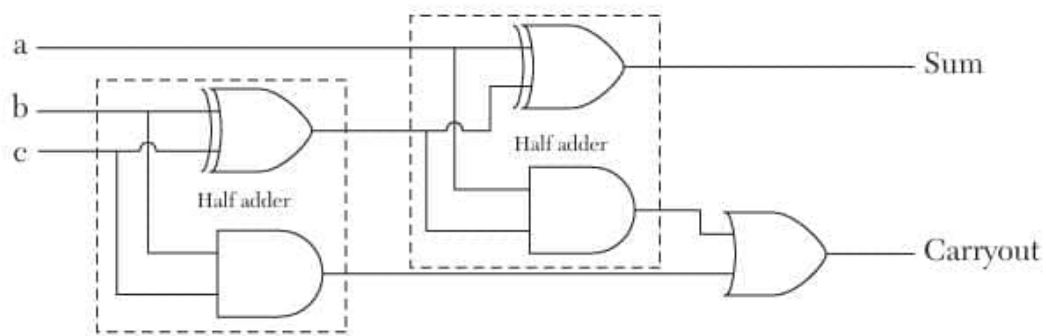


Fig: A full adder built from two half adders

Verilog description

```

module fulladder(a, b, c);
output Sum, Carryout;
input a, b, c;
    assign Sum = (~ a & ~ b & c) | (~ a & b & ~ c) |
                ( a & ~ b & ~ c) | ( a & b & c );
    assign Carryout = (a & b) | (a & c) | (b & c);
endmodule

```

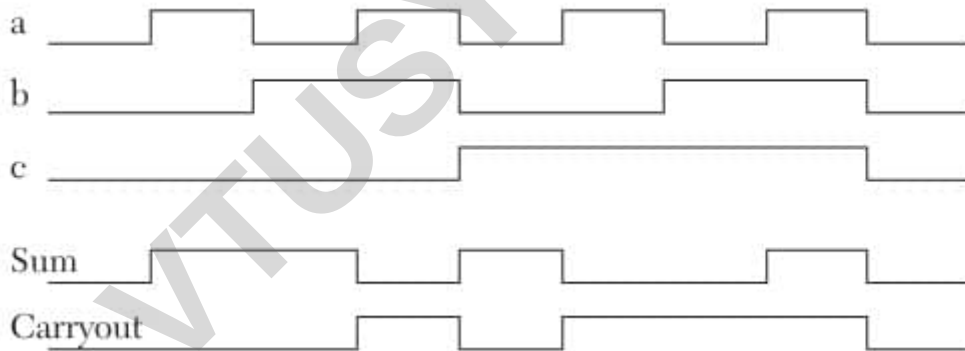


Fig: Simulation waveform of a full adder with no delay time.

FULL SUBTRACTOR:

A full subtractor performs the following operation: $a - b - c = \text{Borrow Diff}$. Borrow and Diff are each one-bit output. The Diff is the difference, and Borrow is the borrow.

For example, $0 - 1 - 0 = 11$. The subtraction is done as follows: $0 - 1$ cannot subtract 1 from 0 because 1 is greater than 0, so borrow 1 from the higher-order bit. Accordingly, this 1 has a weight of 2, so its value is 2; subtract $2 - 1 = 1$. Now, for bit c, $1 - 0 = 1$, so the difference is 1, and the borrow is 1. Table below shows the truth table of a full subtractor.

Input			Output	
a	b	c	Borrow	Diff
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Compare the Diff in Table 2.3 and the Sum in Table 2.2; they are identical, so the Boolean function of the Diff is the same as the sum in Equation for full adder. For the Borrow, draw the K-map as shown in Figure.

a \ bc				
	00	01	11	10
0	0	1	1	1
1	0	0	1	

Fig: K-map for the borrow of a full subtractor

From K-Map, the Boolean functions are:

$$\text{Diff} = \bar{a}\bar{b}c + \bar{a}b\bar{c} + a\bar{b}\bar{c} + abc$$

$$\text{Borrow} = \bar{a}c + \bar{a}b + bc$$

2x1 MULTIPLEXER WITH ACTIVE LOW ENABLE

A 2x1 multiplexer is a combinational circuit; it has two one-bit inputs, a one-bit select line, and a one-bit output. Additional control signals may be added, such as enable.

The output of the basic multiplexer depends on the level of the select line. If the select is high (1), the output is equal to one of the two inputs.

If the select is low (0), the output is equal to the other input. A truth table for a 2x1 multiplexer with active low enable is shown in Table below.

Input		Output
SEL	Gbar	Y
X	H	L
L	L	A
H	L	B

If the enable (Gbar) is high (1), the output is low (0) regardless of the input. When Gbar is low (0), the output is A if SEL is low (0), or the output is B if SEL is high (1). From Table, the Boolean function of the output Y is: $Y = (\overline{S1} \text{ and } A \text{ and } \text{SEL}) \text{ or } (\overline{S1} \text{ and } B \text{ and } \text{SEL})$; S1 is the invert of Gbar

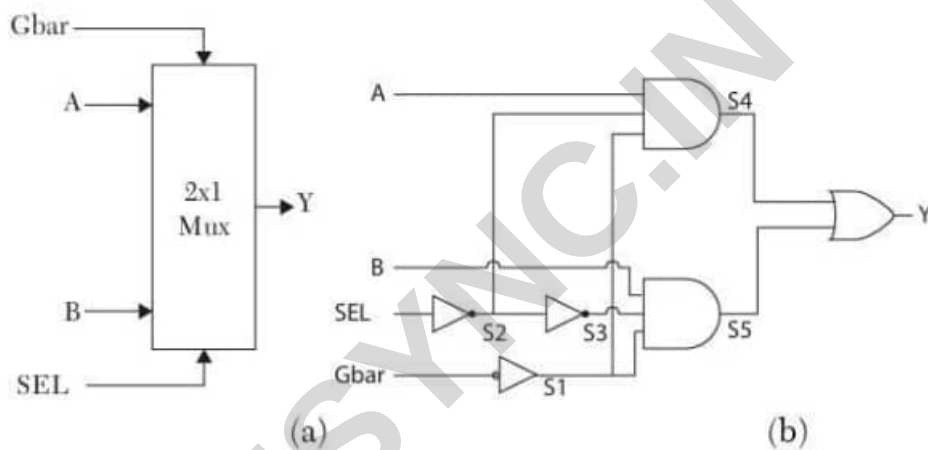


Fig:2x1 Multiplexer. a) Logic symbol. b) Logic diagram.

Verilog Description

```

module mux2x1 (A, B, SEL, Gbar, Y);
input A, B, SEL, Gbar;
output Y;
wire S1, S2, S3, S4, S5;
time dly = 7; // Assume 7 time units delay for all and, or, not operations.
assign #dly Y = S4 | S5; //st1
assign #dly S4 = A & S2 & S1; //st2
assign #dly S5 = B & S3 & S1; //st3
assign #dly S2 = ~ SEL; //st4
assign #dly S3 = ~ S2; //st5

```

```

    assign #dly S1 = ~ Gbar; //st6
endmodule

```

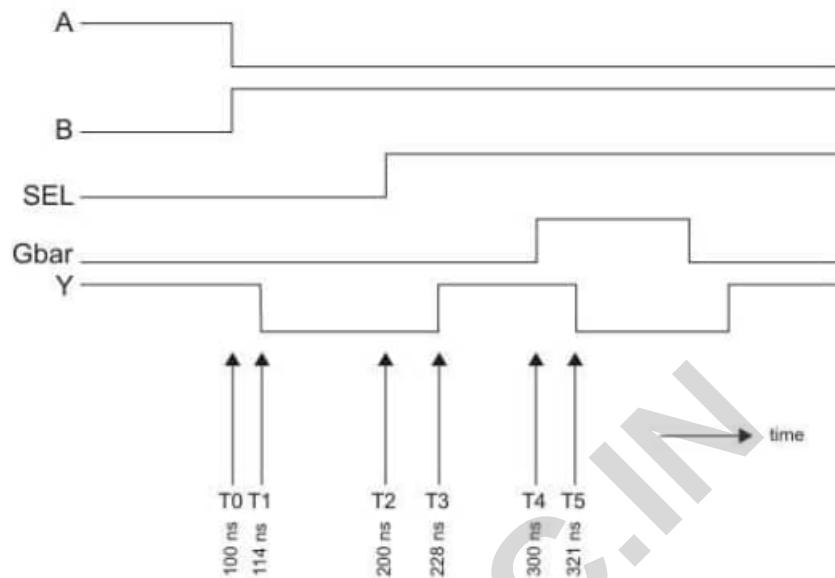


Fig: shows the simulation waveform for the 2x1 multiplexer

2X1 MULTIPLEXER WITH ACTIVE LOW ENABLE USING VERILOG CONDITIONAL OPERATOR (?)

The format of this operator can be written as:

```
assign Y = Conditional-expression ? true-expression : false-expression
```

If the conditional expression is true, the value of the true expression is assigned to Y; if the conditional expression is false, the value of the false expression is assigned to Y.

HDL Code of a 2x1 Multiplexer Using Verilog Conditional Operator:

```

module Mux2x1_conditional(input A,B,SEL,Gbar, output Y );
assign Y = (Gbar) ? 1'b0 : (SEL & B) | (~SEL & A);
endmodule

```

2x4 DECODER

A decoder is a combinational circuit. A 2x4 decoder has two inputs and four outputs. For any input, only one output is active; all others are inactive. For active high output decoders, only one output

is high. The output of n-bit input decoder is 2^n bits. Table 2.5 shows the truth table of the 2x4 decoder.

Inputs		Outputs			
b	a	D3	D2	D1	D0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

The Boolean function of the outputs can be written as:

$$D0 = \bar{a} \bar{b}$$

$$D1 = a \bar{b}$$

$$D2 = \bar{a} b$$

$$D3 = a b$$

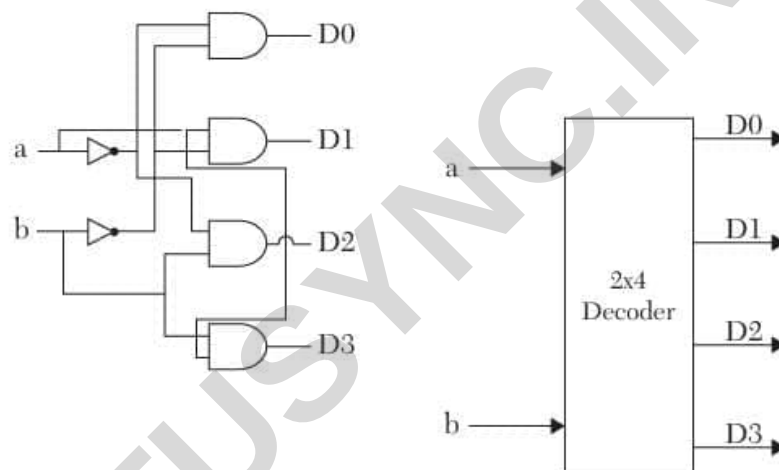


Fig: 2x4 Decoder. a) Logic symbol. b) Logic diagram.

Verilog description

```

module decoder2x4( a, b, D);
input a,b;
output [3:0]D;
    assign D[0] = ~a & ~b;
    assign D[1] = a & ~b;
    assign D[2] = ~a & b;
    assign D[3] = a & b;
endmodule

```

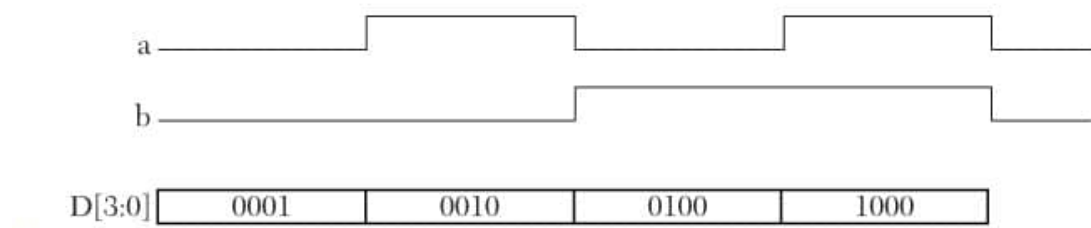


Fig: Simulation Waveform of a 2x4 decoder with no time delay

MODULE – 5

Syllabus: Verilog Behavioral description: Structure, Variable Assignment Statement, Sequential Statements, Loop Statements, Verilog Behavioral Description of Multiplexers (2:1, 4:1, 8:1). (Section 3.1 to 3.4 (only Verilog) of Text 3)

Verilog Structural description: Highlights of Structural description, Organization of structural description, Structural description of ripple carry adder. (Section 4.1 to 4.2 of Text 3)

Text book: 3. HDL Programming VHDL and Verilog by Nazeih M Botros, 2009 reprint, Dream techpress.

Verilog Behavioral Description:

Introduction:

- The behavioral description is a powerful tool to describe systems for which digital logic structures are not known or are hard to generate.
- The behavioral description describes the system by showing how outputs behave with the changes in inputs. In this description, details of the logic diagram of the system are not needed; what is needed is how the output behaves in response to a change in the input. In Verilog, the major behavioral description statements are always and initial.
- With the increasing complexity of digital design, it has become vitally important to make wise design decisions early in a project.
- Designers need to be able to evaluate the trade-offs of various architectures and algorithms before they decide on the optimum architecture and algorithm to implement in hardware.
- Thus, architectural evaluation takes place at an algorithmic level where the designers do not necessarily think in terms of logic gates or data flow but in terms of the algorithm they wish to implement in hardware. They are more concerned about the behavior of the algorithm and its performance.
- Only after the high-level architecture and algorithm are finalized, do designers start focusing on building the digital circuit to implement the algorithm.
- Verilog provides designers the ability to describe design functionality in an algorithmic manner.
- In other words, the designer describes the behavior of the circuit. Thus, behavioral modeling represents the circuit at a very high level of abstraction. Design at this level resembles C

programming more than it resembles digital circuit design. Behavioral Verilog constructs are similar to C language constructs in many ways.

- Verilog is rich in behavioral constructs that provide the designer with a great amount of flexibility.

Structure of Behavioral Description:

Structured Procedures:

There are two structured procedure statements in Verilog:

- **always** and **initial**.

These statements are the two most basic statements in behavioral modeling. All other behavioral statements can appear only inside these structured procedure statements.

Verilog is a concurrent programming language unlike the C programming language, which is sequential in nature. Activity flows in Verilog run in parallel rather than in sequence. Each **always** and **initial** statement represents a separate activity flow in Verilog. Each activity flow starts at simulation time 0. The statements **always** and **initial** cannot be nested. The fundamental difference between the two statements is explained in the following sections.

- **initial Statement**
 - All statements inside an initial statement constitute an initial block.
 - An initial block starts at time 0, executes exactly once during a simulation, and then does not execute again.
 - If there are multiple initial blocks, each block starts to execute concurrently at time 0.
 - Each block finishes execution independently of other blocks. Multiple behavioral statements must be grouped, typically using the keywords **begin** and **end**.
 - If there is only one behavioral statement, grouping is not necessary.
 - This is similar to the **begin-end** blocks in Pascal programming language or the **{ }** grouping in the C programming language.

[Example:](#) illustrates the use of the initial statement.

initial Statement

```
module stimulus;  
reg x,y, a,b, m;
```

```

initial
m = 1'b0; //single statement; does not need to be grouped
initial
begin
#5 a = 1'b1; //multiple statements; need to be grouped
#25 b = 1'b0;
end
initial
begin
#10 x = 1'b0;
#25 y = 1'b1;
end
initial
#50 $finish;
endmodule

```

In the above example, the three initial statements start to execute in parallel at time 0. If a delay #<delay> is seen before a statement, the statement is executed <delay> time units after the current simulation time.

Thus, the execution sequence of the statements inside the initial blocks will be as follows.

time	statement executed
0	m = 1'b0;
5	a = 1'b1;
10	x = 1'b0;
30	b = 1'b0;
35	y = 1'b1;
50	\$finish;

The initial blocks are typically used for initialization, monitoring, waveforms and other processes that must be executed only once during the entire simulation run.

always Statement

- All behavioral statements inside an always statement constitute an always block.
- The always statement starts at time 0 and executes the statements in the always block continuously in a looping fashion.
- This statement is used to model a block of activity that is repeated continuously in a digital circuit.
- An example is a clock generator module that toggles the clock signal every half cycle.
- In real circuits, the clock generator is active from time 0 to as long as the circuit is powered on. [Example](#) illustrates one method to model a clock generator in Verilog.

Example: always Statement

```
module clock_gen (output reg clock);  
  
    //Initialize clock at time zero  
  
    initial  
        clock = 1'b0;  
  
    //Toggle clock every half-cycle (time period = 20)  
  
    always  
        #10 clock = ~clock;  
  
    initial  
        #1000 $finish;  
  
endmodule
```

In [Example](#):

The always statement starts at time 0 and executes the statement `clock = ~clock` every 10 time units. Notice that the initialization of clock has to be done inside a separate initial statement. If we put the initialization of clock inside the always block, clock will be initialized every time the always is entered. Also, the simulation must be halted inside an initial statement. If there is no \$ stop or \$finish statement to halt the simulation, the clock generator will run forever. C programmers might draw an analogy between the always block and an infinite loop. But hardware designers tend to view it as a continuously repeated

activity in a digital circuit starting from power on. The activity is stopped only by power off (\$finish) or by an interrupt (\$stop).

2. Procedural Assignments

- Procedural assignments update values of reg, integer, real, or time variables.
- The value placed on a variable will remain unchanged until another procedural assignment updates the variable with a different value.
- Dataflow Modeling, where one assignment statement can cause the value of the right-hand-side expression to be continuously placed onto the left-hand-side net.
- The syntax for the simplest form of procedural assignment is shown below.

assignment ::= variable_lvalue = [delay_or_event_control] expression

The left-hand side of a procedural assignment <lvalue> can be one of the following:

- A reg, integer, real, or time register variable or a memory element
- A bit select of these variables (e.g., addr[0])
- A part select of these variables (e.g., addr[31:16])
- A concatenation of any of the above
-

There are two types of procedural assignment statements: blocking and nonblocking.

➤ Blocking Assignments

- Blocking assignment statements are executed in the order they are specified in a sequential block.
- A blocking assignment will not block execution of statements that follow in a parallel block..
- The = operator is used to specify blocking assignments.

Example: Blocking Statements

```
reg x, y, z;  
reg [15:0] reg_a, reg_b;  
integer count;  
//All behavioral statements must be inside an initial or always block  
initial  
begin  
    x = 0; y = 1; z = 1; //Scalar assignments
```

```
count = 0; //Assignment to integer variables
reg_a = 16'b0; reg_b = reg_a; //initialize vectors
#15 reg_a[2] = 1'b1; //Bit select assignment with delay
#10 reg_b[15:13] = {x, y, z} //Assign result of concatenation to// part select of a vector
count = count + 1; //Assignment to an integer (increment)
end
```

In [Example](#), the statement $y = 1$ is executed only after $x = 0$ is executed. The behavior in a particular block is sequential in a begin-end block if blocking statements are used, because the statements can execute only in sequence. The statement $\text{count} = \text{count} + 1$ is executed last. The simulation times at which the statements are executed are as follows:

- All statements $x = 0$ through $\text{reg_b} = \text{reg_a}$ are executed at time 0
- Statement $\text{reg_a}[2] = 0$ at time = 15
- Statement $\text{reg_b}[15:13] = \{x, y, z\}$ at time = 25
- Statement $\text{count} = \text{count} + 1$ at time = 25
- Since there is a delay of 15 and 10 in the preceding statements, $\text{count} = \text{count} + 1$ will be executed at time = 25 units

Note that for procedural assignments to registers, if the right-hand side has more bits than the register variable, the right-hand side is truncated to match the width of the register variable. The least significant bits are selected and the most significant bits are discarded. If the right-hand side has fewer bits, zeros are filled in the most significant bits of the register variable.

➤ Nonblocking Assignments

- Nonblocking assignments allow scheduling of assignments without blocking execution of the statements that follow in a sequential block.
- A $\leq$ operator is used to specify nonblocking assignments.

Note that this operator has the same symbol as a relational operator, `less_than_equal_to`.

The operator $\leq$ is interpreted as a relational operator in an expression and as an assignment operator in the context of a nonblocking assignment

To illustrate the behavior of nonblocking statements and its difference from blocking statements, let us consider [Example](#), where we convert some blocking assignments to nonblocking assignments, and observe the behavior.

Example 4-7 Nonblocking Assignments

```
reg x, y, z;
reg [15:0] reg_a, reg_b;
integer count;
//All behavioral statements must be inside an initial or always block
initial
begin
    x = 0; y = 1; z = 1; //Scalar assignments
    count = 0; //Assignment to integer variables
    reg_a = 16'b0; reg_b = reg_a; //Initialize vectors

    reg_a[2] <= #15 1'b1; //Bit select assignment with delay

    reg_b[15:13] <= #10 {x, y, z}; //Assign result of concatenation //to part select
                                   of a vector

    count <= count + 1; //Assignment to an integer (increment)

end
```

In this example, the statements `x = 0` through `reg_b = reg_a` are executed sequentially at time 0. Then the three nonblocking assignments are processed at the same simulation time.

`reg_a[2] = 0` is scheduled to execute after 15 units (i.e., time = 15)
`reg_b[15:13] = {x, y, z}` is scheduled to execute after 10 time units (i.e., time = 10)
`count = count + 1` is scheduled to be executed without any delay (i.e., time = 0)

Thus, the simulator schedules a nonblocking assignment statement to execute and continues to the next statement in the block without waiting for the nonblocking statement to complete execution. Typically, nonblocking assignment statements are executed last in the time step in which they are scheduled, that is, after all the blocking assignments in that time step are executed.

In the example above, we mixed blocking and nonblocking assignments to illustrate their behavior. However, it is recommended that blocking and nonblocking assignments not be mixed in the same always block.

Conditional Statements

Conditional statements are used for making decisions based upon certain conditions. These conditions are used to decide whether or not a statement should be executed. Keywords if and else are used for conditional statements. There are three types of conditional statements. Usage of conditional statements is shown below.

//Type 1 conditional statement. No else statement.

//Statement executes or does not execute.

- **if (<expression>) true_statement ;**

//Type 2 conditional statement. One else statement //Either true_statement or false_statement is evaluated

- **if (<expression>) true_statement ; else false_statement ;**

//Type 3 conditional statement. Nested if-else-if. //Choice of multiple statements. Only one is executed.

- **if (<expression1>) true_statement1 ;
else if (<expression2>) true_statement2 ;

else if (<expression3>) true_statement3 ;

else default_statement ;**

The <expression> is evaluated. If it is true (1 or a non-zero value), the true_statement is executed. However, if it is false (zero) or ambiguous (x), the false_statement is executed. Each true_statement or false_statement can be a single statement or a block of multiple statements. A block must be grouped, typically by using keywords begin and end. A single statement need not be grouped.

Example:

//Execute statements based on ALU control signal.

if (alu_control == 0)

 y = x + z;

else if(alu_control == 1)

```
y = x - z;  
  
else if(alu_control == 2)  
  
    y = x * z;  
  
else  
    $display("Invalid ALU control signal");
```

Multiway Branching

1 case Statement

The keywords case, endcase, and default are used in the case statement..

case (expression)

```
    alternative1: statement1;  
    alternative2: statement2;  
    alternative3: statement3;  
    ...  
    ...  
    default: default_statement;
```

endcase

- Each of statement1, statement2 , default_statement can be a single statement or a block of multiple statements. A block of multiple statements must be grouped by keywords begin and end.
- The expression is compared to the alternatives in the order they are written. For the first alternative that matches, the corresponding statement or block is executed.
- If none of the alternatives matches, the default_statement is executed.
- The default_statement is optional.
- Placing of multiple default statements in one case statement is not allowed.
- The case statements can be nested.

//Execute statements based on the ALU control signal

```
reg [1:0] alu_control;

...

...

case (alu_control)

    2'd0 : y = x + z;

    2'd1 : y = x - z;

    2'd2 : y = x * z;

    default : $display("Invalid ALU control signal");
endcase
```

The case statement can also act like a many-to-one multiplexer.

Example : 4-to-1 Multiplexer with Case Statement

```
module mux4_to_1 (out, i0, i1, i2, i3, s1, s0);

Port declarations from the I/O diagram output out;

input i0, i1, i2, i3; input s1, s0;
reg out;

always @(s1 or s0 or i0 or i1 or i2 or i3)

case ({s1, s0}) //Switch based on concatenation of control signals
    2'd0 : out = i0;
    2'd1 : out = i1;

    2'd2 : out = i2;

    2'd3 : out = i3;

    default: $display("Invalid control signals");
endcase

endmodule
```

The case statement compares 0, 1, x, and z values in the expression and the alternative bit for bit. If the expression and the alternative are of unequal bit width, they are zero filled to match the bit width of the widest of the expression and the alternative. In [Example 4-20](#), we will define a 1-to-4 demultiplexer for which outputs are completely specified, that is, definitive results are provided even for x and z values on the select signal.

2 casex, casez Keywords

- There are two variations of the case statement.
- They are denoted by keywords, casex and casez.
- casez treats all z values in the case alternatives or the case expression as don't cares. All bit positions with z can also be represented by ? in that position.
- casex treats all x and z values in the case item or the case expression as don't cares.
- The use of casex and casez allows comparison of only non-x or -z positions in the case expression and the case alternatives. [Example 4-21](#) illustrates the decoding of state bits in a finite state machine using a casex statement.
- The use of casez is similar. Only one bit is considered to determine the next state and the other bits are ignored.

Example: casex Use

```
reg [3:0] encoding;  
integer state;  
casex (encoding) //logic value x represents a don't care bit.  
4'b1xxx : next_state = 3;  
4'bx1xx : next_state = 2;  
4'bxx1x : next_state = 1;  
4'bxxx1 : next_state = 0;  
default : next_state = 0;  
endcase
```

Thus, an input encoding = 4'b10xz would cause next_state = 3 to be executed.

Loops

There are four types of looping statements in Verilog: while, for, repeat, and forever. The syntax of these loops is very similar to the syntax of loops in the C programming language. All looping statements can appear only inside an initial or always block. Loops may contain delay expressions.

1 While Loop

The keyword while is used to specify this loop. The while loop executes until the while-expression is not true. If the loop is entered when the while-expression is not true, the loop is not executed at all. Any logical expression can be specified with these operators. If multiple statements are to be executed in the loop, they must be grouped typically using keywords begin and end. [Example below](#) illustrates the use of the while loop.

Example : While Loop

```
//Illustration 1: Increment count from 0 to 127. Exit at count 128.
//Display the count variable.
integer count;
initial
begin

    count = 0;

    while (count < 128) //Execute loop till count is 127.,exit at count 128

    begin

        $display("Count = %d", count);

        count = count + 1;

    end

end
```

2 For Loop

The keyword for is used to specify this loop. The for loop contains three parts:

- An initial condition
- A check to see if the terminating condition is true
- A procedural assignment to change value of the control variable

The counter described in [Example 4-22](#) can be coded as a for loop ([Example 4-23](#)). The initialization condition and the incrementing procedural assignment are included in the for loop and do not need to be specified separately. Thus, the for loop provides a more compact loop structure than the while loop. Note, however, that the while loop is more general-purpose than the for loop. The for loop cannot be used in place of the while loop in all situations.

Example: For Loop

```
integer count;
```

```
initial
```

```
    for ( count=0; count < 128; count = count + 1)
```

```
        $display("Count = %d", count);
```

- for loops are generally used when there is a fixed beginning and end to the loop. If the loop is simply looping on a certain condition, it is better to use the while loop.

3 Repeat Loop

- The keyword repeat is used for this loop
- The repeat construct executes the loop a fixed number of times.
- A repeat construct cannot be used to loop on a general logical expression.
- A while loop is used for that purpose.
- A repeat construct must contain a number, which can be a constant, a variable or a signal value.
- However, if the number is a variable or signal value, it is evaluated only when the loop starts and not during the loop execution.

The counter in [Example](#) can be expressed with the repeat loop, as shown in Illustration 1 in [Example](#).

Example: Repeat Loop

```
//Illustration 1 : increment and display count from 0 to 127 integer count;
```

```
initial
begin
    count = 0;
    repeat(128)
    begin
        $display("Count = %d", count);
        count = count + 1;
    end
end
```

4 Forever loop

- The keyword forever is used to express this loop.
- The loop does not contain any expression and executes forever until the \$finish task is encountered.
- The loop is equivalent to a while loop with an expression that always evaluates to true, e.g., while (1).
- A forever loop can be exited by use of the disable statement.

A forever loop is typically used in conjunction with timing control constructs. If timing control constructs are not used, the Verilog simulator would execute this statement infinitely without advancing simulation time and the rest of the design would never be executed. [Example](#) explains the use of the forever statement.

Example:Forever Loop

: Clock generation

//Use forever loop instead of always block

```
reg clock;
```

```
initial
```

```
begin

    clock = 1'b0;

    forever #10 clock = ~clock; //Clock with period of 20 units

end
```

Sequential and Parallel Blocks

Block statements are used to group multiple statements to act together as one. In previous examples, we used keywords **begin** and **end** to group multiple statements. Thus, we used sequential blocks where the statements in the block execute one after another. In this section we discuss the block types: sequential blocks and parallel blocks. We also discuss three special features of blocks: named blocks, disabling named blocks, and nested blocks.

1 Block Types

There are two types of blocks: sequential blocks and parallel blocks.

Sequential blocks

The keywords **begin** and **end** are used to group statements into sequential blocks.

Sequential blocks have the following characteristics:

- The statements in a sequential block are processed in the order they are specified. A statement is executed only after its preceding statement completes execution (except for nonblocking assignments with intra-assignment timing control).
- If delay or event control is specified, it is relative to the simulation time when the previous statement in the block completed execution.

We have used numerous examples of sequential blocks in this book. Two more examples of sequential blocks are given in [Example](#). Statements in the sequential block execute in order. In Illustration 1, the final values are $x = 0$, $y = 1$, $z = 1$, $w = 2$ at simulation time 0. In

Example: Sequential Blocks

//Illustration 1: Sequential block without delay

```
reg x, y;  
  
reg [1:0] z, w;  
  
initial  
  
begin  
  
    x = 1'b0;  
  
    y = 1'b1;  
  
    z = {x, y};  
  
    w = {y, x};  
  
end
```

Parallel blocks

Parallel blocks, specified by keywords fork and join, provide interesting simulation features. Parallel blocks have the following characteristics:

- Statements in a parallel block are executed concurrently.
- Ordering of statements is controlled by the delay or event control assigned to each statement.
- If delay or event control is specified, it is relative to the time the block was entered.

Notice the fundamental difference between sequential and parallel blocks. All statements in a parallel block start at the time when the block was entered. Thus, the order in which the statements are written in the block is not important.\

Example:Parallel Blocks

Parallel blocks with delay.

```
reg x, y;  
reg [1:0] z, w;  
initial  
fork  
  
    x = 1'b0; //completes at simulation time 0  
    #5 y = 1'b1; //completes at simulation time 5  
    #10 z = {x, y}; //completes at simulation time 10  
    #20 w = {y, x}; //completes at simulation time 20  
  
join
```

- Parallel blocks provide a mechanism to execute statements in parallel.

Examples

1 4-to-1 Multiplexer

We can define a 4-to-1 multiplexer with the behavioral case statement.

Example: Behavioral 4-to-1 Multiplexer

```
module mux4_to_1 (out, i0, i1, i2, i3, s1, s0);

output out;
input i0, i1, i2, i3; input s1,
s0;

reg out;

//recompute the signal out if any input signal changes. //All input signals
that cause a recomputation of out to //occur must go into the always @(...)
sensitivity list.
always @(s1 or s0 or i0 or i1 or i2 or i3)
begin

    case ({s1, s0})

        2'b00: out = i0;

        2'b01: out = i1;

        2'b10: out = i2;

        2'b11: out = i3;

        default: out = 1'bx;

    endcase

end

endmodule
```

2) 4-bit Counter-Behavioral 4-bit Counter Description

```
//4-bit Binary      counter

module counter(Q , clock, clear);
```

```
I/O ports

output [3:0] Q;
input clock, clear; //output defined as
register
reg [3:0] Q;
always @( posedge clear    or negedge clock)

begin

    if (clear)

        Q <= 4'd0;    //Nonblocking assignments are recommended

                        //for creating sequential logic such as flipflops

    else

Q <= Q + 1; // Modulo 16 is not necessary because Q is a
// 4-bit value and wraps around.

end

endmodule
```



AJ INSTITUTE OF ENGINEERING & TECHNOLOGY

A Unit of Laxmi Memorial Education Trust ®

(Approved by AICTE, New Delhi, Affiliated to Visvesvaraya Technological University, Belgavi)

Department of Electronics and Communication Engineering

MANUAL ON
COURSE NAME: DIGITAL SYSTEM DESIGN
USING VERILOG

COURSE CODE: BEC302

22 SCHEME

Course Instructor: Ms. Pratheksha Rai N

Assistant Professor

Department of ECE, AJIET



AJ INSTITUTE OF ENGINEERING & TECHNOLOGY

A Unit of Laxmi Memorial Education Trust ®

(Approved by AICTE, New Delhi, Affiliated to Visvesvaraya Technological University, Belgaavi)

Department of Electronics and Communication Engineering

Vision of the institute

“To produce top-quality engineers who are groomed for attaining excellence in their profession and competitive enough to help in the growth of nation and global society.”

Mission of the institute

- M1: To offer affordable high-quality graduate program in engineering with value education and make the students socially responsible.
- M2: To support and enhance the institutional environment to attain research excellence in both faculty and students and to inspire them to push the boundaries of knowledge base.
- M3: To identify the common areas of interest amongst the individuals for the effective industry- institute partnership in a sustainable way by systematically working together.
- M4: To promote the entrepreneurial attitude and inculcate innovative ideas among the engineering professionals.

Department Vision

To be recognized as a center of excellence in the area of Electronics and Communication Engineering by nurturing the young innovative minds into skillful and ethical professionals to cater the industrial and societal needs.

Department Mission

- M1. To establish state-of-the art laboratories to facilitate research and innovation to upgrade the knowledge and skills in healthcare sector and IoT.
- M2. To provide industry interaction for training programs on latest technology.
- M3. To provide ethical and value based education by promoting activities addressing the societal needs.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

- PEO1. Exhibit a desire for lifelong learning through professional and societal activities.
- PEO2. Exhibit and apply their technical skills and knowledge in Electronics and Communication Engineering for industry and societal needs.
- PEO3. Exhibit leadership qualities, professional skills, management skills and ethics needed for successful career.



AJ INSTITUTE OF ENGINEERING & TECHNOLOGY

A Unit of Laxmi Memorial Education Trust ®

(Approved by AICTE, New Delhi, Affiliated to Visvesvaraya Technological University, Belgaumi)

Department of Electronics and Communication Engineering

PROGRAM OUTCOMES (POs)

- 1. Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- 2. Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- 3. Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- 4. Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- 5. Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- 6. The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- 7. Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- 8. Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- 9. Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- 10. Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.



AJ INSTITUTE OF ENGINEERING & TECHNOLOGY

A Unit of Laxmi Memorial Education Trust ®

(Approved by AICTE, New Delhi, Affiliated to Visvesvaraya Technological University, Belgaavi)

Department of Electronics and Communication Engineering

- 11. Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- 12. Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

- PSO1. **Embedded Systems:** Ability to apply the fundamental knowledge of core Electronics and Communication Engineering courses in the analysis, design, and development of integrated electronic systems and healthcare devices.
- PSO2. **Communication Systems:** Ability to apply the fundamental knowledge of signal processing in the analysis, design, and development of communication systems.
- PSO3. **Simulation:** Ability to use modern electronic tools such as MATLAB, Xilinx, Multisim etc, to design and analyze the complex electronics and communication systems.



AJ INSTITUTE OF ENGINEERING & TECHNOLOGY

A Unit of Laxmi Memorial Education Trust ®

(Approved by AICTE, New Delhi, Affiliated to Visvesvaraya Technological University, Belga))

Department of Electronics and Communication Engineering

COURSE OBJECTIVES: This course will enable students to:

Sl. No.	DESCRIPTION
1	To impart the concepts of simplifying Boolean expression using K-map techniques and Quine- McCluskey minimization techniques.
2	To impart the concepts of designing and analysing combinational logic circuits.
3	To impart design methods and analysis of sequential logic circuits.
4	To impart the concepts of Verilog HDL-data flow models for the design of digital systems.
5	To impart the concepts of Verilog HDL- behavioural models for the design of digital systems.

COURSE OUTCOMES (COs): After studying this course, students will be able to:

Sl. No.	DESCRIPTION	PO MAPPING	PSO MAPPING
CO1	Simplify Boolean functions using K-map and Quine-McCluskey minimization technique.	PO1, PO2,PO3, PO5	PSO1,PSO3
CO2	Analyse and design for combinational logic circuits.	PO1, PO2, PO3,PO5	PSO1, PSO3
CO3	Analyse the concepts of Flip Flops (SR, D, T and JK) and to design the synchronous sequential circuits using Flip Flops.	PO1, PO2, PO3,PO5	PSO1,PSO3
CO4	Combinational circuits (adders, subtractors, multiplexers) using Verilog descriptions.	PO1, PO2, PO3,PO5	PSO1,PSO3
CO5	Concepts of Flip Flops (SR, D, T and JK) and to design the synchronous sequential circuits using Verilog descriptions.	PO1, PO2, PO3,PO5	PSO1,PSO3



AJ INSTITUTE OF ENGINEERING & TECHNOLOGY

A Unit of Laxmi Memorial Education Trust ®

(Approved by AICTE, New Delhi, Affiliated to Visvesvaraya Technological University, Belga))

Department of Electronics and Communication Engineering

	PRACTICAL COMPONENT OF IPCC		
1	To simplify the given Boolean expressions and realize using Verilog program.	L3	CO1, CO4
2	To realize Adder/Subtractor (Full/half) circuits using Verilog data flow description.	L3	CO2, CO4
3	To realize 4-bit ALU using Verilog program.	L3	CO2, CO4
4	To realize the following Code converters using Verilog Behavioural description a) Gray to binary and vice versa b) Binary to excess3 and vice versa	L3	CO2, CO5
5	To realize using Verilog Behavioural description: 8:1 mux, 8:3 encoder, Priority encoder	L3	CO2, CO5
6	To realize using Verilog Behavioural description: 1:8 Demux, 3:8 decoder, 2-bit Comparator	L3	CO2, CO5
7	To realize using Verilog Behavioral description: Flip-flops: a) JK type b) SR type c) T type and d) D type	L3	CO3, CO5
8	To realize Counters - up/down (BCD and binary) using Verilog Behavioral description.	L3	CO3, CO5
9	Verilog Program to interface a Stepper motor to the FPGA/CPLD and rotate the motor in the specified direction (by N steps).	L3	CO4, CO5
10	Verilog programs to interface Switches and LEDs to the FPGA/CPLD and demonstrate its Working.	L3	CO4, CO5



A J INSTITUTE OF ENGINEERING AND TECHNOLOGY
MASTER MANUAL

DIGITAL SYSTEM DESIGN USING VERILOG LAB

SUBJECT CODE: BEC302

III-SEMESTER

DEPT. OF ELECTRONICS AND COMMUNICATION ENGINEERING

CONTENTS:

1. Department Vision and Mission, PO's, PEO's, PSO's
2. Syllabus
3. Course Information Sheet
4. Do's and Don't in Lab
5. Format for writing record and observation
6. Steps to Execute in XILINX Software
7. Lab Programs
8. Additional Programs
9. Viva Questions and Answers

DEPARTMENT VISION AND MISSION:**Vision**

To be recognized as a center of excellence in the area of Electronics and Communication Engineering by nurturing the young innovative minds into skillful and ethical professionals to cater the industrial and societal needs.

Mission Statements

M1: To establish state-of-the art laboratories to facilitate research and innovation to upgrade the knowledge and skills in healthcare sector and IoT.

M2: To provide industry interaction for training programs on latest technology

M3: To provide ethical and value based education by promoting activities addressing the societal needs.

PEOs	Description
PEO1	Exhibit a desire for lifelong learning through professional and societal activities.
PEO2	Exhibit and apply their technical skills and knowledge in Electronics and Communication Engineering for industry and societal needs
PEO3	Exhibit leadership qualities, professional skills, management skills and ethics needed for successful career.

S.No.	POs	Description
1	Engineering knowledge	Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.
2	Problem Analysis	Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural science and engineering sciences.
3	Design/development of solutions	Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal and environmental considerations.
4	Conduct investigations of complex problems	Use research based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

S.No.	POs	Description
5	Modern tool usage:	create, select and apply appropriate techniques, resources and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations
6	The engineer and society	Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice
7	Environment sustainability	Understand the impact of the professional engineering solutions in the societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8	Ethics	Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9	Individual and team work	Function effectively as an individual and as a member or leader in diverse teams, and in multidisciplinary settings.
10	Communication	communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions
11	Project management and finance	Demonstrate knowledge and understanding of the engineering and management principles and apply these to ones own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12	Lifelong learning	recognize the need for, and have the preparation and ability to engage in independent and lifelong learning in the broader context of technological change.

PSOs	Description
PSO1	Embedded Systems: Ability to apply the fundamental knowledge of core Electronics and Communication Engineering courses in the analysis, design, development of integrated electronic systems and healthcare devices.
PSO2	Communication Systems: Ability to apply the fundamental knowledge of signal processing in the analysis, design, and development of communication systems.
PSO3	Simulation: Ability to use modern electronic tools such as MATLAB, Xilinx, Multisim etc, to design and analyze the complex electronics and communication systems.

III SEMESTER ELECTRONICS AND COMMUNICATION ENGINEERING

Digital System Design using Verilog- Semester 3

Sub Code	BEC302	CIE Marks	50
Teaching Hours/Week (L:T:P: S)	3:0:2	SEE Marks	50
Total Hours of Pedagogy	40 hours Theory + 8-10 Lab slots	Total Marks	100
Credits	04	Exam Hours	03 Hrs

Course Objectives: This course will enable students to:

1. To impart the concepts of simplifying Boolean expression using K-map techniques and Quine-McCluskey minimization techniques.
2. To impart the concepts of designing and analyzing combinational logic circuits.
3. To impart design methods and analysis of sequential logic circuits.
4. To impart the concepts of Verilog HDL-data flow and behavioral models for the design of digital systems.

Note: Programming can be done using any compiler. Download the programs on a

FPGA/CPLD board and performance testing may be done using 32 channel pattern generator and logic analyzer apart from verification by simulation with tools such as Altera/Modelsim or equivalent.

Course Syllabus

PRACTICAL COMPONENT OF IPCC

Using suitable simulation software, demonstrate the operation of the following circuits:

Sl. No Experiments

1. To simplify the given Boolean expressions and realize using the Verilog program.
2. To realize Adder/Subtractor (Full/half) circuits using Verilog data flow description.
3. To realize 4-bit ALU using the Verilog program.
4. To realize the following Code converters using Verilog Behavioral description
 - a) Gray to binary and vice versa b) Binary to excess3 and vice versa
5. To realize using Verilog Behavioral description: 8:1 mux, 8:3 encoder, Priority encoder
6. To realize using Verilog Behavioral description: 1:8 Demux, 3:8 decoder, 2-bit Comparator
7. To realize using Verilog Behavioral description: Flip-flops: a) JK type b) SR type c) T type and d) D type
8. To realize Counters - up/down (BCD and binary) using Verilog Behavioral description.

Demonstration Experiments (For CIE only – not to be included for SEE)

Use FPGA/CPLD kits for downloading Verilog codes and check the output for interfacing experiments.

9. Verilog Program to interface a Stepper motor to the FPGA/CPLD and rotate the motor in the specified direction (by N steps).
10. Verilog programs to interface Switches and LEDs to the FPGA/CPLD and demonstrate its working.

Course Outcomes:

At the end of the course the student will be able to:

1. Simplify Boolean functions using K-map and Quine-McCluskey minimization technique.
2. Analyze and design for combinational logic circuits.
3. Analyze the concepts of Flip Flops (SR, D, T and JK) and to design the synchronous sequential circuits using Flip Flops.
4. Model Combinational circuits (adders, subtractors, multiplexers) and sequential circuits using Verilog descriptions.

Conduct of Practical Examination:

1. All laboratory experiments are to be included for practical examination.
2. Students are allowed to pick one experiment from the lot.
3. Strictly follow the instructions as printed on the cover page of answer script for breakup of marks.
4. Change of experiment is allowed only once and Marks allotted to the procedure part to be made zero.

DO'S:-

1. Maintain silence and discipline.
2. Before entering, leave the footwear outside the lab.
3. Proper dress code has to be maintained while entering in the Lab.
4. Students should carry lab observation book and record book completed in all aspects.
5. Read and understand logic of the program thoroughly before coming to the laboratory.
6. Enter in the login register before switching on the computers.
7. Use duly filled component issue slips, entering details such as batch names, Component specifications for obtaining components from lab store
8. Students should be at their concerned table; unnecessary movement is restricted.
9. Students should maintain same computer until end of the semester.
10. Report any problems in computers/hardware kits to the faculty member/laboratory technician immediately.
11. Note down the experimental results legibly in the Observation book and get the same verified & signed by the Faculty.
12. After completing the experiments, students should return the hardware kits, switch off the computers, enter logout time and arrange chairs neatly.

DON'Ts:-

1. Do not come late to the Lab.
2. Don't enter the lab without valid ID card.

3. Do not leave the lab without the permission of the Faculty In-Charge.
4. Never eat, drink while working in the laboratory.
5. Do not handle any equipment before reading the Instructions/Instruction manuals.
6. Do not exchange the computers
7. Do not misbehave in the laboratory.
8. Do not alter computer settings/software settings and pen drives should not be connected to computers without permission. Doing so will attract fines.
9. Do not remove anything from the kits/experimental set up without permission.

LIST OF EXPERIMENTS

		Page No:
1	1 To simplify the given Boolean expressions and realize using Verilog program.	16-19
2	To realize Adder/Subtractor (Full/half) circuits using Verilog data flow description.	20-30
3	To realize 4-bit ALU using Verilog program.	31-32
4	To realize the following Code converters using Verilog Behavioral description a) Gray to binary and vice versa b) Binary to excess3 and vice versa	33-43
5	To realize using Verilog Behavioral description: 8:1 mux, 8:3 encoder, Priority encoder	44-57
6	To realize using Verilog Behavioral description: 1:8 Demux, 3:8 decoder, 2-bit Comparator	57-65
7	To realize using Verilog Behavioral description: Flip-flops: a) JK type b) SR type c) T type and d) D type	66-77
8	To realize Counters - up/down (BCD and binary) using Verilog Behavioral description.	78-81
9	Verilog Program to interface a Stepper motor to the FPGA/CPLD and rotate the motor in the specified direction (by N steps).	82-83
10	Verilog programs to interface Switches and LEDs to the FPGA/CPLD and demonstrate its working.	84

SCHEME OF EVALUATION

IA EVALUATION				
Continuous evaluation (15)	Design (3M)	Execution (4M)	Viva (3M)	Record (5M)
Exam (10)	Final exam conducted for 50 marks and scale down to 10 marks			

INTRODUCTION:

Verilog is a hardware description language (HDL) used to model electronic systems. The language supports the design, verification, and implementation of analog, digital, and mixed - signal circuits at various levels of abstraction. HDL is an essential computer aided design (CAD) tool for the modern design and synthesis of digital system. Due to the increase in the power and complexity of digital systems, such systems cannot be easily realized using discrete Integrated Circuits (ICs).

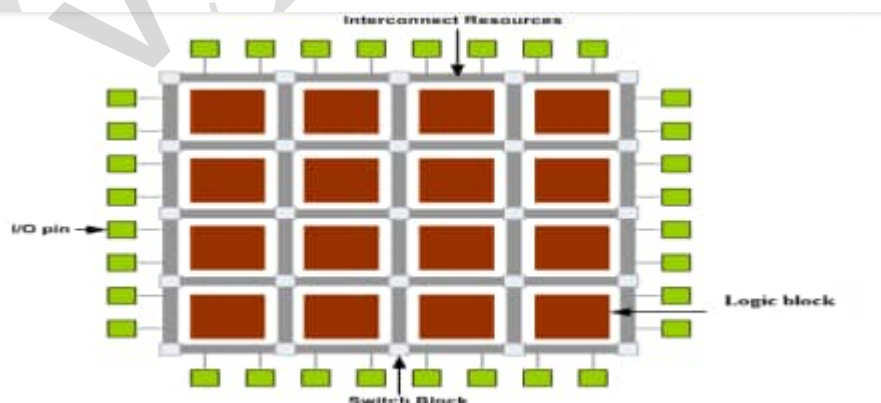
These systems are usually realized using high density programmable chips such as Application Specific Integrated Circuits (ASICs) and Field Programmable Gate Arrays (FPGAs) and required CAD tools.

HDL offers the designer a very efficient tool for implementing and synthesizing designs on chips. The designer uses HDL to describe the system in a computer language code that is similar to several commonly used software languages such as C. Debugging the design is easy because HDL packages implement simulators and test benches.

INTRODUCTION TO FPGA (FIELD PROGRAMMABLE GATE ARRAY)

Field-Programmable Gate Arrays (FPGAs) are integrated circuits that can be configured by the user after manufacturing. They consist of an array of programmable logic blocks (PLBs), interconnected by programmable routing resources. FPGAs offer high flexibility and reconfigurability, making them suitable for a wide range of applications in various industries. Here's the theory behind FPGA technology:

Basic Structure:



Programmable Logic Blocks (PLBs): These are the fundamental building blocks of an FPGA. PLBs typically contain a combination of configurable logic gates, flip-flops, and other elements. Users can program the functionality of PLBs to implement desired logic functions.

Programmable Routing Resources: FPGAs feature a network of programmable interconnects that connect PLBs and other components. The routing resources enable users to establish connections between different logic blocks based on their design requirements.

Input/Output Blocks (IOBs): These blocks provide interfaces for connecting external devices and circuits to the FPGA. IOBs support various standards and protocols, including GPIO (General Purpose Input/Output), LVDS (Low Voltage Differential Signaling), and high-speed serial interfaces like PCIe (Peripheral Component Interconnect Express).

Configuration:

Configuration Memory: FPGAs store configuration data in internal memory elements, such as SRAM (Static Random-Access Memory), flash memory, or anti-fuse elements. Configuration data specifies the functionality of each PLB and the interconnections between them.

Configuration Process: Users configure FPGAs using specialized software tools called Electronic Design Automation (EDA) tools. The design process typically involves writing a hardware description language (HDL) code, such as Verilog or VHDL, synthesizing the code to generate a netlist, and then programming the FPGA with the generated bitstream through a programming interface (such as JTAG or USB).

Flexibility and Reconfigurability:

FPGAs offer high flexibility and reconfigurability compared to fixed-function ASICs (Application-Specific Integrated Circuits). Users can modify the functionality of FPGAs by updating the configuration data, allowing for rapid prototyping, iterative design, and in-field updates.

Partial Reconfiguration: Some advanced FPGAs support partial reconfiguration, where specific regions of the device can be reconfigured dynamically while the rest of the device remains operational. This feature enables dynamic adaptation to changing requirements and runtime optimization.

Applications:

FPGAs are used in a wide range of applications across industries, including telecommunications, automotive, aerospace, consumer electronics, industrial automation, and more.

Common applications include digital signal processing (DSP), embedded systems, real-time processing, high-speed data acquisition, cryptography, machine learning acceleration, and prototyping complex digital systems.

Advantages:

Rapid Prototyping: FPGAs enable rapid prototyping and iterative design cycles, allowing engineers to quickly evaluate and refine hardware designs.

Customization: FPGAs offer the ability to customize hardware functionality to meet specific application requirements, providing a competitive advantage in many industries.

Time-to-Market: FPGAs can shorten time-to-market for new products by facilitating faster development cycles and allowing for in-field updates and enhancements.

Challenges:

Complexity: Designing with FPGAs requires expertise in hardware description languages (HDLs), digital design principles, and FPGA architecture. Complex designs may also require careful consideration of timing constraints and resource utilization.

Resource Constraints: FPGAs have limited resources in terms of logic cells, memory, and I/O pins. Optimizing designs to fit within these constraints while meeting performance requirements can be challenging.

Power Consumption: FPGAs typically consume more power than ASICs or microcontrollers due to their programmable nature and additional overhead associated with configuration and reconfiguration.

Overall, FPGAs offer a versatile platform for implementing custom digital hardware designs, enabling innovation and flexibility in various application domains. Their combination of flexibility, reconfigurability, and performance makes them a valuable tool for engineers and developers seeking to implement complex digital systems.

INTRODUCTION TO XILINX ISE

Xilinx ISE (Integrated Synthesis Environment) is a Computer Aided Design(CAD) tool used for synthesis and analysis of HDL designs, enabling the developer to synthesize (compile) their designs, perform timing analysis and configure the target device with the programmer. This includes intelligent clock gating for dynamic power reduction, design preservation for timing repeatability and a partial reconfiguration option for greater system flexibility, size, power and cost reduction. Xilinx ISE is a design environment for FPGA products and tightly coupled to the architecture of such chips

Xilinx logic block consists of one Look Up Table (LUT) and one Flip-flop. An LUT is used to implement number of different functionality. The input lines to the logic block go into the LUT and enable it. The output of the LUT gives the result of the logic function that it implements and the output of logic block is registered or unregistered output from the LUT.

PROCEDURE:

The Procedure to be followed for Software and Hardware Programs are as follows:

Step 1: Go to Start Menu All Programs Xilinx ISE 13.1i and Select Project Navigator.

Step 2: Go to File Menu and select Close project to close previously opened project if any, and then Select New Project.

Step 3: Enter the Project name and location and Select the Top level module type as HDL.

Step 4: Select the Device family and Device name as Spartan-6 , pin density TQG144, -3 for FPGA.

Step 5: Right click on the source file and select new source followed by Verilog module and Give the file name same as the name of the entity.

Step 6: Define the ports used and their respective directions in the next window that opens.

Step 7: Write the architecture body and the generics etc. in the incomplete Verilog code that opens and save the file after completion of editing.

Step 8: Go to the Process view window and right click on the Synthesize - XST and Select Run. Correct the errors if any.

Step 9: Select and Right click the source file and click on the New Source tab and then select the Test Bench Waveform and give the appropriate file name for the same.

Step 10: Make the alterations in the Clock information and initial length of the test bench if needed.

Step 11: Set or Reset the inputs as required and save the test bench waveform file.

Step 12: Go to Process view and under Xilinx ISE Simulator Right click on the Simulate Behavioral model to see the output for the input conditions for Simulation.

For Interfacing:

Step 13: Make the appropriate connections between the PC and the FPGA kit for the observation of outputs in the FPGA kit and for other Hardware Programming.

Step 14: Select and Right click the source file and click on the New Source tab and then select the Implementation Constraints file and give the appropriate file name for the same.

Step 15: Go to Process view and under User Constraints, double click on the Edit Constraints (Text).

Step 16: Write the code for the user constraints file as required and save the same.

Step 17: Select the main source file and right click on the Implement design in the process view window and select run.

Step 18: Right click on the Generate Programming file in the process view window and select run.

Step 19: Under the Generate Programming file tab, right click on the Configure device (Impact) and click on the Run option.

Step 20: Select the appropriate mode and make changes in the jumper settings of the FPGA Kit as required, select the appropriate.BIT extension file in the pop up window.

Step 21: Right click on the Chip picture in the pop up window and Select “Program”. Debug the errors if it is there. Set the conditions for the inputs using Dip switch and observe the outputs.

FORMAT FOR DIGITAL SYSTEM DESIGN USING VERILOG LAB RECORD BOOK

To be written in the lab record book and presented while entering the lab

Right hand side of the lab record

1. Aim
2. Software/Hardware Requirements
3. Algorithm
4. Verilog Program with comments
5. Inference

Left hand side of the lab record

1. Block Diagram
2. Truth Table
3. Logical Expressions
4. Logic diagram
5. UCF pin details
6. Observation
7. Simulated Waveform

FORMAT FOR DSDV OBSERVATION BOOK

To be written in the observation book and presented while entering the lab

1. Aim
2. Software/Hardware Requirements
3. Block Diagram
4. Truth Table
5. Logical Expressions
6. Logic diagram
7. Verilog Program with comments
8. Expected Waveform

To be signed by faculty present in the Observation Book with Date

Experiment No 1:

1. Write Verilog program to simplify the given Boolean expressions and realize using Verilog program

Objective: To write and simulate the HDL code to realize the given Boolean expressions and realize using Verilog program by simulating the Xilinx ISE Simulator.

Software Required: Xilinx ISE Software

Theory:

A boolean expression is a mathematical expression composed of variables, constants, and logical operators (such as AND, OR, NOT) that evaluates to either true or false. Boolean expressions are fundamental in computer science and are extensively used in digital logic design, programming, and algorithm development.

Algorithm:

STEP 1: Define the module for design block and declare the Ports as inputs and outputs .

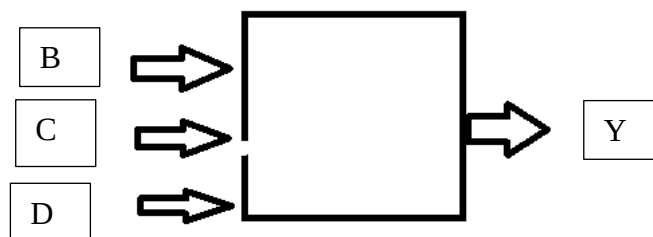
STEP 2: Describe the relationship between the inputs and outputs looking into the expression.

STEP 3: Using step , design the code using Data flow Modelling.

STEP 4: Define the test values for inputs.

Block Diagram:

Boolean expression. $Y = BC + BD$



Boolean expression. $Y = BC + BD$

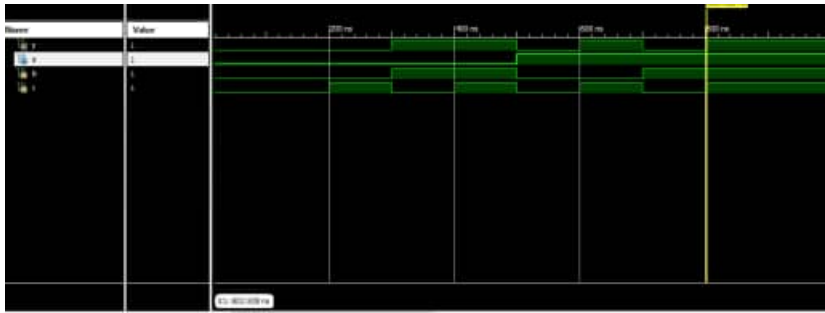
$$Y = B(C + D)$$

Verilog Design code:

```
module be1(Y, B, C, D);  
  
output Y;  
  
input B, C, D;  
  
assign Y = (B & C) | (B & D);  
  
endmodule
```

Verilog testbench code:

```
module be1_tb;  
  
wire Y; reg B,C,D;  
  
be1 uut (.B(B),.C(C),.D(D),.Y(Y));  
  
initial begin  
  
$display("\t\t Boolean Expression1");  
  
B=0;C=0;D=0;#100;  
  
B=0;C=0;D=1;#100;  
  
B=0;C=1;D=0;#100;  
  
B=0;C=1;D=1;#100;  
  
B=1;C=0;D=0;#100;  
  
B=1;C=0;D=1;#100;  
  
B=1;C=1;D=0;#100;  
  
B=1;C=1;D=  
  
end  
  
endmodule
```

Output Waveform:**Example 2:****Boolean Expression: $(ab)+(bc)$** **Design Code:**

```
module exp2(input a,input b,input c,output y);
assign y= (a & b) | (b & c);
endmodule
```

Stimulus Block:

```
module exp3_tb;
// Inputs
reg a;reg b;reg c;
// Outputs
wire y;
// Instantiate the Unit Under Test (UUT)
exp2 uut (.a(a),.b(b),.c(c),.y(y));
initial
begin
// Initialize Inputs
a = 0;b = 0;c = 0;// Wait 100 ns for global reset to finish
a = 0;b = 1;c = 0; :#100; Wait 100 ns for global reset to finish
a = 1;b = 0;c = 0; :#100;// // Wait 100 ns for global reset to finish
```

```
a = 1;b = 1;c = 0; #100;// Wait 100 ns for global reset to finish
```

```
$finish;
```

```
end
```

```
endmodule
```

Result:



Result: The verilog Code for Boolean Expression is designed and simulated.

Experiment No 2:

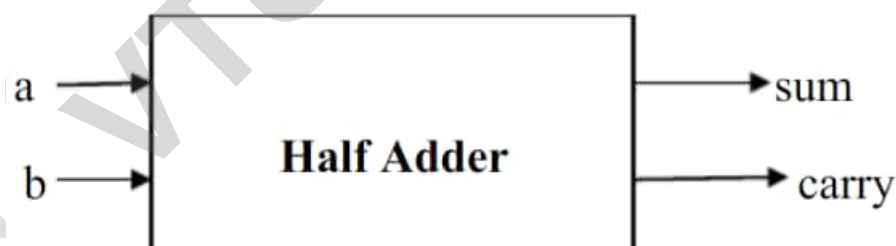
To realize Adder/Subtractor (Full/half) circuits using Verilog data flow description.

OBJECTIVE: To write and simulate the HDL code to realize Adder/Subtractor (Full/half) circuits using Verilog data flow description.

SOFTWARE REQUIRED: Xilinx ISE Software

2.a. HALF ADDER**THEORY:**

- A half adder is a basic digital circuit used to perform addition of two single binary digits. It has two inputs, A and B, representing the two bits to be added, and two outputs, the sum (S) and the carry (C). A and B are the input bits.
- The AND gate takes both A and B as inputs. It produces an output S, which is the sum bit.
- The output S is the result of the bitwise AND operation between A and B.
- The OR gate takes both A and B as inputs. It produces an output C, which is the carry bit. The output C is the result of the bitwise OR operation between A and B.

BLOCK DIAGRAM:**TRUTH TABLE:**

Inputs		Outputs	
a	b	sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

LOGIC EXPRESSION:

$\text{sum} = a \oplus b$

$\text{carry} = a \cdot b$

ALGORITHM:

STEP 1: Define the module for design block and declare the Ports as inputs and outputs .

STEP 2: Define Output as registers for the circuit.

STEP 3: Describe the relationship between the inputs and outputs looking into the truth table.

STEP 4: Design the code using data flow Modelling.

STEP 5: Define the test values for inputs.

VERILOG CODE:**Design Code:****Half Adder :**

```
module half_adder(input a,b, output sum,carry);
    assign sum = a ^ b; // simply using the xor and AND operator for sum and carry
    assign carry = a & b;
endmodule
```

Verilog testbench code:

```
module half_adder_tb(); //testbench module declaration
    reg a, b; //input as reg and output as wire
    wire sum, carry;
    integer i;
    half_adder DUT(a,b,sum,carry); //instantiation of the top module
    //initialization of the inputs
```

```

initial

begin

a = 0;

b = 0;

end

//stimulus generation

initial

begin

for (i = 0; i<4; i = i+1) //using for loop to generate all the four input combinations for a and b

begin {a,b} = i; #10; //using delay to advance the simulation

end

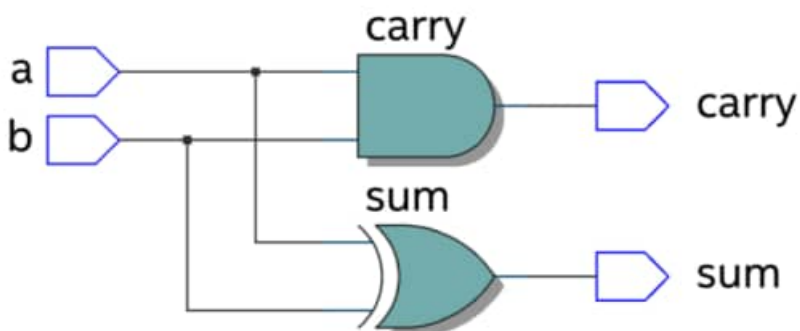
$finish;

endmodule

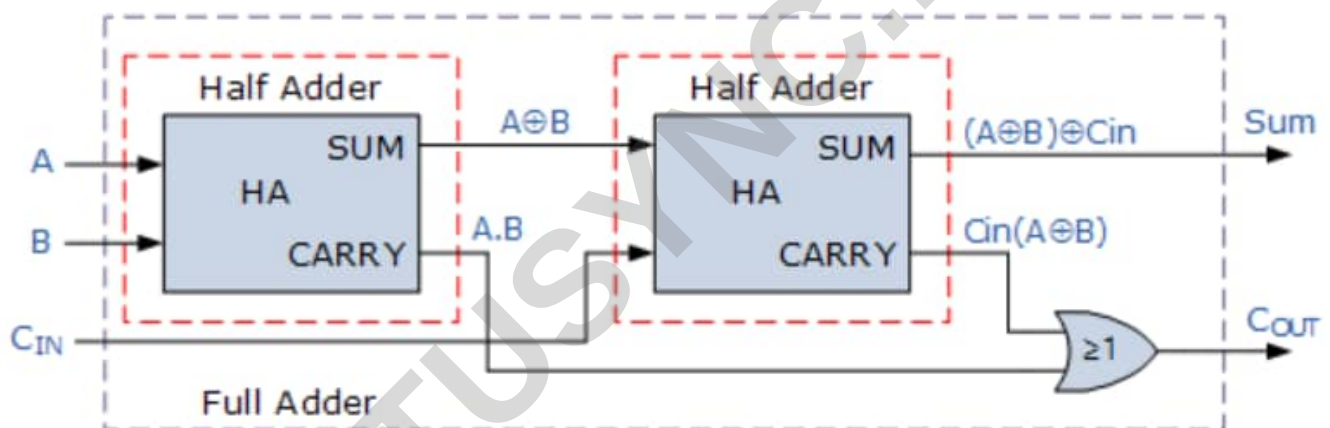
```

Simulation Waveform



RTL Schematic:

2.b) Verilog Code for Full Adder using two Half Adder and one OR gate.

**Design Code**

Full Adder

```

module full_adder(input a, b, c, output sum, carry);

    wire w1, w2, w3; // wire declaration for intermediate connections

    half_adder dut1(a, b, w1, w2); //instantiating the above half adder module 2 times as we need 2 HA.

    half_adder dut2(w1, c, sum, w3);

    or dut3(carry, w2, w3); //using or primitive for generating the carry output

endmodule

```

Simulation Code

```

module full_adder_tb();

reg a,b,c;

wire sum,carry;

integer i;

full_adder dut(a, b, c, sum, carry);

initial

begin

a = 1'b0; b = 1'b0; c = 1'b0;

end

initial

begin

for (i=0;i<8;i=i+1) //using for loop to generate all the 8 input combinations for a and b & c

begin

{a,b,c}=i;

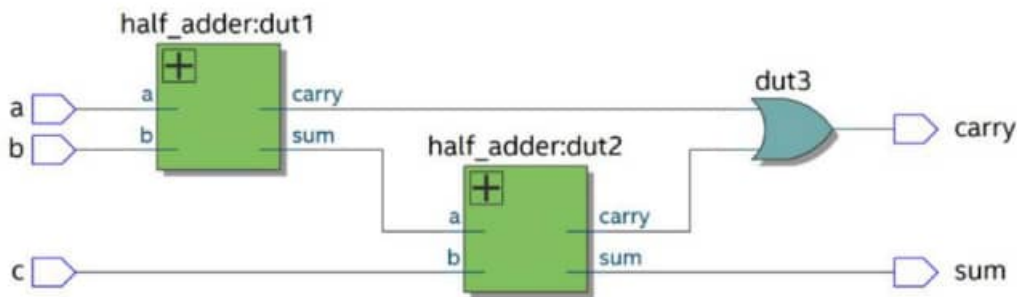
end

endmodule

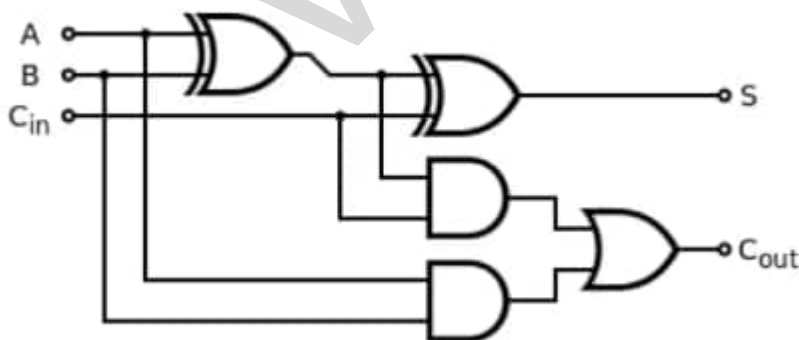
```

Simulation Waveform



RTL Schematic:**2.c) Verilog Code for 1 bit Full Adder using dataflow abstraction.**

THEORY: A full adder is a digital circuit used to add three input bits: A, B, and a carry-in (C_{in}). It produces two output bits: a sum (S) and a carry-out (C_{out}). The full adder is a crucial component in arithmetic logic units (ALUs) and other digital systems where addition is required. Full adders are fundamental components in digital arithmetic circuits, such as adders, subtractors, multipliers, and other complex arithmetic units. They are also used in various other digital systems where addition operations are required.



Inputs			Outputs	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

LOGIC EXPRESSION:

- $S = A \text{ XOR } B \text{ XOR } C_{in}$
- $C_{out} = (A \text{ AND } B) \text{ OR } (C_{in} \text{ AND } (A \text{ XOR } B))$

Design Codes

```

module full_adder1(input a, b, c, output sum, carry);
assign sum = a^b^c; // simply using the boolean equations of sum and carry and assigning them to output.
assign carry = (a&b)|(b&c)|(c&a);
endmodule

```

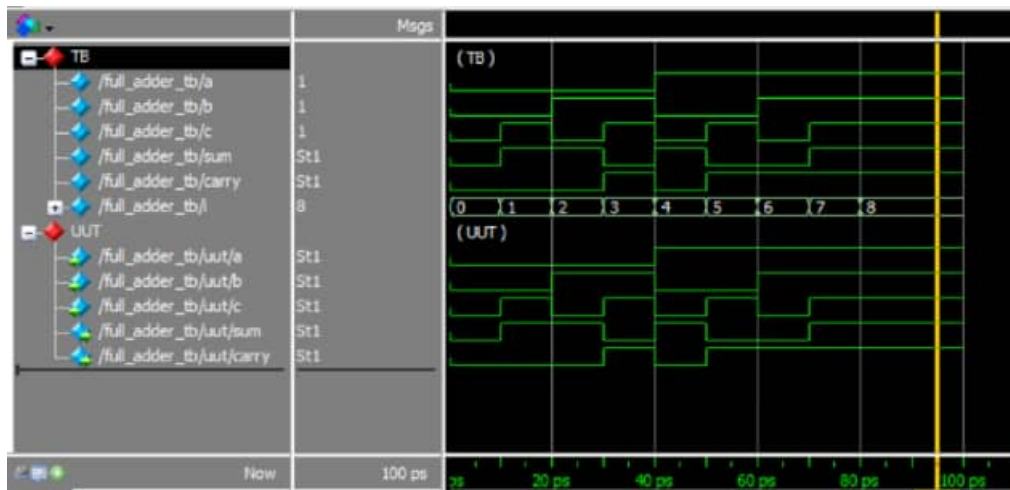
Simulation Code

```

module full_adder_tb();
reg a,b,c;
wire sum,carry;
integer i;
full_adder1 uut(a, b, c, sum, carry);
initial
begin
a = 1'b0;
b = 1'b0;
c = 1'b0;
end
initial
begin
for (i=0;i<8;i=i+1)
begin
{a,b,c}=i;
#10;
end
end
initial
$monitor("Input a=%b, b=%b, c=%b, Output sum =%b, carry=%b",a,b,c,sum,carry);
Initial
#100 $finish;
endmodule

```

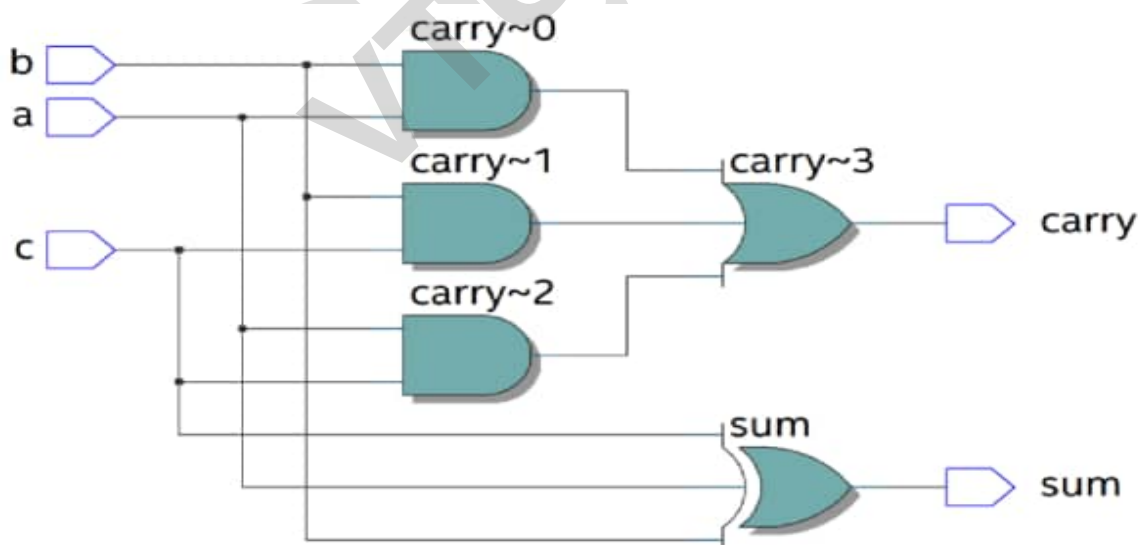
Simulation Waveform



Transcript

```
# Input a=0, b=0, c=0, Output sum =0, carry=0
# Input a=0, b=0, c=1, Output sum =1, carry=0
# Input a=0, b=1, c=0, Output sum =1, carry=0
# Input a=0, b=1, c=1, Output sum =0, carry=1
# Input a=1, b=0, c=0, Output sum =1, carry=0
# Input a=1, b=0, c=1, Output sum =0, carry=1
# Input a=1, b=1, c=0, Output sum =0, carry=1
# Input a=1, b=1, c=1, Output sum =1, carry=1
```

Synthesis Circuit



2.d: Half Subtractor:

Theory:

A half subtractor is a combinational logic circuit used to subtract one binary digit from another, producing a difference and a borrow output. Unlike a full subtractor, which takes into account a borrow input from the previous stage, a half subtractor only considers the two input bits being subtracted.

TRUTH TABLE:

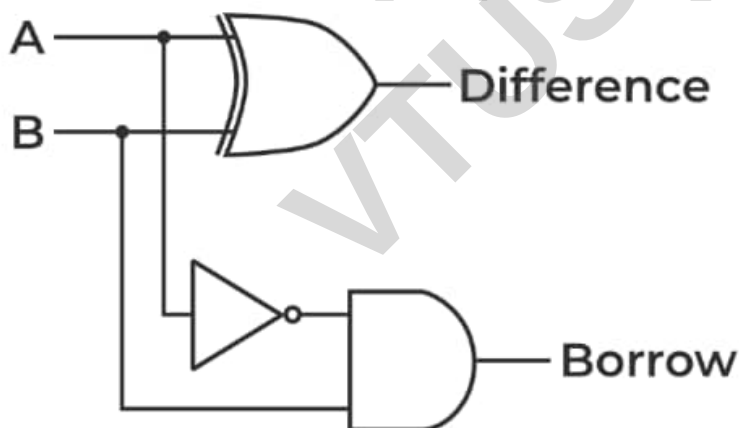
A	B	D	B ₀
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Logic Expression:

$$D = \bar{A}.B + A.\bar{B}$$

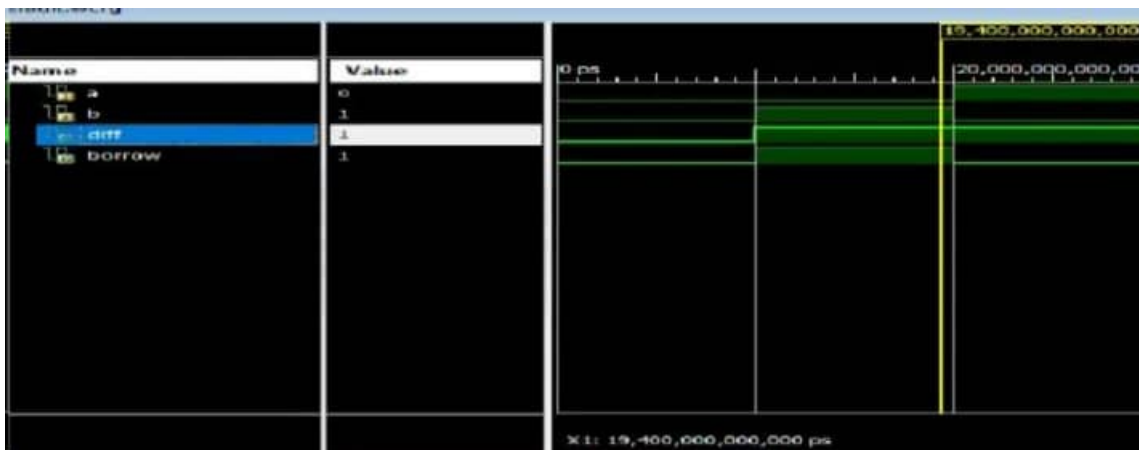
$$B_0 = \bar{A}.B$$

LOGIC DIAGRAM

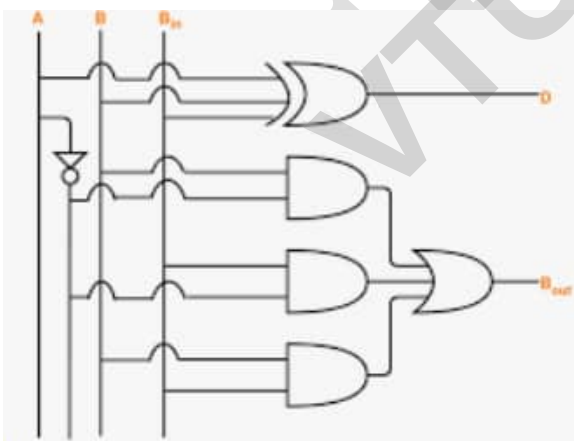


Design Code:

```
module half_subtractor(a,b,diff,borrow);
input a,b;
output diff,borrow;
assign diff=a^b;
assign borrow=~a&b;
endmodule
```

Simulation Output:**2.e Full subtractor**

Theory: A full subtractor is a combinational logic circuit used to subtract one binary digit from another, taking into account both the two input bits to be subtracted and a borrow input from the previous stage. It produces a difference output and a borrow output, just like a half subtractor, but it also considers the borrow input. Full subtractors are used in binary arithmetic circuits, particularly in the design of subtractors, arithmetic logic units (ALUs), and other digital systems where subtraction operations are required. They serve as fundamental building blocks for more complex subtractors and arithmetic units, allowing for efficient subtraction operations in digital systems.

LOGIC DIAGRAM & TRUTHTABLE:

Inputs			Outputs	
A	B	Borrow _{in}	Diff	Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Design Code:

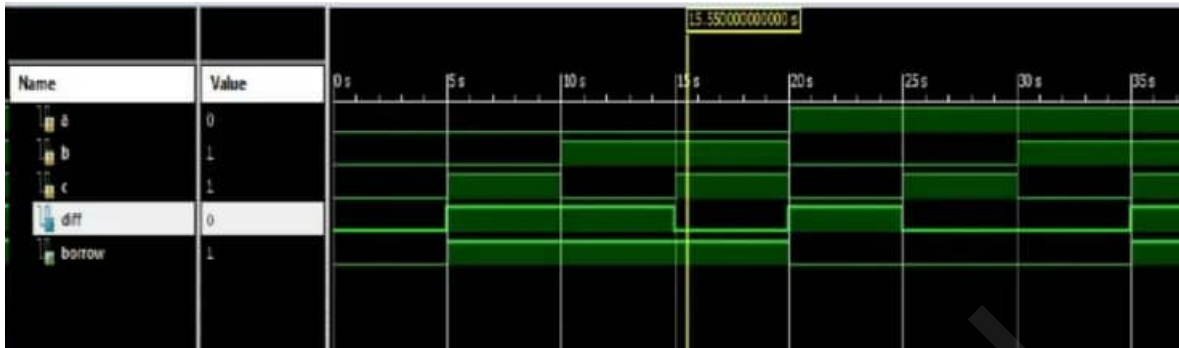
```
module full_subtractor(a,b,c,diff,borrow);
input a,b,c;
```

```

output diff,borrow;
assign diff=(a&~b&~c)|(~a&~b&c)|(~a&b&~c)|(a&b&c);
assign borrow=(~a&c)|(~a&b)|(b&c);
endmodule

```

Simulated Output:



Result: Full/Half Adders and Subtractors are designed using Verilog code and Simulated and verified according to the truth table.

Applications:

Full subtractors are used in digital arithmetic circuits for performing subtraction operations in binary arithmetic. They are essential components in subtracting binary numbers in microprocessors, digital calculators, and other digital computing systems. Arithmetic Logic Units (ALUs) in digital processors and microcontrollers use full subtractors as part of their subtractor circuits. Full subtractors can be used in error detection and correction circuits.

Experiment No 3:

To realize 4-bit ALU using Verilog program.

OBJECTIVE:

To write and simulate the 4-bit ALU considering the A [3:0] and B [3:0] as two inputs and to check the functionality by simulating the design.

Theory: - An Arithmetic Logic Unit (ALU) is a crucial component of a central processing unit (CPU) in a computer. It performs arithmetic and logic operations on binary data, executing instructions provided by the CPU. It can be implemented by using some control inputs to realize all possible arithmetic and logical operations in a single model .

ALGORITHM:

STEP 1: Define the module for design block and declare the Ports as input : A and B as vector [3:0] ,Sel as vector [2:0] and output as vector [7:0] Result.

STEP 2: Describe the relationship between the inputs and outputs using Truth table.

STEP 3: Using step 4, design the code using behavioral modelling.

STEP 4: Define the module for stimulus block and declare inputs as registers and outputs as wires.

STEP 5: Instantiate the design module using connection by order.

STEP 6: Define the test values for inputs.

Truth table:

Select inputs			Output Operation
Sel[2]	Sel[1]	Sel[0]	Y
0	0	0	a+b
0	0	1	a-b
0	1	0	a*b
0	1	1	a/b
1	0	0	~a
1	0	1	a&b
1	1	0	a^b
1	1	1	a==b

Design Block/ Verilog Code:

```

module alu4_bit(a,b,y,sel);
input [3:0]a,b;
input [2:0]sel;
output [7:0]y;
reg [7:0]y;
always@ (a,b,sel)
begin
case(sel)
3'b000:y=a+b;//Arithmetic ADD
3'b001:y=a-b;//
3'b010:y=a*b;
3'b011:y=a|b;//b OR
3'b100:y=~a;//L
3'b101:y=a&b;
3'b110:y=a^b;
3'b111:y=a==b;
endcase
end
endmodule

```

Simulated Output:

Result: The design for 4 bit ALU is done using Verilog Code, and is simulated and executed.

APPLICATIONS : Arithmetic Logic Unit is used extensively for Signal Processing and Control. It can be used in many applications involving arithmetic operations. Many DSP and control applications require a small subset of arithmetic operations that must be computed efficiently.

Experiment No 4

To realize the following Code converters using Verilog Behavioral description:

- a) Gray to binary and vice versa

Theory:

The Gray code, also known as the reflected binary code, is a binary numeral system where two successive values differ in only one bit. The Gray code is often used in applications where it's essential to reduce the possibility of errors during transitions between consecutive values. Converting binary to Gray code involves transforming each binary digit into its corresponding Gray code digit.

Let B_0, B_1, B_2, B_3 be the bits representing the binary numbers, where B_0 is the LSB and B_3 is the MSB, and Let G_3, G_2, G_1, G_0 be the bits representing the gray code of the binary numbers, where G_0 is the LSB and G_3 is the MSB

The truth table for the conversion is-

Binary				Gray Code			
b_3	b_2	b_1	b_0	g_3	g_2	g_1	g_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

To find the corresponding digital circuit, we will use the K-Map technique for each of the gray code bits as output with all of the binary bits as input.

K-map

g0

		b1,b0			
		00	01	11	10
b3,b2	00	0	1	0	1
	01	0	1	0	1
	11	0	1	0	1
	10	0	1	0	1

g1

		b1,b0			
		00	01	11	10
b3,b2	00	0	0	1	1
	01	1	1	0	0
	11	1	1	0	0
	10	0	0	1	1

g2

		b1,b0			
		00	01	11	10
b3,b2	00	0	0	0	0
	01	1	1	1	1
	11	0	0	0	0
	10	1	1	1	1

g3

		b1,b0			
		00	01	11	10
b3,b2	00	0	0	0	0
	01	1	1	1	1
	11	0	0	0	0
	10	1	1	1	1

Simplified expression using K-Map:

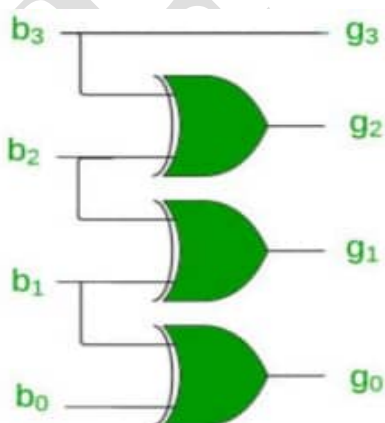
$$g_0 = b_0 b'_1 + b_1 b'_0 = b_0 \oplus b_1$$

$$g_1 = b_2 b'_1 + b_1 b'_2 = b_1 \oplus b_2$$

$$g_2 = b_2 b'_3 + b_3 b'_2 = b_2 \oplus b_3$$

$$g_3 = b_3$$

Logic Diagram:



Grey to binary conversion

TRUTH TABLE:

Gray Code				Binary			
g ₃	g ₂	g ₁	g ₀	b ₃	b ₂	b ₁	b ₀
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	1
1	0	0	0	1	1	1	1
1	0	0	1	1	1	1	0
1	0	1	0	1	1	0	0
1	0	1	1	1	1	0	1
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	1
1	1	1	0	1	0	1	1
1	1	1	1	1	0	1	0

b0**b1**

g ₃ ,g ₂	g ₁ ,g ₀			
	00	01	11	10
00	0	1	0	1
01	1	0	1	0
11	0	1	0	1
10	1	0	1	0

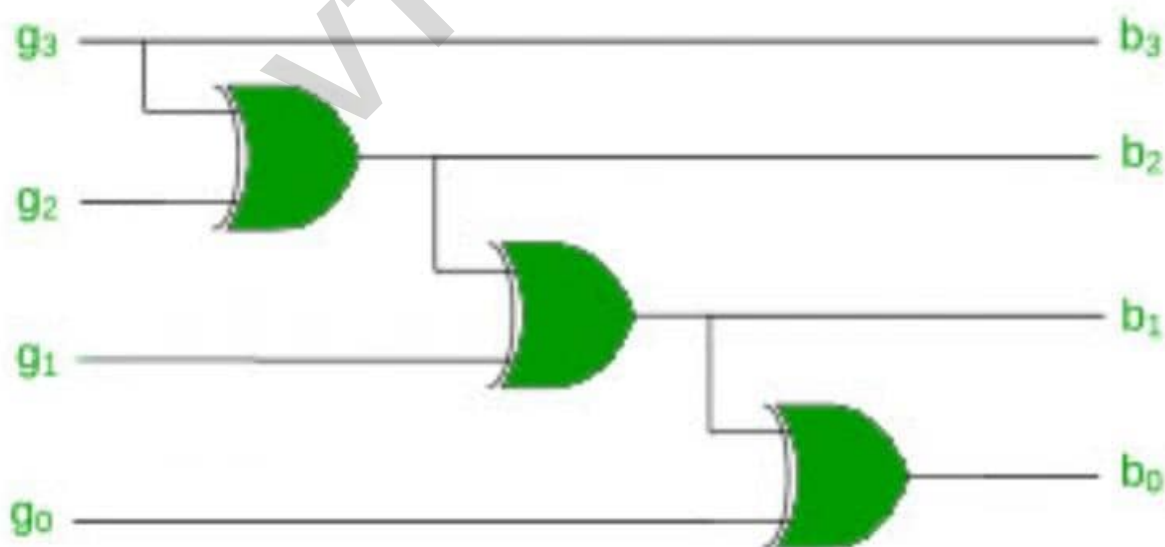
g ₃ ,g ₂	g ₁ ,g ₀			
	00	01	11	10
00	0	0	1	1
01	1	1	0	0
11	0	0	1	1
10	1	1	0	0

b2**b3**

		g_1, g_0			
		00	01	11	10
g_3, g_2	00	0	0	0	0
	01	1	1	1	1
	11	0	0	0	0
	10	1	1	1	1

		g_1, g_0			
		00	01	11	10
g_3, g_2	00	0	0	0	0
	01	0	0	0	0
	11	1	1	1	1
	10	1	1	1	1

$$\begin{aligned}
 b_0 &= g'_3 g'_2 g'_1 g_0 + g'_3 g'_2 g_1 g'_0 + g'_3 g_2 g'_1 g'_0 + g'_3 g_2 g_1 g_0 + g_3 g'_2 g'_1 g'_0 + g_3 g'_2 g_1 g_0 \\
 &\quad + g_3 g_2 g'_1 g_0 + g_3 g_2 g_1 g'_0 \\
 &= g'_3 g'_2 (g'_1 g_0 + g_1 g'_0) + g'_3 g_2 (g'_1 g'_0 + g_1 g_0) + g_3 g'_2 (g'_1 g'_0 + g_1 g_0) \\
 &\quad + g_3 g_2 (g'_1 g_0 + g_1 g'_0) \\
 &= g'_3 g'_2 (g_0 \oplus g_1) + g'_3 g_2 (g_0 \odot g_1) + g_3 g'_2 (g_0 \odot g_1) + g_3 g_2 (g_0 \oplus g_1) \\
 &= (g_0 \oplus g_1)(g_2 \odot g_3) + (g_0 \odot g_1)(g_2 \oplus g_3) \\
 &= g_3 \oplus g_2 \oplus g_1 \oplus g_0 \\
 b_1 &= g'_3 g'_2 g_1 + g'_3 g_2 g'_1 + g_3 g_2 g_1 + g_3 g'_2 g'_1 \\
 &= g'_3 (g'_2 g_1 + g_2 g'_1) + g_3 (g_2 g_1 + g'_2 g'_1) \\
 &= g'_3 (g_2 \oplus g_1) + g_3 (g_2 \odot g_1) \\
 &= g_3 \oplus g_2 \oplus g_1 \\
 b_2 &= g'_3 g_2 + g_3 g'_2 \\
 &= g_3 \oplus g_2 \\
 b_3 &= g_3
 \end{aligned}$$



Design Code:**1.a Binary to Gray**

```

module binarytogray (b3, b2, b1, b0, g3, g2, g1, g0);
input b3, b2, b1, b0;
output g3, g2, g1, g0;
reg g3, g2, g1, g0;
always @(b3, b2, b1, b0)
begin
g0=b1^b0;
g1=b2^b1;
g2=b3^b2;
g3=b3;
end
endmodule

```

1.b.Gray to Binary

```

module graytobinary (g3, g2, g1, g0, b3, b2, b1, b0);
input g3, g2, g1, g0;
output b3, b2, b1, b0;
reg b3, b2, b1, b0;
always @(g3, g2, g1, g0)
begin
b3=g3;
b2=b3^g2;
b1=b2^g1;
b0=b1^g0;
end
endmodule

```

//TEST BENCH for both the code

```

module bin_Gray_tb;

```

```

// Inputs

```

```

reg [2:0] b;

```

```

// Outputs

```

```

wire [2:0] g;

```

```

// Instantiate the Unit Under Test (UUT)

```

```

BIN_GRAY uut (
.g(g),
.b(b)

```

```

);
initial begin

// Initialize Inputs

b = 0;

// Wait 100 ns for global reset to finish

#100;

// Add stimulus here

b=3'b001;
#100;
b=3'b010;
#100;
b=3'b100;
#100;
end

endmodule

```

b) Binary to excess3 and vice versa

Truth Table

B3	B2	B1	B0	E3	E2	E1	E0
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0

1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	1	1	0	1
1	0	1	1	1	1	1	0
1	1	0	0	1	1	1	1
1	1	0	1	x	x	x	x
1	1	1	0	x	x	x	x

E3

$$E3 = B2(B0 + B1) + B3$$

E1

$$E1 = B1'B0' + B1B0$$

E2

$$E2 = B2'(B0 + B1) + B0'B1'B2$$

E0

From the truth table

$$E0 = B0'$$

Excess 3 to binary

Excess-3				BCD			
w	x	y	z	A	B	C	D
0	0	0	0	X	X	X	X
0	0	0	1	X	X	X	X
0	0	1	0	X	X	X	X
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	1
0	1	0	1	0	0	1	0
0	1	1	0	0	0	1	1
0	1	1	1	0	1	0	0
1	0	0	0	0	1	0	1
1	0	0	1	0	1	1	0
1	0	1	0	0	1	1	1
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	1
1	1	0	1	X	X	X	X
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X

K-Map for D:**C:.....****.B:.....A:**

wx \ yz	00	01	11	10
	00	01	11	10
00	X	X	0	X
01	1	0	0	1
11	1	X	X	X
10	1	0	0	1

wx \ yz	00	01	11	10
	00	01	11	10
00	X	X	0	X
01	0	1	0	1
11	0	X	X	X
10	0	1	0	1

wx \ yz	00	01	11	10
	00	01	11	10
00	X	X	0	X
01	0	0	1	0
11	0	X	X	X
10	1	1	0	1

wx \ yz	00	01	11	10
	00	01	11	10
00	X	X	0	X
01	0	0	0	0
11	1	X	X	X
10	0	0	1	0

LOGIC EXPRESSION:

$$A = wx + wyz$$

$$B = x'y' + x'z' + xyz$$

$$C = y'z + yz'$$

$$D = z'$$

B) BINARY TO EXCESS3**Design Code:**

```
module bin_to_excess3(input [3:0] b, output reg [3:0] ex3);
always@*ex3 = b + 3;
endmodule
```

Stimulus Block

```
module bin_to_excess3_tb;
// Inputsreg [3:0] b;
// Outputswire [3:0] ex3;
// Instantiate the Unit Under Test (UUT)
bin_to_excess3 uut (.b(b),.ex3(ex3));
initial
begin
b = 0;#100;
b = 1;#100;
b = 2;#100;
b = 3;#100;
b = 4;#100;
b = 5;#100;
b = 6;#100;
b = 7;#100;
b = 8;#100;
b = 9;#100;
b = 10;#100;
b = 11;#100;
b = 12;#100;
$finish;
end
endmodule
```

d)Excess3 to Binary

```
module ex3tobin(input [3:0] e,output reg[3:0] b);
always@*b = e - 3;
endmodule
```

Stimulus Block

```

module exc3tobin_tb;

reg [3:0] e;

// Outputs

wire [3:0] b;

// Instantiate the Unit Under Test (UUT)

exc3tobin uut (.e(e),.b(b));

initial begin

e = 3;#100;e = 4; #100;
e = 5;#100;e = 6;#100;
e = 7;#100;e = 8;#100;
e = 9;#100;e = 10;#100;
e = 11;#100;e = 12;#100;
e = 13;#100;e = 14;#100;
e = 15;#100;

$finish;

end

endmodule

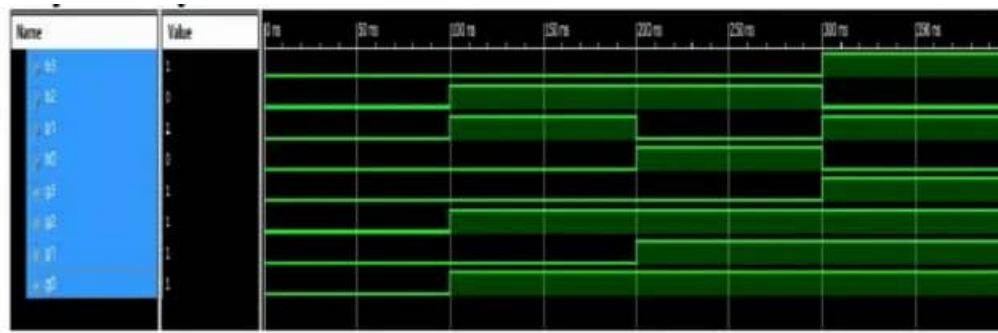
```

RESULT:

Binary to Gray



Gray to Binary



Binary to Excess -3



Excess -3 to Binary

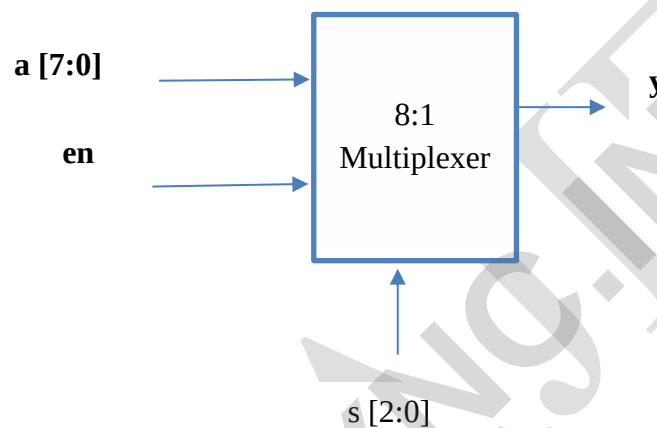


Result: The Verilog code for Code converters are designed, Simulated and executed and verified with the truth table.

APPLICATIONS: Gray codes are widely used to facilitate error correction in digital communications such as digital terrestrial television and some cable TV systems

Experiment No 5:**a. 8 to 1 multiplexer using case statement and if statements**

Theory : A multiplexer (or mux) is a device that selects between several analog or digital input signals and forwards it to a single output line. A multiplexer is also called a data selector. A multiplexer of inputs has select lines, which are used to select which input line to send to the output. Multiplexers are mainly used to increase the amount of data that can be sent over the network within a certain amount of time and bandwidth.

Block Diagram:**Truth Table:**

INPUTS				OUTPUT
en	s[2]	s[1]	s[0]	y
0	X	X	X	0
1	0	0	0	a[0]
1	0	0	1	a[1]
1	0	1	0	a[2]
1	0	1	1	a[3]
1	1	0	0	a[4]
1	1	0	1	a[5]

1	1	1	0	a[6]
1	1	1	1	a[7]

ALGORITHM:

STEP 1: Define the module for design block and declare the Ports as inputs : a as vector [7:0], s as vector [2:0], en and output as y.

STEP 2: Define Output as registers for the circuit according to behavioral modelling.

STEP 3: Describe the relationship between the inputs and outputs using Truth table.

STEP 4: Using step 4, design the code using Behavioral Modelling using case/if-else statements .

STEP 5: Define the module for stimulus block and declare inputs as registers and outputs as wires.

STEP 6: Instantiate the design module using connection by order.

STEP 7: Define the test values for inputs.

Design Block/Verilog Code using Case Statement:

```
module mux81(en, a, s, y);
```

```
input en;
```

```
input [7:0] a;
```

```
input [2:0] s;
```

```
output y;
```

```
reg y;
```

```
always@(en or s or a)
```

```
begin
```

```
if(!en)
```

```
    y = 1'b0;

else

    case(s)

3'b000: y = a[0];

3'b001: y = a[1];

3'b010: y = a[2];

3'b011: y = a[3];

3'b100: y = a[4];

3'b101: y = a[5];

3'b110: y = a[6];

3'b111: y = a[7];

default: y = 1'b0;

    endcase

end

endmodule
```

Verilog Code using if Statement:

```
module mux81(en, a, s, y);
input [7:0]a;
input [2:0]s;
input en;
output y;
reg y;
always @(en,s,a);
begin
if (en==1)
begin
```

```

if (s==000) y=a[0];
else if (s==001) y=a[1];
else if (s==001) y=a[2];
else if (s==001) y=a[3];
else if (s==001) y=a[4];
else if (s==001) y=a[5];
else if (s==001) y=a[6];
else if (s==001) y=a[7];
end
else y=0;
end
end
end module

```

Stimulus block/Test Bench Code:

```

module mux_81_tb();

reg [7:0] a;

reg en;

reg [2:0] s;

wire y;

mux81 UUT (en,a,s,y);

initial
begin
    en=1'b0; s=3'b000; a=8'd0;

    #10 en=1'b1;s=3'b001;a=8'b00000001;

    #10 en=1'b1;s=3'b010;a=8'b00000000;

    #10 en=1'b1;s=3'b011;a=8'b00000100;

    #10 en=1'b1;s=3'b100;a=8'b00000000;

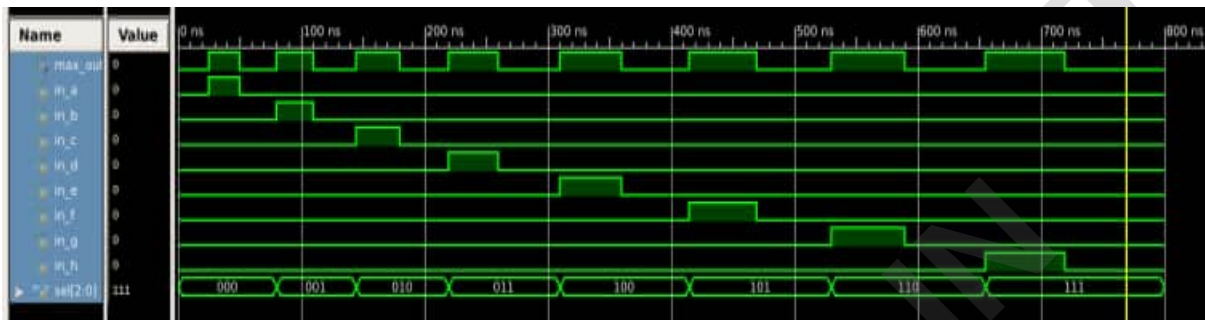
```

```
#10 en=1'b1;s=3'b101;a=8'b00010000;
```

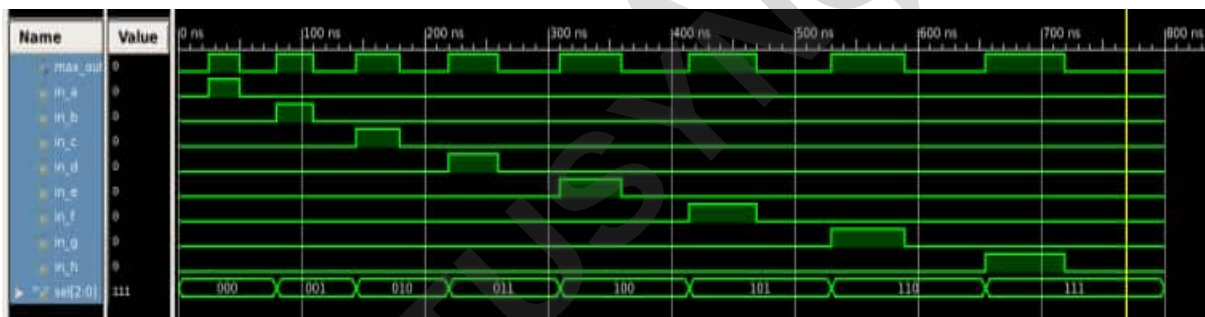
```
end
```

```
endmodule
```

Expected output waveform:



Output Waveform:



APPLICATIONS: Multiplexers are used in building digital semiconductors such as central processing units (CPUs) and graphics controllers. They are also used in communications.cross bar switch, cellphone systems, instrumentation, and any other function where only one transmission channel (e.g a radio transmitter) is available.

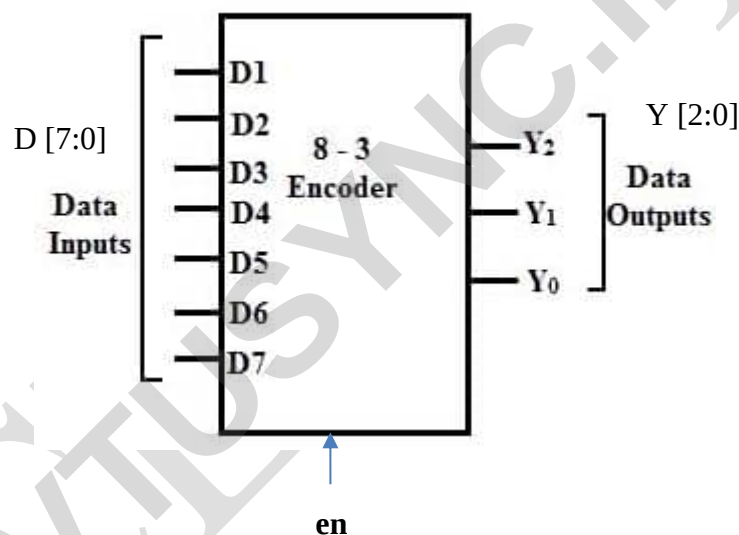
They mostly find in numerous and varied applications in digital systems of all types such as data selection, data routing, operation sequencing, parallel-to-serial conversion, waveform generation and logic-function generation.

b: 8 to 3 encoder without priority (behavioral model)

Theory: An Encoder is a device, circuit, transducer, software program, algorithm or person that converts information from one format or code to another. The purpose of encoder is standardization, speed, secrecy, security, or saving space by shrinking size. Encoders are combinational logic circuits and they are exactly opposite of decoders. They accept one or more inputs and generate a multibit output code.

Encoders perform exactly reverse operation than Decoder. An Encoder has M input and N output lines. Out of M input lines only one is activated at a time and produces equivalent code on output N lines. If a device output code has fewer bits than the input code has, the device is usually called an encoder.

Block diagram:



Truthtable:

INPUTS									OUTPUTS		
en	D[7]	D[6]	D[5]	D[4]	D[3]	D[2]	D[1]	D[0]	Y[2]	Y[1]	Y[0]
0	X	X	X	X	X	X	X	X	Z	Z	Z
1	0	0	0	0	0	0	0	1	0	0	0
1	0	0	0	0	0	0	1	0	0	0	1
1	0	0	0	0	0	1	0	0	0	1	0

1	0	0	0	0	1	0	0	0	0	1	1
1	0	0	0	1	0	0	0	0	1	0	0
1	0	0	1	0	0	0	0	0	1	0	1
1	0	1	0	0	0	0	0	0	1	1	0
1	1	0	0	0	0	0	0	0	1	1	1

ALGORITHM:

STEP 1: Define the module for design block and declare the Ports as input : D as vector [7:0] and output : Y as vector [2:0]

STEP 2: Define Output as registers according to behavioral modelling.

STEP 3: Describe the relationship between the inputs and outputs using Truth table.

STEP 4: Using step 4, design the code using Behavioral Modelling.

STEP 5: Define the module for stimulus block and declare inputs as registers and outputs as wires.

STEP 6: Instantiate the design module using connection by order.

STEP 7: Define the test values for inputs.

Design Block code:

```
module encoder83(D, en, Y);
```

```
    input en;
```

```
    input [7:0] D;
```

```
    output [2:0] Y;
```

```
    reg [2:0] Y;
```

```
    always @ (en or D)
```

```
begin

  if (!en)

    Y = 3'bZZZ;

  else

    case (D)

      8'b00000001 : Y = 3'b000;

      8'b00000010 : Y = 3'b001;

      8'b00000100 : Y = 3'b010;

      8'b00001000 : Y = 3'b011;

      8'b00010000 : Y = 3'b100;

      8'b00100000 : Y = 3'b101;

      8'b01000000 : Y = 3'b110;

      8'b10000000 : Y = 3'b111;

      default: Y = 3'bZZZ;

    endcase

  end

endmodule
```

Stimulus Block / Test bench:

```
module encoder83_tb( );

  reg en;

  reg [7:0] D;

  wire [2:0] Y;
```

```
encoder83 e1 (D, en, Y);
```

```
initial begin
```

```
    en = 1'b0;
```

```
    #20 en = 1'b1; D = 8'b00000001;
```

```
    #20 en = 1'b1; D = 8'b00000010;
```

```
    #20 en = 1'b1; D = 8'b00000100;
```

```
    #20 en = 1'b1; D = 8'b00001000;
```

```
    #20 en = 1'b1; D = 8'b00010000;
```

```
    #20 en = 1'b1; D = 8'b00100000;
```

```
    #20 en = 1'b1; D = 8'b01000000;
```

```
    #20 en = 1'b1; D = 8'b10000000;
```

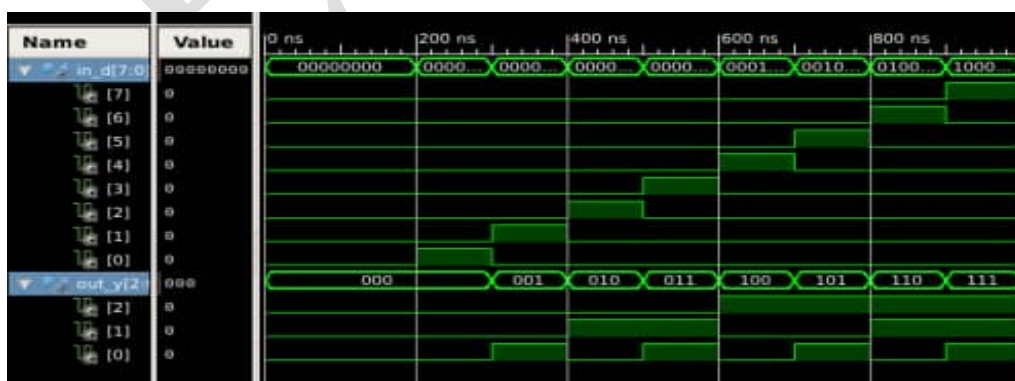
```
end
```

```
endmodule
```

Expected Waveform:

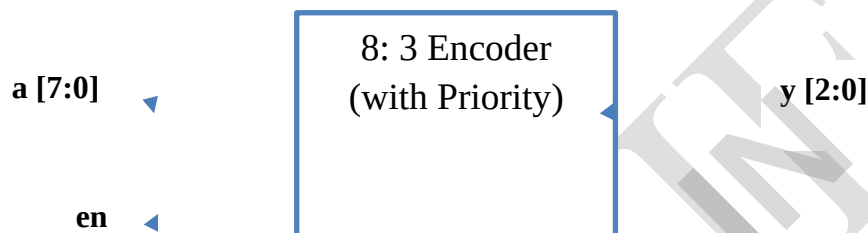


Output Waveform:



c: 8 to 3 encoder with priority (behavioral model)

Theory: An Encoder is a combinational circuit that performs the reverse operation of Decoder. It has maximum of 2^n input lines and 'n' output lines, hence it encodes the information from 2^n inputs into an n-bit code. Encoders are used to translate rotary or linear motion into a digital signal. The priority encoder is the binary representation of the original number starting from zero of the most significant input bit. They are often used to control interrupt requests by acting on the highest priority encoder. If two or more inputs are given at the same time, the input having the highest priority will be taken precedence.

Block Diagram:

Truth Table:

INPUTS									OUTPUTS		
en	a[7]	a[6]	a[5]	a[4]	a[3]	a[2]	a[1]	a[0]	y[2]	y[1]	y[0]
0	X	X	X	X	X	X	X	X	Z	Z	Z
1	0	0	0	0	0	0	0	1	0	0	0
1	0	0	0	0	0	0	1	X	0	0	1
1	0	0	0	0	0	1	X	X	0	1	0
1	0	0	0	0	1	X	X	X	0	1	1
1	0	0	0	1	X	X	X	X	1	0	0
1	0	0	1	X	X	X	X	X	1	0	1
1	0	1	X	X	X	X	X	X	1	1	0
1	1	X	X	X	X	X	X	X	1	1	1

ALGORITHM:

STEP 1: Define the module for design block and declare the Ports as inputs : a as vector [7:0] and outputs : y as vector [2:0].

STEP 2: Define Output as registers for the circuit according to behavioral modelling.

STEP 3: Describe the relationship between the inputs and outputs using Truth table.

STEP 4: Using step 4, design the code using Behavioral Modelling.

STEP 5: Define the module for stimulus block and declare inputs as registers and outputs as wires.

STEP 6: Instantiate the design module using connection by order.

STEP 7: Define the test values for inputs.

Verilog Code:

```

module encoder83pri (a, en, y);

    input en;

    input [7:0] a;

    output [2:0] y;

    reg [2:0] y;

    always @ (en or a)
    begin
        if (!en)
            y = 3'bZZZ;
        else
            casex (a)

                8'b00000001 : y = 3'b000;

                8'b0000001x : y = 3'b001;

```

```

8'b0000001xx : y = 3'b010;

8'b000001xxx : y = 3'b011;

8'b00001xxxx : y = 3'b100;

8'b0001xxxxx : y = 3'b101;

8'b001xxxxxx : y = 3'b110;

8'b01xxxxxxx : y = 3'b111;

default: y = 3'bZZZ;

endcase

end

endmodule

```

Stimulus Block / Test bench:

```

module encoder83pri_tb( );

    reg en;

    reg [7:0] a;

    wire [2:0] y;

    encoder83pri e1 (a, en, y);

    initial begin

        en = 1'b0;

        #20 en = 1'b1; a = 8'b000000001;

        #20 en = 1'b1; a = 8'b00000001X;

        #20 en = 1'b1; a = 8'b0000001XX;

        #20 en = 1'b1; a = 8'b000001XXX;
    end
endmodule

```

```

#20 en = 1'b1; a = 8'b0001XXXX;

#20 en = 1'b1; a = 8'b001XXXXX;

#20 en = 1'b1; a = 8'b01XXXXXX;

#20 en = 1'b1; a = 8'b1XXXXXXX;

end

endmodule

```

Expected Waveform:



Output Waveform:



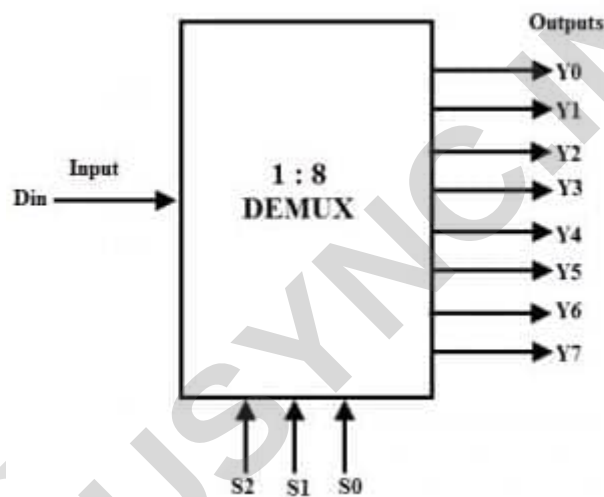
APPLICATIONS : Encoding is used in most wireless control systems to prevent interference. It is useful in web processes, handling and inspection systems that use conveyors and simple speed or position control in high vibration environments.

Experiment no: 6

Aim: To realize using Verilog Behavioral description: 1:8 Demux, 3:8 decoder, 2-bit Comparator

6.a) 1:8 Demux

THEORY: A demultiplexer (often abbreviated as "demux") is a combinational logic circuit that takes a single input line and routes it to one of several possible output lines based on the control signals applied to its select lines. The number of output lines is a power of 2, with 2^n output lines where n is the number of select lines. Demultiplexers are the opposite of multiplexers, which combine multiple inputs onto a single output line.

BLOCK DIAGRAM:**TRUTH TABLE:**

S2	S1	S0	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0
0	0	0	0	0	0	0	0	0	0	D
0	0	1	0	0	0	0	0	0	D	0
0	1	0	0	0	0	0	0	D	0	0
0	1	1	0	0	0	0	D	0	0	0
1	0	0	0	0	0	D	0	0	0	0
1	0	1	0	0	D	0	0	0	0	0

1	1	0	0	D	0	0	0	0	0	0
1	1	1	D	0	0	0	0	0	0	0

Verilog Code:

```
module demux_1_8(y,s,a);
```

```
output reg [7:0]y;
```

```
input [2:0]s;
```

```
input a;
```

```
always @*
```

```
begin y=0;
```

```
case(s)
```

```
3'd0: y[0]=a;
```

```
3'd1: y[1]=a;
```

```
3'd2: y[2]=a;
```

```
3'd3: y[3]=a;
```

```
3'd4: y[4]=a;
```

```
3'd5: y[5]=a;
```

```
3'd6: y[6]=a;
```

```
3'd7: y[7]=a;
```

```
endcase
```

```
end
```

```
endmodule
```

Data flow Description:

```
module Demultiplexer(in,s0,s1,s2,d0,d1,d2,d3,d4,d5,d6, d7);
```

```
input in,s0,s1,s2;
```

```
output d0,d1,d2,d3,d4,d5,d6,d7;
```

```
assign d0=(in & ~s2 & ~s1 &~s0)
```

```
assign d1=(in & ~s2 & ~s1 &s0)
```

```
assign d2=(in & ~s2 & s1 &~s0)
```

```
assign d3=(in & ~s2 & s1 &s0)
```

```
assign d4=(in & s2 & ~s1 &~s0)
```

```
assign d5=(in & s2 & ~s1 &s0)
```

```
assign d6=(in & s2 & s1 &~s0)
```

```
assign d7=(in & s2 & s1 &s0);
```

```
endmodule
```

Test Bench:

```
module TestModule;
```

```
// Inputs reg in; reg s0; reg s1; reg s2;
```

```
// Outputs wire d0; wire d1; wire d2; wire d3; wire d4; wire d5; wire d6; wire d7;
```

```
// Instantiate the Unit Under Test (UUT)
```

```
Demultiplexer uut
```

```
(.in(in),.s0(s0),.s1(s1),.s2(s2),.d0(d0),.d1(d1),.d2(d2),.d3(d3),.d4(d4),.d5(d5),.d6(d6),.d7(d7));
```

```
initial begin
```

```
// Initialize Inputs
```

```

in = 0; s0 = 0; s1 = 0; s2 = 0;

// Wait 100 ns for global reset to finish #100;

in = 1; s0 = 0; s1 = 1; s2 = 0;

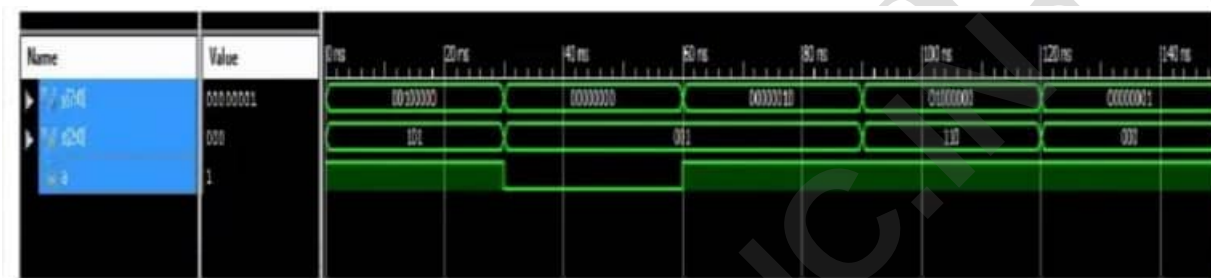
// Wait 100 ns for global reset to finish #100;

// Add stimulus here end

endmodule

```

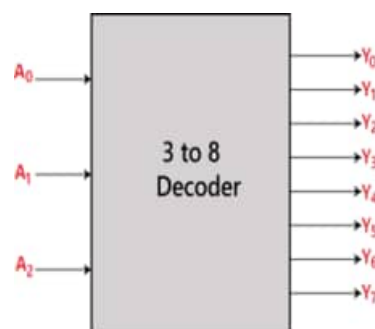
Output Waveform



6.b:3:8 decoder

Theory:

A decoder is a combinational logic circuit that converts binary information from input lines into a corresponding set of output lines. It's essentially the opposite of an encoder, which compresses multiple input lines into fewer output lines. Decoders are widely used in digital systems for tasks such as address decoding, control signal generation, and data routing. Here's the theory behind decoders:



Truth Table

Enable	INPUTS			Outputs							
E	A ₂	A ₁	A ₀	Y ₇	Y ₆	Y ₅	Y ₄	Y ₃	Y ₂	Y ₁	Y ₀
0	x	x	x	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0

Verilog Code

```
module decoder3_to_8( in,out, en); input [2:0] in;
```

```
input en;
```

```
output [7:0] out;
```

```
reg [7:0] out;
```

```
always @( in or en) begin
```

```
if (en)
```

```
begin
```

```
out=8'd0;
```

```
case (in)
```

```
3'b000: out[0]=1'b1;
```

```
3'b001: out[1]=1'b1;
```

```
3'b010: out[2]=1'b1;
```

```
3'b011: out[3]=1'b1;
```

```
3'b100: out[4]=1'b1;

3'b101: out[5]=1'b1;

3'b110: out[6]=1'b1;

3'b111: out[7]=1'b1;

default: out=8'd0; endcase

end else out=8'd0;

end

endmodule
```

Test bench Code

```
Tesmodule decoder_tb;

wire [7:0] out;

reg en;

reg [2:0] in; integer i;

decoder3_to_8 dut(in,out,en);

initial begin

$monitor( "en=%b, in=%d, out=%b ", en, in, out);

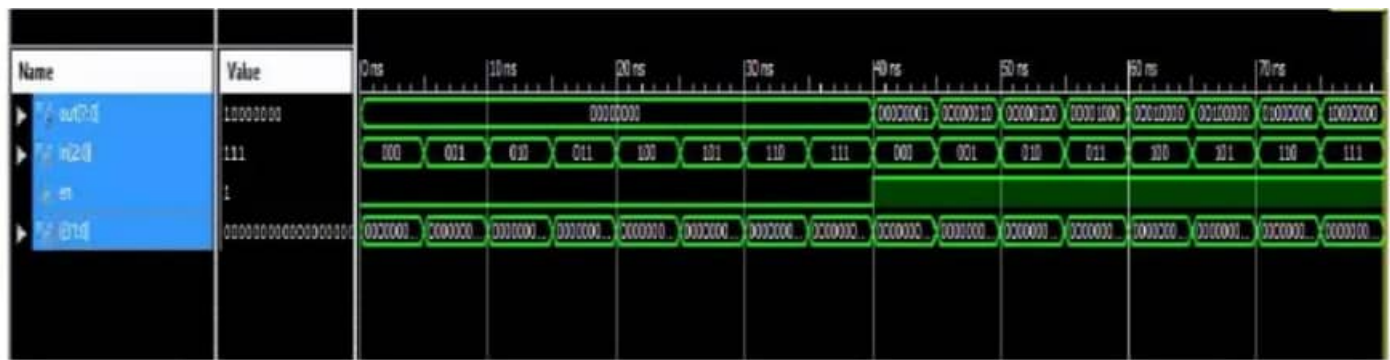
for ( i=0; i<16; i=i+1) begin

{en,in} = i; #1;

end end

endmodule
```

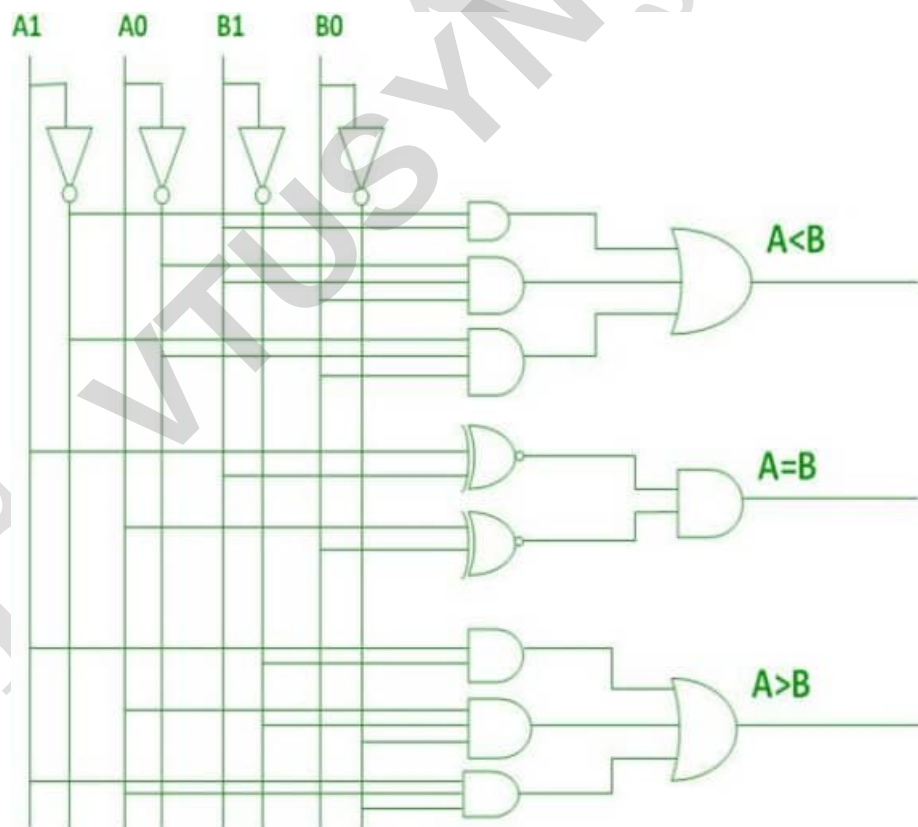
Output Waveform



2-bit Comparator

Theory: A comparator is a fundamental electronic circuit used to compare two input signals and determine their relationship, such as which signal is greater, lesser, or if they are equal. Comparators are widely used in various applications, including analog-to-digital converters, voltage level detection, waveform shaping, and control systems.

Logic Diagram:



Truth Table

INPUT				OUTPUT		
A1	A0	B1	B0	A<B	A=B	A>B
0	0	0	0	0	1	0
0	0	0	1	1	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	1	0

Verilog code for 2-bit comparator

```
module comp2(input [1:0] a, b,output reg equal, greater, lower);
```

```
always @ (a or b)
```

```
begin
```

```
if (a < b)
```

```
begin
```

```
greater = 0; equal = 0; lower = 1;
```

```
end
```

```
else if (a == b
```

```
begin
```

```
greater = 0; equal = 1; lower = 0;
```

```
end
```

```
else
```

```
begin
```

```
greater = 1; equal = 0; lower = 0;
```

```
end
```

```
end
```

```
endmodule
```

Stimulus Block

```
module comp2_tb;
// Inputs
reg [1:0] a;
reg [1:0] b;
// Outputs
wire equal;
wire greater;
wire lower;
// Instantiate the Unit Under Test (UUT)
comp2 uut (.a(a),.b(b),.equal(equal),.greater(greater),.lower(lower));
initial
begin
// Initialize Inputs
a = 0; b = 0;#100;
#100; a = 2; b = 1;
#100; a = 1; b = 2;
#100; a = 3; b = 3;
end
endmodule
```

Output Waveform



Result: The Design for 2 bit Comparator using Verilog code is simulated and executed.

Applications: Comparators find extensive use in a wide range of electronic systems for various tasks such as analog-to-digital conversion, voltage level detection, signal conditioning, and synchronization. Their versatility and simplicity make them essential building blocks in many electronic designs.

Experiment no: 7

Write Verilog code for to realize using Verilog Behavioral description:

Flip-flops:

a) JK type b) SR type c) D type

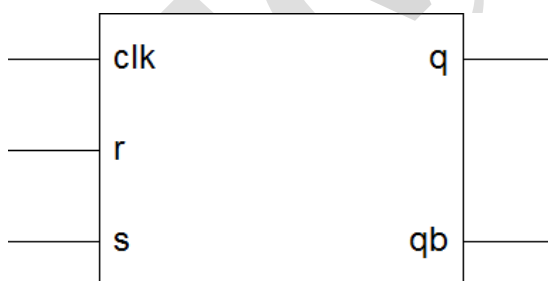
Objective: - Develop and simulate the Verilog code for the following flip-flops SR , JK ,D**Theory:** -

SR flip-flop: A SR flip-flop is the simplest possible memory element. The SR flip flop has two inputs Set and Reset. The SR flip-flop is a basic building block for other flip-flops.

D flip-flop: This is a flip-flop with a delay (D) equal to exactly equal to one cycle of the clock. The defect with SR FF is the indeterminate output when the data inputs at S and R are 1. In order to avoid this the input to R is through an inverter from S so that the input to R is always the complement of S and never same. The S input is redesignated as D.

JK flip-flop: The JK flip-flop is called a “universal flip flop” because the other flip flops like D, SR, T can be derived from it. The “racing or race around condition” takes place in a JK FF when $J=1$ and $K=1$ and $\text{clock}=1$.

TFF

A) SR Flip-flop**a. Block Diagram:****Truth Table:**

INPUTS			OUTPUTS	
clk	s	r	q	qb
↓	X	X	X	X
↑	0	0	q	qb
↑	0	1	0	1
↑	1	0	1	0
↑	1	1	invalid	invalid

ALGORITHM:

STEP 1: Define the module for design block and declare the input and output Ports

STEP 2: Describe the relationship between the inputs and outputs using Truth table.

STEP 3: Using step 4, design the code using behavioral modelling.

STEP 4: Define the module for stimulus block and declare inputs as registers and outputs as wires.

STEP 5: Instantiate the design module using connection by order.

STEP 6: Define the test values for inputs.

Design Block/ Design Code:

```
module sr_ff (clk, s, r, q, qb);
```

```
input clk, s, r;
```

```
output q, qb;
```

```
reg q, qb;
```

```
always @(posedge clk)
```

```
begin

    case ({s, r})

        2'b00: q = q;

        2'b01: q = 0;

        2'b10: q = 1;

        2'b11: q = 1'bz;
```

```
endcase
```

```
    qb = ~ q;
```

```
end
```

```
endmodule
```

Stimulus Block / Test bench

```
module sr_ff_tb( );
```

```
reg clk, s, r;
```

```
wire q, qb;
```

```
sr_ff s1 (clk, s, r, q, qb);
```

```
initial
```

```
begin
```

```
clk = 1'b0;
```

```
forever #10 clk = ~ clk;
```

```
end
```

```
initial
```

```
begin
```

```
    s = 1'b0; r = 1'b0;
```

```
#20 s = 1'b0; r = 1'b1;
```

```
#20 s = 1'b1; r = 1'b0;
```

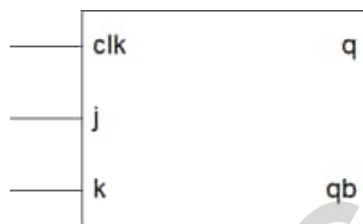
```
#20 s = 1'b1; r = 1'b1;
```

```
end
```

```
endmodule
```

b. JK flip-flop Block Diagram:

Block Diagram:



Truth Table:

INPUTS			OUTPUTS	
clk	j	k	q	qb
↓	X	X	X	X
↑	0	0	q	qb
↑	0	1	0	1
↑	1	0	1	0
↑	1	1	qb	q

Verilog Code:

```
module jk_ff(clk, j, k, q, qb);
```

```
input clk, j, k;

output q, qb;

reg q, qb;

always@(posedge clk)

begin

    case ({j, k})

        2'b00: q = q;

        2'b01: q = 0;

        2'b10: q = 1;

        2'b11: q = ~q;

    endcase

    qb = ~q;

end

endmodule
```

Stimulus Block / Test bench:

```
module jk_ff_tb();

    reg clk, j, k;

    wire q, qb;

    jk_ff j1 (clk, j, k, q, qb);

    initial

    begin

        clk = 1'b0;

        forever #5 clk = ~ clk;
```

```

end

initial

begin

    j = 1'b0; k = 1'b0;

#20 j = 1'b0; k = 1'b1;

#20 j = 1'b1; k = 1'b0;

#20 j = 1'b1; k = 1'b1;

end

endmodule

```

3)D flip-flop

Block Diagram:



Truth Table:

INPUTS		OUTPUTS	
clk	d	q	qb
↓	X	X	X
↑	1	1	0
↑	0	0	1

Verilog Code:

```
module d_ff(clk, d, q, qb );
```

```
input clk, d;

output q, qb;

reg q, qb;

always@(posedge clk)

begin

    q = d;

    qb = ~q;

end

endmodule
```

Stimulus Block / Test bench:

```
module d_ff_tb();

reg clk, d;

wire q, qb;

d_ff d1 (clk, d, q, qb);

initial

begin

    clk = 1'b0;

    forever #5 clk = ~clk;

end

initial

begin

    d = 1'b0;

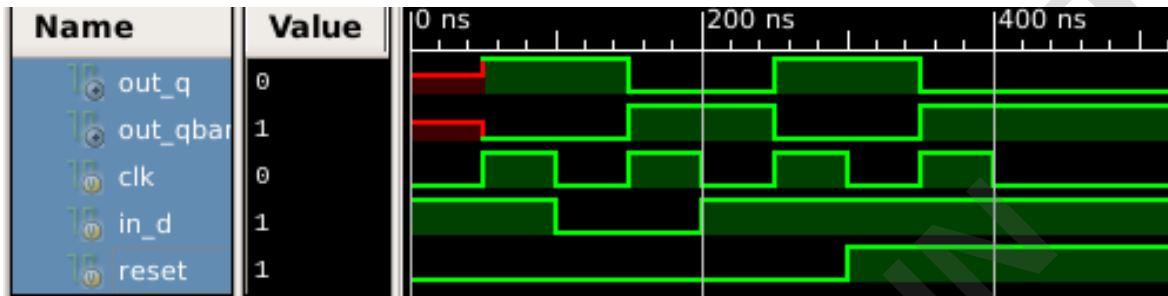
    #20 d = 1'b1;
```

```
end
```

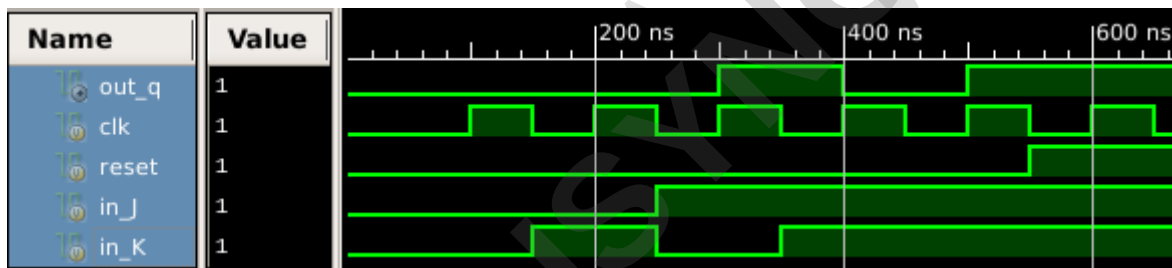
```
endmodule
```

Output waveform:

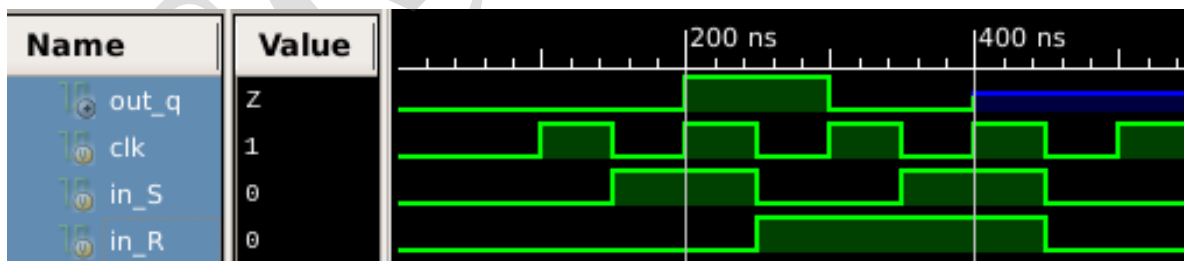
D flipflop



JK flipflop



SR flipflop



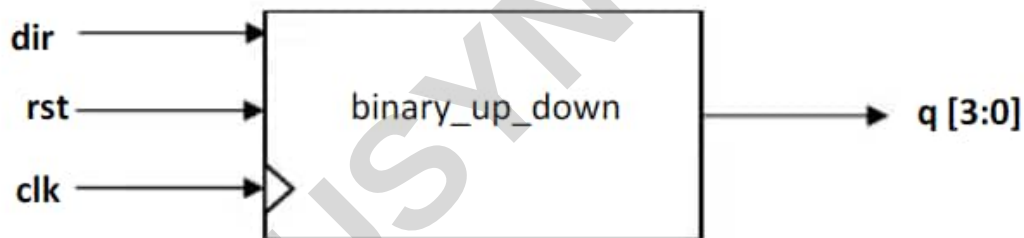
Result: The design for flipflops using Verilog code is simulated and executed.

Applications: Flip-flops are fundamental building blocks in digital electronics, playing crucial roles in data storage, manipulation, and control in various electronic systems and applications. Their versatility and reliability make them indispensable components in modern digital designs.

Experiment no. 8:

Aim: To realize Counters - up/down (BCD and binary) using Verilog Behavioral description.

Theory: - Basically the counters are the sequential blocks where in we use to generate a sequence of numbers with respect to the clock counts which is a control signal provided for the design. Counter can count and store the number of time any particular event or process have occurred, depending on a clock signal. Counters can be modeled of synchronous or Asynchronous types based on the clock applied for the design to execute the logic, Asynchronous counters are those whose output is free from the clock signal. Because the flip flops in asynchronous counters are supplied with different clock signals, there may be delay in producing output. Counters can start counting not only from the zero, In fact it can be initiated to any random value to make a count from some non-zero number and it is also possible to increment the counter with more than one in one clock period.

BINARY UP/DOWN COUNTER:**BLOCK DIAGRAM:****TRUTHABLE**

INPUTS		OUTPUTS			
clk	rst	q[3]	q[2]	q[1]	q[0]
↓	X	0	0	0	0
↑	1	0	0	0	0
↑	0	0	0	0	1
↑	0	0	0	1	0

↑	0	0	0	1	1
↑	0	0	1	0	0
↑	0	0	1	0	1
↑	0	0	1	1	0
↑	0	0	1	1	1
↑	0	1	0	0	0
↑	0	1	0	0	1
↑	0	1	0	1	0
↑	0	1	0	1	1
↑	0	1	1	0	0
↑	0	1	1	0	0
↑	0	1	1	0	1
↑	0	1	1	1	0
↑	0	1	1	1	1

ALGORITHM

STEP 1: Define the module for design block and declare the input and output Ports

STEP 2: Describe the relationship between the inputs and outputs using Truth table.

STEP 3: Using step 4, design the code using behavioral modelling.

STEP 4: Define the module for stimulus block and declare inputs as registers and outputs as wires.

STEP 5: Instantiate the design module using connection by order.

STEP 6: Define the test values for inputs.

Design Block/Design Code

```

module binary_up_down(input clk,input rst,input dir,output reg [3:0] q);

initial q = 0;

always @ (posedge clk)

if (rst)q = 0;

elseif (dir)

q = q + 1;

else q = q - 1;

endmodule

```

Stimulus Block:

```

module binar_up_down_tb;

// Inputs

reg clk;

reg rst;

reg dir;

// Outputs

wire [3:0] q;

// Instantiate the Unit Under Test (UUT)

binary_up_down uut (.clk(clk),.rst(rst),.dir(dir),.q(q));

initial

begin

rst = 1;dir = 1;#100;

rst = 0;#300;d

ir = 0;#250;

$finish;

end

Initial

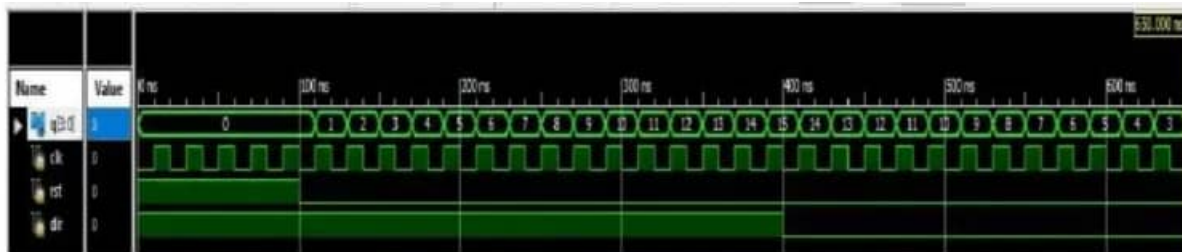
```

```
clk = 0;
```

```
always#10 clk = ~clk;
```

```
endmodule
```

Results:



BCD UP/DOWN COUNTER



TRUTHABLE

INPUTS		OUTPUTS			
clk	rst	q[3]	q[2]	q[1]	q[0]
↓	X	0	0	0	0
↑	1	0	0	0	0
↑	0	0	0	0	1
↑	0	0	0	1	0
↑	0	0	0	1	1
↑	0	0	1	0	0
↑	0	0	1	0	1

↑	0	0	1	1	0
↑	0	0	1	1	1
↑	0	1	0	0	0
↑	0	1	0	0	1
↑	0	0	0	0	0

Design Block:

```
module bcd_up_down(input clk,input rst,input dir,output reg [3:0] q);
```

```
initial q = 0;
```

```
always @ (posedge clk)
```

```
if (rst) q = 0;
```

```
elseif (dir)
```

```
q = (q==9) ? 0 : q + 1;
```

```
else q = (q == 0) ? 9 : q - 1;
```

```
endmodule
```

Stimulus Block:

```
module bcd_up_down_tb;
```

```
// Inputs
```

```
reg clk;
```

```
reg rst;
```

```
reg dir;
```

```
// Outputs
```

```
wire [3:0] q;
```

```
// Instantiate the Unit Under Test (UUT)
```

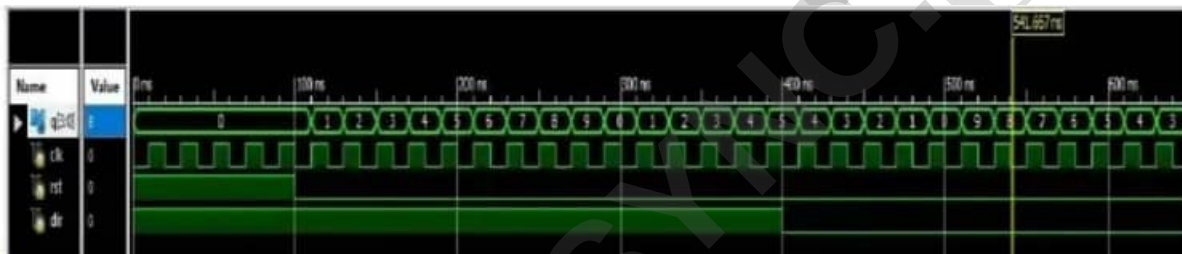
```
bcd_up_down uut (.clk(clk),.rst(rst),.dir(dir),.q(q));
```

```

initial
begin
rst = 1;dir = 1;#100
;rst = 0;#300;
dir = 0;#250;$finish;
end
initial
clk = 0;
always#10 clk = ~ clk;
endmodule

```

Results:



Result: The design for Counters using Verilog code is simulated and executed.

Applications: counters play a crucial role in digital systems, providing the capability to track, control, and manipulate sequences of events or signals in a wide range of applications.

DEMONSTRATION PROGRAMS:

Aim: Design Verilog Program to interface a Stepper motor to the FPGA/CPLD and rotate the motor in the specified direction (by N steps).

Hardware and Software required: FPGA-Spartan 6 Kit, Xilinx ISE

Theory: A stepper motor is a type of electric motor that divides a full rotation into a number of equal steps. It's commonly used in various applications where precise control of rotational motion is required, such as in CNC machines, 3D printers, robotics, and more. There are several theories underlying the operation and control of stepper motors:

Electromagnetic Principle: Like all electric motors, stepper motors operate based on the principle of electromagnetism. They consist of coils (windings) wound around a rotor and stator. When an electric current flows through these coils, a magnetic field is generated, causing the rotor to move.

Step Angle: The step angle of a stepper motor is the angle through which the motor rotates for each step. This is determined by the design of the motor and the number of steps per revolution. For example, a motor with a step angle of 1.8 degrees will move 1.8 degrees for each step.

Block diagram:

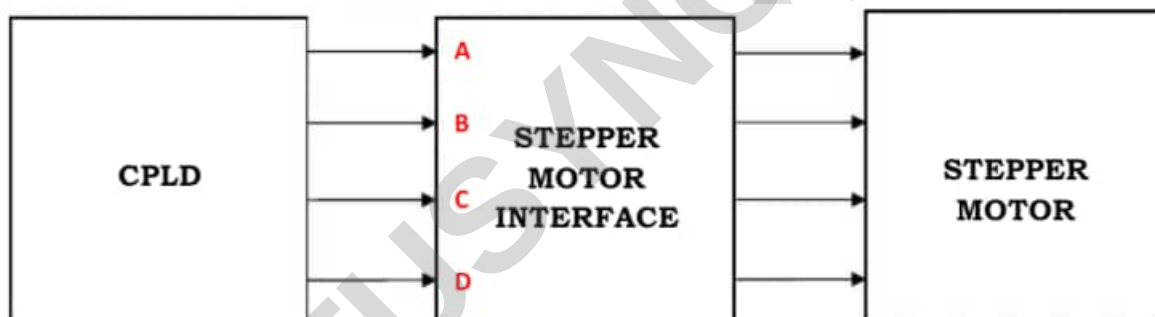


Fig. Connections of CPLD with Stepper Motor Interface and Stepper Motor

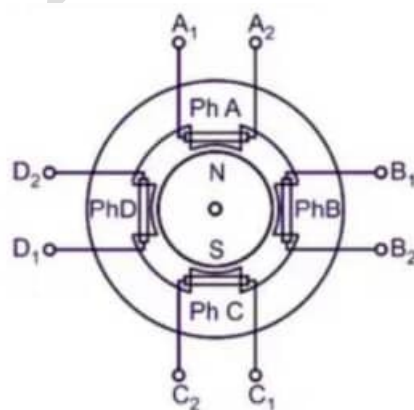


Fig. Cross section of a typical Stepper Motor

Steps	Clockwise rotation Sequence				Steps	Anti-Clockwise rotation Sequence			
	D	C	B	A		D	C	B	A
1	0	0	1	1	1	1	0	0	1
2	0	1	1	0	2	1	1	0	0
3	1	1	0	0	3	0	1	1	0
4	1	0	0	1	4	0	0	1	1

Calculation of the steps for required rotation angle:

Full step excitation

Step angle = 1.8°

No. of steps = Required rotation angle in Degrees / Step angle

Eg: 1. For 360° rotation,

No. of steps = $360 / 1.8 = 200$

2. For 180° rotation,

No. of steps = $180 / 1.8 = 100$

DESIGN CODE:

```
module STP_mtr(clk, dir, d_out);
```

```
input clk;
```

```
input [1:0] dir;
```

```
output reg [3:0] d_out;
```

```
reg [30:0] clk_div = 31'd0;
```

```
reg [7:0] count=8'h0;
```

```
reg [3:0] shift_reg = 4'b1001;
```

```
reg [1:0] state = 2'b00;
```

```
always@(posedge clk)
```

```
begin
```

```
clk_div = clk_div + 1;
```

```
end
```

```
always@(posedge clk_div[16])

begin

if(dir == 2'b10 && state == 2'b00)

begin

shift_reg = {shift_reg[0], shift_reg[3:1]}; // clock wise

count = count + 1;

if (count == 8'd200)

begin

state = state+1;

count = 8'd0;

end

end

else if(dir == 2'b01 && state == 2'b01)

begin

shift_reg = {shift_reg[2:0], shift_reg[3]}; // anti clock

count = count + 1;

if (count == 8'd200)

begin

state = state+1;

count = 8'd0;

end

end

else

begin

shift_reg = shift_reg;

if (state==2'b10)

state =2'b00;
```

```
end  
d_out = shift_reg;  
end  
endmodule
```

UCF FILE

```
NET "clk" LOC = p56;  
NET "dir[0]" LOC = p124;  
NET "dir[1]" LOC= p123;  
NET "d_out[0]" LOC = p29;  
NET "d_out[1]" LOC = p30;  
NET "d_out[2]" LOC = p32;  
NET "d_out[3]" LOC = p33;
```

Result: A Stepper Motor is interfaced to the FPGA Kit and the changes in speed and direction is observed accordingly.

DEMONSTRATION EXPERIMENT-2:

Aim: Design Verilog programs to interface Switches and LEDs to the FPGA/CPLD and demonstrate its working.

Hardware and Software required: FPGA-Spartan 6 Kit, Xilinx ISE

Theory: FPGAs have dedicated I/O pins that can be configured as inputs or outputs based on your design requirements. Switches are typically digital inputs. They can be connected to FPGA input pins.

Each switch is connected to a separate input pin on the FPGA. Connect the switches to the input pins of the FPGA. Typically, switches are connected between the input pin and either VCC (for pull-up configuration) or GND (for pull-down configuration) through a current-limiting resistor to protect the FPGA from excessive current. In your FPGA design, you'll need to create input and output ports corresponding to the switch and LED interfaces.

DESIGN CODE::

```
module blink_LED (input clk, output reg led );
    reg [24:0] counter; // 25-bit counter for timing
    always @(posedge clk) begin
        if (counter == 25000000) begin // Adjust this value for desired blink frequency
            led <= ~led; // Toggle the LED
            counter <= 0; // Reset the counter
        end else begin
            counter <= counter + 1; // Increment the counter
        end
    end
end

endmodule
```

UCF:

NET "led" LOC = P98;

NET "clk" LOC = P56;

ADDITIONAL EXPERIMENTS:**1)Design Ripple carry counter**

Design code:

```
module cntr(q,clk,reset);
```

```
output [3:0]q;
```

```
input clk, reset;
```

```
T_FF tff0(q[0],clk,reset);
```

```
T_FF tff1(q[1],q[0],reset);
```

```
T_FF tff2(q[2],q[1],reset);
```

```
T_FF tff3(q[3],q[2],reset);
```

```
endmodule
```

```
module T_FF(q,clk,reset);
```

```
input clk, reset;
```

```
output q;
```

```
wire d;
```

```
D_FF dff0(q,d,clk,reset);
```

```
not n1(d,q);
```

```
endmodule
```

```
module D_FF(q,d,clk,reset);
```

```
output q;
```

```
input clk,reset,d;
```

```
reg q;
```

```
always@(posedge reset or negedge clk)
```

```
begin
```

```
if(reset)
```

```
q =1'b0;
```

```
else  
q =d;  
end  
endmodule
```

TEST BENCH CODE:

```
module tb;  
    // Inputs  
    reg clk;  
    reg reset;  
    // Outputs  
    wire [3:0] q;  
    // Instantiate the Unit Under Test (UUT)  
    cnrt uut (.clk(clk), .reset(reset), .q(q));  
    initial begin  
        // Initialize Inputs  
        clk = 1'b1;  
        forever #5 clk =~clk;  
        end  
        initial  
        begin  
            reset = 1'b1;  
            #15 reset = 1'b0;  
            #180 reset = 1'b1;  
            #10 reset = 1'b0;  
        end  
    end  
endmodule
```



```
module stimulus;

reg [3:0] a;

reg [3:0] b;

reg c_in;

wire [3:0] sum;

wire c_out;

fulladder uut (

    .a(a),

    .b(b),

    .c_in(c_in),

    .sum(sum),

    .c_out(c_out)

);

initial begin

    // Initialize Inputs

    a=4'd0;b=4'd0;c_in=1'b0;

    #5 a=4'd3; b=4'd4;

    #5 a=4'd2; b=4'd5;

    #5 a=4'd9; b=4'd9;

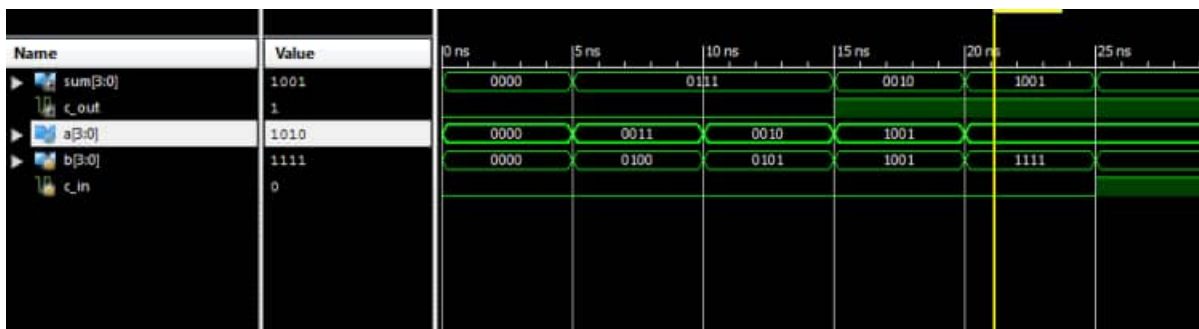
    #5 a=4'd10; b=4'd15;

    #5 a=4'd10; b=4'd5; c_in=1'b1;

end

endmodule
```

Output Waveform:



ADDITIONAL DEMONSTRATION:

Aim: Interface a DC motor to FPGA and write Verilog code to change its speed and direction.

Design Code:

```
module DC_MOTOR(clk,dir,speed,P_DCMEN,P_DCM);
```

```
input clk,dir;
```

```
input [1:0]speed;
```

```
output reg P_DCMEN;
```

```
output reg [1:0] P_DCM ;
```

```
reg [32:0] clk_div = 33'd0;
```

```
always@(posedge clk)
```

```
clk_div = clk_div + 1;
```

```
always@(clk_div)
```

```
begin
```

```
if (speed == 2'b00) P_DCMEN = clk_div[2];
```

```
if (speed == 2'b01) P_DCMEN = clk_div[4];
```

```
if (speed == 2'b10) P_DCMEN = clk_div[8];
```

```
if (speed == 2'b11) P_DCMEN = clk_div[16];
```

```
end
```

```
always@(clk_div)
```

```
begin
```

```
if (dir == 0)
```

```
P_DCM = 2'b01;  
  
else  
  
P_DCM = 2'b10;  
  
end  
  
endmodule
```

UCF:

```
NET "P_DCM[0]" LOC = P82;  
NET "P_DCM[1]" LOC = P83;  
NET "P_DCMEN" LOC = P84;  
NET "dir" LOC = P121 ;  
NET "speed(0)" LOC = P123;  
NET "speed(1)" LOC = P124;  
NET "clk" LOC = P56;
```

Result: A DC Motor is interfaced to the FPGA Kit and the changes in speed and direction is observed accordingly.

VIVA-QUESTIONS AND ANSWERS

1. What is Verilog and how does it differ from VHDL?

Verilog is a hardware description language that comprises structural and behavioural statements. While circuit components, such as gates and microprocessors, possess a design featuring structural statements, programming uses behavioural statements, such as loops and if and then statements. Design or verification engineers can use this language to create textual descriptions for digital logic circuits and define connections between signals.

Simplicity and portability are factors that differentiate Verilog from VHDL. For instance, Verilog is a low-level computing language, which makes it easy to understand. Its ability to run on any type of hardware also makes it highly portable, whereas VHDL possesses more functions and libraries, which helps simplify debugging.

2.What is FPGA?

Answer: FPGA is field-programmable gate array, which is a type of integrated circuit. FPGA makes it easier to customise circuits and fulfil specific design requirements. It has several gates that users can arrange specifically to develop customised digital circuits. Some of the gates present in an FPGA include communication interfaces and a memory controller.

With these gates, users can establish a connection to design distinct circuits and meet particular technological requirements. FPGA is highly flexible, and users can program it to be a microcontroller or a microprocessor. Users can also employ it to effectively design and curate circuit structures to achieve the required hardware architecture goals.

3.What are some key differences between blocking and non-blocking?

Answer: Blocking statements allow users to execute statements in a sequence. For instance, if we have three statements, then blocking allows us to execute the first statement, then the second one and then the third one. In non-blocking statements, we can execute them in a parallel manner. Typically, users can identify the two types of statements with the help of assignment operators. While the notation for a blocking statement is =, the notation for a non-blocking statement is <=.

A blocking assignment statement's behaviour is similar to that of older programming languages, which means that the execution still finishes before moving on to the next statement. A non-blocking operator's evaluation

considers the right-hand side for the present time unit and works with the left-hand side later at the conclusion of the time unit.

Question 4: How Blocking And Non Blocking Statements Get Executed?

Answer: Execution of blocking assignments can be viewed as a one-step process:

1. Evaluate the RHS (right-hand side equation) and update the LHS (left-hand side expression) of the blocking assignment without interruption from any other Verilog statement. A blocking assignment "blocks" trailing assignments in the same always block from occurring until after the current assignment has been completed

Execution of nonblocking assignments can be viewed as a two-step process:

Evaluate the RHS of nonblocking statements at the beginning of the time step.

Update the LHS of nonblocking statements at the end of the time step.

Question 5 Variable And Signal Which Will Be Updated First?

Answer: Signals

Question 6. What Is Sensitivity List?

Answer: The sensitivity list indicates that when a change occurs to any one of elements in the list change, begin...end statement inside that always block will get executed.

Question 7. In A Pure Combinational Circuit Is It Necessary To Mention All The Inputs In Sensitivity List? If Yes, Why?

Answer: Yes in a pure combinational circuit it is necessary to mention all the inputs in sensitivity list otherwise it will result in pre and post synthesis mismatch.

8 What is the difference between wire and reg?

Answer: Wire is the physical connection between Verilog's structural elements, and Verilog requires these elements to function properly. A continuous assignment or gate output defines the value of wire. Reg, or integer, time, real and real-time, is a representation of the abstract data storage element. Users can only assign value to reg in an initial or always statement.

The key difference between reg and wire is that wire requires a connection between a and b to store value. Wire loses value without a connection, $a \rightarrow b$, but reg can hold value even in the absence of a physical connection. While Z is the default value of wire, X is the default value of reg.

Question 9. What Is The Difference Between `===` And `==` ?

Answer:

output of `=="` can be 1, 0 or X.

output of `==="` can only be 0 or 1.

When you are comparing 2 nos using `=="` and if one/both the numbers have one or more bits as "x" then the output would be "X" . But if use `==="` output would be 0 or 1.

e.g `A = 3'b1x0`

`B = 3'b10x`

`A == B` will give X as output.

`A === B` will give 0 as output.

`=="` is used for comparison of only 1's and 0's .It can't compare Xs. If any bit of the input is X output will be X

10.Explain the concept of a flip-flop in Verilog and provide an example of its usage in a real-world application.

Answer: In Verilog, a flip-flop is a fundamental building block used to store and synchronize data in digital circuits. It is a sequential logic element that operates based on a clock signal. There are different types of flip-flops, such as D flip-flop, JK flip-flop, and T flip-flop, each with specific functionality.

Example usage: A common real-world application of a flip-flop is in the design of registers or memory elements. For instance, in a microcontroller or processor, flip-flops are used to store intermediate results, control signals, or temporary data during the execution of instructions.

Question:11: What is the purpose of a testbench in Verilog, and why is it essential during the development process?

Answer:A testbench in Verilog is a module or set of modules used to simulate and verify the functionality of a design. It is an essential part of the development process because it allows developers to create test scenarios and test the correctness of their designs before synthesis and implementation in hardware.

The testbench generates test stimuli to apply to the design and monitors the responses or outputs. By comparing the expected outputs with the actual outputs during simulation, developers can identify design issues, validate the correctness of the hardware description, and ensure the design meets the desired specifications. This verification process helps catch bugs and ensures the design behaves as expected in different scenarios. `==` is used for comparison of X also.

12.Explain the purpose of the always block in Verilog and the difference between always `@(*)` and always `@(posedge clk)`.

Answer:

The always block in Verilog is used to define combinational or sequential logic. It is the fundamental construct used to describe how signals and variables are updated in response to changes in the sensitivity list (the part inside the parenthesis).

always `@(*)`: This is a combinational always block, also known as a sensitivity list or sensitivity to all. The block executes whenever any signal inside the block changes. It is commonly used for combinational logic, where the output depends only on the current values of inputs and not on the past values.

Example:

```
always @(*)
```

```
begin
```

```
    sum = a + b;
```

```
    product = a * b;
```

```
end
```

always `@(posedge clk)`: This is a sequential always block with a positive edge clock trigger. The block executes only when there is a positive edge (rising edge) on the clk signal. It is used to model flip-flops or other sequential elements that change state only on clock edges.

Example:

```
always @(posedge clk)
```

```
begin
```

```
if (reset) // Reset the counter if 'reset' signal is asserted

    counter <= 4'b0;

else

    counter <= counter + 1; // Increment the counter on every clock edge

end
```

The main difference between the two always blocks lies in their behavior:

always @(*) executes whenever any signal inside the block changes and represents combinational logic.

always @(posedge clk) executes only on the positive edge of the clk signal and is used for sequential logic elements like flip-flops.